

文档编号	TY-SUM-20231030A
版 本	A
页 数	536 页

科学计算环境 数学算法替换用户手册

科学计算与系统建模仿真项目组

编制	刘尚	生效日期	2023 年 10 月 30 日
审核	郭俊峰	批准	刘奇

文件变更摘要

日期	版本	变更说明	修订	审核	批准
2023.10.30	V0.1	初版建立	刘尚	郭俊峰	刘奇

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

目录

1. 基本原理	1
1.1 科学计算算法替换原理.....	1
1.2 算法注册/设置接口	4
1.2.1 BLAS 接口	4
1.2.2 LAPACK 接口	5
1.2.3 FFT 接口	5
2. 科学计算算法	7
2.1 BLAS 接口说明	7
2.1.1 Level 1	7
2.1.2 Level 2	37
2.1.3 Level 3	111
2.2 LAPACK 接口说明	150
2.2.1 矩阵分解函数.....	150
2.2.2 线性方程组求解函数.....	177
2.2.3 矩阵求逆函数.....	189
2.2.4 回代求解函数.....	195
2.2.5 特征值问题函数.....	206
2.2.6 其他函数.....	216
2.2.7 Support 函数	265
2.3 FFT 接口说明	267
2.3.1 C2C 变换	267
2.3.2 R2C 变换	297
2.3.3 C2R 变换	325
2.3.4 R2R 变换	351
2.3.5 变换执行函数.....	373
2.3.6 内存函数.....	383
2.3.7 线程函数.....	389
2.3.8 获取 FFT 产品版本信息	396
2.4 SPARSE 接口说明	398
2.4.1 Level 1	398

2.4.2 LeveL 2 和 LeveL 3	416
2.4.3 Support 函数	492
3. 算法替换案例	494
3.1 替换案例一：OpenBLAS@0.3.21	495
3.1.2 切换 BLAS/LAPACK 库	499
3.2 替换案例二：BLIS	500
3.2.1 编译 BLIS 库	500
3.2.2 替换 BLIS 库	500
3.3 KMLBLAS 测试报告	501
3.3.1 测试环境	501
3.3.2 测试	502
3.3.3 函数列表	502
3.3.4 单元测试结果及对比	506
3.3.5 测试结论	521
3.4 切换 FFTW 库	521
3.4.1 构建依赖 KML 库	522
3.4.2 函数库使用	522
3.4.3 KMLFFT 测试报告	523

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

1. 基本原理

1.1 科学计算算法替换原理

科学计算函数库分为两个层次，底层算法和上层应用。通过上层应用来调用底层算法，如图所示：

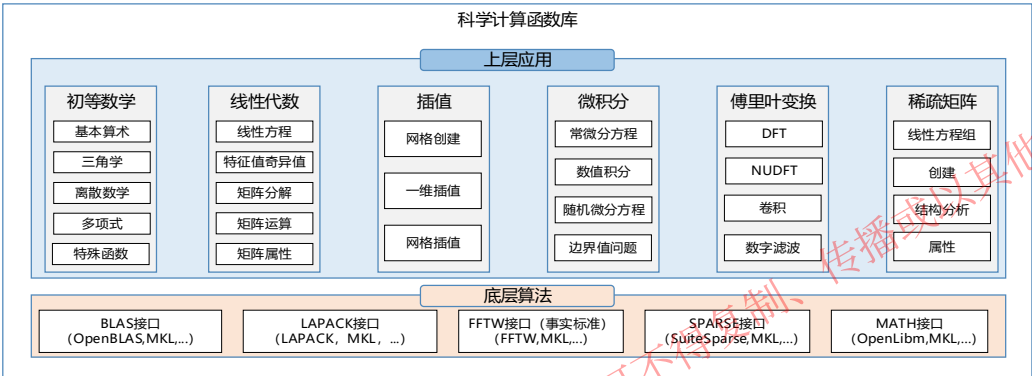


图 1-1 科学计算函数库架构

上层应用包含直接对外暴露、用户可直接调用使用的函数集，其大致分类为初等数学、线性代数、插值、微积分、傅里叶变换、稀疏矩阵等板块函数。

底层算法包含 BLAS/LAPACK 等有关矢量、矩阵乘法、矩阵分解、线性方程组求解等底层核心算法工具集，作为基础数学工具箱内核，支撑整个基础数学算法。

当前科学计算函数库中各子库间的依赖关系如下图所示：

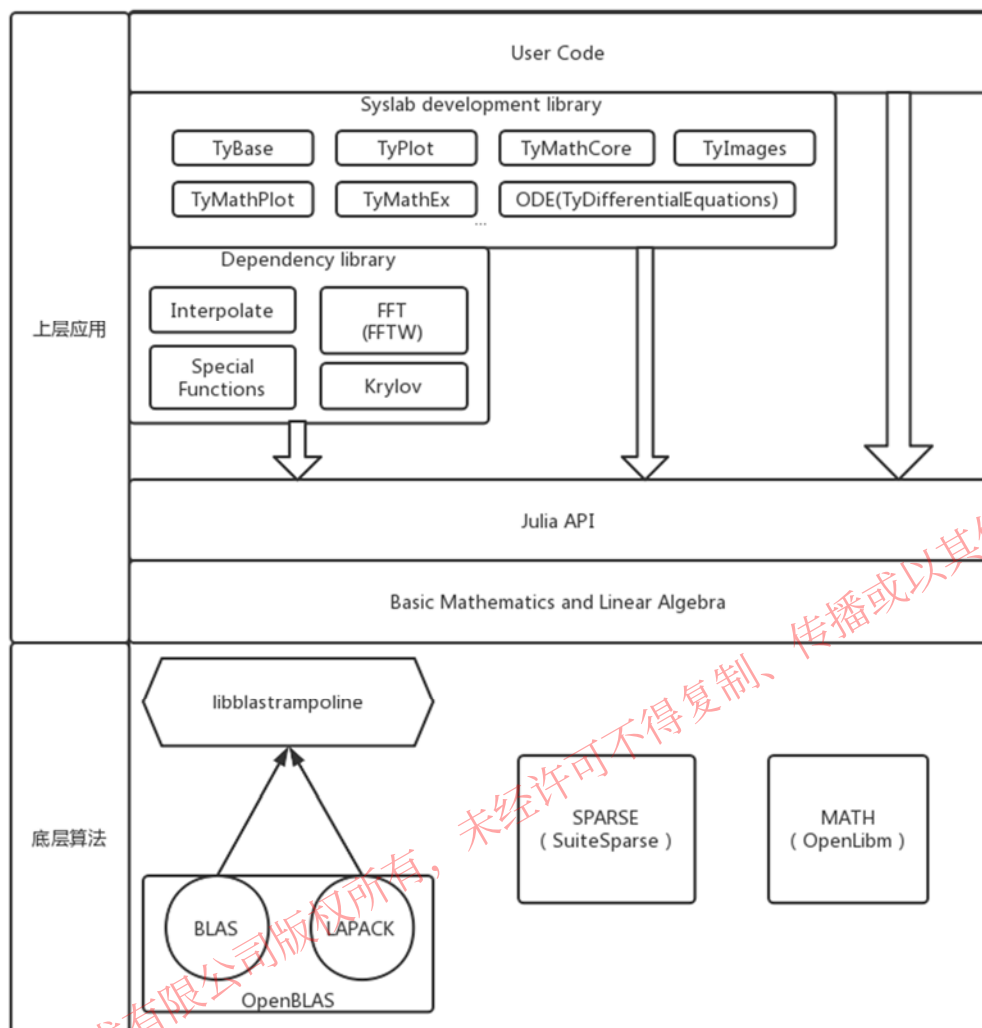


图 1-2 科学计算函数库中各子库的依赖关系

本部分以如何替换平台底层算法中 BLAS 算法库为例，介绍科学计算函数库中底层算法的替换原理和过程。

平台提供了 BinaryBuilder 和 libblastrampoline 两个工具。libblastrampoline 是 BLAS/LAPACK 代理模块，使用 PLT trampolines 提供 BLAS/LAPACK 解复用库，加载时 libblastrampoline 将检查 LBT_DEFAULT_LIBS 环境变量并尝试将对其进行的 BLAS 调用转发到该库，允许用户在运行时动态地选择调用的后端 BLAS/LAPACK 库。BinaryBuilder 是用于构建二进制包的工具库，提供将 C 库编译成动态链接库形式，并封装成_JLL 文件，提供自适应操作系统的调用动态链接库接口。

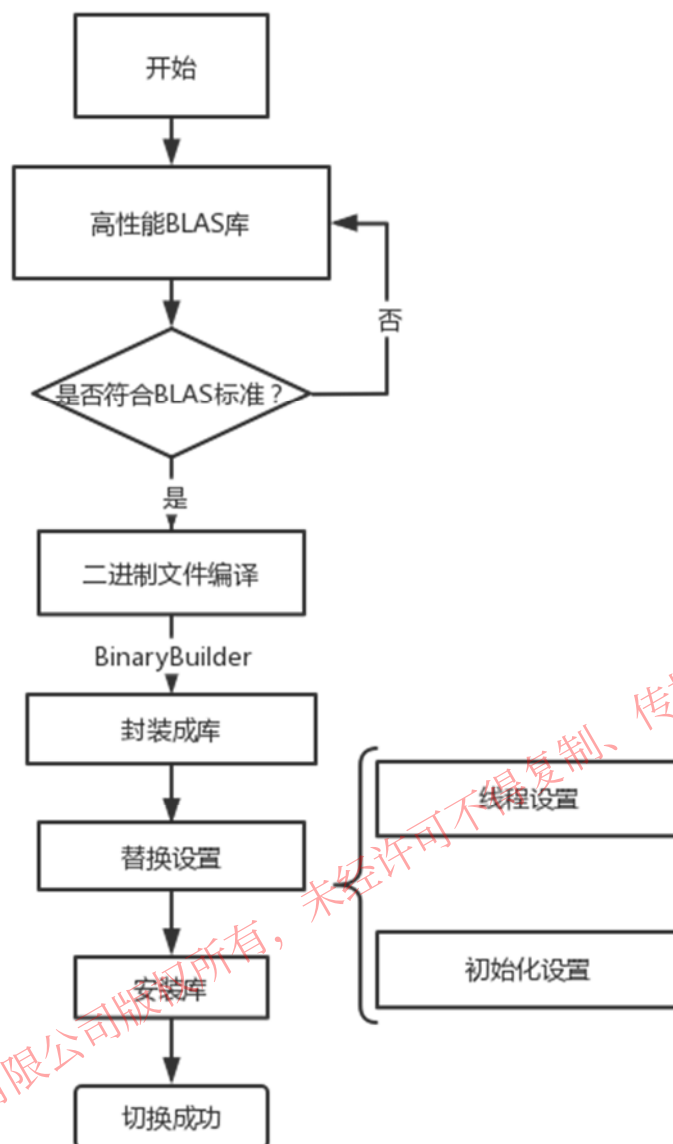


图 1-3 BLAS 替换过程

- 1) 首先利用 BinaryBuilder 将符合 BLAS 标准的 BLAS_{Test} 库编译成 BLAS_{Test}.jll 文件
- 2) 撰写第三方库 BLAS_{Test}.jl 库并依赖 BLAS_{Test}.jll 库
- 3) 方法核心：在 BLAS_{Test} 库 `__init__()` 函数中，利用 `BLAS.lbt_forward(libname, clear, verbose)` 函数，进行转发重定向到 BLAS_{Test}，如果将 `clear` 设置为 1，它将在设置新映射之前清除所有以前的映射，而如果将其设置为 0，它将仅保留给定 `libname` 中不存在的符号。
- 4) 其余接口：`lbt_get_config()`、`lbt_{set,get}_num_threads()` 用于设置、获取线程等内容

- 5) 注意事项：所有 `lbt_*` 函数都应该被认为是线程不安全的。不要尝试同时在两个不同的线程上加载两个 BLAS 库。

构建完成后，使用以下流程进行基本底层 BLAS 切换：

- 1) 复制 BLISBLAS.jl 文件路径 `pkgdir`
- 2) 通过 `using Pkg; Pkg.dev("pkgdir")` 安装库
- 3) `using BLISBLAS` 后利用 `BLAS.get_config` 函数进行查看底层库依赖

```
julia> BLAS.get_config()
LinearAlgebra.BLAS.LBTConfig
Libraries:
├ [ILP64] libopenblas64_.dll
├ [ILP64] libopenblas64_.dll
└ [ILP64] libblis.dll
```

图 1-4 查看依赖库

1.2 算法注册/设置接口

1.2.1 BLAS 接口

1.2.1.1 加载指定 BLAS 算法库

```
LBT_DLLEXPORT int32_t lbt_forward(const char * libname, int32_t
clear, int32_t verbose, const char * suffix_hint);
```

功能	加载给定的 `libname`，在导出列表中查找所有已注册的算法库
参数	[in] libname BLAS 库名 [in] clear 是否清除所有已有映射，非 0 为清除，0 保留 [in] verbose 是否打印调试信息，非 0 打印 [in] suffix_hint 是否用于搜索第一个后缀库中的 BLAS，非 NULL 是
返回	算法库地址

1.2.1.2 设置底层 BLAS 库中的线程数

```
LBT_DLLEXPORT void lbt_set_num_threads(int32_t num_threads);
```

功能	设置底层 BLAS 库中的线程数。如果出现多个库被加载，将它们全部设置为
----	--------------------------------------

	相同的值。
参数	[in] num_threads 线程数

1.2.1.3 获取底层 BLAS 库中的线程数

```
LBT_DLLEXPORT int32_t lbt_get_num_threads();
```

功能	返回底层 BLAS 库配置的线程数。在这种情况下加载多个库，返回所有返回值的最大值。
返回	线程数

1.2.2 LAPACK 接口

1.2.2.1 加载指定 LAPACK 算法库

```
LBT_DLLEXPORT int32_t lbt_forward(const char * libname, int32_t clear, int32_t verbose, const char * suffix_hint);
```

功能	加载给定的 libname，在导出列表中查找所有已注册的算法库
参数	[in] libname LAPACK 库名 [in] clear 是否清除所有已有映射，非 0 为清除，0 保留 [in] verbose 是否打印调试信息，非 0 打印 [in] suffix_hint 是否用于搜索第一个后缀库中的 BLAS/LAPACK，非 NULL 是
返回	算法库地址

1.2.3 FFT 接口

```
FFTW.set_provider!(provider;export_prefs::Bool = false)
```

功能	用于设置 FFT 底层算法库来源的外部函数接口。首选项从加载路径上的 Project.toml 和 LocalPreferences.toml 文件加载。
参数	[in] provider String 算法库名称

	[in] export_prefs Bool export_prefs 选项确定设置的首选项应存储在 LocalPreferences.toml 还是 Project.toml 中。
--	---

FFTW.get_provider()

功能	获取 LocalPreferences.toml 或 Project.toml 中存储的算法库名称
参数	[out] provider String 算法库名称

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

2. 科学计算算法

2.1 BIAS 接口说明

2.1.1 Level 1

2.1.1.1 函数说明

函数族	简介
?rotg	生成 Givens 变换矩阵 具体分为以下两个函数： cblas_srotg:生成单精度浮点型 Givens 变换矩阵 cblas_drotg:生成双精度浮点型 Givens 变换矩阵
?rotmg	生成改进 Givens 变换矩阵 具体分为以下两个函数： cblas_srotmg:生成单精度浮点型改进 Givens 变换矩阵 cblas_drotmg:生成双精度浮点型改进 Givens 变换矩阵
?rot	执行 Givens 变换 $x_i = c * x_i + sy_i$; $y_i = c * y_i - sx_i$; x 和 y 是含有 n 个元素的向量, c 是旋转角的余弦, s 是旋转角的正弦。具体分为以下四个函数： cblas_srot:单精度浮点型 Givens 变换 cblas_drot:双精度浮点型 Givens 变换 cblas_crot:单精度复数型 Givens 变换 cblas_zrot:双精度复数型 Givens 变换
?rotm	执行改进 Givens 变换 $x_i = h_{11} * x_i + h_{12}y_i$; $y_i = h_{21} * x_i + h_{22}y_i$; x 和 y 是含有 n 个元素的向量, h 是变换矩阵元素。具体分为以下两个函数： cblas_srotm:单精度浮点型改进 Givens 变换 cblas_drotm:双精度浮点型改进 Givens 变换
?swap	交换两个向量中的元素。 $x \leftrightarrow y$ 具体分为以下四个函数： cblas_sswap:单精度浮点型向量交换 cblas_dswap:双精度浮点型向量交换 cblas_cswap:单精度复数型向量交换 cblas_zswap:双精度复数型向量交换

函数族	简介
?scal	系数与向量乘积 $x \leftarrow ax$，其中 a 是乘法系数，x 是含有 n 个元素的向量。具体分为以下六个函数： cblas_sscal:单精度浮点型系数与向量乘积 cblas_dscal:双精度浮点型系数与向量乘积 cblas_cscal:单精度复数型系数与向量乘积 cblas_zscal:双精度复数型系数与向量乘积 cblas_csscal:单精度浮点型系数与单精度复数型向量乘积 cblas_zdscal:双精度浮点型系数与双精度复数型向量乘积
?copy	向量复制 $y \leftarrow x$，x 是源向量，y 是目标向量。 具体分为以下四个函数： cblas_scopy:单精度浮点型向量复制 cblas_dcopy:双精度浮点型向量复制 cblas_ccopy:单精度复数型向量复制 cblas_zcopy:双精度复数型向量复制
?axpy	向量缩放后的乘积与另一向量的加和 $y \leftarrow ax + y$ a 是乘法系数，x 和 y 是长度为 n 的向量。 具体分为以下四个函数： cblas_saxpy:单精度浮点型向量缩放后的乘积加和 cblas_daxpy:双精度浮点型向量缩放后的乘积加和 cblas_caxpy:单精度复数型向量缩放后的乘积加和 cblas_zaxpy:双精度复数型向量缩放后的乘积加和
?dot	向量点积 $dot \leftarrow x^T y$ x 和 y 是含有 n 个元素的向量。 具体分为以下两个函数： cblas_sdot:单精度浮点型向量点积 cblas_ddot:双精度浮点型向量点积
?dotc	共轭向量与另一向量的点积 $dotc \leftarrow \sum_{i=1}^n conjg(x_i) * y_i$ x 和 y 是长度为 n 的复数向量。 具体分为以下两个函数： cblas_cdotc:单精度复数型向量点积 cblas_zdotc:双精度复数型向量点积
?dotu	复数向量的点积。 cblas_cdotu:单精度复数型向量点积 cblas_zdotu:双精度复数型向量点积
?sdot	向量点积（输入向量为单精度，以双精度计算） $dot \leftarrow x^T y$ 具体分为以下两个函数： cblas_sdsdot:输出单精度复数型向量点积 cblas_dsdot:输出双精度复数型向量点积

函数族	简介
?nrm2	向量 2 范数 $nrm2 \leftarrow \ x\ _2$, x 是含有 n 个元素的向量。 具体分为以下四个函数： cblas_snrm2:单精度浮点型向量 2 范数 cblas_dnrm2:双精度浮点型向量 2 范数 cblas_scnrm2:单精度复数型向量 2 范数 cblas_dznrm2:双精度复数型向量 2 范数
?asum	向量 1 范数 $asum \leftarrow \ x\ _1$ 具体分为以下四个函数： cblas_sasum:单精度浮点型向量 1 范数 cblas_dasum:双精度浮点型向量 1 范数 cblas_scasum:单精度复数型向量 1 范数 cblas_dzasum:双精度复数型向量 1 范数
i?amax	实数向量返回最大绝对值的索引值，复数向量返回实部绝对值和虚部绝对值之和的最大值索引值。 具体分为以下四个函数： cblas_isamax:单精度浮点型 cblas_idamax:双精度浮点型 cblas_icamax:单精度复数型 cblas_izamax:双精度复数型

2.1.1.2 ?rotg

构造吉文斯旋转平面。

2.1.1.2.1 接口定义

A. C interface:

```
void cblas_srotg(float *a, float *b, float *c, float *s);
```

```
void cblas_drotg(double *a, double *b, double *c, double *s);
```

B. Fortran interface:

```
CALL SROTG(SA, SB, SC, SS)
```

```
CALL DROTG(SA, SB, SC, SS)
```

2.1.1.2.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
a	srotg 中，a 是单精度浮点类型。	笛卡尔坐标系 x 坐标。	输入/输出
	drotg 中，a 是双精度浮点类型。		
b	srotg 中，b 是单精度浮点类型。	笛卡尔坐标系 y 坐标。	输入/输出
	drotg 中，b 是双精度浮点类型。		
c	srotg 中，c 是单精度浮点类型。	吉文斯旋转参数 c。	输出
	drotg 中，c 是双精度浮点类型。		
s	srotg 中，s 是单精度浮点类型。	吉文斯旋转参数 s。	输出
	drotg 中，s 是双精度浮点类型。		

2.1.1.2.3 依赖

```
#include "kblas.h"
```

2.1.1.2.4 示例

A. C interface:

```
float a = 6.0;
float b = -8.0;
float c, s;

cblas_srotg(&a, &b, &c, &s);

/*
 * Output a:-10.000001, b:-1.666667, c:-0.600000, s:0.800000
 */
```

2.1.1.3 ?rotmg

构造一个修改的转换平面。

2.1.1.3.1 接口定义

A. C interface:

```
void cblas_srotmg(float *d1, float *d2, float *b1, const float b2, float *P);
```

```
void cblas_drotmg(double *d1, double *d2, double *b1, const double b2, double *P);
```

B. Fortran interface:

```
CALL SROTMG(D1, D2, X1, X2, PARAM)
```

```
CALL DROTMG(D1, D2, X1, X2, PARAM)
```

2.1.1.3.2 参数

参数名	类型	描述	输入/输出
d1	srotmg 中, d1 是单精度浮点类型。	d1 取值	输入/输出
	drotmg 中, d1 是双精度浮点类型。		
d2	srotmg 中, d2 是单精度浮点类型。	d2 取值	输入/输出
	drotmg 中, d2 是双精度浮点类型。		
b1	srotmg 中, b1 是单精度浮点类型。	b1 取值	输入/输出
	drotmg 中, b1 是双精度浮点类型。		
b2	srotmg 中, b2 是单精度浮点类型。	b2 取值	输入/输出
	drotmg 中, b2 是双精度浮点类型。		
p	srotmg 中, p 是单精度浮点类型。	转换矩阵 H	输出

参数名	类型	描述	输入/输出
	drotmg 中，p 是双精度浮点类型。		

2.1.1.3.3 依赖

```
#include "kblas.h"
```

2.1.1.3.4 示例

A. C interface:

```
double d1 = 10.0, d2 = 1.0, x1 = -2.0, x2 = 4.0;
double p[5];

cblas_drotmg(&d1, &d2, &x1, x2, p);
/**
 * d1:7.142857, d2:0.714286, x1:-2.800000, x2:4.000000
 * param:0.000000, 0.000000, 2.000000, -0.200000, 0.000000
 */
```

2.1.1.4 ?rot

基于平面上的点旋转。 $x_i = c * x_i + sy_i$; $y_i = c * y_i - sx_i$
x和y是含有n个元素的向量，c是旋转角的余弦，s是旋转角的正弦。

2.1.1.4.1 接口定义

A. C interface:

```
void cblas_srot(const BLASINT N, float *X, const BLASINT incX, float *Y, const
BLASINT incY, const float c, const float s);

void cblas_drot(const BLASINT N, double *X, const BLASINT incX, double *Y,
const BLASINT incY, const double c, const double s);

void cblas_csrot(const BLASINT N, float *X, const BLASINT incX, float *Y, const
BLASINT incY, const float c, const float s);

void cblas_zdrot(const BLASINT N, double *X, const BLASINT incX, double *Y,
const BLASINT incY, const double c, const double s);
```


B. Fortran interface:

CALL SROT(N, SX, INCX, SY, INCY, SC, SS)

CALL DROT(N, SX, INCX, SY, INCY, SC, SS)

CALL CSROT(N, SX, INCX, SY, INCY, SC, SS)

CALL ZDROT(N, SX, INCX, SY, INCY, SC, SS)

2.1.1.4.2 参数

参数名	类型	描述	输入/输出
N	整型数	表示 X 向量的元素个数。	输入
X	srot 中, X 是单精度浮点类型。	向量 X, 向量规模至少是 $(1+(n-1)*abs(incx))$ 。	输入/输出
	drot 中, X 是双精度浮点类型。		
	csrot 中, X 是单精度复数类型。		
	zdrot 中, X 是双精度复数类型。		
incX	整型数	表示 x 向量增长步长。	输入
Y	srot 中, Y 是单精度浮点类型。	向量 Y, 向量规模至少是 $(1+(n-1)*abs(incy))$ 。	输入/输出
	drot 中, Y 是双精度浮点类型。		
	csrot 中, Y 是单精度复数类型。		
	zdrot 中, Y 是双精度复数类型。		
incY	整型数	表示 Y 向量增长步。	输入
c	srot, csrot 中, c 是单精度浮点类型。	旋转角的余弦。	输入
	drot, zdrot 中, c 是双精度浮点类型。		

参数名	类型	描述	输入/输出
s	srot, csrot 中, s 是单精度浮点类型。	旋转角的正弦。	输入
	drot, zdrot 中, s 是双精度浮点类型。		

2.1.1.4.3 依赖

```
#include "kblas.h"
```

2.1.1.4.4 示例

A. C interface:

```
int n = 5, incx = 1, incy=1;
float c = 1, s = 1;
/*
 * C = 1, S = 1
 * Input X: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 * Input Y: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};
float y[5] = {-0.302449, -0.164777, 0.268230, -0.222225, 0.053970};
cblas_srot(n, x, incx, y, incy, c, s);
/*
 * Output X: 0.037739, -0.270394, 0.551329, 0.076215, 0.465617
 * Output Y: -0.642636, -0.059160, -0.014870, -0.520665, -0.357677
 */
```

2.1.1.5 ?rotm

基于修改的吉文斯转换。 $x_i = h_{11} * x_i + h_{12}y_i$; $y_i = h_{21} * x_i + h_{22}y_i$, x 和 y 是含有 n 个元素的向量, h 是转换矩阵元素。

2.1.1.5.1 接口定义

A. C interface:

```
void cblas_srotm(const BLASINT N, float *X, const BLASINT incX, float *Y, const
BLASINT incY, const float *P);
```

```
void cblas_drotm(const BLASINT N, double *X, const BLASINT incX, double *Y,
const BLASINT incY, const double *P);
```

B. Fortran interface:

```
CALL SROTM(N, SX, INCX, SY, INCY, DTEMP)
```

```
CALL DROTM(N, SX, INCX, SY, INCY, DTEMP)
```

2.1.1.5.2 参数

参数名	类型	描述	输入/输出
N	整型数	表示 X 向量的元素个数。	输入
X	srotm 中, X 是单精度浮点类型。	向量 X, 向量规模至少是 $(1+(n-1)*abs(incx))$ 。	输入/输出
	drotm 中, X 是双精度浮点类型。		
incX	整型数	表示 x 向量增长步长。	输入
Y	srotm 中, Y 是单精度浮点类型。	向量 Y, 向量规模至少是 $(1+(n-1)*abs(incy))$ 。	输入/输出
	drotm 中, Y 是双精度浮点类型。		
incY	整型数	表示 Y 向量增长步。	输入
P	srotm 中, P 是单精度浮点类型。	转换矩阵 H。	输入

参数名	类型	描述	输入/输出
	drotm 中，P 是双精度浮点类型。		

2.1.1.5.3 依赖

```
#include "kblas.h"
```

2.1.1.5.4 示例

A. C interface:

```
double d1 = 10.0, d2 = 2.0;
double x1 = 2.0, x2 = 1.0;
double p[5];
int n = 5, incx = 1, incy = 1;
double x[5]={1.0, 2.0, 1.0, 3.0, 1.0};
double y[5]={3.0, 1.0, 1.0, 5.0, 1.0};

cblas_drotmg(&d1, &d2, &x1, x2, p);
cblas_drotm(n, x, incx, y, incy, p);
/*
 * Output x:  1.300000,  2.100000, 1.100000,  3.500000, 1.100000
 * Output y:  2.500000,  0.000000, 0.500000,  3.500000, 0.500000
 */
```

2.1.1.6 ?swap

交换两个向量中的元素。

2.1.1.6.1 接口定义

A. C interface:

```
void cblas_sswap(const BLASINT n, float *x, const BLASINT incx, float *y,
const BLASINT incy);
void cblas_dswap(const BLASINT n, double *x, const BLASINT incx, double
```

*y, const BLASINT incy);

void cblas_cswap(const BLASINT n, void *x, const BLASINT incx, void *y,
const BLASINT incy);

void cblas_zswap(const BLASINT n, void *x, const BLASINT incx, void *y,
const BLASINT incy);

B. Fortran interface:

CALL SSWAP(N, SX, INCX, SY, INCY)

CALL DSWAP(N, SX, INCX, SY, INCY)

CALL CSWAP(N, SX, INCX, SY, INCY)

CALL ZSWAP(N, SX, INCX, SY, INCY)

2.1.1.6.2 参数

参数名	类型	描述	输入/输出
n	整型数	表示 x,y 向量的元素个数。	输入
X	sswap 中, x 是双精度浮点类型。	向量 x, 向量规模至少是 (1+(n-1)*abs(incx))。	输入/输出
	dswap 中, x 是单精度浮点类型。		
	cswap 中, x 是单精度复数类型。		
	zswap 中, x 是双精度复数类型。		
incx	整型数	表示 x 向量增长步长。	输入
y	sswap 中, y 是双精度浮点类型。	向量 y, 向量规模至少是 (1+(n-1)*abs(incy))。	输入/输出
	dswap 中, y 是单精度浮点类型。		
	cswap 中, y 是单精度复数类型。		
	zswap 中, y 是双精度复数类型。		

参数名	类型	描述	输入/输出
incy	整型数	表示 y 中向量元增长步长。	输入

2.1.1.6.3 依赖

```
#include "kblas.h"
```

2.1.1.6.4 示例

A. C interface:

```
int n = 5, incx = 1, incy = 1;

/*
 * Input X:  0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 * Input Y: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 * */

float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};
float y[5] = {-0.302449, -0.164777, 0.268230, -0.222225, 0.053970};

cblas_sswap(n, x, incx, y, incy);

/*
 * Output X: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 * Output Y: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 * */
```

2.1.1.7 ?scal

计算向量与系数的乘积，即 $x = \alpha * x$ 。
alpha 是乘法系数，x 是含有 n 个元素的向量。

2.1.1.7.1 接口定义

A. C interface:

```
void cblas_dscal(const BLASINT N, const double alpha, double *X, const BLASINT incX);
```

```
void cblas_sscal(const BLASINT N, const float alpha, float *X, const BLASINT
incX);
```

```
void cblas_cscal(const BLASINT N, const void *alpha, void *X, const BLASINT
incX);
```

```
void cblas_zscal(const BLASINT N, const void *alpha, void *X, const BLASINT
incX);
```

```
void cblas_csscal(const BLASINT N, const double alpha, void *X, const BLASINT
incX);
```

```
void cblas_zdscal(const BLASINT N, const double alpha, void *X, const BLASINT
incX);
```

B. Fortran interface:

```
CALL SSCAL(N, ALPAH, X, INCX)
```

```
CALL DSCAL(N, ALPAH, X, INCX)
```

```
CALL CSCAL(N, ALPAH, X, INCX)
```

```
CALL ZSCAL(N, ALPAH, X, INCX)
```

```
CALL CSSCAL(N, ALPAH, X, INCX)
```

```
CALL ZDSCAL(N, ALPAH, X, INCX)
```

2.1.1.7.2 参数

参数名	类型	描述	输入/输出
N	整型数	表示 x 向量元素个数。	输入
alpha	在 dscal 中, alpha 是双精度浮点类型。 在 sscal 中, alpha 是单精度浮点类型。 在 cscal 中, alpha 是单精度复数类型。 在 zscal 中, alpha 是双精度复数类型。 在 csscal 中, alpha 是单精度浮点类型。	表示系数。	输入

参数名	类型	描述	输入/输出
	在 zdscal 中，alpha 是双精度浮点类型。		
X	在 dscal 中，X 是双精度浮点类型。	向量 X，向量规模至少是 $(1 + (n - 1) * abs(incX))$	输入/输出
	在 sscal 中，X 是单精度浮点类型。		
	在 cscal 中，X 是单精度复数类型。		
	在 zscal 中，X 是双精度复数类型。		
	在 csscal 中，X 是单精度复数类型。		
	在 zdscal 中，X 是双精度复数类型。		
incX	整型数	表示 X 向量增长步长。	输入

2.1.1.7.3 依赖

#include "kblas.h"

2.1.1.7.4 示例

A. C interface:

```
float alpha = 2.0;

int n = 5, incx = 1;

/**
 * X: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 */

float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};

cblas_sscal(n, alpha, x, incx);

/**
 * Output X: 0.680375, -0.211234, 0.566198, 0.596880, 0.823295
```


*/

2.1.1.8 ?copy

向量拷贝，即 $y = x$ 。

x 是源向量，y 是目标向量。

2.1.1.8.1 接口定义

A. C interface:

```
void cblas_scopy(const BLASINT n, const float *x, const BLASINT incx, float *y,
const BLASINT incy);
```

```
void cblas_dcopy(const BLASINT n, const double *x, const BLASINT incx, double
*y, const BLASINT incy);
```

```
void cblas_ccopy(const BLASINT n, const void *x, const BLASINT incx, void *y,
const BLASINT incy);
```

```
void cblas_zcopy(const BLASINT n, const void *x, const BLASINT incx, void *y,
const BLASINT incy);
```

B. Fortran interface:

```
CALL SCOPY(N, X, INCX, Y, INCY)
```

```
CALL DCOPY(N, X, INCX, Y, INCY)
```

```
CALL CCOPY(N, X, INCX, Y, INCY)
```

```
CALL ZCOPY(N, X, INCX, Y, INCY)
```

2.1.1.8.2 参数

参数名	类型	描述	输入/输出
n	整型数	表示向量 x 和 y 的元素个数。	输入
x	在 scopy 中，x 是单精度浮点类型。	向量 x，向量规模至少是 $(1+(n-1)*abs(incx))$ 。	输入
	在 dcopy 中，x 是双精度浮点类型。		
	在 ccopy 中，x 是单精度复数类型。		

参数名	类型	描述	输入/输出
	在 zcopy 中, x 是双精度复数类型。		
incx	整型数	表示 x 向量增长步长。	输入
y	在 scopy 中, y 是单精度浮点类型。	向量 y, 向量规模至少是 $(1+(n-1)*abs(incy))$ 。	输出
	在 dcopy 中, y 是双精度浮点类型。		
	在 ccopy 中, y 是单精度复数类型。		
	在 zcopy 中, y 是双精度复数类型。		
incy	整型数	表示 y 向量增长步。	输入

2.1.1.8.3 依赖

```
#include "kblas.h"
```

2.1.1.8.4 示例

A. C interface:

```
double y[5];
int n = 5, incx = 1, incy = 1;
/*
 *   x: 1, 2, 3, 4, 5
 */
double x[5] = {1, 2, 3, 4, 5};

cblas_dcopy(n, x, incx, y, incy);
/*
 * Output y: 1, 2, 3, 4, 5
 */
```

2.1.1.9 ?axpy

向量缩放后的乘积与另一向量的加和，即 $y = \alpha * x + y$ 。

alpha 是乘法系数，x 和 y 是长度为 n 的向量。

2.1.1.9.1 接口定义

A. C interface:

```
void cblas_saxpy(const BLASINT n, const float alpha, const float *x, const
BLASINT incx, float *y, const BLASINT incy);
```

```
void cblas_daxpy(const BLASINT n, const double alpha, const double *x, const
BLASINT incx, double *y, const BLASINT incy);
```

```
void cblas_caxpy(const BLASINT n, const void *alpha, const void *x, const
BLASINT incx, void *y, const BLASINT incy);
```

```
void cblas_zaxpy(const BLASINT n, const void *alpha, const void *x, const
BLASINT incx, void *y, const BLASINT incy);
```

B. Fortran interface:

```
CALL SAXPY(N, ALPHA, X, INCX, Y, INCY)
```

```
CALL DAXPY(N, ALPHA, X, INCX, Y, INCY)
```

```
CALL CAXPY(N, ALPHA, X, INCX, Y, INCY)
```

```
CALL ZAXPY(N, ALPHA, X, INCX, Y, INCY)
```

2.1.1.9.2 参数

参数名	类型	描述	输入/输出
n	整型数	表示 x 向量元素个数。	输入
alpha	在 daxpy 中是双精度浮点类型。	表示乘法系数。	输入
	在 saxpy 中是单精度浮点类型。		
	在 caxpy 中是单精度复数类型。		
	在 zaxpy 中是双精度复数类型。		

参数名	类型	描述	输入/输出
x	在 daxpy 中, x 是双精度浮点类型。	向量 x, 向量规模至少是 $(1+(n-1)*abs(incx))$ 。	输入
	在 saxpy 中, x 是单精度浮点类型。		
	在 caxpy 中, x 是单精度复数类型。		
	在 zaxpy 中, x 是双精度复数类型。		
incx	整型数	表示 x 向量增长步长。	输入
y	在 daxpy 中, y 是双精度浮点类型。	向量 y, 向量规模至少是 $(1+(n-1)*abs(incy))$ 。	输入/输出
	在 saxpy 中, y 是单精度浮点类型。		
	在 caxpy 中, y 是单精度复数类型。		
	在 zaxpy 中, y 是双精度复数类型。		
incy	整型数	表示 y 向量增长步长。	输入

2.1.1.9.3 依赖

```
#include "kblas.h"
```

2.1.1.9.4 示例

A. C interface:

```
float alpha = 2.0;
int n = 5, incx = 1, incy = 1;
/**
 *   X: f 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 *   Y: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 */
```

```
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};
float y[5] = {-0.302449, -0.164777, 0.268230, -0.222225, 0.053970};

cblas_saxpy(n, alpha, x, incx, y, incy);

/**
 *   Output Y: 0.377927, -0.376011, 0.834428, 0.374655, 0.877264
 */
```

2.1.1.10 ?dot

计算向量与向量的点积，即 $res = \sum_{i=1}^n x_i * y_i$
x 和 y 是含有 n 个元素的向量。

2.1.1.10.1 接口定义

A. C interface:

```
float cblas_sdot(const BLASINT n, const float *x, const BLASINT incx, const float
*y, const BLASINT incy);
```

```
double cblas_ddot(const BLASINT n, const double *x, const BLASINT incx, const
double *y, const BLASINT incy);
```

B. Fortran interface:

```
RES = SDOT(N, X, INCX, Y, INCY)
```

```
RES = DDOT(N, X, INCX, Y, INCY)
```

2.1.1.10.2 参数

参数名	类型	描述	输入/输出
n	整型数	表示 x 向量元素个数。	输入
x	在 ddot 中，x 是双精度浮点类型。	向量 x，向量规模至少是 $(1 + (n - 1) * abs(incx))$ 。	输入
	在 sdot 中，x 是单精度浮点类型。		

参数名	类型	描述	输入/输出
incx	整型数	表示 x 向量增长步长。	输入
y	在 ddot 中, y 是双精度浮点类型。	向量 y, 向量规模至少是 $(1 + (n - 1) * abs(incy))$ 。	输入
	在 sdot 中, y 是单精度浮点类型。		
incy	整型数	表示 y 向量增长步长。	输入

2.1.1.10.3 依赖

```
#include "kblas.h"
```

2.1.1.10.4 示例

A. C interface:

```
float res;

int n = 5, incx = 1, incy = 1;

/**
 *   X: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 *   Y: -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};
float y[5] = {-0.302449, -0.164777, 0.268230, -0.222225, 0.053970};

res = cblas_sdot(n, x, incx, y, incy);

/**
 *   Output:
 *
 *   -0.053655
 */
```

2.1.1.11 ?dotc

计算共轭向量与另一向量的点积, 即 $res = \sum_{i=1}^n conjg(x_i) * y_i$ 。

x 和 y 是长度为 n 的复数向量。

2.1.1.11.1 接口定义

A. C interface:

```
float _Complex cblas_cdotc(const BLASINT n, const void *x, const BLASINT incx,
const void *y, const BLASINT incy);
```

```
void cblas_cdotc_sub(const BLASINT n, const void *x, const BLASINT incx, const
void *y, const BLASINT incy, void *ret);
```

```
double _Complex cblas_zdotc(const BLASINT n, const void *x, const BLASINT
incx, const void *y, const BLASINT incy);
```

```
void cblas_zdotc_sub(const BLASINT n, const void *x, const BLASINT incx, const
void *y, const BLASINT incy, void *ret);
```

B. Fortran interface:

```
RES = CDOTC(N, CX, INCX, CY, INCY)
```

```
RES = ZDOTC(N, CX, INCX, CY, INCY)
```

2.1.1.11.2 参数

参数名	类型	描述	输入/输出
n	整型数	表示 x 、 y 向量元素个数。	输入
x	在 $cdotc$ 中, x 是单精度复数类型。 在 $zdotc$ 中, x 是双精度复数类型。	向量 x , 向量规模至少是 $(1 + (n - 1) * abs(incx))$ 。	输入
$incx$	整型数	表示 x 向量增长步长。	输入
y	在 $cdotc$ 中, y 是单精度复数类型。 在 $zdotc$ 中, y 是双精度复数类型。	向量 y , 向量规模至少是 $(1 + (n - 1) * abs(incy))$ 。	输入
$incy$	整型数	表示 y 向量增长步长。	输入

参数名	类型	描述	输入/输出
ret	在 cdotc 中，ret 是单精度复数类型。	计算结果。	输出
	在 zdotc 中，ret 是双精度复数类型。		

2.1.1.11.3 依赖

```
#include "kblas.h"
```

2.1.1.11.4 示例

A. C interface:

```
int n = 5, incx = 1, incy = 1;

/**
 *   X:
 *
 *   0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
 *
 *   -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 *
 *   Y:
 *
 *   -0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
 *
 *   0.416195, 0.135712, 0.217297, -0.358397, 0.106969
 */
float x[10] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
               -0.302449, -0.164777, 0.268230, -0.222225, 0.053970};
float y[10] = {-0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
               0.416195, 0.135712, 0.217297, -0.358397, 0.106969};

float _Complex res = cblas_cdotc(n, x, incx, y, incy);

/**
 *   Output:  0.126042, 0.317067
 *
 */
```


2.1.1.12 ?dotu

计算两个复数向量的点积。

2.1.1.12.1 接口定义

A. C interface:

```
float _Complex cblas_cdotu(const BLASINT n, const void *x, const BLASINT incx,
const void *y, const BLASINT incy);
```

```
void cblas_cdotu_sub(const BLASINT n, const void *x, const BLASINT incx, const
void *y, const BLASINT incy, void *ret);
```

```
double _Complex cblas_zdotu(const BLASINT n, const void *x, const BLASINT
incx, const void *y, const BLASINT incy);
```

```
void cblas_zdotu_sub(const BLASINT n, const void *x, const BLASINT incx, const
void *y, const BLASINT incy, void *ret);
```

B. Fortran interface:

```
RES = CDOTU(N, CX, INCX, CY, INCY)
```

```
RES = ZDOTU(N, CX, INCX, CY, INCY)
```

2.1.1.12.2 参数

参数名	类型	描述	输入/输出
n	整型数	表示 x,y 向量元素个数。	输入
x	在 cdotu 中, x 是单精度复数类型。	向量 x, 向量规模至少是 (1+(n-1)*abs(incx))。	输入
	在 zdotu 中, x 是双精度复数类型。		
incx	整型数	表示 x 中向量元素的增量。	输入
y	在 cdotu 中, y 是单精度复数类型。	向量 y, 向量规模至少是 (1+(n-1)*abs(incy))。	输入
	在 zdotu 中, y 是双精度复数类型。		

参数名	类型	描述	输入/输出
incy	整型数	表示 y 向量增长步长。	输入
ret	在 cdotc 中，ret 是单精度复数类型。	计算结果。	输出
	在 zdotc 中，ret 是双精度复数类型。		

2.1.1.12.3 依赖

```
#include "kblas.h"
```

2.1.1.12.4 示例

A. C interface:

```
int n = 5, incx = 1, incy = 1;

/**
 *   X:
 *   0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
 *   -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 *   Y:
 *   -0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
 *   0.416195, 0.135712, 0.217297, -0.358397, 0.106969
 */
float x[10] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
               -0.302449, -0.164777, 0.268230, -0.222225, 0.053970};
float y[10] = {-0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
               0.416195, 0.135712, 0.217297, -0.358397, 0.106969};

float _Complex res = cblas_cdotu(n, x, incx, y, incy);

/**
 *   Output:  0.268904 0.001699
 */
```

*/

2.1.1.13 ?sdot

扩展精度点积，输入为单精度，以双精度计算，sdsdot 输出单精度，dsdot 输出双精度。

2.1.1.13.1 接口定义

A. C interface:

```
float cblas_sdsdot(const BLASINT n, const float alpha, const float *x, const
BLASINT incx, const float *y, const BLASINT incy);
```

```
double cblas_dsdot(const BLASINT n, const float *x, const BLASINT incx, const
float *y, const BLASINT incy);
```

B. Fortran interface:

```
RES = SDSDOT(N, ALPHA, SX, INCX, SY, INCY)
```

```
RES = DSDOT(N, SX, INCX, SY, INCY)
```

2.1.1.13.2 参数

参数名	类型	描述	输入/输出
n	整型数	表示 x,y 向量元素个数。	输入
alpha	在 dasum 中，x 是双精度浮点类型。	单精度标量，加到点积中返回。	输入
x	在 sasum 中，x 是单精度浮点类型。	向量 x，向量规模至少是 $(1+(n-1)*abs(incx))$ 。	输入
incx	在 scasum 中，x 是单精度复数类型。	表示 x 中向量元素的增量。	输入
y	在 zasum 中，x 是双精度复数类型。	向量 y，向量规模至少是 $(1+(n-1)*abs(incy))$ 。	输入
incy	整型数	表示 x 向量增长步长。	输入

2.1.1.13.3 依赖

```
#include "kblas.h"
```

2.1.1.13.4 示例

A. C interface:

```
int n = 10, incx = 1, incy = 1;

/**
 *   X:
 *   0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
 *   -0.302449, -0.164777, 0.268230, -0.222225, 0.053970
 *   Y:
 *   -0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
 *   0.416195, 0.135712, 0.217297, -0.358397, 0.106969
 */
float x[10] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647,
               -0.302449, -0.164777, 0.268230, -0.222225, 0.053970};
float y[10] = {-0.022603, 0.128871, -0.135216, 0.013401, 0.452230,
               0.416195, 0.135712, 0.217297, -0.358397, 0.106969};

double res = cblas_dsdot(n, x, incx, y, incy);

/**
 *   Output: 0.126042
 */
```

2.1.1.14 ?nrm2

计算向量的欧几里得范数，即 $res = ||x||$ 。

x 是含有 n 个元素的向量， $||x||$ 相当于 $\sqrt{|x_1|^2 + |x_2|^2 + \cdots + |x_n|^2}$

2.1.1.14.1 接口定义

A. C interface:

```
float cblas_snorm2 (const BLASINT N, const float *X, const BLASINT incX);
double cblas_dnorm2 (const BLASINT N, const double *X, const BLASINT incX);
float cblas_scnrm2 (const BLASINT N, const float *X, const BLASINT incX);
double cblas_dznrm2 (const BLASINT N, const double *X, const BLASINT incX);
```

B. Fortran interface:

```
RES = SNRM2(N, X, INCX)
RES = DNRM2(N, X, INCX)
RES = SCNRM2(N, X, INCX)
RES = DZNRM2(N, X, INCX)
```

2.1.1.14.2 参数

参数名	类型	描述	输入/输出
n	整型数	表示 x 向量元素个数。	输入
x	在 snrm2 中，X 是单精度浮点类型。	向量 x， 向量规模至少是 (1+(n-1)*abs(incX))。	输入/输出
	在 dnorm2 中，X 是双精度浮点类型。		
	在 scnrm2 中，X 是单精度复数类型。		
	在 dznrm2 中，X 是双精度复数类型。		
incX	整型数	表示 x 向量增长步长。	输入

2.1.1.14.3 依赖

```
#include "kblas.h"
```

2.1.1.14.4 示例

A. C interface:

```
int n = 5, incx = 1;

float res;

/**
 *   X: 0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 */

float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};

res = cblas_snorm2(n, x, incx);

/**
 *   Output: 0.682310
 */
```

2.1.1.15 ?asum

向量元素绝对值求和（对于复数，则实部和虚部分别取绝对值后再求和）。

2.1.1.15.1 接口定义

A. C interface:

```
float cblas_sasum(const BLASINT n, const float *x, const BLASINT incx);
double cblas_dasum(const BLASINT n, const double *x, const BLASINT incx);
float cblas_scasum(const BLASINT n, const void *x, const BLASINT incx);
double cblas_dzasum(const BLASINT n, const void *x, const BLASINT incx);
```

B. Fortran interface:

```
RES = SASUM(N, X, INCX)
RES = DASUM(N, X, INCX)
RES = SCASUM(N, X, INCX)
RES = DZASUM(N, X, INCX)
```

2.1.1.15.2 参数

参数名	类型	描述	输入/输出
n	整型数	表示 x 向量元素个数。	输入
x	在 dasum 中，x 是双精度浮点类型。	向量 x， 向量规模至少是 (1+(n-1)*abs(incx))。	输入
	在 sasum 中，x 是单精度浮点类型。		
	在 scasum 中，x 是单精度复数类型。		
	在 zasum 中，x 是双精度复数类型。		
incx	整型数	表示 x 向量增长步长。	输入

2.1.1.15.3 依赖

#include "kblas.h"

2.1.1.15.4 示例

A. C interface:

```
int n = 5, incx = 1, incy = 1;
/*
 * Input X:  0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 */
float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};

float res = cblas_sasum(n, x, incx);
/*
 * Output : 1.438991
 */
```

2.1.1.16 l?amax

实数向量返回最大绝对值的索引值，复数向量返回实部绝对值和虚部绝对值之

和的最大值索引值。

2.1.1.16.1 接口定义

A. C interface:

CBLAS_INDEX cblas_isamax(const BLASINT n, const float *x, const BLASINT incx);

CBLAS_INDEX cblas_idamax(const BLASINT n, const double *x, const BLASINT incx);

CBLAS_INDEX cblas_icamax(const BLASINT n, const void *x, const BLASINT incx);

CBLAS_INDEX cblas_izamax(const BLASINT n, const void *x, const BLASINT incx);

B. Fortran interface:

INDEX = ISAMAX(N, X, INCX)

INDEX = IDAMAX(N, X, INCX)

INDEX = ICAMAX(N, X, INCX)

INDEX = IZAMAX(N, X, INCX)

2.1.1.16.2 参数

参数名	类型	描述	输入/输出
n	整型数	表示 x 向量元素个数。	输入
x	在 isamax 中, x 是双精度浮点类型。	向量 x, 向量规模至少是 $(1+(n-1)*abs(incx))$ 。	输入
	在 idamax 中, x 是单精度浮点类型。		
	在 icamax 中, x 是单精度复数类型。		
	在 izamax 中, x 是双精度复数类型。		
incx	整型数	表示 x 向量增长步长。	输入

2.1.1.16.3 依赖

```
#include "kblas.h"
```

2.1.1.16.4 示例

A. C interface:

```
int n = 5, incx = 1;

int res;

/**
 *   X:  0.340188, -0.105617, 0.283099, 0.298440, 0.411647
 */

float x[5] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647};

res = cblas_isamax(n, x, incx);

/**
 * Output:  4
 */
```

2.1.2 Level 2

2.1.2.1 函数说明

函数族	简介
?gemv	$y \leftarrow \alpha * A * x + \beta * y$ or $y \leftarrow \alpha * A^T * x + \beta * y$ A 为常规矩阵; 具体分为以下四个函数: cblas_sgemv: 单精度浮点型 cblas_dgemv: 双精度浮点型 cblas_cgemv: 单精度复数型 cblas_zgemv: 双精度复数型

函数族	简介
?gbmv	$y \leftarrow \alpha * A * x + \beta * y$ or $y \leftarrow \alpha * A^T * x + \beta * y$ A 为带状矩阵; 具体分为以下四个函数: cblas_sgbmv: 单精度浮点型 cblas_dgbmv: 双精度浮点型 cblas_cgbmv: 单精度复数型 cblas_zgbmv: 双精度复数型
?symv	$y \leftarrow \alpha * A * x + \beta * y$ A 为对称矩阵; 具体分为以下两个函数: cblas_ssylv: 单精度浮点型 cblas_dsylv: 双精度浮点型
?sbmv	$y \leftarrow \alpha * A * x + \beta * y$ A 为带状对称矩阵; 具体分为以下两个函数: cblas_ssbmv: 单精度浮点型 cblas_dsbmv: 双精度浮点型
?spmv	$y \leftarrow \alpha * A * x + \beta * y$ A 为 packed 对称矩阵; 具体分为以下两个函数: cblas_sspmv: 单精度浮点型 cblas_dspmv: 双精度浮点型
?trmv	$x \leftarrow A * x$ or $x \leftarrow A^T * x$ A 为三角矩阵; 具体分为以下四个函数: cblas_strmv: 单精度浮点型 cblas_dtrmv: 双精度浮点型 cblas_ctrmv: 单精度复数型 cblas_ztrmv: 双精度复数型
?tbmv	$x \leftarrow A * x$ or $x \leftarrow A^T * x$ A 为带状三角矩阵; 具体分为以下四个函数: cblas_stbmv: 单精度浮点型 cblas_dtbmv: 双精度浮点型 cblas_ctbmv: 单精度复数型 cblas_ztbmv: 双精度复数型
?tpmv	$x \leftarrow A * x$ or $x \leftarrow A^T * x$ A 为 packed 三角矩阵; 具体分为以下四个函数: cblas_stpmv: 单精度浮点型 cblas_dtpmv: 双精度浮点型 cblas_ctpmv: 单精度复数型 cblas_ztpmv: 双精度复数型

函数族	简介
?trsv	$x \leftarrow A^{-1} * x$ or $x \leftarrow A^{-T} * x$ A 为三角矩阵; 具体分为以下四个函数: cblas_strsv: 单精度浮点型 cblas_dtrsv: 双精度浮点型 cblas_ctrsv: 单精度复数型 cblas_ztrsv: 双精度复数型
?tbsv	$x \leftarrow A^{-1} * x$ or $x \leftarrow A^{-T} * x$ A 为带状三角矩阵; 具体分为以下四个函数: cblas_stbsv: 单精度浮点型 cblas_dtbsv: 双精度浮点型 cblas_ctbsv: 单精度复数型 cblas_ztbsv: 双精度复数型
?tpsv	$x \leftarrow A^{-1} * x$ or $x \leftarrow A^{-T} * x$ A 为 packed 三角矩阵; 具体分为以下四个函数: cblas_stpsv: 单精度浮点型 cblas_dtpsv: 双精度浮点型 cblas_ctpsv: 单精度复数型 cblas_ztpsv: 双精度复数型
?ger	$A \leftarrow \alpha * x * y^T + A$, A 为常规矩阵; 具体分为以下两个函数: cblas_sger: 单精度浮点型 cblas_dger: 双精度浮点型
?syr	$A \leftarrow \alpha * x * x^T + A$ A 为对称矩阵; 具体分为以下两个函数: cblas_ssyr: 单精度浮点型 cblas_dsyr: 双精度浮点型
?spr	$A \leftarrow \alpha * x * x^T + A$ A 为 packed 对称矩阵; 具体分为以下两个函数: cblas_sspr: 单精度浮点型 cblas_dspr: 双精度浮点型
?syr2	$A \leftarrow \alpha * x * y^T + \alpha * y * x^T + A$ A 为对称矩阵; 具体分为以下两个函数: cblas_ssyr2: 单精度浮点型 cblas_dsyr2: 双精度浮点型

函数族	简介
?spr2	$A \leftarrow \alpha * x * y^T + \alpha * y * x^T + A$ A 为 packed 对称矩阵; 具体分为以下两个函数: cblas_sspr2: 单精度浮点型 cblas_dspr2: 双精度浮点型
?hpr	$A \leftarrow \alpha * x * \text{conj}(x^T) + A$ A 为三角展开埃尔米特矩阵; 具体分为以下两个函数: cblas_chpr: 单精度复数型 cblas_zhpr: 双精度复数型
?her	$A \leftarrow \alpha * x * \text{conj}(x^T) + A$ A 为 n 阶埃尔米特矩阵; 具体分为以下两个函数: cblas_cher: 单精度复数型 cblas_zher: 双精度复数型
?hpr2	$A \leftarrow \alpha * x * \text{conj}(y^T) + \text{conj}(\alpha) * y * \text{conj}(x^T) + A$ A 为三角展开埃尔米特矩阵; 具体分为以下两个函数: cblas_chpr2: 单精度复数型 cblas_zhpr2: 双精度复数型
?her2	$A \leftarrow \alpha * x * \text{conj}(y^T) + \text{conj}(\alpha) * y * \text{conj}(x^T) + A$ A 为 n 阶埃尔米特矩阵; 具体分为以下两个函数: cblas_cher2: 单精度复数型 cblas_zher2: 双精度复数型
?hpmv	$y \leftarrow \alpha * A * x + \beta * y$ A 是三角展开的埃尔米特矩阵; 具体分为以下两个函数: cblas_chpmv: 单精度复数型 cblas_zhpmv: 双精度复数型
?hemv	$y \leftarrow \alpha * A * x + \beta * y$ A 是 n 阶埃尔米特矩阵; 具体分为以下两个函数: cblas_chemv: 单精度复数型 cblas_zhemv: 双精度复数型
?hbm	$y \leftarrow \alpha * A * x + \beta * y$ A 是 n 阶埃尔米特带状矩阵; 具体分为以下两个函数: cblas_chbm: 单精度复数型 cblas_zhbm: 双精度复数型

函数族	简介
?gerc	$A \leftarrow \alpha * x * y^T + A$ A 为常规矩阵; 具体分为以下两个函数: cblas_cgerc: 单精度复数型 cblas_zgerc: 双精度复数型
?geru	$A \leftarrow \alpha * x * conj(y^T) + A$ A 为常规矩阵; 具体分为以下两个函数: cblas_cgeru: 单精度复数型 cblas_zgeru: 双精度复数型

2.1.2.2 ?gemv

向量与矩阵的乘积。

$$y = \alpha * A * x + \beta * y \text{ or } y = \alpha * A^T * x + \beta * y \text{ or } y = \alpha * conjg(A^T) * x + \beta * y$$

对于 Fortran 接口另有:

$$y = \alpha * A * conj(x) + \beta * y \text{ or } y = \alpha * A^T * conj(x) + \beta * y \text{ or } y = \alpha * conjg(A^T) * conj(x) + \beta * y$$

alpha 和 beta 是乘法系数, x 和 y 是向量, A 是 m*n 的一般矩阵。

2.1.2.2.1 接口定义

A. C interface:

```
void cblas_sgemv(const enum CBLAS_ORDER order, const enum
CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const float
alpha, const float *a, const BLASINT lda, const float *x, const BLASINT incx, const
float beta, float *y, const BLASINT incy);
```

```
void cblas_dgemv(const enum CBLAS_ORDER order, const enum
CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const double
alpha, const double *a, const BLASINT lda, const double *x, const BLASINT incx,
const double beta, double *y, const BLASINT incy);
```

```
void cblas_cgemv(const enum CBLAS_ORDER order, const enum
CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const void
*alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const
void *beta, void *y, const BLASINT incy);
```

```
void cblas_zgemv(const enum CBLAS_ORDER order, const enum
CBLAS_TRANSPOSE trans, const BLASINT m, const BLASINT n, const void
```

*alpha, const void *a, const BLASINT lda, const void *x, const BLASINT incx, const void *beta, void *y, const BLASINT incy);

B. Fortran interface:

CALL SGEMV(TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL DGEMV(TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL CGEMV(TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL ZGEMV(TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

2.1.2.2.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
trans	枚举类型 CBLAS_TRANSPOSE	表示矩阵 A 的操作。 如果 trans=CblasNoTrans 则 $y = \alpha * A * x + \beta * y$ 如果 trans=CblasTrans, 则 $y = \alpha * A^T * x + \beta * y$ 如果 trans=CblasConjTrans, 则 $y = \alpha * conjg(A^T) * x + \beta * y$ 对于 Fortran 接口另有: 如果 TransA=R, 则 $y = \alpha * A * conj(x) + \beta * y$ 如果 TransA=S, 则 $y = \alpha * A^T * conj(x) + \beta * y$ 如果 TransA=D, 则 $y = \alpha * conjg(A^T) * conj(x) + \beta * y$	输入
m	整型数	表示矩阵 A 的行数, 要求 m 大于或等于零。	输入

参数名	类型	描述	输入/输出
n	整型数	表示矩阵 A 的列数，要求 n 大于或等于零。	输入
alpha	在 sgemv 中，alpha 是单精度浮点类型。	表示乘法系数。	输入
	在 dgemv 中，alpha 是双精度浮点类型。		
	在 cgemv 中，alpha 是单精度复数类型。		
	在 zgemv 中，alpha 是双精度复数类型。		
a	在 sgemv 中，a 是单精度浮点类型。	矩阵，矩阵规模是 lda*n。	输入
	在 dgemv 中，a 是双精度浮点类型。		
	在 cgemv 中，a 是单精度复数类型。		
	在 zgemv 中，a 是双精度复数类型。		
lda	整型数	矩阵 A 中主维度长度，如果矩阵 A 为列存，要求 lda 大于或等于 max(1, m)，否则要求大于或等于 max(1, n)。	输入
x	在 sgemv 中，x 是单精度浮点类型。	向量 x。 如果 trans=CblasNoTrans，则向量规模至少是 (1+(n-1)*abs(incx))。 否则，向量规模至少是 (1+(m-1)*abs(incy))。	输入
	在 dgemv 中，x 是双精度浮点类型。		
	在 cgemv 中，x 是单精度复数类型。		
	在 zgemv 中，x 是双精度复数类型。		
incx	整型数	表示 x 中向量增长步长，不能为零。	输入

参数名	类型	描述	输入/输出
beta	在 sgemv 中，beta 是单精度浮点类型。	乘法系数。	输入
	在 dgemv 中，beta 是双精度浮点类型。		
	在 cgemv 中，beta 是单精度复数类型。		
	在 zgemv 中，beta 是双精度复数类型。		
y	在 sgemv 中，y 是单精度浮点类型。	向量 y。 如果 trans=CblasNoTrans，则 向量规模至少是 $(1+(m-1)*abs(incy))$ 。 否则，向量规模至少是 $(1+(n-1)*abs(incy))$ 。	输入/输出
	在 dgemv 中，y 是双精度浮点类型。		
	在 cgemv 中，y 是单精度复数类型。		
	在 zgemv 中，y 是双精度复数类型。		
incy	整型数	表示 y 中向量增长步长，不能为零。	输入

2.1.2.2.3 依赖

#include "kblas.h"

2.1.2.2.4 示例

A. C interface:

```
int m = 3, n = 3, lda = 3, incx = 1, incy = 1;
float alpha = 1.0, beta = 2.0;
/**
 * A:
 *    0.340188, 0.298440, -0.164777
 *   -0.105617, 0.411647, 0.268230
 *    0.283099, -0.302449, -0.222225
 * X:
```



```

*      0.053970, -0.022603, 0.128871
*   Y:
*      -0.135216, 0.013401, 0.452230
*/

float a[9] = {0.340188, 0.298440, -0.164777,
              -0.105617, 0.411647, 0.268230,
              0.283099, -0.302449, -0.222225};

float x[3] = {0.053970, -0.022603, 0.128871};
float y[3] = {-0.135216, 0.013401, 0.452230};

cblas_sgemv(CblasColMajor,CblasNoTrans, m, n, alpha, a, lda, x,
incx, beta, y, incy);

/**
*   Output Y: -0.213202, -0.005373, 0.860866
*
*/

```

2.1.2.3 ?gbmv

向量与带状矩阵乘积。

$$y = \alpha * A * x + \beta * y \text{ or } y = \alpha * A^T * x + \beta * y \text{ or } y = \alpha * \text{conjg}(A^T) * x + \beta * y$$

对于 Fortran 接口另有：

$$y = \alpha * A * \text{conj}(x) + \beta * y \text{ or } y = \alpha * A^T * \text{conj}(x) + \beta * y \text{ or } y = \alpha * \text{conjg}(A^T) * \text{conj}(x) + \beta * y$$

alpha 和 beta 是乘法系数，x 和 y 是向量，A 是 m*n 的带状矩阵。

2.1.2.3.1 接口定义

A. C interface:

```

void cblas_sgbmv(const enum CBLAS_ORDER order, const enum
CBLAS_TRANSPOSE TransA, const BLASINT M, const BLASINT N, const
BLASINT KL, const BLASINT KU, const float alpha, const float *A, const
BLASINT lda, const float *X, const BLASINT incX, const float beta, float *Y, const
BLASINT incY);

```

```
void cblas_dgbmv(const enum CBLAS_ORDER order, const enum
CBLAS_TRANSPOSE TransA, const BLASINT M, const BLASINT N, const
BLASINT KL, const BLASINT KU, const double alpha, const double *A, const
BLASINT lda, const double *X, const BLASINT incX, const double beta, double *Y,
const BLASINT incY);
```

```
void cblas_cgbmv(const enum CBLAS_ORDER order, const enum
CBLAS_TRANSPOSE TransA, const BLASINT M, const BLASINT N, const
BLASINT KL, const BLASINT KU, const void *alpha, const void *A, const
BLASINT lda, const void *X, const BLASINT incX, const void *beta, void *Y, const
BLASINT incY);
```

```
void cblas_zgbmv(const enum CBLAS_ORDER order, const enum
CBLAS_TRANSPOSE TransA, const BLASINT M, const BLASINT N, const
BLASINT KL, const BLASINT KU, const void *alpha, const void *A, const
BLASINT lda, const void *X, const BLASINT incX, const void *beta, void *Y, const
BLASINT incY);
```

B. Fortran interface:

```
CALL SGBMV(TRANS, M, N, KL, KU, ALPHA, A, LDA, X, INCX, BETA, Y,
INCY)
```

```
CALL DGBMV(TRANS, M, N, KL, KU, ALPHA, A, LDA, X, INCX, BETA, Y,
INCY)
```

```
CALL CGBMV(TRANS, M, N, KL, KU, ALPHA, A, LDA, X, INCX, BETA, Y,
INCY)
```

```
CALL ZGBMV(TRANS, M, N, KL, KU, ALPHA, A, LDA, X, INCX, BETA, Y,
INCY)
```

2.1.2.3.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

参数名	类型	描述	输入/输出
trans	枚举类型 CBLAS_TRANSPOSE	矩阵 A 为常规矩阵，转置矩阵或共轭矩阵。 如果 transA=CblasNoTrans 则 $y = \alpha * A * x + \beta * y$ 如果 transA=CblasTrans, 则 $y = \alpha * A^T * x + \beta * y$ 如果 transA=CblasConjTrans, 则 $y = \alpha * conjg(A^T) * x + \beta * y$ 对于 Fortran 接口另有: 如果 TransA=R, 则 $y = \alpha * A * conj(x) + \beta * y$ 如果 TransA=S, 则 $y = \alpha * A^T * conj(x) + \beta * y$ 如果 TransA=D, 则 $y = \alpha * conjg(A^T) * conj(x) + \beta * y$	输入
M	整型数	表示矩阵 A 的行数，要求 M 大于或等于零。	输入
N	整型数	表示矩阵 A 的列数，要求 N 大于或等于零。	输入
KL	整型数	表示 A 的次对角阶数，要求 KL 大于或等于零。	输入
KU	整型数	表示 A 的超对角阶数，要求 KU 大于或等于零。	输入
alpha	在 sgbmV 中，alpha 是单精度浮点类型。	表示乘法系数。	输入
	在 dgbmv 中，alpha 是双精度浮点类型。		

参数名	类型	描述	输入/输出
	在 cgbmv 中, alpha 是单精度复数类型。		
	在 zgbmv 中, alpha 是双精度复数类型。		
A	在 sgbmv 中, A 是单精度浮点类型。	带状矩阵, 矩阵规模是 $lda*N$ 。	输入
	在 dgbmv 中, A 是双精度浮点类型。		
	在 cgbmv 中, A 是单精度复数类型。		
	在 zgbmv 中, A 是双精度复数类型。		
ida	整型数	矩阵 A 中主维度长度, 要求 lda 大于或等于 $(KL+KU+1)$ 。	输入
X	在 sgbmv 中, X 是单精度浮点类型。	向量 X。 如果 $TransA=CblasNoTrans$, 则向量规模至少是 $(1+(N-1)*abs(incX))$ 。 否则, 向量规模至少是 $(1+(M-1)*abs(incX))$ 。	输入
	在 dgbmv 中, X 是双精度浮点类型。		
	在 cgbmv 中, X 是单精度复数类型。		
	在 zgbmv 中, X 是双精度复数类型。		
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
beta	在 sgbmv 中, beta 是单精度浮点类型。	乘法系数。	输入
	在 dgbmv 中, beta 是双精度浮点类型。		
	在 cgbmv 中, beta 是单精度复数类型。		
	在 zgbmv 中, beta 是双精度复数类型。		

参数名	类型	描述	输入/输出
Y	在 sgbmV 中，Y 是单精度浮点类型。	向量 Y。 如果 TransA=CblasNoTrans，则 向量规模至少是 $(1+(M-1)*abs(incY))$ 。 否则，向量规模至少是 $(1+(N-1)*abs(incY))$ 。	输入/输出
	在 dgbmv 中，Y 是双精度浮点类型。		
	在 cgbmv 中，Y 是单精度复数类型。		
	在 zgbmv 中，Y 是双精度复数类型。		
incY	整型数	表示 Y 中向量增长步长，不能为零。	输入

2.1.2.3.3 依赖

```
#include "kblas.h"
```

2.1.2.3.4 示例

A. C interface:

```
int m = 5, n = 4;
float alpha = 2.0;

/**
 *      | .      .    2.0  2.0 |
 *      | .    1.0  2.0  3.0 |
 *      | 1.0  2.0  2.0  4.0 |
 *  A  = | 1.0  3.0  2.0  5.0 |
 *      | 1.0  4.0  2.0   .  |
 *      | 1.0  5.0   .    .  |
 *      | .      .    .    .  |
 *      | .      .    .    .  |
 */

float a[32] = {0, 0, 1.0, 1.0, 1.0, 1.0, 0, 0,
               0, 1.0, 2.0, 3.0, 4.0, 5.0, 0, 0,
```

```
                2.0, 2.0, 2.0, 2.0, 2.0, 0, 0, 0,
                2.0, 3.0, 4.0, 5.0, 0, 0, 0, 0};

float x[4] = {2.0, 2.0, 3.0, 4.0};
float y[5] = {1.0, 1.0, 13.0, 4.0, 5.0};

cblas_sgbmv(CblasColMajor,CblasNoTrans, m, n, 3, 2, alpha, a, 8, x,
1, 10.0, y, 1);

/**
 *  Output y: 30.0, 50.0, 182.0, 104.0, 122.0
 */
```

2.1.2.4 ?symv

向量与对称矩阵乘积

$$y = \alpha * A * x + \beta * y$$

alpha 和 beta 是乘法系数，x 和 y 是向量，A 是 n*n 的带状矩阵。

2.1.2.4.1 接口定义

A. C interface:

void cblas_ssymv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const float alpha, const float *A, const BLASINT lda, const float *X, const BLASINT incX, const float beta, float *Y, const BLASINT incY);

void cblas_dsymv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO Uplo, const BLASINT N, const double alpha, const double *A, const BLASINT lda, const double *X, const BLASINT incX, const double beta, double *Y, const BLASINT incY);

B. Fortran interface:

CALL SSYMV(UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL DSYMV(UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

2.1.2.4.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角部分。 如果 Uplo=CblasLower, 则使用 A 的下三角部分。	输入
N	整型数	表示矩阵 A 的阶数, 要求 N 大于或等于零。	输入
alpha	在 ssymv 中, alpha 是单精度浮点类型。 在 dsymv 中, alpha 是双精度浮点类型。	表示系数。	输入
A	在 ssymv 中, A 是单精度浮点类型。 在 dsymv 中, A 是双精度浮点类型。	对称矩阵 A(lda, n)。	输入
lda	整型数	矩阵 A 中每列存储的地址距离, 要求 lda 大于或等于 max(1, n)。	输入
X	在 ssymv 中, X 是单精度浮点类型。 在 dsymv 中, X 是双精度浮点类型。	向量 X, 向量规模至少是 (1+(N-1)*abs(incX))。	输入
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
beta	在 ssymv 中, beta 是单精度浮点类型。 在 dsymv 中, beta 是双精度浮点类型。	乘法系数。	输入
Y	在 ssymv 中, Y 是单精度浮点类型。	向量 Y, 向量规模至少是 (1+(N-1)*abs(incY))。	输入/输出

参数名	类型	描述	输入/输出
	在 dsymv 中，Y 是双精度浮点类型。		
incY	整型数	表示 Y 中向量增长步长，不能为零。	输入

2.1.2.4.3 依赖

```
#include "kblas.h"
```

2.1.2.4.4 示例

A. C interface:

```
int n = 3, lda = 3;

float alpha = 1.0, beta = 1.0;

int incx = 1, incy = 1;

/*
 * A = | 0.0  *  * |
 *      | 2.0 3.0 * |
 *      | 0.0 4.0 1.0 |
 */

float a[9] = {0, 2.0, 0, 0, 3.0, 4.0, 0, 0, 1.0};
float x[3] = {5.0, 2.0, 1.0};
float y[3] = {1.0, 1.0, 3.0};

cblas_ssymv(CblasColMajor,CblasLower, n, alpha, a, lda, x, incx,
beta, y, incy);

/*
 * Output y: 5.000000, 21.000000, 12.000000
 */
```


2.1.2.5 ?sbmv

向量与对称带状矩阵乘积

$$y = \alpha * A * x + \beta * y$$

alpha 和 beta 是乘法系数，x 和 y 是向量，A 是 n*n 的对称带状矩阵。

2.1.2.5.1 接口定义

A. C interface:

```
void cblas_ssbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const BLASINT K, const float alpha, const float *A, const
BLASINT lda, const float *X, const BLASINT incX, const float beta, float *Y, const
BLASINT incY);
```

```
void cblas_dsbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const BLASINT K, const double alpha, const double *A,
const BLASINT lda, const double *X, const BLASINT incX, const double beta,
double *Y, const BLASINT incY);
```

B. Fortran interface:

```
CALL SSBMV(UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
```

```
CALL DSBMV(UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
```

2.1.2.5.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角部分。 如果 Uplo=CblasLower, 则使用 A 的下三角部分。	输入
N	整型数	表示矩阵 A 的阶数, 要求 N 大于或等于零。	输入

参数名	类型	描述	输入/输出
K	整型数	表示矩阵 A 的超对角阶数，要求 K 大于或等于零。	输入
alpha	在 ssbmV 中，alpha 是单精度浮点类型。	表示系数。	输入
	在 dsbmV 中，alpha 是双精度浮点类型。		
A	在 ssbmV 中，A 是单精度浮点类型。	对称带状矩阵，矩阵规模是 lda*n。	输入
	在 dsbmV 中，A 是双精度浮点类型。		
lda	整型数	矩阵 A 中每列存储的地址距离，要求 lda 大于或等于 (k+1)。	输入
X	在 ssbmV 中，X 是单精度浮点类型。	向量 X，向量规模至少是 $(1+(N-1)*abs(incX))$ 。	输入
	在 dsbmV 中，X 是双精度浮点类型。		
incX	整型数	表示 X 中向量增长步长，不能为零。	输入
beta	在 ssbmV 中，beta 是单精度浮点类型。	乘法系数。	输入
	在 dsbmV 中，beta 是双精度浮点类型。		
Y	在 ssbmV 中，Y 是单精度浮点类型。	向量 Y，向量规模至少是 $(1+(N-1)*abs(incY))$ 。	输入/输出
	在 dsbmV 中，Y 是双精度浮点类型。		
incY	整型数	表示 Y 中向量增长步长，不能为零。	输入

2.1.2.5.3 依赖

```
#include "kblas.h"
```

2.1.2.5.4 示例

A. C interface:

```
int n = 4, k = 2;

float alpha = 2.0;

/**
 *      | .      .  2.0  2.0 |
 *      | .   1.0  2.0  3.0 |
 *      | 1.0  2.0  2.0  4.0 |
 *  A  = | 1.0  3.0  2.0  5.0 |
 *      | 1.0  4.0  2.0  6.0 |
 *      | 1.0  5.0  2.0  2.0 |
 *      | .      .      .   |
 *      | .      .      .   |
 */

float a[32] = {0, 0, 1.0, 1.0, 1.0, 1.0, 0, 0,
               0, 1.0, 2.0, 3.0, 4.0, 5.0, 0, 0,
               2.0, 2.0, 2.0, 2.0, 2.0, 2.0, 0, 0,
               2.0, 3.0, 4.0, 5.0, 6.0, 2.0, 0, 0};

float x[4] = {2.0, 2.0, 3.0, 4.0};

float y[4] = {1.0, 1.0, 13.0, 4.0};

cblas_ssbmv(CblasColMajor,CblasUpper, n, k, alpha, a, 8, x, 1, 1.0,
y, 1);

/**
 * Output y = |21.0, 41.0, 65.0, 62.0|
 */
```

2.1.2.6 ?spmv

向量与压缩的对称矩阵乘积

$$y = \alpha * A * x + \beta * y$$

alpha 和 beta 是缩放系数，x 和 y 是含有 n 个元素的向量，A 是 n 阶压缩的对称矩阵。

2.1.2.6.1 接口定义

A. C interface:

```
void cblas_sspmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const float alpha, const float *Ap, const float *X, const
BLASINT incX, const float beta, float *Y, const BLASINT incY);
```

```
void cblas_dspmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const double alpha, const double *Ap, const double *X,
const BLASINT incX, const double beta, double *Y, const BLASINT incY);
```

B. Fortran interface:

```
CALL SSPMV(UPLO, N, ALPHA, AP, X, INCX, BETA, Y, INCY)
```

```
CALL DSPMV(UPLO, N, ALPHA, AP, X, INCX, BETA, Y, INCY)
```

2.1.2.6.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 存储展开方式（上三角或下三角）。 如果 Uplo=CblasUpper，则使用 A 的上三角部分。 如果 Uplo=CblasLower，则使用 A 的下三角部分。	输入
N	整型数	表示矩阵 A 的阶数，要求 N 大于或等于零。	输入
alpha	在 sspmv 中，alpha 是单精度复数类型。	表示乘法系数。	输入

参数名	类型	描述	输入/输出
	在 dspmv 中, alpha 是双精度复数类型。		
Ap	在 sspmv 中, Ap 是单精度复数类型。	压缩的对称矩阵, 矩阵规模至少是 $(n*(n+1)/2)$ 。	输入
	在 dspmv 中, Ap 是双精度复数类型。		
X	在 sspmv 中, X 是单精度复数类型。	向量 X, 向量规模至少是 $(1+(N-1)*abs(incX))$ 。	输入
	在 dspmv 中, X 是双精度复数类型。		
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
beta	在 sspmv 中, beta 是单精度复数类型。	乘法系数。	输入
	在 dspmv 中, beta 是双精度浮点类型。		
Y	在 sspmv 中, Y 是单精度复数类型。	向量 Y, 向量规模至少是 $(1+(N-1)*abs(incY))$ 。	输入/输出
	在 dspmv 中, Y 是双精度复数类型。		
incY	整型数	表示 Y 中向量增长步长, 不能为零。	输入

2.1.2.6.3 依赖

```
#include "kblas.h"
```

2.1.2.6.4 示例

A. C interface:

```
int n = 3;
float alpha = 1.0, beta = 1.0;
int incx = 1, incy = 1;
```

```

/*
 * A = | 2.0  1.0  3.0 |
 *      | 1.0  6.0  9.0 |
 *      | 3.0  9.0  2.0 |
 */

float a[6] = {2.0, 1.0, 3.0, 6.0, 9.0, 2.0};
float x[3] = {1.0, 1.0, 1.0};
float y[3] = {3.0, 2.0, 2.0};

cblas_sspmv(CblasColMajor,CblasLower, n, alpha, a, x, incx, beta, y,
incy);

/*
 * Output y = |9.0, 18.0, 16.0|
 */

```

2.1.2.7 ?trmv

向量与三角矩阵乘积。

$x = A * x$ or $x = A^T * x$ or $x = conjg(A^T) * x$

其中，x 为包含 n 个元素的向量，A 是 n*n 的单元或非单元三角矩阵。

2.1.2.7.1 接口定义

A. C interface:

```
void cblas_strmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const float *A, const BLASINT lda, float *X, const BLASINT
incX);
```

```
void cblas_dtrmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const double *A, const BLASINT lda, double *X, const
BLASINT incX);
```

```
void cblas_ctrmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const void *A, const BLASINT lda, void *X, const BLASINT
incX);
```

```
void cblas_ztrmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const void *A, const BLASINT lda, void *X, const BLASINT
incX);
```

B. Fortran interface:

```
CALL STRMV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)
```

```
CALL DTRMV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)
```

```
CALL CTRMV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)
```

```
CALL ZTRMV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)
```

2.1.2.7.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 存储展开方式（上三角或下三角）。 如果 Uplo=CblasUpper，则使用 A 的上三角部分。 如果 Uplo=CblasLower，则使用 A 的下三角部分。	输入
TransA	枚举类型 CBLAS_TRANSPOSE	矩阵 A 为常规矩阵，转置矩阵或共轭矩阵。 如果 trans=CblasNoTrans，则 $x = A * x$ 如果 trans=CblasTrans，则 $x = A^T * x$ 如果 trans=CblasConjTrans，则 $x = \text{conjg}(A^T) * x$	输入

参数名	类型	描述	输入/输出
Diag	枚举类型 CBLAS_DIAG	表示 A 是否是单元三角阵。 如果 Diag=CblasUnit, 则 A 是单元三角矩阵。 如果 Diag=CblasNonUnit, 则 A 不是单元三角矩阵。	输入
A	在 strmv 中, A 是单精度浮点类型。 在 dtrmv 中, A 是双精度浮点类型。 在 ctrmv 中, A 是单精度复数类型。 在 ztrmv 中, A 是双精度复数类型。	三角矩阵, 矩阵规模至少是 lda*N。	输入
lda	整型数	矩阵 A 中每列存储的地址距离, 要求 lda 大于或等于 max(1, N)。	输入
X	在 strmv 中, X 是单精度浮点类型。 在 dtrmv 中, X 是双精度浮点类型。 在 ctrmv 中, X 是单精度复数类型。 在 ztrmv 中, X 是双精度复数类型。	向量 X, 向量规模至少是 (1+(N-1)*abs(incX))。	输入/输出
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入

2.1.2.7.3 依赖

```
#include "kblas.h"
```


2.1.2.7.4 示例

A. C interface:

```
int n = 4, incx = 1;

float x[4] = {4.0, 3.0, 2.0, 1.0};

/**
 *      | .  2.0  1.0  1.0 |
 *  A   = | .  .  1.0  2.0 |
 *      | .  .  .  1.0 |
 *      | .  .  .  .  |
 **/

float a[16] = {0, 0, 0, 0,
               2.0, 0, 0, 0,
               1.0, 1.0, 0, 0,
               1.0, 2.0, 1.0, 0};

cblas_strmv(CblasColMajor, CblasUpper, CblasTrans, CblasUnit, n, a,
4, x, incx);

/**
 * Output x = |4.0, 11.0, 9.0, 13.0|
 **/
```

2.1.2.8 ?tbmv

向量与三角带状矩阵乘积。

$$x = A * x \text{ or } x = A^T * x \text{ or } x = \text{conjg}(A^T) * x$$

其中，x 为包含 n 个元素的向量，A 是 n*n 的单元或非单元三角带状矩阵。

2.1.2.8.1 接口定义

A. C interface:

```
void cblas_stbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
```

```
const BLASINT N, const BLASINT K, const float *A, const BLASINT lda, float *X,
const BLASINT incX);
```

```
void cblas_dtbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const BLASINT K, const double *A, const BLASINT lda, double
*X, const BLASINT incX);
```

```
void cblas_ctbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const BLASINT K, const void *A, const BLASINT lda, void *X,
const BLASINT incX);
```

```
void cblas_ztbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const BLASINT K, const void *A, const BLASINT lda, void *X,
const BLASINT incX);
```

B. Fortran interface:

```
CALL STBMV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)
```

```
CALL DTBMV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)
```

```
CALL CTBMV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)
```

```
CALL ZTBMV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)
```

2.1.2.8.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 存储展开方式（上三角或下三角）。 如果 Uplo=CblasUpper，则使用 A 的上三角部分。 如果 Uplo=CblasLower，则使用 A 的下三角部分。	输入

参数名	类型	描述	输入/输出
TransA	枚举类型 CBLAS_TRANSPOSE	矩阵 A 为常规矩阵，转置矩阵或共轭矩阵。 如果 trans=CblasNoTrans，则 $x = A * x$ 如果 trans=CblasTrans，则 $x = A^T * x$ 如果 trans=CblasConjTrans，则 $x = conjg(A^T) * x$	输入
Diag	枚举类型 CBLAS_DIAG	表示 A 是否是单元三角阵。 如果 Diag=CblasUnit，则 A 是单元三角矩阵。 如果 Diag=CblasNonUnit，则 A 不是单元三角矩阵。	输入
N	整型数	表示矩阵 A 的阶数，要求 N 大于或等于零。	输入
K	整型数	表示矩阵 A 的超对角阶数，要求 K 大于或等于零。	输入
A	在 stbmV 中，A 是单精度浮点类型。 在 dtbmV 中，A 是双精度浮点类型。 在 ctbmV 中，A 是单精度复数类型。 在 ztbmV 中，A 是双精度复数类型。	三角带状矩阵 A(lda, n)。	输入
lda	整型数	矩阵 A 中每列存储的地址距离，要求 lda 大于或等于 (k+1)。	输入

参数名	类型	描述	输入/输出
X	在 stbm _v 中，X 是单精度浮点类型。	向量 X，向量规模至少是 (1+(N-1)*abs(incX))。	输入/输出
	在 dtbm _v 中，X 是双精度浮点类型。		
	在 ctbm _v 中，X 是单精度复数类型。		
	在 ztbm _v 中，X 是双精度复数类型。		
incX	整型数	表示 X 中向量增长步长，不能为零。	输入

2.1.2.8.3 依赖

```
#include "kblas.h"
```

2.1.2.8.4 示例

A. C interface:

```
int n = 7, k = 3, lda = 5, incx = 1;

/**
 *      | . . . 2.0 2.0 3.0 1.0 |
 *      | . . 2.0 1.0 2.0 4.0 0.0 |
 *      | . 2.0 2.0 3.0 4.0 2.0 1.0 |
 * A = | 1.0 3.0 3.0 3.0 3.0 3.0 3.0 |
 *      | . . . . . . . |
 */

float a[35] = {0, 0, 0, 1.0, 0,
               0, 0, 2.0, 3.0, 0,
               0, 2.0, 2.0, 3.0, 0,
               2.0, 1.0, 3.0, 3.0, 0,
               2.0, 2.0, 4.0, 3.0, 0,
               3.0, 4.0, 2.0, 3.0, 0,
```

```

        1.0, 0.0, 1.0, 3.0, 0};

float x[7] = {2.0, 2.0, 1.0, 2.0, 1.0, 3.0, 4.0};

cblas_stbmv(CblasColMajor,CblasUpper, CblasNoTrans, CblasNonUnit, n,
k, a, lda, x, incx);

/**
 * Output x = |12.0, 12.0, 20.0, 26.0, 9.0, 13.0, 12.0|
 */

```

2.1.2.9 ?tpmv

向量与三角带状矩阵乘积。

$$x = A * x \text{ or } x = A^T * x \text{ or } x = \text{conjg}(A^T) * x$$

其中, x 为包含 n 个元素的向量, A 是 n*n 的单元或非单元按照三角展开的三角矩阵。

2.1.2.9.1 接口定义

A. C interface:

```
void cblas_stpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const float *Ap, float *X, const BLASINT incX);
```

```
void cblas_dtpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const double *Ap, double *X, const BLASINT incX);
```

```
void cblas_ctpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const void *Ap, void *X, const BLASINT incX);
```

```
void cblas_ztpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const void *Ap, void *X, const BLASINT incX);
```

B. Fortran interface:

```
CALL STPMV(UPLO, TRANS, DIAG, N, AP, X, INCX)
```

```
CALL DTPMV(UPLO, TRANS, DIAG, N, AP, X, INCX)
```

```
CALL CTPMV(UPLO, TRANS, DIAG, N, AP, X, INCX)
```

CALL ZTPMV(UPLO, TRANS, DIAG, N, AP, X, INCX)

2.1.2.9.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角部分。 如果 Uplo=CblasLower, 则使用 A 的下三角部分。	输入
TransA	枚举类型 CBLAS_TRANSPOSE	矩阵 A 为常规矩阵, 转置矩阵或共轭矩阵。 如果 trans=CblasNoTrans, 则 $x = A * x$ 如果 trans=CblasTrans, 则 $x = A^T * x$ 如果 trans=CblasConjTrans, 则 $x = conjg(A^T) * x$	输入
Diag	枚举类型 CBLAS_DIAG	表示 A 是否是单元三角阵。 如果 Diag=CblasUnit, 则 A 是单元三角矩阵。 如果 Diag=CblasNonUnit, 则 A 不是单元三角矩阵。	输入
N	整型数	表示矩阵 A 的阶数, 要求 N 大于或等于零。	输入
Ap	在 stpmv 中, Ap 是单精度浮点类型。 在 dtpmv 中, Ap 是双精度浮点类型。	压缩的三角矩阵, 矩阵规模至少是 $(N*(N+1)/2)$ 。	输入

参数名	类型	描述	输入/输出
	在 ctpmv 中，Ap 是单精度复数类型。		
	在 ztpmv 中，Ap 是双精度复数类型。		
X	在 stpmv 中，X 是单精度浮点类型。	向量 X，向量规模至少是 $(1+(N-1)*abs(incX))$ 。	输入/输出
	在 dtpmv 中，X 是双精度浮点类型。		
	在 ctpmv 中，X 是单精度复数类型。		
	在 ztpmv 中，X 是双精度复数类型。		
incX	整型数	表示 X 中向量增长步长，不能为零。	输入

2.1.2.9.3 依赖

```
#include "kblas.h"
```

2.1.2.9.4 示例

A. C interface:

```
int n = 4, incx = 1;
float x[4] = {4.0, 3.0, 2.0, 1.0};

/**
 *      |  1.0 1.0  1.0  3.0 |
 * A    = |  .   3.0  3.0  2.0 |
 *      |  .   .   2.0  3.0 |
 *      |  .   .   .   1.0 |
 */
float ap[10] = {1.0, 1.0, 3.0, 1.0, 3.0, 2.0, 3.0, 2.0, 3.0, 1.0};
```

```

cblas_stpmv(CblasColMajor, CblasUpper, CblasTrans, CblasNonUnit, n,
ap, x, incx);

/**
 * Output x = |4.0, 13.0, 17.0, 25.0|
 */

```

2.1.2.10 ?trsv

线性方程组求解，系数矩阵是三角矩阵。

$$A * x = b \text{ or } A^T * x = b \text{ or } \text{conjg}(A^T) * x = b$$

其中，x 和 b 为包含 n 个元素的向量，A 是 n*n 的单元或非单元的三角矩阵。

2.1.2.10.1 接口定义

A. C interface:

```

void cblas_strsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const float *A, const BLASINT lda, float *X, const BLASINT
incX);

```

```

void cblas_dtrsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const double *A, const BLASINT lda, double *X, const
BLASINT incX);

```

```

void cblas_ctrsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const void *A, const BLASINT lda, void *X, const BLASINT
incX);

```

```

void cblas_ztrsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const void *A, const BLASINT lda, void *X, const BLASINT
incX);

```

B. Fortran interface:

CALL STRSV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

CALL DTRSV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

CALL CTRSV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

CALL ZTRSV(UPLO, TRANS, DIAG, N, A, LDA, X, INCX)

2.1.2.10.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角部分。 如果 Uplo=CblasLower, 则使用 A 的下三角部分。	输入
TransA	枚举类型 CBLAS_TRANSPOSE	矩阵 A 为常规矩阵, 转置矩阵或共轭矩阵。 如果 trans=CblasNoTrans, 则 $x = A * x$ 如果 trans=CblasTrans, 则 $x = A^T * x$ 如果 trans=CblasConjTrans, 则 $x = conjg(A^T) * x$	输入
Diag	枚举类型 CBLAS_DIAG	表示 A 是否是单元三角阵。 如果 Diag=CblasUnit, 则 A 是单元三角矩阵。 如果 Diag=CblasNonUnit, 则 A 不是单元三角矩阵。	输入
N	整型数	表示矩阵 A 的阶数, 要求 N 大于或等于零。	输入
A	在 strsv 中, A 是单精度浮点类型。 在 dtrsv 中, A 是双精度浮点类型。	三角矩阵, 矩阵规模至少是 lda*N。	输入

参数名	类型	描述	输入/输出
	在 ctrsv 中，A 是单精度复数类型。		
	在 ztrsv 中，A 是双精度复数类型。		
lda	整型数	矩阵 A 中每列存储的地址距离，要求 lda 大于或等于 max(1, N)。	输入
X	在 strsv 中，X 是单精度浮点类型。	向量 X，向量规模至少是 (1+(N-1)*abs(incX))。	输入/输出
	在 dtrsv 中，X 是双精度浮点类型。		
	在 ctrsv 中，X 是单精度复数类型。		
	在 ztrsv 中，X 是双精度复数类型。		
incX	整型数	表示 X 中向量增长步长，不能为零。	输入

2.1.2.10.3 依赖

```
#include "kblas.h"
```

2.1.2.10.4 示例

A. C interface:

```
int n = 4, lda = 4, incx = 1;

float x[4] = {4.0, 3.0, 2.0, 1.0};

/**
 *      |  1.0 1.0  1.0  3.0 |
 * A    = |  .   3.0  3.0  2.0 |
 *      |  .   .   2.0  3.0 |
 *      |  .   .   .   1.0 |
 */
```

```

float ap[16] = {1.0, 0, 0, 0, 1.0, 3.0, 0, 0, 1.0, 3.0, 2.0, 0, 3.0,
2.0, 3.0, 1.0};

cblas_strsv(CblasColMajor, CblasUpper, CblasTrans, CblasNonUnit, n,
ap, lda, x, incx);

/**
 * Output x = |4.000000, -0.333333, -0.500000, -8.833333|
 */

```

2.1.2.11 ?tbsv

线性方程组求解，稀疏矩阵为三角带状矩阵。

$$x = \text{conjg}(A^T) * x \text{ or } A^T * x = b \text{ or } \text{conjg}(A^T) * x = b$$

其中，x 和 b 为包含 n 个元素的向量，A 为 n*n 的单元或非单元、上三角或下三角的带状矩阵。

2.1.2.11.1 接口定义

A. C interface:

```
void cblas_stbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const blasint N, const blasint K, const float *A, const blasint lda, float *X, const
blasint incX);
```

```
void cblas_dtbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const blasint N, const blasint K, const double *A, const blasint lda, double *X, const
blasint incX);
```

```
void cblas_ctbsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const blasint N, const blasint K, const void *A, const blasint lda, void *X, const
blasint incX);
```

```
void cblas_ztbsv(const enum CBLAS_ORDER order, const enum
CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA, const enum
CBLAS_DIAG Diag, const blasint N, const blasint K, const void* A, const blasint lda,
void *X, const blasint incX);
```

B. Fortran interface:

CALL STBSV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

CALL DTBSV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

CALL CTBSV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

CALL ZTBSV(UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)

2.1.2.11.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角部分。 如果 Uplo=CblasLower, 则使用 A 的下三角部分。	输入
TransA	枚举类型 CBLAS_TRANSPOSE	矩阵 A 为常规矩阵, 转置矩阵或共轭矩阵。 如果 trans=CblasNoTrans, 则 $x = A * x$ 如果 trans=CblasTrans, 则 $x = A^T * x$ 如果 trans=CblasConjTrans, 则 $x = conjg(A^T) * x$	输入
Diag	枚举类型 CBLAS_DIAG	表示 A 是否是单元三角阵。 如果 Diag=CblasUnit, 则 A 是单元三角矩阵。 如果 Diag=CblasNonUnit, 则 A 不是单元三角矩阵。	输入
N	整型数	表示矩阵 A 的阶数, 要求 N 大于或等于零。	输入

参数名	类型	描述	输入/输出
K	整型数	表示矩阵 A 的超对角阶数，要求 K 大于或等于零。	输入
A	在 stbsv 中，A 是单精度浮点类型。	三角带状矩阵 A(lda, n)。	输入
	在 dtbsv 中，A 是双精度浮点类型。		
	在 ctbsv 中，A 是单精度复数类型。		
	在 ztbsv 中，A 是双精度复数类型。		
lda	整型数	矩阵 A 中每列存储的地址距离，要求 lda 大于或等于 (k+1)。	输入
X	在 stbsv 中，X 是单精度浮点类型。	向量 X，向量规模至少是 (1+(N-1)*abs(incX))。	输入/输出
	在 dtbsv 中，X 是双精度浮点类型。		
	在 ctbsv 中，X 是单精度复数类型。		
	在 ztbsv 中，X 是双精度复数类型。		
incX	整型数	表示 X 中向量增长步长，不能为零。	输入

2.1.2.11.3 依赖

```
#include "kblas.h"
```

2.1.2.11.4 示例

A. C interface:

```
int n = 7, k = 3, lda = 5, incx = 1;
```

```

/**
 *      | .      .      .      2.0  2.0  3.0 1.0 |
 *      | .      .      2.0  1.0  2.0  4.0 0.0 |
 *      | .      2.0  2.0  3.0  4.0  2.0 1.0 |
 *  A = | 1.0  3.0  3.0  3.0  3.0  3.0 3.0 |
 *      | .      .      .      .      .      .      |
 */

float a[35] = {0, 0, 0, 1.0, 0,
               0, 0, 2.0, 3.0, 0,
               0, 2.0, 2.0, 3.0, 0,
               2.0, 1.0, 3.0, 3.0, 0,
               2.0, 2.0, 4.0, 3.0, 0,
               3.0, 4.0, 2.0, 3.0, 0,
               1.0, 0.0, 1.0, 3.0, 0};

float x[7] = {12.0, 12.0, 20.0, 26.0, 9.0, 13.0, 12.0};

cblas_stbsv(CblasColMajor, CblasUpper, CblasNoTrans, CblasNonUnit,
n, k, a, lda, x, incx);

/**
 * Output x = |2.000000 2.000000 1.000000 2.000000 1.000000
3.000000 4.000000|
 */

```

2.1.2.12 ?tpsv

线性方程组求解，系数矩阵是按照三角展开的三角矩阵。

$$A * x = b \text{ or } A^T * x = b \text{ or } \text{conjg}(A^T) * x = b$$

其中，x 和 b 为包含 n 个元素的向量，A 为 n*n 的单元或非单元按照三角展开的三角矩阵。

2.1.2.12.1 接口定义

A. C interface:

```
void cblas_stpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const float *Ap, float *X, const BLASINT incX);
```

```
void cblas_dtpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const double *Ap, double *X, const BLASINT incX);
```

```
void cblas_ctpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const void *Ap, void *X, const BLASINT incX);
```

```
void cblas_ztpsv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE TransA, const enum CBLAS_DIAG Diag,
const BLASINT N, const void *Ap, void *X, const BLASINT incX);
```

B. Fortran interface:

```
CALL STPSV(UPLO, TRANS, DIAG, N, AP, X, INCX)
```

```
CALL DTPSV(UPLO, TRANS, DIAG, N, AP, X, INCX)
```

```
CALL CTPSV(UPLO, TRANS, DIAG, N, AP, X, INCX)
```

```
CALL ZTPSV(UPLO, TRANS, DIAG, N, AP, X, INCX)
```

2.1.2.12.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角部分。 如果 Uplo=CblasLower, 则使用 A 的下三角部分。	输入

参数名	类型	描述	输入/输出
TransA	枚举类型 CBLAS_TRANSPOSE	矩阵 A 为常规矩阵，转置矩阵或共轭矩阵。 如果 trans=CblasNoTrans，则 $x = A * x$ 如果 trans=CblasTrans，则 $x = A^T * x$ 如果 trans=CblasConjTrans，则 $x = conjg(A^T) * x$	输入
Diag	枚举类型 CBLAS_DIAG	表示 A 是否是单元三角阵。 如果 Diag=CblasUnit，则 A 是单元三角矩阵。 如果 Diag=CblasNonUnit，则 A 不是单元三角矩阵。	输入
N	整型数	表示矩阵 A 的阶数，要求 N 大于或等于零。	输入
Ap	在 stpsv 中，Ap 是单精度浮点类型。 在 dtpsv 中，Ap 是双精度浮点类型。 在 ctpsv 中，Ap 是单精度复数类型。 在 ztpsv 中，Ap 是双精度复数类型。	压缩的三角矩阵，矩阵规模至少是 $(N*(N+1)/2)$ 。	输入
X	在 stpsv 中，X 是单精度浮点类型。 在 dtpsv 中，X 是双精度浮点类型。 在 ctpsv 中，X 是单精度复数类型。	向量 X，向量规模至少是 $(1+(N-1)*abs(incX))$ 。 获得方程解后更新输出。	输入/输出

参数名	类型	描述	输入/输出
	在 ztpsv 中，X 是双精度复数类型。		
incX	整型数	表示 X 中向量增长步长，不能为零。	输入

2.1.2.12.3 依赖

```
#include "kblas.h"
```

2.1.2.12.4 示例

A. C interface:

```
int  n = 4, incx = 1;
float x[4] = {4.0, 3.0, 2.0, 1.0};
/**
 *      |  1.0 1.0  1.0  3.0 |
 * A    = |  .   3.0  3.0  2.0 |
 *      |  .   2.0  3.0  .
 *      |  .   .   .   1.0 |
 */
float ap[10] = {1.0, 1.0, 3.0, 1.0, 3.0, 2.0, 3.0, 2.0, 3.0, 1.0};

cblas_stpsv(CblasColMajor, CblasUpper, CblasTrans, CblasNonUnit, n,
ap, x, incx);
/**
 * Output x = |4.000000, -0.333333, -0.500000, -8.833333|
 */
```

2.1.2.13 ?ger

一般矩阵秩 1 更新， $A = \alpha * x * y^T + A$
其中，alpha 是乘法系数，A 是 m*n 的一般矩阵，x 是含有 m 个元素的向量，y 是含有 n 个元素的向量。

2.1.2.13.1 接口定义

A. C interface:

```
void cblas_sger (const enum CBLAS_ORDER order, const BLASINT M, const
BLASINT N, const float alpha, const float *X, const BLASINT incX, const float *Y,
const BLASINT incY, float *A, const BLASINT lda);
```

```
void cblas_dger (const enum CBLAS_ORDER order, const BLASINT M, const
BLASINT N, const double alpha, const double *X, const BLASINT incX, const
double *Y, const BLASINT incY, double *A, const BLASINT lda);
```

B. Fortran interface:

```
CALL SGER(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)
```

```
CALL DGER(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)
```

2.1.2.13.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
M	整型数	矩阵 A 的行数。	输入
N	整型数	矩阵 A 的列数。	输入
alpha	在 sger 中, alpha 为单精度浮点类型。 在 dger 中, alpha 为双精度浮点类型。	乘法系数。	输入
X	在 sger 中, X 为单精度浮点类型。 在 dger 中, X 为双精度浮点类型。	矩阵 X, 向量规模至少是 $(1+(m-1)*abs(incx))$ 。	输入
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
Y	在 sger 中, Y 为单精度浮点类型。 在 dger 中, Y 为双精度浮点类型。	矩阵 Y, 向量规模至少是 $(1+(n-1)*abs(incy))$ 。	输入

参数名	类型	描述	输入/输出
incY	整型数	向量 Y 的增长步长，不能为零。	输入
A	在 sger 中，A 为单精度浮点类型。	矩阵 A(lda, n)。	输出
	在 dger 中，A 为双精度浮点类型。		
lda	整型数	矩阵 A 中主维度长度，如果矩阵 A 为列存，要求 lda 大于或等于 max(1, m)，否则要求大于或等于 max(1, n)。	输入

2.1.2.13.3 依赖

```
#include "kblas.h"
```

2.1.2.13.4 示例

A. C interface:

```
int m = 4, n = 4, lda = 4;
float alpha = 1.0;
int incx = 1, incy = 1;
/**
 * X:
 *   0.340188, -0.105617, 0.283099, 0.298440
 * Y:
 *   0.411647, -0.302449, -0.164777, 0.268230
 *
 * Input A:
 *   -0.222225    -0.135216     0.135712    -0.483699
 *   0.053970     0.013401     0.217297    -0.257113
 *   -0.022603     0.452230    -0.358397    -0.362768
```

```

*      0.128871      0.416195      0.106969      0.304177
*/

float x[4] = {0.340188, -0.105617, 0.283099, 0.298440};
float y[4] = {0.411647, -0.302449, -0.164777, 0.268230};
float a[16] = {-0.222225, -0.135216, 0.135712, -0.483699,
               0.053970, 0.013401, 0.217297, -0.257113,
               -0.022603, 0.452230, -0.358397, -0.362768,
               0.128871, 0.416195, 0.106969, 0.304177};

cblas_sger(CblasColMajor, m, n, alpha, x, incx, y, incy, a, lda);
/**
 * Output A:
 *
 *      -0.082188      -0.048920      -0.078658      0.220120
 *      -0.178693      0.045345      0.469633      0.387865
 *      0.252249      0.131674      -0.405045      0.182905
 *      -0.360847      -0.347376      -0.411944      0.384228
 */

```

2.1.2.14 ?syr

对称矩阵秩 1 更新, $A = \alpha * x * x^T + A$

其中, alpha 为乘法系数, x 为包含 n 个元素的向量, A 为 n*n 的对称矩阵。

2.1.2.14.1 接口定义

A. C interface:

```
void cblas_ssyr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const float alpha, const float *X, const BLASINT incX,
float *A, const BLASINT lda);
```

```
void cblas_dsyr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const double alpha, const double *X, const BLASINT incX,
double *A, const BLASINT lda);
```

B. Fortran interface:

```
CALL SSYR(UPLO, N, ALPHA, X, INCX, A, LDA)
```

CALL DSYR(UPLO, N, ALPHA, X, INCX, A, LDA)

2.1.2.14.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角。 如果 Uplo=CblasLower, 则使用 A 的下三角。	输入
N	整型数	向量 X 的元素个数。	输入
alpha	在 ssyr 中, alpha 为单精度浮点类型。 在 dsyr 中, alpha 为双精度浮点类型。	乘法系数。	输入
X	在 ssyr 中, X 为单精度浮点类型。 在 dsyr 中, X 为双精度浮点类型。	矩阵 X, 长度至少 $1+(n-1)*abs(incX)$ 。	输入
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
A	在 ssyr 中, A 为单精度浮点类型。 在 dsyr 中, A 为双精度浮点类型。	矩阵 A(lda, n)。	输入/输出
lda	整型数	矩阵 A 的主维度长度, 要求 lda 大于或等于 $\max(1, n)$ 。	输入

2.1.2.14.3 依赖

#include "kblas.h"

2.1.2.14.4 示例

A. C interface:

```
int n = 3, lda = 4, incx = 1;

float alpha = 1.0;

float x[3] = {1.0, 3.0, 2.0};

/**
 *
 *      | 3.0  14.0  12.0 |
 *      A  = | .   16.0  17.0 |
 *      | .   .   13.0 |
 *      | .   .   .   |
 */

float a[12] = {3.0, 0, 0, 0, 14.0, 16.0, 0, 0, 12, 17.0, 13.0, 0};

cblas_ssypr(CblasColMajor, CblasUpper, n, alpha, x, incx, a, lda);

/**
 *
 *      | 4.0  17.0  14.0 |
 *      Output A = | .   25.0  23.0 |
 *      | .   .   17.0 |
 *      | .   .   .   |
 */
```

2.1.2.15 ?spr

对称展开矩阵秩 1 更新, $A = \alpha * x * x^T + A$

其中, alpha 为乘法系数, x 为包含 n 个元素的向量, A 为 n*n 的三角展开对称矩阵。

2.1.2.15.1 接口定义

A. C interface:

```
void cblas_sspr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const float alpha, const float *X, const BLASINT incX,
float *Ap);
```

```
void cblas_dspr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const double alpha, const double *X, const BLASINT incX,
double *Ap);
```

B. Fortran interface:

```
CALL SSPR(UPLO, N, ALPHA, X, INCX, AP)
```

```
CALL DSPR(UPLO, N, ALPHA, X, INCX, AP)
```

2.1.2.15.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角。 如果 Uplo=CblasLower, 则使用 A 的下三角。	输入
N	整型数	向量 X 的元素个数。	输入
alpha	在 sspr 中, alpha 为单精度浮点类型。 在 dspr 中, alpha 为双精度浮点类型。	乘法系数。	输入
X	在 sspr 中, X 为单精度浮点类型。 在 dspr 中, X 为双精度浮点类型	矩阵 X, 长度至少 $1+(n-1)*abs(incX)$ 。	输入
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
Ap	在 sspr 中, Ap 为单精度浮点类型。 在 dspr 中, Ap 为双精度浮点类型。	矩阵 A	输出

2.1.2.15.3 依赖

```
#include "kblas.h"
```

2.1.2.15.4 示例

A. C interface:

```
int n = 3;

float alpha = 1.0;

int incx = 1;

float x[3] = {2.0, 2.0, 1.0};

/**
 *          | 3.0  1.0  2.0 |
 *      A = | 1.0  6.0  3.0 |
 *          | 2.0  3.0  3.0 |
 */

float a[6] = {3.0, 1.0, 2.0, 6.0, 3.0, 3.0};

cblas_sspr(CblasColMajor, CblasLower, n, alpha, x, incx, a);

/**
 * Output a = |7.0, 5.0, 4.0, 10.0, 5.0, 4.0|
 */
```

2.1.2.16 ?syr2

对称矩阵秩 2 更新, $A = \alpha * x * y^T + \alpha * y * x^T + A$

其中, alpha 为乘法系数, x 和 y 为包含 n 个元素的向量, A 为 n*n 的对称矩阵。

2.1.2.16.1 接口定义

A. C interface:

```
void cblas_ssyr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const float alpha, const float *X, const BLASINT incX,
const float *Y, const BLASINT incY, float *A, const BLASINT lda);
```



```
void cblas_dsyr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const double alpha, const double *X, const BLASINT incX,
const double *Y, const BLASINT incY, double *A, const BLASINT lda);
```

B. Fortran interface:

```
CALL SSYR2(UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA)
```

```
CALL DSyr2(UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA)
```

2.1.2.16.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角。 如果 Uplo=CblasLower, 则使用 A 的下三角。	输入
N	整型数	向量 X 的元素个数。	输入
alpha	在 ssyr2 中, alpha 为单精度复数类型。 在 dsyr2 中, alpha 为双精度复数类型。	乘法系数。	输入
X	在 ssyr2 中, X 为单精度复数类型。 在 dsyr2 中, X 为双精度复数类型。	矩阵 X, 长度至少 $1+(n-1)*abs(incX)$ 。	输入
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
Y	在 ssyr2 中, Y 为单精度复数类型。 在 dsyr2 中, Y 为双精度复数类型。	矩阵 Y, 长度至少 $1+(n-1)*abs(incY)$ 。	输入
incY	整型数	向量 Y 的增长步长, 不能为零。	输入

参数名	类型	描述	输入/输出
A	在 ssyr2 中，A 为单精度复数类型。	矩阵 A(lda, n)。	输入/输出
	在 dsyr2 中，A 为双精度复数类型。		
lda	整型数	矩阵 A 的主维度长度，要求 lda 大于或等于 max(1, n)。	输入

2.1.2.16.3 依赖

```
#include "kblas.h"
```

2.1.2.16.4 示例

A. C interface:

```
int n = 3, lda = 4;
float alpha = 1.0;
int incx = 1, incy = 1;
float x[3] = {1.0, 3.0, 2.0};
float y[3] = {2.0, 3.0, 2.0};

/**
 *      | 3.0  1.0  1.0 |
 *  A   = | .   16.0 10.0 |
 *      | .   .   1.0 |
 *      | .   .   .   |
 */
float a[12] = {3.0, 0, 0, 0, 1.0, 16.0, 0, 0, 1.0, 10.0, 1.0, 0};

cblas_ssy2(CblasColMajor,CblasUpper, n, alpha, x, incx, y, incy, a,
lda);

/**
 *      | 7.0  10.  7.0 |
```

```

*           | .  34.  22.0|
* Output A = | .      .  9.0 |
*           | .      .      . |
*/

```

2.1.2.17 ?spr2

对称展开矩阵秩 1 更新, $A = \alpha * x * y^T + \alpha * y * x^T + A$

其中, alpha 为乘法系数, x 和 y 为包含 n 个元素的向量, A 为 n*n 的三角展开对称矩阵。

2.1.2.17.1 接口定义

A. C interface:

```
void cblas_sspr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const float alpha, const float *X, const BLASINT incX,
const float *Y, const BLASINT incY, float *A);
```

```
void cblas_dspr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const double alpha, const double *X, const BLASINT incX,
const double *Y, const BLASINT incY, double *A);
```

B. Fortran interface:

```
CALL SSPR2(UPLO, N, ALPHA, X, INCX, Y, INCY, AP)
```

```
CALL DSPR2(UPLO, N, ALPHA, X, INCX, Y, INCY, AP)
```

2.1.2.17.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	对称矩阵展开存储方式 (上三角或下三角展开) 如果 Uplo=CblasUpper, 则使用 A 的上三角。 如果 Uplo=CblasLower, 则使用 A 的下三角。	输入
N	整型数	向量 X 的元素个数。	输入

参数名	类型	描述	输入/输出
alpha	在 sspr2 中，alpha 为单精度复数类型。	乘法系数。	输入
	在 dspr2 中，alpha 为双精度复数类型。		
X	在 sspr2 中，X 为单精度复数类型。	矩阵 X，长度至少 $1+(n-1)*abs(incX)$ 。	输入
	在 dspr2 中，X 为双精度复数类型。		
incX	整型数	表示 X 中向量增长步长，不能为零。	输入
Y	在 sspr2 中，Y 为单精度复数类型。	矩阵 Y，长度至少 $1+(n-1)*abs(incY)$ 。	输入
	在 dspr2 中，Y 为双精度复数类型。		
incY	整型数	向量 Y 的增长步长，不能为零。	输入
A	在 sspr2 中，A 为单精度复数类型。	矩阵 A 三角存储。	输入/输出
	在 dspr2 中，A 为双精度复数类型。		

2.1.2.17.3 依赖

```
#include "kblas.h"
```

2.1.2.17.4 示例

A. C interface:

```
float alpha = 1.0;

int n = 3, incx = 1, incy = 1;

float x[3] = {2.0, 2.0, 1.0};

float y[3] = {1.0, 2.0, 1.0};

/**
```

```

*           | 3.0  1.0  2.0 |
*      A = | 1.0  6.0  3.0 |
*           | 2.0  3.0  3.0 |
*/

float a[6] = {3.0, 1.0, 2.0, 6.0, 3.0, 3.0};

cblas_sspr2(CblasColMajor,CblasLower, n, alpha, x, incx, y, incy,
a);

/**
 *  Output a = |7.0, 7.0, 5.0, 14.0, 7.0, 5.0|
 */
```

2.1.2.18 ?hpr

三角展开的埃尔米特矩阵秩 1 更新， $A = \alpha * x * conj(x^T) + \alpha * x + A$
其中，alpha 是缩放系数，x 是含有 n 个元素的向量，A 是三角展开的埃尔米特矩阵。

2.1.2.18.1 接口定义

A. C interface:

```
void cblas_chpr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const float alpha, const void *X, const BLASINT incX,
void *A);
void cblas_zhpr(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const double alpha, const void *X,const BLASINT incX,
void *A);
```

B. Fortran interface:

```
CALL CHPR(UPLO, N, ALPHA, X, INCX, AP)
CALL ZHPR(UPLO, N, ALPHA, X, INCX, AP)
```

2.1.2.18.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角。 如果 Uplo=CblasLower, 则使用 A 的下三角。	输入
N	整型数	向量 X 的元素个数。	输入
alpha	在 chpr 中, alpha 为单精度复数类型。	乘法系数。	输入
	在 zhpr 中, alpha 为双精度复数类型。		
X	在 chpr 中, X 为单精度复数类型。	矩阵 X, 长度至少 $1+(n-1)*abs(incX)$ 。	输入
	在 zhpr 中, X 为双精度复数类型。		
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
A	在 chpr 中, A 为单精度复数类型。	矩阵 A, 矩阵规模至少是 $(n*(n+1)/2)$ 。	输入/输出
	在 zhpr 中, A 为双精度复数类型。		

2.1.2.18.3 依赖

```
#include "kblas.h"
```

2.1.2.18.4 示例

A. C interface:

```
%
```

```
int n = 3;
```

```
float alpha = 1.0;
```

```

int incx = 1;

/**
 *          | (2.0, 0.0) (13.0, 5.0) (12.0, -13.0) |
 *      A = | (13.0, -5.0) (3.0, 0.0) (4.0, -18.0) |
 *          | (12.0, 13.0) (4.0, 18.0) (2.0, 0.0) |
 */

float ap[12] = {2.0, 0, 13.0, -5.0, 12.0, 13.0, 3.0, 0, 4.0, 18.0,
2.0, 0};

float x[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};

cblas_chpr(CblasColMajor, CblasLower, n, alpha, x, incx, ap);

/**
 * Output ap = |(10.0, 0.0), (39.0, 25.0), (10.0, 27.0), (200.0,
0.0), (50.0, 71.0), (27.0, 0.0) |
 */

```

2.1.2.19 ?her

埃尔米特矩阵秩 1 更新, $A = \alpha * x * conj(x^T) + A$

其中, alpha 是缩放系数, x 是含有 n 个元素的向量, A 是埃尔米特矩阵。

2.1.2.19.1 接口定义

A. C interface:

```
void cblas_cher(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const float alpha, const void *X, const BLASINT incX,
void *A, const BLASINT lda);
```

```
void cblas_zher(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const double alpha, const void *X, const BLASINT incX,
void *A, const BLASINT lda);
```

B. Fortran interface:

```
CALL CHER(UPLO, N, ALPHA, X, INCX, A, LDA)
```

```
CALL ZHER(UPLO, N, ALPHA, X, INCX, A, LDA)
```

2.1.2.19.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角。 如果 Uplo=CblasLower, 则使用 A 的下三角。	输入
N	整型数	向量 X 的元素个数。	输入
alpha	在 cher 中, alpha 为单精度浮点类型。	乘法系数。	输入
	在 zher 中, alpha 为双精度浮点类型。		
X	在 cher 中, X 为单精度复数类型。	矩阵 X, 长度至少 $1+(n-1)*abs(incX)$ 。	输入
	在 zher 中, X 为双精度复数类型。		
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
A	在 cher 中, A 为单精度复数类型。	矩阵 A(lda, n)。	输入/输出
	在 zher 中, A 为双精度复数类型。		
lda	整型数	矩阵 A 的主维度长度, 要求 lda 大于或等于 $\max(1, n)$ 。	输入

2.1.2.19.3 依赖

```
#include "kblas.h"
```


2.1.2.19.4 示例

A. C interface:

```
%
int n = 3;
float alpha = 2.9;
int incx = 1;
/**
 *          | (1.0, 0.0) (13.0, 5.0) (12.0, 0) |
 *      A = | (13.0, -5.0) (2.0, 0.0) (4.0, -18.0) |
 *          | (12.0, 0) (4.0, 18.0) (3.0, 0.0) |
 */
float ap[18] = {1.0, 0, 13.0, -5.0, 12.0, 0, 13.0, 5.0, 2.0, 0, 4.0,
18.0, 12.0, 0, 4.0, -18.0, 3.0, 0};
float x[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};

cblas_cher(CblasColMajor, CblasLower, n, alpha, x, incx, ap, 3);
/**
 *          | (24.200001, 0.000000) (13.000000  5.000000)
(12.000000  0.000000) |
 * Output A = | (88.400002, 82.000000) (573.300049  0.000000)
(4.000000  -18.000000) |
 *          | (6.200001, 40.600002) (137.400009 171.700012)
(75.500000  0.000000) |
 */
```

2.1.2.20 ?hpr2

埃尔米特展开矩阵秩 2 更新, $A = \alpha * x * conj(x^T) + \alpha * x + A$

其中, alpha 是缩放系数, x,y 是含有 n 个元素的向量, A 是 n*n 的埃尔米特矩阵。

2.1.2.20.1 接口定义

A. C interface:

```
void cblas_chpr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const void *alpha, const void *X, const BLASINT incX,
const void *Y, const BLASINT incY, void *Ap);
```

```
void cblas_zhpr2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const void *alpha, const void *X, const BLASINT incX,
const void *Y, const BLASINT incY, void *Ap);
```

B. Fortran interface:

```
CALL CHPR2(UPLO, N, ALPHA, X, INCX, Y, INCY, AP)
```

```
CALL ZHPR2(UPLO, N, ALPHA, X, INCX, Y, INCY, AP)
```

2.1.2.20.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	埃尔米特矩阵展开存储方式（上三角或下三角展开） 如果 Uplo=CblasUpper, 则使用 A 的上三角。 如果 Uplo=CblasLower, 则使用 A 的下三角。	输入
N	整型数	向量 X 的元素个数。	输入
alpha	在 chpr2 中, alpha 为单精度复数类型。	乘法系数。	输入
	在 zhpr2 中, alpha 为双精度复数类型。		
X	在 chpr2 中, X 为单精度复数类型。	矩阵 X, 长度至少 $1+(n-1)*abs(incX)$ 。	输入
	在 zhpr2 中, X 为双精度复数类型。		

参数名	类型	描述	输入/输出
incX	整型数	表示 X 中向量增长步长，不能为零。	输入
Y	在 cher2 中，Y 为单精度复数类型。	矩阵 Y，长度至少 $1+(n-1)*abs(incY)$ 。	输入
	在 zher2 中，Y 为双精度复数类型。		
incY	整型数	向量 Y 的增长步长，不能为零。	输入
A	在 chpr 中，A 为单精度复数类型。	矩阵 A，矩阵规模至少是 $(n*(n+1)/2)$ 。	输入/输出
	在 zhpr 中，A 为双精度复数类型。		

2.1.2.20.3 依赖

```
#include "kblas.h"
```

2.1.2.20.4 示例

A. C interface:

```
%
int n = 3;
float alpha[2] = {1.0, 2.9};
int incx = 1, incy = 1;
/**
 *          | (2.0, 0.0) (13.0, 5.0) (12.0, -13.0) |
 *      A = | (13.0, -5.0) (2.0, 0.0) (4.0, -18.0) |
 *          | (12.0, 13.0) (4.0, 18.0) (1.0, 0.0) |
 */
float ap[12] = {2.0, 0, 13.0, -5.0, 12.0, 13.0, 2.0, 0, 4.0, 18.0,
1.0, 0};

float x[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};
float y[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};
```

```

cblas_chpr2(CblasColMajor, CblasLower, n, alpha, x, incx, y, incy,
ap);

/**
 * Output ap = |(18.0, 0.0), (65.0, 55.0), (8.0, 41.0), (396.0,
0.0), (96.0, 124.0), (51.0, 0.0)|
 */

```

2.1.2.21 ?her2

埃尔米特矩阵秩 2 更新, $A = \alpha * x * conj(y^T) + conj(\alpha) * y * conj(x^T) + A$
 其中, alpha 是缩放系数, x,y 是含有 n 个元素的向量, A 是埃尔米特矩阵。

2.1.2.21.1 接口定义

A. C interface:

```

void cblas_cher2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const void *alpha, const void *X, const BLASINT incX,
const void *Y, const BLASINT incY, void *A, const BLASINT lda);

```

```

void cblas_zher2(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const void *alpha, const void *X, const BLASINT incX,
const void *Y, const BLASINT incY, void *A, const BLASINT lda);

```

B. Fortran interface:

```
CALL CHER2(UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA)
```

```
CALL ZHER2(UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA)
```

2.1.2.21.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

参数名	类型	描述	输入/输出
Uplo	枚举类型 CBLAS_UPLO	埃尔米特矩阵存储方式 (上三角或下三角)。 如果 Uplo=CblasUpper, 则 使用 A 的上三角。 如果 Uplo=CblasLower, 则 使用 A 的下三角。	输入
N	整型数	向量 X 的元素个数。	输入
alpha	在 cher2 中, alpha 为单 精度复数类型。	乘法系数。	输入
	在 zher2 中, alpha 为双 精度复数类型。		
X	在 cher2 中, X 为单精 度复数类型。	矩阵 X, 长度至少 $1+(n-1)*abs(incX)$ 。	输入
	在 zher2 中, X 为双精 度复数类型。		
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
Y	在 cher2 中, Y 为单精 度复数类型。	矩阵 Y, 长度至少 $1+(n-1)*abs(incY)$ 。	输入
	在 zher2 中, Y 为双精 度复数类型。		
incY	整型数	向量 Y 的增长步长, 不能 为零。	输入
A	在 cher2 中, A 为单精 度复数类型。	矩阵 A(lda, n)。	输出
	在 zher2 中, A 为双精 度复数类型。		
lda	整型数	矩阵 A 的主维度长度, 要 求 lda 大于或等于 $\max(1, n)$ 。	输入

2.1.2.21.3 依赖

```
#include "kblas.h"
```

2.1.2.21.4 示例

A. C interface:

```
int n = 3;

float alpha[2] = {1.0, 2.9};

int incx = 1, incy = 1;

/**
 *          | (1.0, 0.0) (13.0, 5.0) (12.0, 0) |
 *      A = | (13.0, -5.0) (2.0, 0.0) (4.0, -18.0) |
 *          | (12.0, 0) (4.0, 18.0) (3.0, 0.0) |
 */

float ap[18] = {1.0, 0, 13.0, -5.0, 12.0, 0, 13.0, 5.0, 2.0, 0, 4.0,
18.0, 12.0, 0, 4.0, -18.0, 3.0, 0};

float x[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};
float y[6] = {2.0, -2.0, 14.0, 1, 3.0, 4.0};

cblas_cher2(CblasColMajor, CblasLower, n, alpha, x, incx, y, incy,
ap, 3);

/**
 *          | 17.000000 0.000000      13.000000 5.000000
12.000000 0.000000 |
 *      Output A = | 65.000000 55.000000      396.000000 0.000000
4.000000 -18.000000 |
 *          | 8.000000 28.000000      96.000000 124.000000
53.000000 0.000000 |
 */
```

2.1.2.22 ?hpmv

向量与三角展开的埃尔米特矩阵乘积, $y = \alpha * A * x + \beta * y$

其中, alpha 和 beta 是缩放系数, x 和 y 是含有 n 个元素的向量, A 是三角展开的埃尔米特矩阵。

2.1.2.22.1 接口定义

A. C interface:

```
void cblas_chpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const void *alpha, const void *Ap, const void *X, const
BLASINT incX, const void *beta, void *Y, const BLASINT incY);
```

```
void cblas_zhpmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const void *alpha, const void *Ap, const void *X, const
BLASINT incX, const void *beta, void *Y, const BLASINT incY);
```

B. Fortran interface:

```
CALL CHPMV(UPLO, N, ALPHA, AP, X, INCX, BETA, Y, INCY)
```

```
CALL ZHPMV(UPLO, N, ALPHA, AP, X, INCX, BETA, Y, INCY)
```

2.1.2.22.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示矩阵 A 的存储展开方式（上三角或下三角）。 如果 Uplo=CblasUpper，则使用 A 的上三角。 如果 Uplo=CblasLower，则使用 A 的下三角。	输入
N	整型数	表示矩阵 A 的阶数，要求 N 大于或等于零。	输入
alpha	在 chpmv 中，alpha 是单精度复数类型。	乘法系数。	输入
	在 zhpmv 中，alpha 是双精度复数类型。		
Ap	在 chpmv 中，Ap 是单精度复数类型。	压缩的埃尔米特矩阵，矩阵规模至少是 $(n*(n+1)/2)$ 。	输入
	在 zhpmv 中，Ap 是双精度复数类型。		

参数名	类型	描述	输入/输出
X	在 chpmv 中, X 是单精度复数类型。	矩阵 X, 长度至少 $1+(n-1)*abs(incX)$ 。	输入
	在 zhpmv 中, X 是双精度复数类型。		
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
Y	在 chpmv 中, Y 是单精度复数类型。	矩阵 Y, 长度至少 $1+(n-1)*abs(incY)$ 。	输入/输出
	在 zhpmv 中, Y 是双精度复数类型。		
incY	整型数	向量 Y 的增长步长, 不能为零。	输入
beta	在 chpmv 中, beta 是单精度复数类型。	乘法系数。	输入
	在 zhpmv 中, beta 是双精度复数类型。		

2.1.2.22.3 依赖

```
#include "kblas.h"
```

2.1.2.22.4 示例

A. C interface:

```
int n = 3;

float alpha[2] = {1.0, 0}, beta[2] = {1.0, 0};

int incx = 1, incy = 1;

/*          | (1.0, 0) (3.0, 3.0) (3.0, -3.0) |
*      A =  | (3.0, -3.0) (2.0, 0.0) (3.0, -1.0) |
*          | (3.0, 3.0) (3.0, 1.0) (3.0, 0.0) |
*/

float ap[12] = {1.0, 0, 3.0, -3.0, 3.0, 3.0, 2.0, 0, 3.0, 1.0, 3.0,
0};
```



```

float x[6] = {3.0, 2.0, 1.0, 2.0, -1.0, 1.0};
float y[6] = {1.0, 2.0, 3.0, 2.0, -1.0, 1.0};

cblas_chpmv(CblasColMajor,CblasLower, n, alpha, ap, x, incx, beta,
y, incy);

/*
 * Output y = |1.000000, 19.000000, 18.000000, 7.000000, 0.000000,
26.000000|
 */

```

2.1.2.23 ?hemv

向量与三角展开的埃尔米特矩阵乘积, $y = \alpha * A * x + \beta * y$
alpha 和 beta 是乘法系数, x 和 y 是含有 n 个元素的向量, A 是 n 阶埃尔米特矩阵。

2.1.2.23.1 接口定义

A. C interface:

```
void cblas_chemv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const void *alpha, const void *A, const BLASINT lda,
const void *X, const BLASINT incX, const void *beta, void *Y, const BLASINT
incY);
```

```
void cblas_zhemv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const void *alpha, const void *A, const BLASINT lda,
const void *X, const BLASINT incX, const void *beta, void *Y, const BLASINT
incY);
```

B. Fortran interface:

CALL CHEMV(UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

CALL ZHEMV(UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)

2.1.2.23.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入

参数名	类型	描述	输入/输出
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角。 如果 Uplo=CblasLower, 则使用 A 的下三角。	输入
N	整型数	表示矩阵 A 的阶数, 要求 N 大于或等于零。	输入
alpha	在 chemv 中, alpha 是单精度复数类型。	表示系数。	输入
	在 zhemv 中, alpha 是双精度复数类型。		
A	在 chemv 中, A 是单精度复数类型。	埃尔米特矩阵 A(lda, n)。	输入
	在 zhemv 中, A 是双精度复数类型。		
lda	整型数	矩阵 A 中主维度长度, 要求 lda 大于或等于(k+1)。	输入
X	在 chemv 中, X 是单精度复数类型。	矩阵 X, 长度至少 $1+(n-1)*abs(incX)$ 。	输入
	在 zhemv 中, X 是双精度复数类型。		
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
Y	在 chemv 中, Y 是单精度复数类型。	矩阵 Y, 长度至少 $1+(n-1)*abs(incY)$ 。	输入/输出
	在 zhemv 中, Y 是双精度复数类型。		
incY	整型数	向量 Y 的增长步长, 不能为零。	输入
beta	在 chemv 中, beta 是单精度复数类型。	乘法系数。	输入

参数名	类型	描述	输入/输出
	在 zhemv 中，beta 是双精度复数类型。		

2.1.2.23.3 依赖

```
#include "kblas.h"
```

2.1.2.23.4 示例

A. C interface:

```
int n = 3, lda = 3;
float alpha[2] = {1.0, 0}, beta[2] = {1.0, 0};
int incx = 1, incy = 1;

/*      | (21.0, * )      *      |
*      A = | (1.0, -1.0) (3.0, * )      *      |
*      | (1.0, 3.0) (2.0, 2.0) (2.0, * ) |
*/

float a[18] = {21.0, 0, 1.0, -1.0, 1.0, 3.0, 0, 0, 3.0, 0, 2.0, 2.0,
0, 0, 0, 0, 2.0, 0};

float x[6] = {3.0, 2.0, 1.0, 2.0, -1.0, 1.0};
float y[6] = {1.0, 2.0, 3.0, 2.0, -1.0, 1.0};

cblas_chemv(CblasColMajor,CblasLower, n, alpha, a, lda, x, incx,
beta, y, incy);

/*
* Output y = | 65.0, 51.0, 11.0, 11.0, -8.0, 20.0|
*/
```

2.1.2.24 ?hbm

向量与埃尔米特带状矩阵乘积， $y = \alpha * A * x + \beta * y$
alpha 和 beta 是乘法系数，x 和 y 是含有 n 个元素的向量，A 是 n 阶埃尔米特带状矩阵。

2.1.2.24.1 接口定义

A. C interface:

```
void cblas_chbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const
BLASINT lda, const void *X, const BLASINT incX, const void *beta, void *Y, const
BLASINT incY);
```

```
void cblas_zhbmv(const enum CBLAS_ORDER order, const enum CBLAS_UPLO
Uplo, const BLASINT N, const BLASINT K, const void *alpha, const void *A, const
BLASINT lda, const void *X, const BLASINT incX, const void *beta, void *Y, const
BLASINT incY);
```

B. Fortran interface:

```
CALL CHBMV(UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
```

```
CALL ZHBMV(UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
```

2.1.2.24.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 展开方式（上三角或下三角）。 如果 Uplo=CblasUpper, 则使用 A 的上三角。 如果 Uplo=CblasLower, 则使用 A 的下三角。	输入
N	整型数	表示矩阵 A 的阶数, 要求 N 大于或等于零。	输入
K	整型数	表示矩阵 A 的超对角阶数, 要求 K 大于或等于零。	输入
alpha	在 chbmv 中, alpha 是单精度复数类型。 在 zhbmv 中, alpha 是双精度复数类型。	表示系数。	输入

参数名	类型	描述	输入/输出
A	在 chbmV 中, A 是单精度复数类型。	埃尔米特带状矩阵 A(lda, n)。	输入
	在 zhbmV 中, A 是双精度复数类型。		
lda	整型数	矩阵 A 中主维度长度, 要求 lda 大于或等于(k+1)。	输入
X	在 chbmV 中, X 是单精度复数类型。	矩阵 X, 长度至少 1+(n-1)*abs(incX)。	输入
	在 zhbmV 中, X 是双精度复数类型。		
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
Y	在 chbmV 中, Y 是单精度复数类型。	矩阵 Y, 长度至少 1+(n-1)*abs(incY)。	输入/输出
	在 zhbmV 中, Y 是双精度复数类型。		
incY	整型数	向量 Y 的增长步长, 不能为零。	输入
beta	在 chbmV 中, beta 是单精度复数类型。	乘法系数。	输入
	在 zhbmV 中, beta 是双精度复数类型。		

2.1.2.24.3 依赖

```
#include "kblas.h"
```

2.1.2.24.4 示例

A. C interface:

```
int n = 3, k = 1, lda = 3;
float alpha[2] = {1.0, 0}, beta[2] = {1.0, 0};
int incx = 1, incy = 1;
```

```

/*
 *          | 1.0, 0.0  7.0, 0.0, 6.0, 0 |
 *      ap = | 3.0,-5.0  4.0, 8.0      * |
 *          |      *      *      * |
 */

float ap[18] = {1.0, 0, 3.0, -5.0, 0, 0, 7.0, 0, 4.0, 8.0, 0.0, 0.0,
6.0, 0, 0, 0, 0, 0};

float x[6] = {3.0, 0.0, 2.0, 0.0, 1.0, 0.0};
float y[6] = {1.0, 0.0, 2.0, 0.0, 3.0, 0.0};

cblas_chbmv(CblasColMajor,CblasLower, n, k, alpha, ap, lda, x, incx,
beta, y, incy);

/*
 * Output y = | 10.0, 10.0, 29.0, -23.0, 17.0, 16.0|
 */

```

2.1.2.25 ?gerc

一般矩阵秩 1 更新, $A = A + \alpha * x * y^T$

alpha 是乘法系数, x 是含有 m 个元素的向量, y 是含有 n 个元素的向量, A 是 m*n 的一般矩阵。

2.1.2.25.1 接口定义

A. C interface:

```
void cblas_cgerc(const enum CBLAS_ORDER order, const BLASINT M, const
BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y,
const BLASINT incY, void *A, const BLASINT lda);
```

```
void cblas_zgerc(const enum CBLAS_ORDER order, const BLASINT M, const
BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y,
const BLASINT incY, void *A, const BLASINT lda);
```

B. Fortran interface:

```
CALL CGERC(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)
```

CALL ZGERC(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)

2.1.2.25.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
M	整型数	矩阵 A 的行数。	输入
N	整型数	矩阵 A 的列数。	输入
alpha	在 cgerc 中, alpha 为单精度复数类型。 在 zgerc 中, alpha 为双精度复数类型。	乘法系数。	输入
A	在 cgerc 中, A 为单精度复数类型。 在 zgerc 中, A 为双精度复数类型。	矩阵 A(lda, n)。	输出
lda	整型数	矩阵 A 中主维度长度, 如果矩阵 A 为列存, 要求 lda 大于或等于 $\max(1, m)$, 否则要求大于或等于 $\max(1, n)$ 。	输入
X	在 cgerc 中, X 为单精度复数类型。 在 zgerc 中, X 为双精度复数类型。	矩阵 X, 长度至少 $1+(n-1)*\text{abs}(\text{incX})$ 。	输入
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入
Y	在 cgerc 中, Y 为单精度复数类型。 在 zgerc 中, Y 为双精度复数类型。	矩阵 Y, 长度至少 $1+(n-1)*\text{abs}(\text{incY})$ 。	输入
incY	整型数	向量 Y 的增长步长, 不能为零。	输入

2.1.2.25.3 依赖

```
#include "kblas.h"
```

2.1.2.25.4 示例

A. C interface:

```
int m = 2, n = 2, lda = 2;

float alpha[2] = {1.0, 2.0};

int incx = 1, incy = 1;

float x[4] = {1.0, 2.0, 3.0, 4.0};
float y[4] = {2.0, 3.0, 4.0, 2.0};
float a[8] = {-1.0, 2.0, 2.0, 2.0, 3.0, 4.0, 5.0, 6.0};

cblas_cgerc(CblasColMajor, m, n, alpha, x, incx, y, incy, a, lda);

/**
 *   Output A:
 *
 *           (5.000000, 19.000000), (-1.000000, 26.000000)
 *           (22.000000, 37.000000), (5.000000, 56.000000)
 */
```

2.1.2.26?geru

一般共轭矩阵秩 1 更新, $A = A + \alpha * x * conj(y^T)$

alpha 是乘法系数, x 是含有 m 个元素的向量, y 是含有 n 个元素的向量, A 是 m*n 的一般矩阵。

2.1.2.26.1 接口定义

A. C interface:

```
void cblas_cgeru(const enum CBLAS_ORDER order, const BLASINT M, const
BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y,
const BLASINT incY, void *A, const BLASINT lda);
```



```
void cblas_zgeru(const enum CBLAS_ORDER order, const BLASINT M, const
BLASINT N, const void *alpha, const void *X, const BLASINT incX, const void *Y,
const BLASINT incY, void *A, const BLASINT lda);
```

B. Fortran interface:

```
CALL CGERU(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)
```

```
CALL ZGERU(M, N, ALPHA, X, INCX, Y, INCY, A, LDA)
```

2.1.2.26.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
M	整型数	矩阵 A 的行数。	输入
N	整型数	矩阵 A 的列数。	输入
alpha	在 cgeru 中, alpha 为单精度复数类型。	乘法系数。	输入
	在 zgeru 中, alpha 为双精度复数类型。		
A	在 cgeru 中, A 为单精度复数类型。	矩阵 A(lda, n)。	输出
	在 zgeru 中, A 为双精度复数类型。		
lda	整型数	矩阵 A 中主维度长度, 如果矩阵 A 为列存, 要求 lda 大于或等于 $\max(1, m)$, 否则要求大于或等于 $\max(1, n)$ 。	输入
X	在 cgeru 中, X 为单精度复数类型。	矩阵 X, 长度至少 $1+(n-1)*\text{abs}(\text{incX})$ 。	输入
	在 zgeru 中, X 为双精度复数类型。		
incX	整型数	表示 X 中向量增长步长, 不能为零。	输入

参数名	类型	描述	输入/输出
Y	在 cgeru 中，Y 为单精度复数类型。	矩阵 Y，长度至少 $1+(n-1)*abs(incY)$ 。	输入
	在 zgeru 中，Y 为双精度复数类型。		
incY	整型数	向量 Y 的增长步长，不能为零。	输入

2.1.2.26.3 依赖

```
#include "kblas.h"
```

2.1.2.26.4 示例

A. C interface:

```
int m = 4, n = 4, lda = 4;
float alpha[2] = {1.0, 2.0};
int incx = 1, incy = 1;
/**
 * X:
 *    0.340188, -0.105617, 0.283099, 0.298440, 0.411647, -0.302449,
-0.164777, 0.268230
 * Y:
 *    -0.222225, 0.053970, -0.022603, 0.128871, -0.135216, 0.013401,
0.452230, 0.416195
 * A:
 *    0.135712, 0.217297, -0.343321, -0.099056, 0.112640, -
0.203968, 0.026745,    0.269914
 *    -0.358397, 0.106969, -0.370210, -0.391191, 0.137552, 0.024287,
-0.099771, 0.391529
 *    -0.483699, -0.257113, 0.498924, -0.281743, -0.006417,
0.472775, -0.216685, -0.147542
 *    -0.362768, 0.304177, 0.012932, 0.339112, -0.207483, 0.271358,
0.307725, 0.419026
```

```

    */

    float x[8] = {0.340188, -0.105617, 0.283099, 0.298440, 0.411647, -
0.302449, -0.164777, 0.268230};

    float y[8] = {-0.222225, 0.053970, -0.022603, 0.128871, -0.135216,
0.013401, 0.452230, 0.416195};

    float a[32] = {0.135712, 0.217297, -0.343321, -0.099056, 0.112640,
-0.203968, 0.026745, 0.269914,
                    -0.358397, 0.106969, -0.370210, -0.391191, 0.137552,
0.024287, -0.099771, 0.391529,
                    -0.483699, -0.257113, 0.498924, -0.281743, -
0.006417, 0.472775, -0.216685, -0.147542,
                    -0.362768, 0.304177, 0.012932, 0.339112, -0.207483,
0.271358, 0.307725, 0.419026};

    cblas_cgeru(CblasColMajor, m, n, alpha, x, incx, y, incy, a, 4);
/**
 * Output A:
 *
 *   -0.017847  0.119331  -0.444931  0.165040  -0.565962  -0.327440  -
0.352610  0.793599
 *
 *   -0.320256  -0.308135  -0.474544  -0.451172  0.529765  -0.402861  -
0.488827  0.599533
 *
 *   -0.141372  -0.264850  0.047453  0.143518   -0.150850  0.415971
0.035456  0.929981
 *
 *    0.185887  0.245696  -0.076018  0.302546  -0.121045  -0.148647
0.016127  0.099442
 */

```

2.1.3 Level 3

2.1.3.1 函数说明

函数族	简介
-----	----

函数族	简介
?gemm	$C \leftarrow \alpha * op(A) * op(B) + \beta * C,$ and $op(X) = X \text{ or } X^T$ A/B/C 为常规矩阵; 具体分为以下四个函数: cblas_sgemm: 单精度浮点型 cblas_dgemm: 双精度浮点型 cblas_cgemm: 单精度复数型 cblas_zgemm: 双精度复数型
?hemm	$C \leftarrow \alpha * A * B + \beta * C$ or $C \leftarrow \alpha * B * A + \beta * C$ A 为埃尔米特矩阵, B/C 为常规矩阵; 具体分为以下两个函数: cblas_chemm: 单精度复数型 cblas_zhemm: 双精度复数型
?symm	$C \leftarrow \alpha * A * B + \beta * C$ or $C \leftarrow \alpha * B * A + \beta * C$ A 为对称矩阵, B/C 为常规矩阵; 具体分为以下四个函数: cblas_ssymb: 单精度浮点型 cblas_dsymb: 双精度浮点型 cblas_csymb: 单精度复数型 cblas_zsymb: 双精度复数型
?syrk	$C \leftarrow \alpha * A * A^T + \beta * C$ or $C \leftarrow \alpha * A^T * A + \beta * C$ A 为对称矩阵, C 为常规矩阵; 具体分为以下四个函数: cblas_ssyrk: 单精度浮点型 cblas_dsyrk: 双精度浮点型 cblas_csyrk: 单精度复数型 cblas_zsyrk: 双精度复数型
?syr2k	$C \leftarrow \alpha AB^T + \alpha BA^T + \beta C$ or $C \leftarrow \alpha A^T B + \alpha B^T A + \beta C$ C 为对称矩阵, A/B 为常规矩阵; 具体分为以下四个函数: cblas_ssyr2k: 单精度浮点型 cblas_dsyr2k: 双精度浮点型 cblas_csyr2k: 单精度复数型 cblas_zsyr2k: 双精度复数型

函数族	简介
?trmm	$B \leftarrow \alpha * op(A) * B$ $\text{or } B \leftarrow \alpha * B * op(A)$ $\text{and } op(A) = A \text{ or } A^T$ A 为三角矩阵, B 为常规矩阵; 具体分为以下四个函数: cblas_strmm: 单精度浮点型 cblas_dtrmm: 双精度浮点型 cblas_ctrmm: 单精度复数型 cblas_ztrmm: 双精度复数型
?trsm	$B \leftarrow \alpha * op(A^{-1}) * B$ $\text{or } B \leftarrow \alpha * B * op(A^{-1}),$ $\text{and } op(A) = A \text{ or } A^T$ A 为三角矩阵, B 为常规矩阵; 具体分为以下四个函数: cblas_strsm: 单精度浮点型 cblas_dtrsm: 双精度浮点型 cblas_ctrsm: 单精度复数型 cblas_ztrsm: 双精度复数型
?herk	$C \leftarrow \alpha * A * A^H + \beta * C$ $\text{or } C \leftarrow \alpha A^H A + \beta C$ C 为埃尔米特矩阵; 一种情况矩阵 A 为 n*k, 第二种情况矩阵 A 为 k*n; 具体分为以下两个函数: cblas_cherk: 单精度复数型 cblas_zherk: 双精度复数型
?her2k	$C \leftarrow \alpha * A * B^H + conjg(\alpha) * B * A^H + \beta * C$ $\text{or } C \leftarrow \alpha A^H B + conjg(\alpha) * B^H * A + \beta * C$ C 为埃尔米特矩阵; 第一种情况矩阵 A, B 为 n*k, 第二种情况矩阵 A, B 为 k*n; 具体分为以下两个函数: cblas_cher2k: 单精度复数型 cblas_zher2k: 双精度复数型

2.1.3.2 ?gemm

矩阵乘以矩阵。 $C = \alpha * op(A) * op(B) + \beta * C$

op(X)可取值: $op(X) = X, op(X) = X^T, op(X) = conjg(X^T),$

alpha 和 beta 是乘法系数, op(A)为 m*k 矩阵, op(B)为 k*n 矩阵, C 为 m*n 矩阵。

2.1.3.2.1 接口定义

A. C interface:

```
void cblas_sgemm(const enum CBLAS_ORDER Order, const enum
CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const
BLASINT M, const BLASINT N, const BLASINT K, const float alpha, const float
*A, const BLASINT lda, const float *B, const BLASINT ldb, const float beta, float
*C, const BLASINT ldc);
```

```
void cblas_dgemm(const enum CBLAS_ORDER Order, const enum
CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const
BLASINT M, const BLASINT N, const BLASINT K, const double alpha, const
double *A, const BLASINT lda, const double *B, const BLASINT ldb, const double
beta, double *C, const BLASINT ldc);
```

```
void cblas_cgemm(const enum CBLAS_ORDER Order, const enum
CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const
BLASINT M, const BLASINT N, const BLASINT K, const void *alpha, const void
*A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void
*C, const BLASINT ldc);
```

```
void cblas_zgemm(const enum CBLAS_ORDER Order, const enum
CBLAS_TRANSPOSE TransA, const enum CBLAS_TRANSPOSE TransB, const
BLASINT M, const BLASINT N, const BLASINT K, const void *alpha, const void
*A, const BLASINT lda, const void *B, const BLASINT ldb, const void *beta, void
*C, const BLASINT ldc);
```

B. Fortran interface:

```
CALL SGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C,
LDC)
```

```
CALL DGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA,
C, LDC)
```

```
CALL CGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C,
LDC)
```

```
CALL ZGEMM(TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C,
LDC)
```

2.1.3.2.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
transA	枚举类型 CBLAS_TRANSPOSE	矩阵 A 为常规矩阵，转置矩阵或共轭矩阵。 如果 TransA = CblasNoTrans, $op(A) = A$ 。 如果 TransA = CblasTrans, $op(A) = A^T$ 。 如果 TransA = CblasConjTrans, $op(A) = conjg(A^T)$ 。	输入
transB	枚举类型 CBLAS_TRANSPOSE	矩阵 B 为常规矩阵，转置矩阵或共轭矩阵。 如果 TransB = CblasNoTrans, $op(B) = B$ 。 如果 TransB = CblasTrans, $op(B) = B^T$ 。 如果 TransB = CblasConjTrans, $op(B) = conjg(B^T)$ 。	输入
M	整型数	矩阵 $op(A)$ 和矩阵 C 的行。	输入
N	整型数	矩阵 $op(B)$ 和矩阵 C 的列。	输入
K	整型数	矩阵 $op(A)$ 的列和矩阵 $op(B)$ 的行。	输入
alpha	在 sgemm 中, alpha 为单精度浮点类型。	表示乘法系数。	输入

参数名	类型	描述	输入/输出
	在 <code>dgemm</code> 中, <code>alpha</code> 为双精度浮点类型。		
	在 <code>cgemm</code> 中, <code>alpha</code> 为单精度复数类型。		
	在 <code>zgemm</code> 中, <code>alpha</code> 为双精度复数类型。		
A	在 <code>sgemm</code> 中, A 为单精度浮点类型。	矩阵 A	输入
	在 <code>dgemm</code> 中, A 为双精度浮点类型。		
	在 <code>cgemm</code> 中, A 为单精度复数类型。		
	在 <code>zgemm</code> 中, A 为双精度复数类型。		
lda	整型数	矩阵为列存, <code>TransA = CblasNoTrans</code> , lda 至少 $\max(1, m)$, 否则 $\max(1, k)$ 。	输入
		矩阵为行存, <code>TransA = CblasNoTrans</code> , lda 至少 $\max(1, k)$, 否则 $\max(1, m)$ 。	
B	在 <code>sgemm</code> 中, B 为单精度浮点类型。	矩阵 B	输入
	在 <code>dgemm</code> 中, B 为双精度浮点类型。		
	在 <code>cgemm</code> 中, B 为单精度复数类型。		
	在 <code>zgemm</code> 中, B 为双精度复数类型。		
ldb	整型数	矩阵为列存, <code>TransB = CblasNoTrans</code> , ldb 至少 $\max(1, k)$, 否则 $\max(1, n)$ 。	输入

参数名	类型	描述	输入/输出
		矩阵为行存，TransB = CblasNoTrans，ldb 至少 $\max(1, n)$ ，否则 $\max(1, k)$ 。	
beta	在 sgemm 中，beta 为单精度浮点类型。	乘法系数。	输入
	在 dgemm 中，beta 为双精度浮点类型。		
	在 cgemm 中，beta 为单精度复数类型。		
	在 zgemm 中，beta 为双精度复数类型。		
C	在 sgemm 中，C 为单精度浮点类型。	矩阵 C。	输入/输出
	在 dgemm 中，C 为双精度浮点类型。		
	在 cgemm 中，C 为单精度复数类型。		
	在 zgemm 中，C 为双精度复数类型。		
ldc	整型数	矩阵为列存，ldc 至少 $\max(1, m)$ ，否则 $\max(1, n)$ 。	输入

2.1.3.2.3 依赖

#include "kblas.h"

2.1.3.2.4 示例

A. C interface:

```
int m = 4, k = 3, n = 4, lda = 4, ldb = 3, ldc = 4;

float alpha = 1.0, beta = 2.0;

/*
```

```

* A:
*      0.340188,      0.411647,      -0.222225,
*      -0.105617,     -0.302449,      0.053970,
*      0.283099,      -0.164777,     -0.022603,
*      0.298440,      0.268230,      0.128871,
* B:
*      -0.135216,      0.416195,      -0.358397,     -0.257113,
*      0.013401,      0.135712,      0.106969,     -0.362768,
*      0.452230,      0.217297,      -0.483699,      0.304177,
* C:
*      -0.343321,      0.498924,      0.112640,     -0.006417,
*      -0.099056,     -0.281743,     -0.203968,      0.472775,
*      -0.370210,      0.012932,      0.137552,     -0.207483,
*      -0.391191,      0.339112,      0.024287,      0.271358,
*/

float a[12] = {0.340188, -0.105617, 0.283099,
               0.298440, 0.411647, -0.302449,
               -0.164777, 0.268230, -0.222225,
               0.053970, -0.022603, 0.128871};

float b[12] = {-0.135216, 0.013401, 0.452230, 0.416195,
               0.135712, 0.217297, -0.358397, 0.106969,
               -0.483699, -0.257113, -0.362768, 0.304177};

float c[16] = {-0.343321, -0.099056, -0.370210, -0.391191,
               0.498924, -0.281743, 0.012932, 0.339112,
               0.112640, -0.203968, 0.137552, 0.024287,
               -0.006417, 0.472775, -0.207483, 0.271358};

cblas_sgemm(CblasColMajor,CblasNoTrans,CblasNoTrans, m, n, k, alpha,
a, lda, b, ldb, beta, c, ldc);

/*
* Output C:

```

```

*      -0.827621      1.147010      0.254881      -0.317229
*      -0.163476      -0.636762      -0.428542      1.098841
*      -0.791128      0.116416      0.166949      -0.434854
*      -0.760862      0.866839      -0.092028      0.407877
*
*/

```

2.1.3.3 ?hemm

计算矩阵 A 为埃尔米特矩阵乘法。 $C = \alpha * A * B + \beta * C$ or $C = \alpha * B * A + \beta * C$

alpha 和 beta 是乘法系数，A 为埃尔米特矩阵，B 和 C 为 m*n 一般矩阵。

2.1.3.3.1 接口定义

A. C interface:

```

void cblas_chemm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const
void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb,
const void *beta, void *C, const BLASINT ldc);

```

```

void cblas_zhemm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const
void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb,
const void *beta, void *C, const BLASINT ldc);

```

B. Fortran interface:

CALL CHEMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL ZHEMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

2.1.3.3.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Side	枚举类型 CBLAS_SIDE	埃尔米特矩阵 A 在矩阵 B 的左边还是右边。	输入

参数名	类型	描述	输入/输出
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角部分。 如果 Uplo=CblasLower, 则使用 A 的下三角部分。	输入
M	整型数	矩阵 C 的行。 A*B 为矩阵 A 的行。 B*A 为矩阵 B 的行。	输入
N	整型数	矩阵 C 的列。 A*B 为矩阵 B 的列。 B*A 为矩阵 A 的列。	输入
alpha	在 chemm 中, alpha 为单精度复数类型。 在 zhemm 中, alpha 为双精度复数类型。	表示乘法系数。	输入
A	在 chemm 中, A 为单精度复数类型。 在 zhemm 中, A 为双精度复数类型。	埃尔米特矩阵 A(lda, ka)。 如果 Side = CblasLeft, ka = m, 否则 ka = n。	输入
lda	整型数	如果 Side = CblasLeft, lda 至少 max(1, m), 否则至少 max(1, n)。	输入
B	在 chemm 中, B 为单精度复数类型。 在 zhemm 中, B 为双精度复数类型。	矩阵 B(ldb, n)。	输入

参数名	类型	描述	输入/输出
ldb	整型数	矩阵为列存，ldb 至少 $\max(1, m)$ 。	输入
		矩阵为行存，ldb 至少 $\max(1, n)$ 。	
beta	在 chemm 中，beta 为单精度复数类型。	乘法系数。	输入
	在 zhemm 中，beta 为双精度复数类型。		
C	在 chemm 中，C 为单精度复数类型。	矩阵 C(ldc, n)。	输入/输出
	在 zhemm 中，C 为双精度复数类型。		
ldc	整型数	矩阵为列存，ldc 至少 $\max(1, m)$ 。	输入
		矩阵为行存，ldc 至少 $\max(1, n)$ 。	

2.1.3.3.3 依赖

```
#include "kblas.h"
```

2.1.3.3.4 示例

A. C interface:

```
int m = 2, n = 3, lda = 4, ldb = 3, ldc = 5;

float alpha[2] = {2.0, 3.0}, beta[2] = {1.0, 6.0};

/**
 *
 *
 *          | (2.0, . )      .      .      |
 *      A   = | (13.0, -2.0) (2.0, . )      .      |
 *          | (-2.0, -6.0) (2.0, 14.0) (2.0, . ) |
 *          |      .      .      .      |
 *
 */
```

```

*          | (2.0, 2.0) (-13.0, 12.0) (3.0, 3.0) |
*      B   = | (2.0, 6.0) (4.0, -5.0) (-11.0, -4.0) |
*          |      .              .              .      |
*
*          | (3.0, 6.0) (-1.0, 6.0) (1.0, 7.0) |
*          | (-11.0, 8.0) (1.0, 1.0) (-14.0, 2.0) |
*      C   = |      .              .              .      |
*          |      .              .              .      |
*          |      .              .              .      |
*
*/

float a[24] = {2.0, 0, 13.0, -2.0, -2.0, -6.0, 0, 0,
              0, 0, 2.0, 0, 2.0, 14.0, 0, 0,
              0, 0, 0, 0, 2.0, 0, 0, 0};

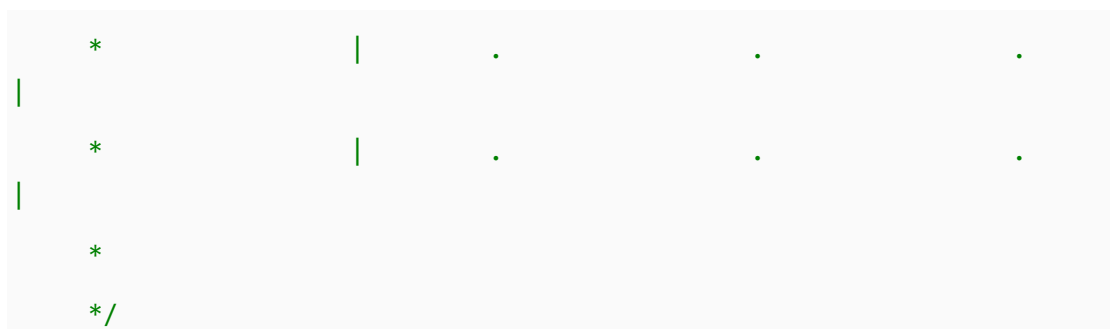
float b[18] = {2.0, 2.0, 2.0, 6.0, 0, 0,
              -13.0, 12.0, 4.0, -5.0, 0, 0,
              3.0, 3.0, -11.0, -4.0, 0, 0};

float c[30] = {3.0, 6.0, -11.0, 8.0, 0, 0, 0, 0, 0, 0,
              -1.0, 6.0, 1.0, 1.0, 0, 0, 0, 0, 0, 0,
              1.0, 7.0, -14.0, 2.0, 0, 0, 0, 0, 0, 0};

cblas_chemm(CblasColMajor, CblasRight, CblasLower, m, n, alpha, a,
lda, b, ldb, beta, c, ldc);

/**
 * Output C
 *
*          | (-777.0, -39.0) (-423.0, 84.0) (-437.0,
849.0) |
*
*          | (-10.0, 100.0) (337.0, -5.0) (-52.0, -602.0)
|
*      C   = |      .              .              .
|

```



2.1.3.4 ?symm

计算矩阵 A 为对称矩阵的矩阵乘法。 $C = \alpha * A * B + \beta * C$ or $C = \alpha * B * A + \beta * C$

alpha 和 beta 是乘法系数，A 为对称矩阵，B 和 C 为 m*n 一般矩阵。

2.1.3.4.1 接口定义

A. C interface:

```
void cblas_ssymm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const
float alpha, const float *A, const BLASINT lda, const float *B, const BLASINT ldb,
const float beta, float *C, const BLASINT ldc);
```

```
void cblas_dsymm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const
double alpha, const double *A, const BLASINT lda, const double *B, const
BLASINT ldb, const double beta, double *C, const BLASINT ldc);
```

```
void cblas_csymm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const
void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb,
const void *beta, void *C, const BLASINT ldc);
```

```
void cblas_zsymm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const BLASINT M, const BLASINT N, const
void *alpha, const void *A, const BLASINT lda, const void *B, const BLASINT ldb,
const void *beta, void *C, const BLASINT ldc);
```

B. Fortran interface:

```
CALL SSYMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)
```

```
CALL DSYMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)
```

```
CALL CSYMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)
```

CALL ZSYMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

CALL ZHEMM(SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

2.1.3.4.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Side	枚举类型 CBLAS_SIDE	对称矩阵 A 在矩阵 B 的左边还是右边。 Side = CblasLeft, $C = \alpha * A * B + \beta * C$ Side=CblasRight, $C = \alpha * B * A + \beta * C$	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角部分。 如果 Uplo=CblasLower, 则使用 A 的下三角部分。	输入
M	整型数	矩阵 C 的行。 A*B 为矩阵 A 的行。 B*A 为矩阵 B 的行。	输入
N	整型数	矩阵 C 的列。 A*B 为矩阵 B 的列。 B*A 为矩阵 A 的列。	输入
alpha	在 ssymm 中, alpha 为单精度浮点类型。	表示乘法系数。	输入
	在 dsymm 中, alpha 为双精度浮点类型。		
	在 csymm 中, alpha 为单精度复数类型。		
	在 zsymm 中, alpha 为双精度复数类型。		

参数名	类型	描述	输入/输出
A	在 ssymm 中, A 为单精度浮点类型。	矩阵 A(lda, ka)。 如果 Side = CblasLeft, ka = m, 否则 ka = n。	输入
	在 dsymm 中, A 为双精度浮点类型。		
	在 csymm 中, A 为单精度复数类型。		
	在 zsymm 中, A 为双精度复数类型。		
lda	整型数	如果 Side = CblasLeft, lda 至少 max(1, m), 否则至少 max(1, n)。	输入
B	在 ssymm 中, B 为单精度浮点类型。	矩阵 B(ldb, n)。	输入
	在 dsymm 中, B 为双精度浮点类型。		
	在 csymm 中, B 为单精度复数类型。		
	在 zsymm 中, B 为双精度复数类型。		
ldb	整型数	矩阵为列存, ldb 至少 max(1, m)。	输入
		矩阵为行存, ldb 至少 max(1, n)。	
beta	在 ssymm 中, B 为单精度浮点类型。	乘法系数。	输入
	在 dsymm 中, B 为双精度浮点类型。		
	在 csymm 中, B 为单精度复数类型。		
	在 zsymm 中, B 为双精度复数类型。		
C	在 ssymm 中, C 为单精度浮点类型。	矩阵 C。	输入/输出

参数名	类型	描述	输入/输出
	在 dsymm 中，C 为双精度浮点类型。		
	在 csymm 中，C 为单精度复数类型。		
	在 zsymm 中，C 为双精度复数类型。		
ldc	整型数	矩阵为列存，ldc 至少 $\max(1, m)$ 。	输入
		矩阵为行存，ldc 至少 $\max(1, n)$ 。	

2.1.3.4.3 依赖

```
#include "kblas.h"
```

2.1.3.4.4 示例

A. C interface:

```
int m = 3, n = 6, lda = 4, ldb = 3, ldc = 5;
float alpha = 2.0, beta = 2.0;

/**
 *
 *      | 4.0   .   .   |
 * A(4 * 3) = | 3.0  4.0   .   |
 *      | 6.0 -7.0 -1.0 |
 *      | .   .   .   |
 *
 *
 *      | -1.0 -3.0  2.0  2.0 -1.0  8.0 |
 * B(3 * 6) = | 2.0  3.0  0.0  4.0  2.0 -2.0 |
 *      | 11.0 -2.0 -21.0 -1.0 -1.0  5.0 |
 *
 *
 *      | 3.0  2.0 -1.0  3.0 -1.0 -11.0|
 *      | 9.0 13.0  5.0  5.0  3.0 -5.0 |
 * C(5 * 6) = | -1.0 -6.0 -3.0  3.0 -1.0 32.0|
```

```

*           | . . . . . |
*           | . . . . . |
*/

float a[12] = {4.0, 3.0, 6.0, 0,
              3.0, 4.0, -7.0, 0,
              6.0, -7.0, -1.0, 0};

float b[18] = {-1.0, 2.0, 11.0,
              -3.0, 3.0, -2.0,
              2.0, 0, -21.0,
              2.0, 4.0, -1.0,
              -1.0, 2.0, -1.0,
              8.0, -2.0, 5.0};

float c[30] = {6.0, 9.0, -1.0, 0, 0,
              2.0, 13.0, -6.0, 0, 0,
              -1.0, 5.0, -3.0, 0, 0,
              3.0, 5.0, 3.0, 0, 0,
              -1.0, 3.0, -1.0, 0, 0,
              -11.0, -5.0, 32.0, 0, 0};

cblas_ssymm(CblasColMajor,CblasLeft,CblasLower, m, n, alpha, a, lda,
b, ldb, beta, c, ldc);
/**
 * Output C
 *
 *           | 148.0 -26.0 -238.0 34.0 -10.0 90.0 |
 *           | -126.0 60.0 316.0 68.0 30.0 -48.0 |
 * C(5 * 6) = | -64.0 -86.0 60.0 -24.0 -40.0 178.0 |
 *           | . . . . . |
 *           | . . . . . |
 *
 */

```

2.1.3.5 ?syrrk

对称矩阵秩 k 更新。 $C = \alpha * A * A^T + \beta * C$ or $C = \alpha * A^T * A + \beta * C$

其中 alpha, beta 为乘法系数, C 为 n*n 的对称矩阵; 第一种情况矩阵 A 为 n*k, 第二种情况矩阵 A 为 k*n 一般矩阵。

2.1.3.5.1 接口定义

A. C interface:

```
void cblas_ssyrrk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const float alpha, const float *A, const BLASINT lda, const float beta, float *C,
const BLASINT ldc);
```

```
void cblas_dsyrrk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const double alpha, const double *A, const BLASINT lda, const double beta,
double *C, const BLASINT ldc);
```

```
void cblas_csyrrk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const void *alpha, const void *A, const BLASINT lda, const void *beta, void *C,
const BLASINT ldc);
```

```
void cblas_zsyrrk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const void *alpha, const void *A, const BLASINT lda, const void *beta, void *C,
const BLASINT ldc);
```

B. Fortran interface:

```
CALL SSYRK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)
```

```
CALL DSYRK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)
```

```
CALL CSYRK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)
```

```
CALL ZSYRK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)
```

2.1.3.5.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	矩阵 C 存储方式。	输入
Trans	枚举类型 CBLAS_TRANSPOSE	矩阵 A 和 B 为常规矩阵，转置矩阵或共轭矩阵。 当 Trans = CblasNoTrans, $C = \alpha * A * A^T + \beta * C$ 。 当 Trans = CblasTrans/CblasConjTrans, $C = \alpha * A^T * A + \beta * C$ 。	输入
N	整型数	矩阵 C 的主序长度。	输入
K	整型数	当 Trans = CblasNoTrans, K 为矩阵 A 的列，否则为矩阵 A 的行。	输入
alpha	在 ssyrk 中，alpha 为单精度浮点类型。 在 dsyrk 中，alpha 为双精度浮点类型。 在 csyrk 中，alpha 为单精度复数类型。 在 zsyrk 中，alpha 为双精度复数类型。	表示乘法系数。	输入
A	在 ssyrk 中，A 为单精度浮点类型。 在 dsyrk 中，A 为双精度浮点类型。 在 csyrk 中，A 为单精度复数类型。 在 zsyrk 中，A 为双精度复数类型。	矩阵 A(lda, ka)。 当 Trans = CblasNoTrans, ka = k，否则 ka = n。	输入

参数名	类型	描述	输入/输出
lda	整型数	矩阵为列存, Trans = CblasNoTrans, lda 至少 $\max(1, n)$, 否则至少 $\max(1, k)$ 。	输入
		矩阵为行存, Trans = CblasNoTrans, lda 至少 $\max(1, k)$, 否则至少 $\max(1, n)$ 。	
beta	在 ssyrk 中, beta 为单精度浮点类型。	乘法系数。	输入
	在 dsyrk 中, beta 为双精度浮点类型。		
	在 csyrk 中, beta 为单精度复数类型。		
	在 zsyrk 中, beta 为双精度复数类型。		
C	在 ssyrk 中, C 为单精度浮点类型。	对称矩阵 C。	输入/输出
	在 dsyrk 中, C 为双精度浮点类型。		
	在 csyrk 中, C 为单精度复数类型。		
	在 zsyrk 中, C 为双精度复数类型。		
ldc	整型数	ldc 至少 $\max(1, n)$ 。	输入

2.1.3.5.3 依赖

```
#include "kblas.h"
```

2.1.3.5.4 示例

A. C interface:

```
int n = 3, lda = 3, k = 2, ldc = 3;
```

```

float alpha = 2.0, beta = 2.0;

/**
 *
 *      | 4.0  1.0  |
 *  A(3 * 2) = | 3.0  4.0  |
 *
 *      | .    .    |
 *
 *
 *      | 3.0  2.0  .|
 *  C(3 * 3) = | -1.0 -6.0 .|
 *
 *      | .    .    .|
 */

float a[6] = {4.0, 3.0, 0, 1.0, 4.0, 0};
float c[9] = {3.0, -1.0, 0, 2.0, -6.0, 0, 0, 0, 0};

cblas_ssyrrk(CblasColMajor, CblasUpper, CblasNoTrans, n, k, alpha, a,
lda, beta, c, ldc);

/**
 * Output C
 *
 *      | 40.0  36.0  .|
 *  C(3 * 3) = | -1.0  38.0 .|
 *
 *      | .    .    .|
 */

```

2.1.3.6 ?syr2k

对称矩阵 C 秩 2 更新。 $C = \alpha * A * B^T + \alpha * B * A^T + \beta * C$ or $C = \alpha * A^T * B + \alpha * B^T * A + \beta * C$

alpha, beta 为乘法系数, C 为 n*n 的对称矩阵; 第一种情况矩阵 A, B 为 n*k, 第二种情况矩阵 A, B 为 k*n 一般矩阵。

2.1.3.6.1 接口定义

A. C interface:

```
void cblas_ssy2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const float alpha, const float *A, const BLASINT lda, const float *B, const
BLASINT ldb, const float beta, float *C, const BLASINT ldc);
```

```
void cblas_dsy2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const double alpha, const double *A, const BLASINT lda, const double *B, const
BLASINT ldb, const double beta, double *C, const BLASINT ldc);
```

```
void cblas_csy2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const void *alpha, const void *A, const BLASINT lda, const void *B, const
BLASINT ldb, const void *beta, void *C, const BLASINT ldc);
```

```
void cblas_zsy2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const void *alpha, const void *A, const BLASINT lda, const void *B, const
BLASINT ldb, const void *beta, void *C, const BLASINT ldc);
```

B. Fortran interface:

```
CALL SSYR2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)
```

```
CALL DSYR2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)
```

```
CALL CSYR2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)
```

```
CALL ZSYR2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)
```

2.1.3.6.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	矩阵 C 存储方式。	输入

参数名	类型	描述	输入/输出
Trans	枚举类型 CBLAS_TRANSPOSE	矩阵 A 和 B 为常规矩阵，转置矩阵或共轭矩阵。 当 Trans = CblasNoTrans, $C = \alpha * A * B^T + \alpha * B * A^T + \beta * C$ 当 Trans = CblasTrans/CblasConjTrans, $C = \alpha * A^T * B + \alpha * B^T * A + \beta * C$	输入
N	整型数	矩阵 C 的主序长度。	输入
K	整型数	当 Trans = CblasNoTrans, K 为矩阵 A 和 B 的列，否则为矩阵 A 和 B 的行。	输入
alpha	在 ssyr2k 中, alpha 为单精度浮点类型。 在 dsyr2k 中, alpha 为双精度浮点类型。 在 csyr2k 中, alpha 为单精度复数类型。 在 zsyr2k 中, alpha 为双精度复数类型。	表示乘法系数。	输入
A	在 ssyr2k 中, A 为单精度浮点类型。 在 dsyr2k 中, A 为双精度浮点类型。 在 csyr2k 中, A 为单精度复数类型。 在 zsyr2k 中, A 为双精度复数类型。	矩阵 A(lda, ka)。 当 Trans = CblasNoTrans, ka = k, 否则 ka = n。	输入
lda	整型数	矩阵为列存, Trans = CblasNoTrans, lda 至少 max(1, n), 否则至少 max(1, k)。	输入

参数名	类型	描述	输入/输出
		矩阵为行存, Trans = CblasNoTrans, lda 至少 $\max(1, k)$, 否则至少 $\max(1, n)$ 。	
B	在 ssyr2k 中, B 为单精度浮点类型。	矩阵 B(ldb, kb)。 当 Trans = CblasNoTrans, kb = k, 否则 kb = n。	输入
	在 dsyr2k 中, B 为双精度浮点类型。		
	在 csyr2k 中, B 为单精度复数类型。		
	在 zsyr2k 中, B 为双精度复数类型。		
ldb	整型数	矩阵为列存, Trans = CblasNoTrans, ldb 至少 $\max(1, n)$, 否则至少 $\max(1, k)$ 。	输入
		矩阵为行存, Trans = CblasNoTrans, ldb 至少 $\max(1, k)$, 否则至少 $\max(1, n)$ 。	
beta	在 ssyr2k 中, beta 为单精度浮点类型。	乘法系数。	输入
	在 dsyr2k 中, beta 为双精度浮点类型。		
	在 csyr2k 中, beta 为单精度复数类型。		
	在 zsyr2k 中, beta 为双精度复数类型。		
C	在 ssyr2k 中, C 为单精度浮点类型。	对称矩阵 C。	输入/输出
	在 dsyr2k 中, C 为双精度浮点类型。		

参数名	类型	描述	输入/输出
	在 csyr2k 中，C 为单精度复数类型。		
	在 zsyr2k 中，C 为双精度复数类型。		
ldc	整型数	ldc 至少 $\max(1, n)$ 。	输入

2.1.3.6.3 依赖

```
#include "kblas.h"
```

2.1.3.6.4 示例

A. C interface:

```
int n = 2, lda = 3, k = 2, ldb = 3, ldc = 3;

float alpha = 2.0, beta = 2.0;

/**
 *      | 4.0  1.0 |
 *  A(3 * 2) = | 3.0  4.0 |
 *      | .    .  |
 *
 *  B(2 * 2)  = | 2.0  3.0 |
 *      | 11.0 -2.0 |
 *
 *      | 3.0  2.0 |
 *  C(3 * 2)  = | -1.0 -6.0 |
 *      | .    .  |
 */

float a[6] = {4.0, 3.0, 0, 1.0, 4.0, 0};
float b[4] = {2.0, 11.0, 3.0, -2.0};
float c[6] = {3.0, -1.0, 0, 2.0, -6.0, 0};
```

```

cblas_ssyr2k(CblasColMajor, CblasUpper, CblasNoTrans, n, k, alpha,
a, lda, b, ldb, beta, c, ldc);

/**
 * Output C
 *
 *      | 30.0  96.0 |
 * C(3 * 2) = | -1.0  184.0 |
 *      | .      .  |
 *
 */

```

2.1.3.7 ?trmm

矩阵 A 为三角矩阵的矩阵乘法。 $B = \alpha * op(A) * B$ or $B = \alpha * B * op(A)$

alpha 为乘法系数，B 为 m*n 的一般矩阵，A 为三角矩阵。

2.1.3.7.1 接口定义

A. C interface:

```
void cblas_strmm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA,
const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const float
alpha, const float *A, const BLASINT lda, float *B, const BLASINT ldb);
```

```
void cblas_dtrmm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA,
const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const
double alpha, const double *A, const BLASINT lda, double *B, const BLASINT ldb);
```

```
void cblas_ctrmm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA,
const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const void
*alpha, const void *A, const BLASINT lda, void *B, const BLASINT ldb);
```

```
void cblas_ztrmm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA,
const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const void
*alpha, const void *A, const BLASINT lda, void *B, const BLASINT ldb);
```

B. Fortran interface:

CALL STRMM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

CALL DTRMM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

CALL CTRMM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

CALL ZTRMM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)

2.1.3.7.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Side	枚举类型 CBLAS_SIDE	矩阵 A 在矩阵 B 的左边还是右边。 Side=CblasLeft, $B = \alpha * op(A) * B$ 。 Side=CblasRight, $B = \alpha * B * op(A)$ 。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 Uplo=CblasUpper, 则使用 A 的上三角部分。 如果 Uplo=CblasLower, 则使用 A 的下三角部分。	输入
TranA	枚举类型 CBLAS_TRANSPOSE	矩阵 A 是否为常规矩阵, 转置矩阵或共轭矩阵。 当 TranA = CblasNoTrans, $op(A) = A$ 。 当 TranA = CblasTrans, $op(A) = A^T$ 。 当 TranA = CblasConjTrans, $op(A) = conjg(A^T)$ 。	输入

参数名	类型	描述	输入/输出
Diag	枚举类型 CBLAS_DIAG	是否为单位三角矩阵。	输入
M	整型数	矩阵 B 的行。	输入
N	整型数	矩阵 B 的列。	输入
alpha	在 strmm 中, alpha 为单精度浮点类型。 在 dtrmm 中, alpha 为双精度浮点类型。 在 ctrmm 中, alpha 为单精度复数类型。 在 ztrmm 中, alpha 为双精度复数类型。	表示乘法系数。	输入
A	在 strmm 中, A 为单精度浮点类型。 在 dtrmm 中, A 为双精度浮点类型。 在 ctrmm 中, A 为单精度复数类型。 在 ztrmm 中, A 为双精度复数类型。	矩阵(lda, k)。 当 Side = CblasLeft, k = m。 当 Side = CblasRight, k = n。	输入
lda	整型数	矩阵为列存, side = CblasLeft, lda 至少 max(1, m), 否则至少 max(1, n)。 矩阵为行存, Trans = CblasNoTrans, lda 至少 max(1, n), 否则至少 max(1, m)。	输入
B	在 strmm 中, B 为单精度浮点类型。 在 dtrmm 中, B 为双精度浮点类型。 在 ctrmm 中, B 为单精度复数类型。	矩阵 B	输入/输出

参数名	类型	描述	输入/输出
	在 ztrmm 中，B 为双精度复数类型。		
ldb	整型数	矩阵为列存，ldb 至少 $\max(1, m)$ ，否则至少 $\max(1, n)$ 。	输入

2.1.3.7.3 依赖

```
#include "kblas.h"
```

2.1.3.7.4 示例

A. C interface:

```
int m = 3, n = 3, lda = 3, ldb = 3;

float alpha = 1.0;

/**
 *      | 4.0  1.0  2.0 |
 *  A   = | .   1.0  3.0 |
 *      | .   .   1.0 |
 *
 *  B    = | 2.0  3.0  1.0|
 *      | 11.0 -2.0  2.0|
 *      | 1.0  1.0  6.0|
 */

float a[9] = {4.0, 0, 0, 1.0, 1.0, 0, 2.0, 3.0, 1.0};
float b[9] = {2.0, 11.0, 1.0, 3.0, -2.0, 1.0, 1.0, 2.0, 6.0};

cblas_strmm(CblasColMajor, CblasRight, CblasUpper, CblasNoTrans,
CblasNonUnit, m, n, alpha, a, lda, b, ldb);

/**
 * Output B
 *
 *      | 8.000000      5.000000      14.000000|
```

```

*      B = | 44.000000      9.000000      18.000000 |
*          | 4.000000      2.000000      11.000000 |
*/

```

2.1.3.8 ?trsm

三角矩阵方程求解，方程是以下一种形式： $op(A) * X = \alpha * B$ or $X * op(A) = \alpha * B$; $op(A) = A$ or $op(A) = A^T$ or $op(A) = conjg(A^T)$

alpha 为乘法系数，X 和 B 为 m*n 的一般矩阵，A 为三角矩阵。

2.1.3.8.1 接口定义

A. C interface:

```

void cblas_strsm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA,
const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const float
alpha, const float *A, const BLASINT lda, float *B, const BLASINT ldb);

```

```

void cblas_dtrsm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA,
const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const
double alpha, const double *A, const BLASINT lda, double *B, const BLASINT ldb);

```

```

void cblas_ctrsm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA,
const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const void
*alpha, const void *A, const BLASINT lda, void *B, const BLASINT ldb);

```

```

void cblas_ztrsm(const enum CBLAS_ORDER Order, const enum CBLAS_SIDE
Side, const enum CBLAS_UPLO Uplo, const enum CBLAS_TRANSPOSE TransA,
const enum CBLAS_DIAG Diag, const BLASINT M, const BLASINT N, const void
*alpha, const void *A, const BLASINT lda, void *B, const BLASINT ldb);

```

B. Fortran interface:

```
CALL STRSM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)
```

```
CALL DTRSM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)
```

```
CALL CTRSM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)
```

```
CALL ZTRSM(SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B, LDB)
```


2.1.3.8.2 参数

参数名	类型	描述	输入/输出
Order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Side	枚举类型 CBLAS_SIDE	矩阵 A 在矩阵 B 的左边还是右边。 $\text{Side}=\text{CblasLeft}, \text{op}(A) * X = \alpha * B$ 。 $\text{Side}=\text{CblasRight}, X * \text{op}(A) = \alpha * B$ 。	输入
Uplo	枚举类型 CBLAS_UPLO	表示使用矩阵 A 的上三角或下三角。 如果 $\text{Uplo}=\text{CblasUpper}$, 则使用 A 的上三角部分。 如果 $\text{Uplo}=\text{CblasLower}$, 则使用 A 的下三角部分。	输入
TransA	枚举类型 CBLAS_TRANSPOSE	矩阵 A 为常规矩阵, 转置矩阵或共轭矩阵。 当 $\text{TransA} = \text{CblasNoTrans}$, $\text{op}(A) = A$ 。 当 $\text{TransA} = \text{CblasTrans}$, $\text{op}(A) = A^T$ 。 当 $\text{TransA} = \text{CblasConjTrans}$, $\text{op}(A) = \text{conjg}(A^T)$ 。	输入

参数名	类型	描述	输入/输出
Diag	枚举类型 CBLAS_DIAG	表示 A 是否是单元三角阵。 如果 Diag=CblasUnit, 则 A 是单元三角矩阵。 如果 Diag=CblasNonUnit, 则 A 不是单元三角矩阵。。	输入
M	整型数	矩阵 B 的行。	输入
N	整型数	矩阵 B 的列。	输入
alpha	在 strsm 中, alpha 为单精度浮点类型。 在 dtrsm 中, alpha 为双精度浮点类型。 在 ctrsm 中, alpha 为单精度复数类型。 在 ztrsm 中, alpha 为双精度复数类型。	表示乘法系数。	输入
A	在 strsm 中, A 是单精度浮点类型。 在 dtrsm 中, A 是双精度浮点类型。 在 ctrsm 中, A 是单精度复数类型。 在 ztrsm 中, A 是双精度复数类型。	三角矩阵 A(lda, N)。	输入
lda	整型数	矩阵 A 中每列存储的地址距离, 要求 lda 大于或等于 max(1, N)。	输入
B	在 strsm 中, B 为单精度浮点类型。 在 dtrsm 中, B 为双精度浮点类型。	矩阵 B	输入/输出

参数名	类型	描述	输入/输出
	在 ctrsm 中, B 为单精度复数类型。		
	在 ztrsm 中, B 为双精度复数类型。		
ldb	整型数	矩阵为列存, ldb 至少 $\max(1, m)$, 否则至少 $\max(1, n)$ 。	输入

2.1.3.8.3 依赖

```
#include "kblas.h"
```

2.1.3.8.4 示例

A. C interface:

```
int m = 3, n = 3, lda = 3, ldb = 3;
float alpha = 1.0;

/**
 *
 *      A      = | 4.0   1.0   2.0 |
 *               |   .   1.0   3.0 |
 *               |   .   .   1.0 |
 *
 *      B      = | 2.0   3.0   1.0|
 *               | 11.0  -2.0  2.0|
 *               | 1.0   1.0   6.0|
 */

float a[9] = {4.0, 0, 0, 1.0, 1.0, 0, 2.0, 3.0, 1.0};
float b[9] = {2.0, 11.0, 1.0, 3.0, -2.0, 1.0, 1.0, 2.0, 6.0};

cblas_strsm(CblasColMajor, CblasRight, CblasUpper, CblasNoTrans,
CblasNonUnit, m, n, alpha, a, lda, b, ldb);

/**
```

```

* Output B
*          | 0.500000      2.500000      -7.500000|
*   B =    | 2.750000      -4.750000      10.750000|
*          | 0.250000      0.750000      3.250000 |
*/

```

2.1.3.9 ?herk

埃尔米特矩阵秩 k 更新。 $C = \alpha * A * A^H + \beta * C$ or $C = \alpha * A^H * A + \beta * C$

其中 alpha, beta 为实乘法系数, C 为 n*n 的埃尔米特矩阵; 第一种情况矩阵 A 为 n*k, 第二种情况矩阵 A 为 k*n。

2.1.3.9.1 接口定义

A. C interface:

```
void cblas_cherk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const float alpha, const void *A, const BLASINT lda, const float beta, void *C,
const BLASINT ldc);
```

```
void cblas_zherk(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const double alpha, const void *A, const BLASINT lda, const double beta, void *C,
const BLASINT ldc);
```

B. Fortran interface:

```
CALL CHERK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)
```

```
CALL ZHERK(UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)
```

2.1.3.9.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	矩阵 C 存储方式。	输入

参数名	类型	描述	输入/输出
Trans	枚举类型 CBLAS_TRANSPOSE	矩阵 A 和 B 为常规矩阵， 转置矩阵或共轭矩阵。 当 Trans = CblasNoTrans， $C = \alpha * A * A^H + \beta * C$ 。 当 Trans = CblasConjTrans， $C = \alpha * A^H * A + \beta * C$ 。	输入
N	整型数	矩阵 C 的阶数。	输入
K	整型数	当 Trans = CblasNoTrans， 矩阵 A 列。	输入
		当 Trans = CblasConjTrans， 矩阵 A 行。	
alpha	在 cherk 中，alpha 为单精度实数类型。	表示乘法系数。	输入
	在 zherk 中，alpha 为双精度实数类型。		
A	在 cherk 中，A 为单精度复数类型。	矩阵 A(lda, ka)。 如果 Trans = CblasNoTrans，ka = k，否则 ka = n。	输入
	在 zherk 中，A 为双精度复数类型。		
lda	整型数	矩阵为列存，Trans = CblasNoTrans，lda 至少 max(1, n)，否则至少 max(1, k)。	输入
		矩阵为行存，Trans = CblasNoTrans，lda 至少 max(1, k)，否则至少 max(1, n)。	
beta	在 cherk 中，beta 为单精度实数类型。	乘法系数。	输入

参数名	类型	描述	输入/输出
	在 zherk 中, beta 为双精度类型。		
C	在 cherk 中, C 为单精度复数类型。	矩阵 C(ldc, n)。	输入/输出
	在 zherk 中, C 为双精度复数类型。		
ldc	整型数	ldc 至少 max(1, n)。	输入

2.1.3.9.3 依赖

```
#include "kblas.h"
```

2.1.3.9.4 示例

A. C interface:

```
int n = 2, lda = 2, k = 2, ldc = 2;
float alpha = 2.0, beta = 2.0;
/**
 *
 *      | 4.0 1.0  1.0 2.0 |
 *  A(2 * 2)  =  | 3.0 2.0  4.0 1.0 |
 *
 *      | 3.0 1.0  2.0 1.0 |
 *  C(2 * 2)  =  | -1.0 1.0 -6.0 2.0 |
 */
float a[8] = {4.0, 1.0, 3.0, 2.0, 1.0, 2.0, 4.0, 1.0};
float c[8] = {3.0, 1.0, -1.0, 1.0, 2.0, 1.0, -6.0, 2.0};

cblas_cherk(CblasColMajor, CblasUpper, CblasConjTrans, n, k, alpha,
a, lda, beta, c, ldc);
/**
 * Output C
 *
 *      | (66.0, 0.0) (44.0, 6.0) |
```

```

*      C(2 * 2)      = | (-1.0, 1.0) (32.0, 0.0) |
*
*/

```

2.1.3.10 ?her2k

埃尔米特矩阵秩 2 更新。 $C = \alpha * A * B^H + \text{conjg}(\alpha) * B * A^H + \beta * C$ or $C = \alpha * A^H * B + \text{conjg}(\alpha) * B^H * A + \beta * C$

其中 alpha, beta 为乘法系数, C 为 n*n 的埃尔米特矩阵; 第一种情况矩阵 A, B 为 n*k, 第二种情况矩阵 A, B 为 k*n。

2.1.3.10.1 接口定义

A. C interface:

```
void cblas_cher2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const void *alpha, const void *A, const BLASINT lda, const void *B, const
BLASINT ldb, const float beta, void *C, const BLASINT ldc);
```

```
void cblas_zher2k(const enum CBLAS_ORDER Order, const enum CBLAS_UPLO
Uplo, const enum CBLAS_TRANSPOSE Trans, const BLASINT N, const BLASINT
K, const void *alpha, const void *A, const BLASINT lda, const void *B, const
BLASINT ldb, const double beta, void *C, const BLASINT ldc);
```

B. Fortran interface:

```
CALL CHER2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)
```

```
CALL ZHER2K(UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)
```

2.1.3.10.2 参数

参数名	类型	描述	输入/输出
order	枚举类型 CBLAS_ORDER	表示矩阵是行主序或列主序。	输入
Uplo	枚举类型 CBLAS_UPLO	矩阵 C 存储方式。	输入

参数名	类型	描述	输入/输出
Trans	枚举类型 CBLAS_TRANSPOSE	矩阵 A 和 B 为常规矩阵， 转置矩阵或共轭矩阵。 当 Trans = CblasNoTrans， $C = \alpha * A * B^H + conjg(\alpha) * B * A^H + \beta * C$ 。 当 Trans = CblasConjTrans， $C = \alpha * A^H * B + conjg(\alpha) * B^H * A + \beta * C$ 。	输入
N	整型数	矩阵 C 的行。	输入
K	整型数	当 Trans = CblasNoTrans， 表示矩阵 A、B 的列。	输入
		当 Trans = CblasConjTrans， 表示矩阵 A、B 的行。	
alpha	在 cher2k 中，alpha 为单精度复数类型。	表示乘法系数。	输入
	在 zher2k 中，alpha 为双精度复数类型。		
A	在 cher2k 中，A 为单精度复数类型。	矩阵 A(lda, ka)。 如果 Trans = CblasNoTrans，ka = k，否则 ka = n。	输入
	在 zher2k 中，A 为双精度复数类型。		
lda	整型数	矩阵为列存，如果 Trans = CblasNoTrans，lda 至少 max(1, n)，否则至少 max(1, k)。	输入
		矩阵为列存，如果 Trans = CblasNoTrans，lda 至少 max(1, k)，否则至少 max(1, n)。	
B	在 cher2k 中，B 为单精度复数类型。	矩阵 B(lb, kb)。	输入

参数名	类型	描述	输入/输出
	在 zher2k 中，B 为双精度复数类型。	Trans = CblasNoTrans, kb = k, 否则 kb = n。	
ldb	整型数	矩阵为列存，如果 Trans = CblasNoTrans, ldb 至少 max(1, n), 否则至少 max(1, k)。	输入
		矩阵为列存，如果 Trans = CblasNoTrans, ldb 至少 max(1, k), 否则至少 max(1, n)。	
beta	在 cher2k 中，beta 为单精度复数类型。	乘法系数。	输入
	在 zher2k 中，beta 为双精度复数类型。		
C	在 cher2k 中，C 为单精度复数类型。	矩阵 C(ldc, n)。	输入/输出
	在 zher2k 中，C 为双精度复数类型。		
ldc	整型数	ldc 至少 max(1, n)。	输入

2.1.3.10.3 依赖

#include "kblas.h"

2.1.3.10.4 示例

A. C interface:

```
int n = 2, lda = 2, k = 2, ldb =3, ldc = 3;
float alpha[2] = {2.0, 1.0}, beta = 2.0;
/**
 *          | 4.0 1.0   1.0 2.0 |
 *   A(2 * 2) = | 3.0 2.0   4.0 1.0 |
 *
 */
```

```

*      B(2 * 2)   = | 2.0 1.0   3.0  2.0 |
*                  | 11.0 2.0 -2.0  1.0 |
*
*                  | 3.0 1.0   2.0  1.0 |
*      C(3 * 2)   = | -1.0 1.0  -6.0 2.0 |
*                  |   .           .   |
*/

float a[8] = {4.0, 1.0, 3.0, 2.0, 1.0, 2.0, 4.0, 1.0};
float b[8] = {2.0, 1.0, 11.0, 2.0, 3.0, 2.0, -2.0, 1.0};
float c[12] = {3.0, 1.0, -1.0, 1.0, 0, 0, 2.0, 1.0, -6.0, 2.0, 0,
0};

cblas_cher2k(CblasColMajor, CblasUpper, CblasConjTrans, n, k, alpha,
a, lda, b, ldb, beta, c, ldc);

/**
 * Output C
 *
 *      C(3 * 2)   = | (218.0, 0.0) (123.0, -27.0) |
*                  | (-1.0 1.0)   (46.0, 0.0)   |
*                  |   .           .           |
*/

```

2.2 LAPACK 接口说明

2.2.1 矩阵分解函数

2.2.1.1 函数说明

函数族	简介
-----	----

函数族	简介
?getrf	计算矩阵 A 的 LU 分解，允许行交换（partial pivoting）。分解结果为 $A = PLU$，其中 P 为置换矩阵、L 为下三角矩阵或下阶梯矩阵且对角线为 1，U 是上三角矩阵或上阶梯矩阵。 具体分为以下四个函数： sgetrf: 单精度浮点型 dgetrf: 双精度浮点型 cgetrf: 单精度复数型 zgetrf: 双精度复数型
?geqrf	计算矩阵的 QR 分解，即 $A=Q*R$ 具体分为以下四个函数： sgeqrf: 单精度浮点型 dgeqrf: 双精度浮点型 cgeqrf: 单精度复数型 zgeqrf: 双精度复数型
?gerqf	计算矩阵的 RQ 分解，即 $A=R*Q$。 具体分为以下四个函数： sgerqf: 单精度浮点型 dgerqf: 双精度浮点型 cgerqf: 单精度复数型 zgerqf: 双精度复数型
?geqlf	计算矩阵的 QL 分解，即 $A=Q*L$。 具体分为以下四个函数： sgeqlf: 单精度浮点型 dgeqlf: 双精度浮点型 cgeqlf: 单精度复数型 zgeqlf: 双精度复数型
?gelqf	计算矩阵的 LQ 分解，即 $A=L*Q$。 具体分为以下四个函数： sgelqf: 单精度浮点型 dgelqf: 双精度浮点型 cgelqf: 单精度复数型 zgelqf: 双精度复数型
?potrf	计算对称正定矩阵或者 Hermite 正定矩阵的 Cholesky 分解，即 $A = LL^T$ 或者 $A = UU^T$, U 上三角矩阵，L 下三角矩阵。 具体分为以下四个函数： spotrf: 单精度浮点型 dpotrf: 双精度浮点型 cpotrf: 单精度复数型 zpotrf: 双精度复数型

函数族	简介
?pttrf	计算实（共轭）对称正定三对角矩阵 A 的 LDL^H 或 $U^H DU$ 分解，其中 D 为对角块矩阵，L 为下三角矩阵，U 为上三角矩阵。 具体分为以下四个函数： spttrf: 单精度浮点型 dpttrf: 双精度浮点型 cpttrf: 单精度复数型 zpttrf: 双精度复数型
?gttrf	计算一般三对角矩阵 A 的 LU 分解。 具体分为以下四个函数： sgttrf: 单精度浮点型 dgttrf: 双精度浮点型 cgttrf: 单精度复数型 zgttrf: 双精度复数型
?sptrf	计算压缩对称矩阵的 LDL^H 或 $U^H DU$ 分解。 具体分为以下四个函数： ssptrf: 单精度浮点型 dsptrf: 双精度浮点型 csptrf: 单精度复数型 zsptrf: 双精度复数型

2.2.1.2 ?getrf

计算矩阵 A 的 LU 分解，允许行交换（partial pivoting）。分解结果为 $A=PLU$ ，其中 P 为置换矩阵、 L 为下三角矩阵或下阶梯矩阵且对角线为 1， U 是上三角矩阵或上阶梯矩阵。

2.2.1.2.1 接口定义

A. C interface:

```
void sgetrf_(const int *m, const int *n, float *a, const int *lda, int *ipiv, int *info);
void dgetrf_(const int *m, const int *n, double *a, const int *lda, int *ipiv, int *info);
void cgetrf_(const int *m, const int *n, float _Complex *a, const int *lda, int *ipiv, int *info);
void zgetrf_(const int *m, const int *n, double _Complex *a, const int *lda, int *ipiv, int *info);
```

B. Fortran interface:

```
SGETRF(m, n, a, lda, ipiv, info)
```

DGETRF(m, n, a, lda, ipiv, info)

CGETRF(m, n, a, lda, ipiv, info)

ZGETRF(m, n, a, lda, ipiv, info)

2.2.1.2.2 参数

参数名	类型	描述	输入/输出
m	整型数	矩阵 A 的行数。	输入
n	整型数	矩阵 A 的列数。	输入
a	在 sgetrf 中为单精度浮点型数组。 在 dgetrf 中为双精度浮点型数组。 在 cgetrf 中为单精度复数型数组。 在 zgetrf 中为双精度复数型数组。	调用前保存矩阵 A。 调用后保存分解结果 L 和 U，不保存 L 的对角线元素（均为 1）。	输入/输出
lda	整型数	A 的 leading dimension 大小，要求 $lda \geq \max(1, n)$ 。	输入
ipiv	整型数组	?getrf 得到的主元索引，长度为 $\min(m, n)$ ： $1 \leq ipiv \leq \min(m, n)$ ，表示分解中矩阵第 i 行与第 $ipiv[i-1]$ 行交换。	输出
info	整型	执行结果： 等于 0：成功。 小于 0：第 -info 个参数值不合法。 大于 0：矩阵 U 对角线上第 info 个元素为 0，矩阵分解完成但 U 是奇异的，会导致求解线性方程组时出现除以零的错误。	输出

2.2.1.2.3 依赖

```
#include "klapack.h"
```

2.2.1.2.4 示例

A. C interface:

```
int m = 4;
int n = 4;
int lda = 4;
int ipiv[4];
int info = 0;
/*
 * A (stored in column-major):
 *  1.80  2.88  2.05 -0.89
 *  5.25 -2.95 -0.95 -3.80
 *  1.58 -2.69 -2.90 -1.04
 * -1.11 -0.66 -0.59  0.80
 */
double a[] = {1.80,  5.25, 1.58, -1.11,
               2.88, -2.95, -2.69, -0.66,
               2.05, -0.95, -2.90, -0.59,
               -0.89, -3.80, -1.04,  0.80};
dgetrf_(&m, &n, a, &lda, ipiv, &info);
/*
 * Output:
 *  5.2500 -2.9500 -0.9500 -3.8000
 *  0.3429  3.8914  2.3757  0.4129
 *  0.3010 -0.4631 -1.5139  0.2948
 * -0.2114 -0.3299  0.0047  0.1314
 */
```

2.2.1.3 ?geqrf

计算矩阵的 QR 分解，即 $A = Q \times R$ 。

2.2.1.3.1 接口定义

A. C interface:

```
void dgeqrf_(const int *m, const int *n, double *a, const int *lda, double *tau, double
*work, const int *lwork, int *info);
```

```
void sgeqrf_(const int *m, const int *n, float *a, const int *lda, float *tau, float *work,
const int *lwork, int *info);
```

```
void cgeqrf_(const int *m, const int *n, float _Complex *a, const int *lda, float
_Complex *tau, float _Complex *work, const int *lwork, int *info);
```

```
void zgeqrf_(const int *m, const int *n, double _Complex *a, const int *lda, double
_Complex *tau, double _Complex *work, const int *lwork, int *info);
```

B. Fortran interface:

```
DGEQRF(m, n, a, lda, tau, work, lwork, info);
```

```
SGEQRF(m, n, a, lda, tau, work, lwork, info);
```

```
CGEQRF(m, n, a, lda, tau, work, lwork, info);
```

```
ZGEQRF(m, n, a, lda, tau, work, lwork, info);
```

2.2.1.3.2 参数

参数名	类型	描述	输入/输出
m	整型数	矩阵 A 的行数。	输入
n	整型数	矩阵 A 的列数。	输入
a	在 sgeqrf 中为单精度浮点型数组。 在 dgeqrf 中为双精度浮点型数组。 在 cgeqrf 中为单精度复数型数组。 在 zgeqrf 中为双精度复数型数组。	调用前保存待分解的矩阵 A。调用后对角线以及对角线以上的部分保存大小为 $\min(m,n)*n$ 的矩阵 R (当 $m \geq n$ 时 R 为上三角矩阵), 对角线以下元素和 tau 共同表示正交矩阵 Q (参见说明)。	输入/输出
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, m)$ 。	输出
tau	在 sgeqrf 中为单精度浮点型数组。	初等反射的系数, 长度为 $\min(m,n)$ (参见说明)。	输出

参数名	类型	描述	输入/输出
	在 dgeqrf 中为双精度浮点型数组。		
	在 cgeqrf 中为单精度复数型数组。		
	在 zgeqrf 中为双精度复数型数组。		
work	在 sgeqrf 中为单精度浮点型数组。	临时存储空间，使用 lwork=-1 调用后 work[0]为最优的 lwork 值。	输出
	在 dgeqrf 中为双精度浮点型数组。		
	在 cgeqrf 中为单精度复数型数组。		
	在 zgeqrf 中为双精度复数型数组。		
lwork	整数型	work 数组的长度。 lwork=-1 时查询最优 work 大小，结果保存在 work[0]中，否则要求 $lwork \geq n$ 。	输入
info	整数型	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合法。	输出

2.2.1.3.3 依赖

#include "klapack.h"

2.2.1.3.4 示例

A. C interface:

```
int m = 6;

int n = 4;

int lda = 6;

int info = 0;
```



```

double tau[4];

double *work = NULL;

double qwork;

int lwork = -1;

/*
 * A (6x4, stored in column-major):
 *  2.229  1.273  0.087  0.035
 *  8.667  4.317  4.091  3.609
 *  0.205  7.810  2.553  6.507
 *  2.758  2.911  8.791  5.051
 *  8.103  1.396  1.317  4.738
 *  8.859  3.161  0.808  5.972
 */

double a[] = {2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
              1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
              0.035, 3.609, 6.507, 5.051, 4.738, 5.972};

/* Query optimal work size */
dgeqrf_(&m, &n, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);

/* Calculate QR */
dgeqrf_(&m, &n, a, &lda, tau, work, &lwork, &info);
free(work);

/*
 * Output:
 *  tau
 *  1.1464 1.0946 1.3670 1.7415

```

```

* A output (stored in column-major)
*  -15.2274 -5.8577 -5.1387 -9.0573
*   0.4965 -8.2070 -4.5806 -6.5291
*   0.0117  0.8600  7.4527  1.3953
*   0.1580  0.1986 -0.6634 -3.5188
*   0.4642 -0.2130 -0.0183  0.2805
*   0.5075 -0.0510  0.1506  0.2642
*/

```

2.2.1.4 ?gerqf

计算矩阵的 RQ 分解，即 $A = R \times Q$ 。

2.2.1.4.1 接口定义

A. C interface:

```
void sgerqf_(const int *M, const int *N, float *A, const int *LDA, float *TAU, float
*WORK, const int *LWORK, int *INFO);
```

```
void dgerqf_(const int *M, const int *N, double *A, const int *LDA, double *TAU,
double *WORK, const int *LWORK, int *INFO);
```

```
void cgerqf_(const int *M, const int *N, float _Complex *A, const int *LDA, float
_Complex *TAU, float _Complex *WORK, const int *LWORK, int *INFO);
```

```
void zgerqf_(const int *M, const int *N, double _Complex *A, const int *LDA,
double _Complex *TAU, double _Complex *WORK, const int *LWORK, int *INFO);
```

B. Fortran interface:

```
DGERQF(m, n, a, lda, tau, work, lwork, info);
```

```
SGERQF(m, n, a, lda, tau, work, lwork, info);
```

```
CGERQF(m, n, a, lda, tau, work, lwork, info);
```

```
ZGERQF(m, n, a, lda, tau, work, lwork, info);
```

2.2.1.4.2 参数

参数名	类型	描述	输入/输出
m	整型数	矩阵 A 的行数。	输入
n	整型数	矩阵 A 的列数。	输入

参数名	类型	描述	输入/输出
a	在 sgerqf 中为单精度浮点型数组。	调用前保存待分解的矩阵 A。 调用后对角线以及对角线以上的部分保存大小为 $\min(m,n)*n$ 的矩阵 R (当 $m \geq n$ 时 R 为上三角矩阵), 对角线以下元素和 tau 共同表示正交矩阵 Q (参见说明)。	输入/输出
	在 dgerqf 中为双精度浮点型数组。		
	在 cgerqf 中为单精度复数型数组。		
	在 zgerqf 中为双精度复数型数组。		
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, m)$ 。	输出
tau	在 sgerqf 中为单精度浮点型数组。	初等反射的系数, 长度为 $\min(m,n)$ (参见说明)。	输出
	在 dgerqf 中为双精度浮点型数组。		
	在 cgerqf 中为单精度复数型数组。		
	在 zgerqf 中为双精度复数型数组。		
work	在 sgerqf 中为单精度浮点型数组。	临时存储空间, 使用 lwork=-1 调用后 work[0]为最优的 lwork 值。	输出
	在 dgerqf 中为双精度浮点型数组。		
	在 cgerqf 中为单精度复数型数组。		
	在 zgerqf 中为双精度复数型数组。		
lwork	整数型	work 数组的长度。 lwork=-1 时查询最优 work 大小, 结果保存在 work[0]中, 否则要求 $lwork \geq n$ 。	输入
info	整数型	执行结果: 等于 0: 成功。 小于 0: 第-info 个参数值不合法。	输出

2.2.1.4.3 依赖

```
#include "klapack.h"
```

2.2.1.4.4 示例

A. C interface:

```
int m = 6;

int n = 4;

int lda = 6;

int info = 0;

double tau[4];

double *work = NULL;

double qwork;

int lwork = -1;

/*
 * A (6x4, stored in column-major):
 * 2.229  1.273  0.087  0.035
 * 8.667  4.317  4.091  3.609
 * 0.205  7.810  2.553  6.507
 * 2.758  2.911  8.791  5.051
 * 8.103  1.396  1.317  4.738
 * 8.859  3.161  0.808  5.972
 */

double a[] = {2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
              1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
              0.035, 3.609, 6.507, 5.051, 4.738, 5.972};

/* Query optimal work size */
dgeqrf_(&m, &n, a, &lda, tau, &qwork, &lwork, &info);

if (info != 0) {
    return ERROR;
}
```

```

}

lwork = (int)qwork;

work = (double *)malloc(sizeof(double) * lwork);

/* Calculate QR */

dgeqrf_(&m, &n, a, &lda, tau, work, &lwork, &info);

free(work);

/*
 * Output:
 * tau
 *   1.1464  1.0946  1.3670  1.7415
 * A output (stored in column-major)
 *  -15.2274 -5.8577 -5.1387 -9.0573
 *   0.4965 -8.2070 -4.5806 -6.5291
 *   0.0117  0.8600  7.4527  1.3953
 *   0.1580  0.1986 -0.6634 -3.5188
 *   0.4642 -0.2130 -0.0183  0.2805
 *   0.5075 -0.0510  0.1506  0.2642
 */

```

2.2.1.5.1 ?geqlf

计算矩阵的QL分解，即 $A = Q \times L$ 。

2.2.1.5.1 接口定义

A. C interface:

```
void sgeqlf_(const int *M, const int *N, float *A, const int *LDA, float *TAU, float
*WORK, const int *LWORK, int *INFO);
```

```
void dgeqlf_(const int *M, const int *N, double *A, const int *LDA, double *TAU,
double *WORK, const int *LWORK, int *INFO);
```

```
void cgeqlf_(const int *M, const int *N, float _Complex *A, const int *LDA, float
_Complex *TAU, float _Complex *WORK, const int *LWORK, int *INFO);
```

```
void zgeqlf_(const int *M, const int *N, double _Complex *A, const int *LDA,
double _Complex *TAU, double _Complex *WORK, const int *LWORK, int *INFO);
```

B. Fortran interface:

DGEQLF(m, n, a, lda, tau, work, lwork, info);

SGEQLF(m, n, a, lda, tau, work, lwork, info);

CGEQLF(m, n, a, lda, tau, work, lwork, info);

ZGEQLF(m, n, a, lda, tau, work, lwork, info);

2.2.1.5.2 参数

参数名	类型	描述	输入/输出
m	整型数	矩阵 A 的行数。	输入
n	整型数	矩阵 A 的列数。	输入
a	在 sgeqlf 中为单精度浮点型数组。 在 dgeqlf 中为双精度浮点型数组。 在 cgeqlf 中为单精度复数型数组。 在 zgeqlf 中为双精度复数型数组。	调用前保存待分解的矩阵 A。 调用后对角线以及对角线以上的部分保存大小为 $\min(m,n)*n$ 的矩阵 R (当 $m \geq n$ 时 R 为上三角矩阵), 对角线以下元素和 tau 共同表示正交矩阵 Q (参见说明)。	输入/输出
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, m)$ 。	输出
tau	在 sgeqlf 中为单精度浮点型数组。 在 dgeqlf 中为双精度浮点型数组。 在 cgeqlf 中为单精度复数型数组。 在 zgeqlf 中为双精度复数型数组。	初等反射的系数, 长度为 $\min(m,n)$ (参见说明)。	输出
work	在 sgeqlf 中为单精度浮点型数组。 在 dgeqlf 中为双精度浮点型数组。	临时存储空间, 使用 $lwork=-1$ 调用后 work[0]为最优的 lwork 值。	输出

参数名	类型	描述	输入/输出
	在 cgeqlf 中为单精度复数型数组。		
	在 zgeqlf 中为双精度复数型数组。		
lwork	整数型	work 数组的长度。 lwork=-1 时查询最优 work 大小，结果保存在 work[0]中，否则要求 $\text{lwork} \geq n$ 。	输入
info	整数型	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合法。	输出
说明： 分解结果矩阵 Q 通过一系列初等反射乘积表示：$Q = H(1) * H(2) * \dots * H(k), k = \min(m, n)$。 $H(i) = I - \tau * v * v'$。 τ 为标量，v 为向量且前 i-1 个元素为 0，第 i 个元素为 1，剩余元素保存在 a 的第 i 列中（a 的下三角部分）。			

2.2.1.5.3 依赖

```
#include "klapack.h"
```

2.2.1.5.4 示例

A.C interface:

```
int m = 6;

int n = 4;

int lda = 6;

int info = 0;

double tau[4];

double *work = NULL;

double qwork;

int lwork = -1;

/*
```

```

* A (6x4, stored in column-major):
*   2.229  1.273  0.087  0.035
*   8.667  4.317  4.091  3.609
*   0.205  7.810  2.553  6.507
*   2.758  2.911  8.791  5.051
*   8.103  1.396  1.317  4.738
*   8.859  3.161  0.808  5.972
*/

double a[] = {2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
              1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
              0.035, 3.609, 6.507, 5.051, 4.738, 5.972};

/* Query optimal work size */
dgeqlf_(&m, &n, a, &lida, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate QL */
dgeqlf_(&m, &n, a, &lida, tau, work, &lwork, &info);
free(work);

/*
* Output:
* tau
*   1.628275      1.498542      1.123799      1.506551
* A output (stored in column-major)
*   -0.199388     -0.434210      8.949191     -3.693237
*   -1.136543     -11.698489     -0.184444     -0.181795
*   -0.517264      4.452845      0.152113     -9.045264
*   -0.009043     -0.309964      0.056353     -0.824829

```



```

* 6.975979      -7.366574      0.001971      0.203192
* 0.366353      0.284379      0.266756      -11.789537
*/

```

2.2.1.6 ?gelqf

计算矩阵的 LQ 分解，即 $A = L \times Q$ 。

2.2.1.6.1 接口定义

A. C interface:

```
void sgelqf_(const int *M, const int *N, float *A, const int *LDA, float *TAU, float
*WORK, const int *LWORK, int *INFO);
```

```
void dgelqf_(const int *M, const int *N, double *A, const int *LDA, double *TAU,
double *WORK, const int *LWORK, int *INFO);
```

```
void cgelqf_(const int *M, const int *N, float _Complex *A, const int *LDA, float
_Complex *TAU, float _Complex *WORK, const int *LWORK, int *INFO);
```

```
void zgelqf_(const int *M, const int *N, double _Complex *A, const int *LDA,
double _Complex *TAU, double _Complex *WORK, const int *LWORK, int *INFO);
```

B. Fortran interface:

```
DGELQF(m, n, a, lda, tau, work, lwork, info);
```

```
SGELQF(m, n, a, lda, tau, work, lwork, info);
```

```
CGELQF(m, n, a, lda, tau, work, lwork, info);
```

```
ZGELQF(m, n, a, lda, tau, work, lwork, info);
```

2.2.1.6.2 参数

参数名	类型	描述	输入/输出
m	整型数	矩阵 A 的行数。	输入
n	整型数	矩阵 A 的列数。	输入
a	在 sgelqf 中为单精度浮点型数组。 在 dgelqf 中为双精度浮点型数组。	调用前保存待分解的矩阵 A。 调用后对角线以及对角线以上的部分保存大小为 $\min(m,n)*n$ 的矩阵 R（当 $m \geq n$ 时 R 为上三角	输入/输出

参数名	类型	描述	输入/输出
	在 <code>cgelqf</code> 中为单精度复数型数组。	矩阵), 对角线以下元素和 <code>tau</code> 共同表示正交矩阵 <code>Q</code> (参见说明)。	
	在 <code>zgelqf</code> 中为双精度复数型数组。		
<code>lda</code>	整型数	<code>A</code> 的 leading dimension 大小, 要求 $lda \geq \max(1, m)$ 。	输出
<code>tau</code>	在 <code>sgelqf</code> 中为单精度浮点型数组。	初等反射的系数, 长度为 $\min(m, n)$ (参见说明)。	输出
	在 <code>dgelqf</code> 中为双精度浮点型数组。		
	在 <code>cgelqf</code> 中为单精度复数型数组。		
	在 <code>zgelqf</code> 中为双精度复数型数组。		
<code>work</code>	在 <code>sgelqf</code> 中为单精度浮点型数组。	临时存储空间, 使用 <code>lwork=-1</code> 调用后, <code>work[0]</code> 为最优的 <code>lwork</code> 值。	输出
	在 <code>dgelqf</code> 中为双精度浮点型数组。		
	在 <code>cgelqf</code> 中为单精度复数型数组。		
	在 <code>zgelqf</code> 中为双精度复数型数组。		
<code>lwork</code>	整数型	<code>work</code> 数组的长度。 <code>lwork=-1</code> 时查询最优 <code>work</code> 大小, 结果保存在 <code>work[0]</code> 中, 否则要求 $lwork \geq n$ 。	输入
<code>info</code>	整数型	执行结果: 等于 0: 成功。 小于 0: 第- <code>info</code> 个参数值不合法。	输出

参数名	类型	描述	输入/输出
说明： 分解结果矩阵 Q 通过一系列初等反射乘积表示：$Q = H(1) * H(2) * \dots * H(k), k = \min(m, n)$。 $H(i) = I - \tau * v * v'$。 τ 为标量， v 为向量且前 $i-1$ 个元素为 0， 第 i 个元素为 1， 剩余元素保存在 a 的第 i 列中（a 的下三角部分）。			

2.2.1.6.3 依赖

```
#include "klapack.h"
```

2.2.1.6.4 示例

A. C interface:

```
int m = 6;

int n = 4;

int lda = 6;

int info = 0;

double tau[4];

double *work = NULL;

double qwork;

int lwork = -1;

/*
 * A (6x4, stored in column-major):
 *  2.229  1.273  0.087  0.035
 *  8.667  4.317  4.091  3.609
 *  0.205  7.810  2.553  6.507
 *  2.758  2.911  8.791  5.051
 *  8.103  1.396  1.317  4.738
 *  8.859  3.161  0.808  5.972
 */

double a[] = {2.229, 8.667, 0.205, 2.758, 8.103, 8.859,
              1.273, 4.317, 7.810, 2.911, 1.396, 3.161,
```

```

0.087, 4.091, 2.553, 8.791, 1.317, 0.808,
0.035, 3.609, 6.507, 5.051, 4.738, 5.972};

/* Query optimal work size */
dgelqf_(&m, &n, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate LQ */
dgelqf_(&m, &n, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * tau
 *   1.867784      1.115696      1.413422      0.000000
 * A output (stored in column-major)
 *  -2.568611     -9.848324     -4.223656     -4.202616
 *  -7.832678     -9.363028      0.265340      5.150248
 *   5.402586      9.567440      4.194706      4.480431
 *   0.018134     -0.653528     -7.929047     -1.156667
 *   1.133611     -0.463361      0.007295     -0.604570
 *   0.644209     -2.887734      3.399866      4.103192
 */

```

2.2.1.7 ?potrf

计算对称正定矩阵或者 Hermite 正定矩阵的 Cholesky 分解，即 $A = LL^T$ 或者 $A = U^T U$ ，U 上三角矩阵，L 下三角矩阵。

2.2.1.7.1 接口定义

A. C interface:

```

void spotrf_(const char *uplo, const int *n, float *a, const int *lda, int *info);
void dpotrf_(const char *uplo, const int *n, double *a, const int *lda, int *info);
void cpotrf_(const char *uplo, const int *n, float _Complex *a, const int *lda, int
*info);
void zpotrf_(const char *uplo, const int *n, double _Complex *a, const int *lda, int
*info);

```

B. Fortran interface:

```

SPOTRF( uplo, n, a, lda, info);
DPOTRF( uplo, n, a, lda, info);
CPOTRF( uplo, n, a, lda, info);
ZPOTRF( uplo, n, a, lda, info);

```

2.2.1.7.2 参数

参数名	类型	描述	输入/输出
uplo	字符型	'U': 保存 A 的上三角矩阵。	输入
		'L': 保存 A 的下三角矩阵。	
n	整型数	对称矩阵 A 的行数或列数。	输入
a	在 spotrf 中为单精度浮点型数组。	调用前保存待分解的矩阵 A。 当 uplo = 'U', 调用后对角线以及对角线以上的部分保存上三角矩阵 U。 当 uplo = 'L', 调用后对角线以及对角线以下的部分保存下三角矩阵 L。	输入/输出
	在 dpotrf 中为双精度浮点型数组。		
	在 cpotrf 中为单精度复数型数组。		
	在 zpotrf 中为双精度复数型数组。		
lda	整型数	A 的 leading dimension 大小, 要求 $lda \geq \max(1, m)$ 。	输入
info	整数型	执行结果: 等于 0: 退出成功。 小于 0: 第-info 个参数值不合法。 大于 0: A 中 info 大小的主子式非正定, 无法完成分解。	输出

2.2.1.7.3 依赖

```
#include "klapack.h"
```

2.2.1.7.4 示例

A. C interface:

```
char uplo = 'L'
int n = 4;
int lda = 4;
int info = 0;
/*
 * A (stored in column-major):
 *   231.8009    33.9545    9.4153    11.1156
 *   33.9454    162.2902    6.6684    53.2023
 *   9.4153     6.6684   100.3220    71.5384
 *   11.1156    53.2023    71.5384   123.4425
 */
double a[] = {231.8009,    33.9545,    9.4143,    11.1156,
               33.9495,   162.2902,    6.6684,    53.2033,
               9.4143,     6.6684,   100.322,    71.5384,
               11.1156,    53.2033,    71.5384,   106.5638 };
dpotrf_(&uplo, &n, a, &lda, &info);
/*
 * Output:
 *   15.2250    33.9495    9.4143    11.1156
 *   2.2302    12.5426    6.6684    53.2033
 *   0.6183     0.4217    9.9881    71.5384
 *   0.7301     4.112    6.9436     6.396
 */
```

2.2.1.8 ?pttrf

计算实（共轭）对称正定三对角矩阵 A 的 LDL^* 或 U^*DU 分解，其中 D 为对角块矩阵， L 为下三角矩阵， U 为上三角矩阵。

2.2.1.8.1 接口定义

A. C interface:

```
void sptrf_(const int *N, float *D, float *E, int *INFO);
void dpttrf_(const int *N, double *D, double *E, int *INFO);
void cpttrf_(const int *N, float *D, float _Complex *E, int *INFO);
void zpttrf_(const int *N, double *D, double _Complex *E, int *INFO);
```

B. Fortran interface:

```
SPTTRF(N, D, E, INFO);
DPTTRF(N, D, E, INFO);
CPTTRF(N, D, E, INFO);
ZPTTRF(N, D, E, INFO);
```

2.2.1.8.2 参数

参数名	类型	描述	输入/输出
N	整型数	矩阵 A 的维数， $N \geq 0$ 。	输入
D	整型数	对称矩阵 A 的行数或列数。	输入/输出
E	在 sptrf/cpttrf 中为单精度浮点型数组。	输入时，为三对角矩阵 A 的对角元素，维度为 N 。	输入/输出
	在 dpttrf/zpttrf 中为双精度浮点型数组。	输出时，为分解后的对角元素 D 。	
INFO	整型	执行结果： 等于 0：退出成功。 小于 0：第-INFO 个参数值不合法。 大于 0： A 中 INFO 大小的主子式非正定，无法完成分解。	输出

2.2.1.8.3 依赖

```
#include "klapack.h"
```

2.2.1.8.4 示例

A. C interface:

```
const int n = 4;

double d[] = {103.608696, 103.739130, 103.347826, 100.652174};
double e[] = {0.352941, 0.000000, 0.823529};
int info = 0;
dpttrf_(&n, d, e, &info);

/*
 * Output:
 * d: 103.608696      103.737928      103.347826      100.645612
 * e:  0.003406       0.000000       0.007969
 */
```

2.2.1.9 ?gttrf

计算一般三对角矩阵 A 的 LU 分解。

2.2.1.9.1 接口定义

A. C interface:

```
void sgtrf(const int *N, float *DL, float *D, float *DU, float *DU2, int *IPIV, int *INFO);
```

```
void dgtrf(const int *N, double *DL, double *D, double *DU, double *DU2, int *IPIV, int *INFO);
```

```
void cgtrf(const int *N, float _Complex *DL, float _Complex *D, float _Complex *DU, float _Complex *DU2, int *IPIV, int *INFO);
```

```
void zgtrf(const int *N, double _Complex *DL, double _Complex *D, double _Complex *DU, double _Complex *DU2, int *IPIV, int *INFO);
```

B. Fortran interface:

SGTTRF(N,DL,D,DU,DU2,IPIV,INFO);

DGTTRF(N,DL,D,DU,DU2,IPIV,INFO);

CGTTRF(N,DL,D,DU,DU2,IPIV,INFO);

ZGTTRF(N,DL,D,DU,DU2,IPIV,INFO);

2.2.1.9.2 参数

参数名	类型	描述	输入/输出
N	整型数	矩阵 A 的维数， $N \geq 0$ 。	输入
DL	在 sgttrf 中为单精度浮点型数组。	输入时，为三对角矩阵 A 的次对角元素，维度为 N-1。 输出时，为 L 矩阵。	输入/输出
	在 dgttrf 中为双精度浮点型数组。		
	在 cgttrf 中为单精度复数型数组。		
	在 zgttrf 中为双精度复数型数组。		
D	在 sgttrf 中为单精度浮点型数组。	输入时，为三对角矩阵 A 的对角元素，维度为 N。 输出时，为 U 的对角元素。	输入/输出
	在 dgttrf 中为双精度浮点型数组。		
	在 cgttrf 中为单精度复数型数组。		
	在 zgttrf 中为双精度复数型数组。		
DU	在 sgttrf 中为单精度浮点型数组。	输入时，为三对角矩阵 A 的超对角元素，维度为 N-1。 输出时，为 U 的第一超对角元素。	输入、输出
	在 dgttrf 中为双精度浮点型数组。		
	在 cgttrf 中为单精度复数型数组。		
	在 zgttrf 中为双精度复数型数组。		
DU2	在 sgttrf 中为单精度浮点型数组。	输出时，为 U 的第二超对角元素。	输出

参数名	类型	描述	输入/输出
	在 <code>dgtrf</code> 中为双精度浮点型数组。		
	在 <code>cgtrf</code> 中为单精度复数型数组。		
	在 <code>zgtrf</code> 中为双精度复数型数组。		
IPIV	整型数	转置数组。	输出
INFO	整型数	等于 0: 成功退出。 小于 0: 表示第-INFO 个参数非法。 大于 0: 表示 $U(\text{INFO}, \text{INFO})$ 为零, 分解失败。	输出

2.2.1.9.3 依赖

`#include "klapack.h"`

2.2.1.9.4 示例

A. C interface:

```
const int n = 4;
double dl[] = {0.923077, 0.153846, 0.153846};
double d[] = {0.521739, 0.043478, 0.304348, 0.086957};
double du[] = {0.924528, 0.150943, 0.566038};
double du2[] = {0.986301, 0.397260};
double ipiv[4];
int info = 0;
dgtrf_(&n, dl, d, du, du2, ipiv, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
```

```

* Output:
*   d:    0.923077      0.899954      0.318932      -0.186088
*   dl:    0.565217      0.170949      0.482378
*   du:    0.043478      -0.085316      0.566038
*   du2:   0.150943      0.000000
*   ipiv:  2          2          3          4
*/

```

2.2.1.10 ?sptrf

计算压缩对称矩阵的 LDL^* 或 U^*DU 分解。

2.2.1.10.1 接口定义

A. C interface:

```

void ssptrf_(const char *uplo, const int *n, float *ap, int *ipiv, int *info);
void dsptrf_(const char *uplo, const int *n, double *ap, int *ipiv, int *info);
void csptrf_(const char *uplo, const int *n, float _Complex *ap, int *ipiv, int *info);
void zsptrf_(const char *uplo, const int *n, double _Complex *ap, int *ipiv, int *info);

```

B. Fortran interface:

```

SSPTRF(UPLO, N, AP, IPIV, INFO);
DSPTRF(UPLO, N, AP, IPIV, INFO);
CSPTRF(UPLO, N, AP, IPIV, INFO);
ZSPTRF(UPLO, N, AP, IPIV, INFO);

```

2.2.1.10.2 参数

参数名	类型	描述	输入/输出
UPLO	字符数	'U': 保存 A 的上三角矩阵。	输入
		'L': 保存 A 的下三角矩阵。	
N	整型数	矩阵 A 的维数, $N \geq 0$ 。	输入
AP	在 ssptrf 中为单精度浮点型数组。	输入时, 为压缩对称矩阵 A, 大小为 $N*(N+1)/2$ 。	输入/输出

参数名	类型	描述	输入/输出
	在 dsptf 中为双精度浮点型数组。	输出时，块对角矩阵 D，和用于获得因子 U 或 L 的乘法器，存储为上或下三角矩阵覆盖 A。	
	在 csptf 中为单精度复数型数组。		
	在 zsptf 中为双精度复数型数组。		
IPIV	整型数	转置数组，大小为 N，表示对角矩阵 D 的块结构和交换细节。 若 $IPIV(k) > 0$ ，第 k 与 IPIV(k) 的行列交换，且 D(k,k) 是 1*1 的对角线块。 若 $UPLO='U'$ 且 $IPIV(k)=IPIV(k-1) < 0$ ，则第 k-1 与 -IPIV(k) 的行列交换，且 D(k-1:k, k-1:k) 是 2*2 的对角线块。 若 $UPLO='L'$ 且 $IPIV(k)=IPIV(k+1) < 0$ ，则第 k+1 与 -IPIV(k) 的行列交换，且 D(k:k+1;k:k+1) 是 2*2 的对角线块。	输出
INFO	整型数	等于 0：成功退出。 小于 0：表示第 -INFO 个参数非法。 大于 0：表示 U(INFO, INFO) 为零，分解失败。	输出

2.2.1.10.3 依赖

#include "klapack.h"

2.2.1.10.4 示例

A. C interface:

```
const char uplo = 'U';  
const int n = 4;
```

```

double ap[] = {0.521739, 0.043478, 0.304348, 0.130435, 0.304348,
0.043478, 0.652174, 0.086957, 0.521739, 0.086957};

double ipiv[4];

int info = 0;

dsptf_(&uplo, &n, ap, ipiv, &info);

if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
*   ap: -0.129534   1.024339   0.302185   -0.492537   -0.044777   -
0.728261   1.250000   0.083333   0.250000   0.521739
*   ipiv: 1       2       1       1
*/

```

2.2.2 线性方程组求解函数

2.2.2.1 函数说明

函数族	简介
?ppsv	求解线性方程组 $A * X = B$。其中 A 为实对称或共轭对称的 $n*n$ 正定矩阵，以压缩方式存储；X 和 B 为 $n*nrhs$ 矩阵。计算矩阵 A 的 Cholesky 分解，并通过分解结果求解线性方程组。 具体分为以下四个函数： sppsv: 单精度浮点型 dppsv: 双精度浮点型 cppsv: 单精度复数型 zppsv: 双精度复数型

函数族	简介
?gesv	求解线性方程组$A * X = B$。其中 A 为 $n*n$ 的矩阵，X 和 B 为 $n*nrhs$ 矩阵。求解过程中，对 A 使用具有部分旋转和行交换的 LU 分解，将其分解为$A = P * L * U$的形式，其中 P 是置换矩阵，L 是单位下三角矩阵，U 是上三角矩阵。并通过 LU 分解的结果来求解线性方程组。 具体分为以下四个函数： sgesv: 单精度浮点型 dgesv: 双精度浮点型 cgesv: 单精度复数型 zgesv: 双精度复数型
?ptsv	求解线性方程组，其中系数矩阵 A 为实（共轭）对称正定三对角矩阵。 具体分为以下四个函数： sptsv: 单精度浮点型 dptsv: 双精度浮点型 cptsv: 单精度复数型 zptsv: 双精度复数型
?gtsv	求解线性方程组$A * X = B$，其中系数矩阵 A 为一般三对角矩阵。 具体分为以下四个函数： sgtsv: 单精度浮点型 dgtsv: 双精度浮点型 cgtsv: 单精度复数型 zgtsv: 双精度复数型

2.2.2.2 ?ppsv

求解线性方程组 $AX = B$ 。其中 A 为实对称或共轭对称的 $n \times n$ 正定矩阵，以压缩方式存储； X 和 B 为 $n \times n$ rhs 矩阵。计算矩阵 A 的 Cholesky 分解，并通过分解结果求解线性方程组。

2.2.2.2.1 接口定义

A. C interface:

```
void dppsv_(const char *uplo, const int *n, const int *nrhs, double *ap, double *b,
const int *ldb, int *info);
```

```
void sppsv_(const char *uplo, const int *n, const int *nrhs, float *ap, float *b, const
int *ldb, int *info);
```

```
void cppsv_(const char *uplo, const int *n, const int *nrhs, float _Complex *ap, float
_Complex *b, const int *ldb, int *info);
```

```
void zppsv_(const char *uplo, const int *n, const int *nrhs, double _Complex *ap,
double _Complex *b, const int *ldb, int *info);
```

B. Fortran interface:

```
DPPSV(uplo, n, nrhs, ap, b, ldb, info);
```

```
SPPSV(uplo, n, nrhs, ap, b, ldb, info);
```

```
CPPSV(uplo, n, nrhs, ap, b, ldb, info);
```

```
ZPPSV(uplo, n, nrhs, ap, b, ldb, info);
```

2.2.2.2.2 参数

参数名	类型	描述	输入/输出
uplo	字符型	'U': 保存 A 的上三角矩阵。	输入
		'L': 保存 A 的下三角矩阵。	
n	整型数	对称矩阵 A 的行数或列数。	输入
nrhs	整型数	右侧项的数量，即矩阵 B 的列数，要求 $nrhs \geq 0$ 。	输入
ap	在 sppsv 中为单精度浮点型数组。	调用前以压缩格式保存对称矩阵 A，见数据结构中关于压缩存储矩阵的说明。 调用后以压缩格式保存 A 的 Cholesky 分解结果 (U 或 L)。	输入/输出
	在 dppsv 中为双精度浮点型数组。		
	在 cppsv 中为单精度复数型数组。		
	在 zppsv 中为双精度复数型数组。		
b	在 sppsv 中为单精度浮点型数组。	调用前为右侧矩阵 B。 调用后为解矩阵 X。	输入/输出
	在 dppsv 中为双精度浮点型数组。		
	在 cppsv 中为单精度复数型数组。		
	在 zppsv 中为双精度复数型数组。		

参数名	类型	描述	输入/输出
ldb	整型数	B 的 leading dimension 大小，要求 $\text{ldb} \geq \max(1, n)$ 。	输入
info	整型数	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合法。 大于 0：A 中 info 大小的顺序主子式非正定，无法完成分解。	输出

2.2.2.2.3 依赖

```
#include "klapack.h"
```

2.2.2.2.4 示例

A. C interface:

```
char uplo = 'L';
int n = 5;
int nrhs = 2;
int ldb = 5;
int info = 0;
/*
 * A (stored in column-major):
 *   178.535    33.2929    9.2538    11.1156    21.5482
 *   33.2929   156.5955    5.1031    55.771    41.3063
 *    9.2538    5.1031   100.5198    73.8396    23.04
 *   11.1156    55.771    73.8396   106.5638    56.19
 *   21.5482    41.3063    23.04     56.19   114.5161
 */
double ap[] = {178.535, 33.2929, 9.2538, 11.1156, 21.5482,
               156.5955, 5.1031, 55.771, 41.3063,
               100.5198, 73.8396, 23.04,
```



```

                                106.5638, 56.19,
                                114.5161});

double b[] = {0.374, 6.242, 5.593, 2.456, 5.222,
              6.906, 2.31 , 3.948, 6.403, 5.664};

dppsv_(&uplo, &n, &nrhs, ap, b, &ldb, &info);
/*
 * Output X:
 *  -0.0186   0.0373
 *   0.0770  -0.0235
 *   0.1536  -0.0208
 *  -0.1574   0.0726
 *   0.0677   0.0195
 */

```

2.2.2.3 ?gesv

求解线性方程组 $AX = B$ 。其中 A 为 $n \times n$ 矩阵, X 和 B 为 $n \times n$ rhs 矩阵。求解过程中, 对使用具有部分旋转和行交换的 LU 分解, 将其分解为 $A = P * L * U$ 的形式, 其中 P 是置换矩阵, L 是单位下三角矩阵, U 是上三角矩阵。并通过 LU 分解的结果来求解线性方程组。

2.2.2.3.1 接口定义

A. C interface:

```
void sgesv_(const int *n,const int *nrhs,float *a,const int *lda,int *ipiv, float *b, const
int *ldb,int *info);
```

```
void dgesv_(const int *n,const int *nrhs,double *a,const int *lda,int *ipiv, double *b,
const int *ldb,int *info);
```

```
void cgesv_(const int *n,const int *nrhs,float_Complex *a,const int *lda,int *ipiv,
float_Complex *b, const int *ldb,int *info);
```

```
void zgesv_(const int *n,const int *nrhs, double_Complex *a,const int *lda,int *ipiv,
double_Complex *b, const int *ldb,int *info);
```

B. Fortran interface:

SGESV(n, nrhs, a, lda, ipiv, b, ldb, info);

DGESV(n, nrhs, a, lda, ipiv, b, ldb, info);

CGESV(n, nrhs, a, lda, ipiv, b, ldb, info);

ZGESV(n, nrhs, a, lda, ipiv, b, ldb, info);

2.2.2.3.2 参数

参数名	类型	描述	输入/输出
n	整型数	矩阵 A 的阶数，要求 $n \geq 0$ 。	输入
nrhs	整型数	右侧项的数量，即矩阵 B 的列数，要求 $nrhs \geq 0$ 。	输入
a	在 sgesv 中为单精度浮点型数组。 在 dgesv 中为双精度浮点型数组。 在 cgesv 中为单精度复数型数组。 在 zgesv 中为双精度复数型数组。	矩阵维度为(lda,n)。 输入： $n \times n$ 的系数矩阵 A。 输出： 储存了中的和，其中的单位对角线元素不存储。	输入/输出
lda	整数型	A 的 leading dimension 大小，要求 $lda \geq \max(1, n)$ 。	输入
ipiv	整数型数组	数组维度为 n。 存放了置换矩阵 P 的主元索引，即矩阵第 i 行与第 ipiv(i)行交换。	输出
b	在 sgesv 中为单精度浮点型数组。 在 dgesv 中为双精度浮点型数组。 在 cgesv 中为单精度复数型数组。 在 zgesv 中为双精度复数型数组。	矩阵维度为(ldb,nrhs)。 输入： 的右端矩阵 B。 输出： 在 info=0 的情况下，为的解矩阵 X。	输入/输出

参数名	类型	描述	输入/输出
ldb	整型数	B 的 leading dimension 大小，要求 $\text{ldb} \geq \max(1, n)$ 。	输入
info	整型数	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合法。 大于 0： $U(\text{info}, \text{info}) = 0$ 分解已经完成，但是因为奇异，所以无法完成求解。	输出

2.2.2.3.3 依赖

```
#include "klapack.h"
```

2.2.2.3.4 示例

A. C interface:

```
int n = 5;

int nrhs = 2;

int lda = 5;
int ldb = 5;
int ipiv[5];
int info = 0;

double a[]={72.1673 , 66.1857 , 64.7644 , 28.0199 , 91.4151,
            6.5180 , 62.8483 , 72.4323 , 46.5760 , 8.6928,
            28.9821 , 42.1828 , 18.6437 , 99.8612 , 35.6972,
            67.9812 , 5.0880 , 85.5035 ,79.2945 , 54.5920,
            28.6869 , 49.7512 , 7.5186 ,28.6929 , 84.6041};

double b[]={ 9.4532 , 1.5204 , 2.2127 , 0.9891, 7.1778,
            6.8955 , 7.2465, 3.5019 , 8.2268, 3.5287};

dgesv_(&n,&nrhs,a,&lda,ipiv,b,&ldb,&info);

/*
```

```

* Output:
* 0.17538061067669766      0.16637572403098155
* -0.11183914210321674    -3.7758100714851153E-002
* 5.5415093265516101E-002  0.15550950667724139
* -1.4901096204948673E-002 -7.3593964815566168E-002
* -0.10693481055391466    -0.15230871441871899
*/

```

2.2.2.4 ?ptsv

求解线性方程组，其中系数矩阵 A 为实（共轭）对称正定三对角矩阵。

2.2.2.4.1 接口定义

A. C interface:

```
void sptsv_(const int *N, const int *NRHS, float *D, float *E, float *B, const int
*LDB, int *INFO);
```

```
void dptsv_(const int *N, const int *NRHS, double *D, double *E, double *B, const
int *LDB, int *INFO);
```

```
void cptsv_(const int *N, const int *NRHS, float *D, float _Complex *E, float
_Complex *B, const int *LDB, int *INFO);
```

```
void zptsv_(const int *N, const int *NRHS, double *D, double _Complex *E, double
_Complex *B, const int *LDB, int *INFO);
```

B. Fortran interface:

```
SPTSV(N, NRHS, D, E, B, LDB, INFO);
```

```
DPTSV(N, NRHS, D, E, B, LDB, INFO);
```

```
CPTSV(N, NRHS, D, E, B, LDB, INFO);
```

```
ZPTSV(N, NRHS, D, E, B, LDB, INFO);
```

2.2.2.4.2 参数

参数名	类型	描述	输入/输出
N	整型数	矩阵 A 的阶数，要求 $n \geq 0$ 。	输入
NRHS	整型数	右侧项的数量，即矩阵 B 的列数，要求 $NRHS \geq 0$ 。	输入

参数名	类型	描述	输入/输出
D	在 sptsv/cptsv 中为单精度浮点型数组。	输入时，为三对角矩阵 A 的对角元素，维度为 N。	输入/输出
	在 dptsv/zptsv 中为双精度浮点型数组。	输出时，为分解后的对角元素 D。	
E	在 sptsv 中为单精度浮点型数组。	输入时，为三对角矩阵 A 的次对角元素，维度为 N-1。	输入/输出
	在 dptsv 中为双精度浮点型数组。	输出时，为分解后矩阵 A 的次对角元素。	
	在 cptsv 中为单精度复数型数组。		
	在 zptsv 中为双精度复数型数组。		
B	在 sptsv 中为单精度浮点型数组。	输入时，为右端项矩阵，维度为 (LDB, NRHS)。	输入/输出
	在 dptsv 中为双精度浮点型数组。	输出时，为结果 X，维度为 (LDB, NRHS)。	
	在 cptsv 中为单精度复数型数组。		
	在 zptsv 中为双精度复数型数组。		
LDB	整型数	矩阵 B 的主维。	输入
INFO	整型数	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合法。 大于 0：第 INFO 阶顺序主子式不正定。	输出

2.2.2.4.3 依赖

```
#include "klapack.h"
```

2.2.2.4.4 示例

A. C interface:

%

```

const int n = 4;
const int ldb = n;
const int nrhs = 1;

double d[] = {7.608696, 7.739130, 7.347826, 4.652174};
double e[] = {0.352941, 0.000000, 0.823529};
double b[] = {0.300000, 0.100000, 0.700000, 0.200000};

int info = 0;

dptsv_(&n, &nrhs, d, e, b, &ldb, &info);
/*
* Output:
*   d: 7.608696      7.722759      7.347826      4.559874
*   e: 0.046387      0.000000      0.112078
*   b: 0.038912      0.011147      0.092279      0.026655
*/

```

2.2.2.5 ?gtsv

求解线性方程组 $AX = B$ 。其中系数矩阵 A 为一般三对角矩阵， X 和 B 为 $n \times n$ rhs 矩阵。

2.2.2.5.1 接口定义

A. C interface:

```
void sgtsv(const int *N, const int *NRHS, float *DL, float *D, float *DU, float *B,
const int *LDB, int *INFO);
```

```
void dgtsv(const int *N, const int *NRHS, double *DL, double *D, double *DU,
double *B, const int *LDB, int *INFO);
```

```
void cgtsv(const int *N, const int *NRHS, float _Complex *DL, float _Complex *D,
float _Complex *DU, float _Complex *B, const int *LDB, int *INFO);
```

```
void zgtsv(const int *N, const int *NRHS, double _Complex *DL, double _Complex
*D, double _Complex *DU, double _Complex *B, const int *LDB, int *INFO);
```

B. Fortran interface:

SGTSV(N, NRHS, DL, D, DU, B, LDB, INFO);

DGTSV(N, NRHS, DL, D, DU, B, LDB, INFO);

CGTSV(N, NRHS, DL, D, DU, B, LDB, INFO);

ZGTSV(N, NRHS, DL, D, DU, B, LDB, INFO);

2.2.2.5.2 参数

参数名	类型	描述	输入/输出
N	整型数	矩阵 A 的阶数，要求 $n \geq 0$ 。	输入
NRHS	整型数	右侧项的数量，即矩阵 B 的列数，要求 $NRHS \geq 0$ 。	输入
DL	在 sgtsv 中为单精度浮点型数组。	输入时，为三对角矩阵 A 的次对角元素，维度为 N-1。 输出时，为 U 的第二超对角元素。	输入/输出
	在 dgtsv 中为双精度浮点型数组。		
	在 cgtsv 中为单精度复数型数组。		
	在 zgtsv 中为双精度复数型数组。		
D	在 sgtsv 中为单精度浮点型数组。	输入时，为三对角矩阵 A 的对角元素，维度为 N。 输出时，为 U 的对角元素。	输入/输出
	在 dgtsv 中为双精度浮点型数组。		
	在 cgtsv 中为单精度复数型数组。		
	在 zgtsv 中为双精度复数型数组。		
DU	在 sgtsv 中为单精度浮点型数组。	输入时，为三对角矩阵 A 的超对角元素，维度为 N-1。 输出时，为 U 的第一超对角元素。	输入/输出
	在 dgtsv 中为双精度浮点型数组。		
	在 cgtsv 中为单精度复数型数组。		

参数名	类型	描述	输入/输出
	在 zgtsv 中为双精度复数型数组。		
B	在 sgesv 中为单精度浮点型数组。	矩阵维度为(ldb,nrhs)。 输入：的右端矩阵 B。 输出：在 info=0 的情况下，为的解矩阵 X。	输入/输出
	在 dgesv 中为双精度浮点型数组。		
	在 cgesv 中为单精度复数型数组。		
	在 zgesv 中为双精度复数型数组。		
LDB	整型数	矩阵 B 的主维。	输入
INFO	整型数	执行结果： 等于 0：成功。 小于 0：第-INFO 个参数值不合法。 大于 0： $U(info,info) = 0$	输出

2.2.2.5.3 依赖

```
#include "klapack.h"
```

2.2.2.5.4 示例

A. C interface:

```
const int n = 4;
const int ldb = n;
const int nrhs = 1;
double dl[] = {0.923077, 0.153846, 0.153846};
double d[] = {0.521739, 0.043478, 0.304348, 0.086957};
double du[] = {0.924528, 0.150943, 0.566038};
double b[] = {0.986301, 0.397260, 0.958904, 0.520548};
int info = 0;
```



```
dgtsv_(&n, &nrhs, dl, d, du, b, &ldb, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
*   d:  0.923077      0.899954      0.318932      -0.186088
*   dl: 0.150943      0.000000      0.153846
*   du: 0.043478      -0.085316      0.566038
*   b:  -0.239541      1.201996      3.750506      -0.649209
*/
```

2.2.3 矩阵求逆函数

2.2.3.1 函数说明

函数族	简介
?getri	根据?getrf得到的 LU 分解结果计算逆矩阵。 具体分为以下四个函数： sgetri: 单精度浮点型 dgetri: 双精度浮点型 cgetri: 单精度复数型 zgetri: 双精度复数型
?potri	计算对称正定矩阵的逆。 具体分为以下四个函数： spotri: 单精度浮点型 dpotri: 双精度浮点型 cpotri: 单精度复数型 zpotri: 双精度复数型

2.2.3.2 ?getri

根据?getrf得到的 LU 分解结果计算逆矩阵。

2.2.3.2.1 接口定义

A. C interface:

```
void sgetri_(const int *n, float *a, const int *lda, const int *ipiv, float *work, const int *lwork, int *info);
```

```
void dgetri_(const int *n, double *a, const int *lda, const int *ipiv, double *work, const int *lwork, int *info);
```

```
void cgetri_(const int *n, float _Complex *a, const int *lda, const int *ipiv, float _Complex *work, const int *lwork, int *info);
```

```
void zgetri_(const int *n, double _Complex *a, const int *lda, const int *ipiv, double _Complex *work, const int *lwork, int *info);
```

B. Fortran interface:

```
SGETRI(n, a, lda, ipiv, work, lwork, info)
```

```
DGETRI(n, a, lda, ipiv, work, lwork, info)
```

```
CGETRI(n, a, lda, ipiv, work, lwork, info)
```

```
ZGETRI(n, a, lda, ipiv, work, lwork, info)
```

2.2.3.2.2 参数

参数名	类型	描述	输入/输出
n	整型数	矩阵 A 的行数或列数。	输入
a	在 sgetri 中为单精度浮点型数组。	调用前保存矩阵 A 的 LU 分解结果（由?getrf 得到）。调用后保存原矩阵 A 的逆矩阵。	输入/输出
	在 dgetri 中为双精度浮点型数组。		
	在 cgetri 中为单精度复数型数组。		
	在 zgetri 中为双精度复数型数组。		
lda	整型数	A 的 leading dimension 大小，要求 $lda \geq \max(1, n)$ 。	输入
ipiv	整型数	?getrf 得到的主元索引，长度为 n: $1 \leq ipiv \leq n$ ，表示分解中矩阵第 i 行与第 ipiv[i-1]行交换。	输入

参数名	类型	描述	输入/输出
work	在 sgetri 中为单精度浮点型数组。	临时存储空间，使用 lwork=-1 调用后 work[0]为最优的 lwork 值。	输出
	在 dgetri 中为双精度浮点型数组。		
	在 cgetri 中为单精度复数型数组。		
	在 zgetri 中为双精度复数型数组。		
lwork	整型数	work 数组的长度。 lwork=-1 时查询最优 work 大小，结果保存在 work[0]中，否则要求 lwork $\geq$ n。	输入
info	整型数	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合法。 大于 0： $U(info,info) = 0$ ，矩阵不可逆。	输出

2.2.3.2.3 依赖

```
#include "klapack.h"
```

2.2.3.2.4 示例

A. C interface:

```
int n = 4;

int lda = 4;

int info = 0;

double *work = NULL;

double qwork;

int lwork = -1;

/*
 * Origin A:
```

```

*   1.80   2.88   2.05  -0.89
*   5.25  -2.95  -0.95  -3.80
*   1.58  -2.69  -2.90  -1.04
*  -1.11  -0.66  -0.59   0.80

* After LU factorization (via getrf, stored in column-major):
*   5.2500  -2.9500  -0.9500  -3.8000
*   0.3429   3.8914   2.3757   0.4129
*   0.3010  -0.4631  -1.5139   0.2948
*  -0.2114  -0.3299   0.0047   0.1314
*/

double a[] = {5.2500, 0.3429,  0.3010, -0.2114,
              -2.9500, 3.8914, -0.4631, -0.3299,
              -0.9500, 2.3757, -1.5139,  0.0047,
              -3.8000, 0.4129,  0.2948,  0.1314};

int ipiv[] = {2, 2, 3, 4};

/* Query optimal work size */
dgetri_(&n, a, &lدا, ipiv, &qwork, &lwork, &info);

if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);

/* Calculate inversion */
dgetri_(&n, a, &lدا, ipiv, work, &lwork, &info);
free(work);

/*
* Output:
*   1.7718  0.5753  0.0844  4.8145
*  -0.1174 -0.4455  0.4113 -1.7122
*   0.1798  0.4526 -0.6675  1.4820
*   2.4941  0.7644 -0.0358  7.6104

```

*/

2.2.3.3 ?potri

计算对称正定矩阵的逆。

2.2.3.3.1 接口定义

A. C interface:

```
void spotri_(const char *UPLO, const int *N, const float *A, const int *LDA, int
*INFO);
```

```
void dpotri_(const char *UPLO, const int *N, const double *A, const int *LDA, int
*INFO);
```

```
void cpotri_(const char *UPLO, const int *N, const float _Complex *A, const int
*LDA, int *INFO);
```

```
void zpotri_(const char *UPLO, const int *N, const double _Complex *A, const int
*LDA, int *INFO);
```

B. Fortran interface:

```
SPOTRI(UPLO, N, A, LDA, INFO);
```

```
DPOTRI(UPLO, N, A, LDA, INFO);
```

```
CPOTRI(UPLO, N, A, LDA, INFO);
```

```
ZPOTRI(UPLO, N, A, LDA, INFO);
```

2.2.3.3.2 参数

参数名	类型	描述	输入/输出
UPLO	字符型	'U': 保存 A 的上三角矩阵。	输入
		'L': 保存 A 的下三角矩阵。	
N	整型数	矩阵 A 的维数, $n \geq 0$ 。	输入
A	在 spotri 中为单精度浮点型数组。	调用前保存矩阵 A 的 Cholesky 分解结果 (由?potrf 得到)。 调用后保存原矩阵 A 的逆矩阵。	输入/输出
	在 dpotri 中为双精度浮点型数组。		
	在 cpotri 中为单精度复数型数组。		

参数名	类型	描述	输入/输出
	在 zpotri 中为双精度复数型数组。		
LDA	整型	A 的 leading dimension 大小，lda $\geq \max(1,n)$ 。	输入
INFO	整型数	执行结果： 等于 0：成功。 小于 0：第-INFO 个参数值不合法。 大于 0： $U(info,info) = 0$, 矩阵不可逆。	输出

2.2.3.3.3 依赖

```
#include "klapack.h"
```

2.2.3.3.4 示例

A.C interface:

```
char uplo = 'L'
int n = 4;
int lda = n;
int info = 0;
/*
* A: After Cholesky factorization (via potrf, stored in column-major):
* 231.8009    33.9545    9.4153    11.1156
* 33.9454    162.2902    6.6684    53.2023
* 9.4153     6.6684   100.3220    71.5384
* 11.1156    53.2023    71.5384   123.4425
```

```

*/
double a[] = {231.8009,   33.9545,   9.4143,   11.1156,
              33.9495,  162.2902,   6.6684,   53.2033,
              9.4143,   6.6684,  100.322,   71.5384,
              11.1156,   53.2033,   71.5384,  106.5638 };

dpotri_(&uplo, &n, a, &lda, &info);
/*
* Output:
*   0.000020  -0.000006  -0.000005   0.000002
*  33.949500   0.000046   0.000015  -0.000026
*   9.414300   6.668400   0.000144  -0.000063
*  11.115600  53.203300  71.538400   0.000088
*/

```

2.2.4 回代求解函数

2.2.4.1 函数说明

函数族	简介
?pttrs	求解三对角方程 $AX=B$，其中系数矩阵 A 由?pttrf 分解而来。 具体分为以下四个函数： spttrs: 单精度浮点型 dpttrs: 双精度浮点型 cpttrs: 单精度复数型 zpttrs: 双精度复数型
?ptts2	求解三对角方程 $AX=B$，其中系数矩阵 A 由?pttrf 分解而来。 具体分为以下四个函数： sptts2: 单精度浮点型 dptts2: 双精度浮点型 cptts2: 单精度复数型 zptts2: 双精度复数型

函数族	简介
?gttrs	求解三对角方程 $A * X = B$，或 $A^H * T * X = B$，或 $A^H * H * X = B$，其中系数矩阵 A 由?gttrf 分解而来。 具体分为以下四个函数： sgttrs: 单精度浮点型 dgttrs: 双精度浮点型 cgttrs: 单精度复数型 zgttrs: 双精度复数型
?trtrs	求解三角方程 $A * X = B$，或 $A^H * T * X = B$。 具体分为以下四个函数： strtrs: 单精度浮点型 dtrtrs: 双精度浮点型 ctrtrs: 单精度复数型 ztrtrs: 双精度复数型

2.2.4.2 ?pttrs

求解三对角方程 $AX=B$ ，其中系数矩阵 A 由?pttrf 分解而来。

2.2.4.2.1 接口定义

A. C interface:

```
void spttrs_(const int *N, const int *NRHS, float *D, float *E, float *B, const int
*LDB, int *INFO);
```

```
void dpttrs_(const int *N, const int *NRHS, double *D, double *E, double *B, const
int *LDB, int *INFO);
```

```
void cpttrs_(const char *UPLO, const int *N, const int *NRHS, float *D, float
_Complex *E, float _Complex *B, const int *LDB, int *INFO);
```

```
void zpttrs_(const char *UPLO, const int *N, const int *NRHS, double *D, double
_Complex *E, double _Complex *B, const int *LDB, int *INFO);
```

B. Fortran interface:

```
SPTTRS(N, NRHS, D, E, B, LDB, INFO);
```

```
DPTTRS(N, NRHS, D, E, B, LDB, INFO);
```

```
CPTTRS(UPLO, N, NRHS, D, E, B, LDB, INFO);
```

```
ZPTTRS(UPLO, N, NRHS, D, E, B, LDB, INFO);
```


2.2.4.2.2 参数

参数名	类型	描述	输入/输出
UPLO	字符型	'U': $A=U^{**}H^{*}D^{*}U$	输入
		'L': $A=L^{*}D^{*}L^{**}H$	
N	整型数	矩阵 A 的维数， $N \geq 0$ 。	输入
NRHS	整型数	矩阵 B 的列数， $NRHS \geq 0$ 。	输入
D	在 spttrs/cpttrs 中为单精度浮点型数组。	三对角矩阵 A 的对角元素，维度为 N。	输入
	在 dpttrs/zpttrs 中为双精度浮点型数组。		
E	在 spttrs 中为单精度浮点型数组。	三对角矩阵 A 的次对角元素，维度为 N-1。	输入
	在 dpttrs 中为双精度浮点型数组。		
	在 cpttrs 中为单精度复数型数组。		
	在 zpttrs 中为双精度复数型数组。		
B	在 spttrs 中为单精度浮点型数组。	输入时，为右端项矩阵，维度为 (LDB, NRHS)。	输入/输出
	在 dpttrs 中为双精度浮点型数组。	输出时，为解 X，维度为(LDB, NRHS)。	
	在 cpttrs 中为单精度复数型数组。		
	在 zpttrs 中为双精度复数型数组。		
LDB	整数型	矩阵 B 的主维。	输入
INFO	整型数	执行结果： 等于 0：成功。 小于 0：第-INFO 个参数值不合法。 大于 0： $U(info,info) = 0$, 矩阵不可逆。	输出

2.2.4.2.3 依赖

```
#include "klapack.h"
```

2.2.4.2.4 示例

A. C interface:

```
const int n = 4;
const int ldb = n;
const int nrhs = 1;
double d[] = {7.608696, 7.739130, 7.347826, 4.652174};
double e[] = {0.352941, 0.000000, 0.823529};
double b[] = {0.300000, 0.100000, 0.700000, 0.200000};
int info = 0;

dpttrs_(&n, &nrhs, d, e, b, &ldb, &info);
/*
* Output:
*   b: 0.039697      -0.000760      0.161909      -0.080924
*/
```

2.2.4.3 ?pttrs2

求解三对角方程 $AX=B$ ，其中系数矩阵 A 由?pttrf 分解而来。

2.2.4.3.1 接口定义

A. C interface:

```
void sptts2_(const int *N, const int *NRHS, float *D, float *E, float *B, const int
*LDB);
```

```
void dptts2_(const int *N, const int *NRHS, double *D, double *E, double *B, const
int *LDB);
```

```
void cptts2_(const int *IUPLO, const int *N, const int *NRHS, float *D, float
_Complex *E, float _Complex *B, const int *LDB);
```

```
void zptts2_(const int *IUPLO, const int *N, const int *NRHS, double *D, double
_Complex *E, double _Complex *B, const int *LDB);
```

B. Fortran interface:

```
SPTTS2(N, NRHS, D, E, B, LDB);
```

```
DPTTS2(N, NRHS, D, E, B, LDB);
```

```
CPTTS2(IUPLO, N, NRHS, D, E, B, LDB);
```

```
ZPTTS2(IUPLO, N, NRHS, D, E, B, LDB);
```

2.2.4.3.2 参数

参数名	类型	描述	输入/输出
IUPLO (复数 类型特 有)	整数型	1: $A=U^{**H}*D*U$	输入
		0: $A=L*D*L^{**H}$	
N	整数型	矩阵 A 的维数, $N \geq 0$ 。	输入
NRHS	整数型	矩阵 B 的列数, $NRHS \geq 0$ 。	输入
D	在 sptts2/cptts2 中为 单精度浮点型数组。	三对角矩阵 A 的对角元素, 维 度为 N。	输入
	在 dptts2/zptts2 中为 双精度浮点型数组。		
E	在 spttrs2 中为单精度 浮点型数组。	三对角矩阵 A 的次对角元素, 维度为 N-1。	输入
	在 dpttrs2 中为双精 度浮点型数组。		
	在 cpttrs2 中为单精 度复数型数组。		
	在 zpttrs2 中为双精 度复数型数组。		
B	在 spttrs2 中为单精度 浮点型数组。	输入时, 为右端项矩阵, 维度为 (LDB, NRHS)。 输出时, 为解 X, 维度为(LDB, NRHS)。	输入/输出
	在 dpttrs2 中为双精 度浮点型数组。		
	在 cpttrs2 中为单精 度复数型数组。		

参数名	类型	描述	输入/输出
	在 <code>zpttrs2</code> 中为双精度复数型数组。		
LDB	整数型	矩阵 <code>B</code> 的主维。	输入

2.2.4.3.3 依赖

```
#include "klapack.h"
```

2.2.4.3.4 示例

A. C interface:

```
const int n = 4;
const int ldb = n;
const int nrhs = 1;
double d[] = {7.608696,7.739130, 7.347826,4.652174};
double e[] = {0.352941,0.000000,0.823529};
double b[] = {0.300000,0.100000,0.700000,0.200000};

dptts2_(&n, &nrhs, d, e, b, &ldb);

/*
 * Output:
 * b: 0.039697      -0.000760      0.161909      -0.080924
 */
```

2.2.4.4 ?gttrs

求解三对角方程 $A * X = B$ ，或 $A^{**T} * X = B$ ，或 $A^{**H} * X = B$ ，其中系数矩阵 `A` 由?gttrf 分解而来。

2.2.4.4.1 接口定义

A. C interface:

```
void sgtrtrs(const char *TRANS, const int *N, const int *NRHS, float *DL, float *D,
float *DU, float *DU2, int *IPIV, float *B, const int *LDB, int *INFO);
```

```

void dgttrs(const char *TRANS, const int *N, const int *NRHS, double *DL, double
*D, double *DU, double *DU2, int *IPIV, double *B, const int *LDB, int *INFO);

void cgttrs(const char *TRANS, const int *N, const int *NRHS, float _Complex *DL,
float _Complex *D, float _Complex *DU, float _Complex *DU2, int *IPIV, float
_Complex *B, const int *LDB, int *INFO);

void zgttrs(const char *TRANS, const int *N, const int *NRHS, double _Complex
*DL, double _Complex *D, double _Complex *DU, double _Complex *DU2, int
*IPIV, double _Complex *B, const int *LDB, int *INFO);

```

B. Fortran interface:

```

SGTTRS(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO);
DGTTRS(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO);
CGTTRS(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO);
ZGTTRS(TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO);

```

2.2.4.4.2 参数

参数名	类型	描述	输入/输出
TRANS	字符型	'N', $A * X = B$	输入
		'T', $A ** T * X = B$	
		'C', $A ** T * X = B$ (共轭转置)	
N	整数型	矩阵 A 的维数, $N \geq 0$ 。	输入
NRHS	整数型	矩阵 B 的列数, $NRHS \geq 0$ 。	输入
DL	在 sgttrs 中为单精度浮点型数组。	LU 分解中的 L 部分, 维度为 N-1。	输入
	在 dgttrs 中为双精度浮点型数组。		
	在 cgttrs 中为单精度复数型数组。		
	在 zgttrs 中为双精度复数型数组。		
D	在 sgttrs 中为单精度浮点型数组。	U 的对角元素, 维度为 N。	输入
	在 dgttrs 中为双精度浮点型数组。		

参数名	类型	描述	输入/输出
	在 <code>cgtrfs</code> 中为单精度复数型数组。		
	在 <code>zgtrfs</code> 中为双精度复数型数组。		
DU2	在 <code>sgtrfs</code> 中为单精度浮点型数组。	为 U 的第二超对角元素。	输入
	在 <code>dgttrs</code> 中为双精度浮点型数组。		
	在 <code>cgtrfs</code> 中为单精度复数型数组。		
	在 <code>zgtrfs</code> 中为双精度复数型数组。		
IPIV	整数型数组	转置数组。	输入
B	在 <code>sgtrfs</code> 中为单精度浮点型数组。	右端项矩阵，维度为(LDB, NRHS)。	输入/输出
	在 <code>dgttrs</code> 中为双精度浮点型数组。		
	在 <code>cgtrfs</code> 中为单精度复数型数组。		
	在 <code>zgtrfs</code> 中为双精度复数型数组。		
LDB	整数型	矩阵 B 的主维。	输入
INFO	整数型	等于 0：成功退出。 小于 0：表示第-INFO 个参数非法。	输出

2.2.4.4.3 依赖

```
#include "klapack.h"
```

2.2.4.4.4 示例

A. C interface:

```
const char trans = 'N';
```

```

const int n = 4;
const int ldb = n;
const int nrhs = 1;
double dl[] = {0.565217, 0.170949, 0.482378};
double d[] = {0.923077, 0.899954, 0.318932, -0.186088};
double du[] = {0.043478, -0.085316, 0.566038};
double du2[] = {0.150943, 0.000000};
double b[] = {0.301370, 0.835616, 0.109589, 0.041096};
int ipiv[] = {2,2,3,4};
int info = 0;
dgttrs_(&trans, &n, &nrhs, dl, d, du, du2, ipiv, b, &ldb, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

/*
* Output:
*   b:  0.882521      -0.172061      0.188567      0.138984
*/

```

2.2.4.5 ?trtrs

求解三角方程 $A * X = B$, 或 $A^{**T} * X = B$ 。

2.2.4.5.1 接口定义

A. C interface:

```
void dtrtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const
int *nrhs, const double *a, const int *lda, double *b, const int *ldb, int *info);
```

```
void strtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const
int *nrhs, const float *a, const int *lda, float *b, const int *ldb, int *info);
```

```
void ctrtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const
int *nrhs, const float _Complex *a, const int *lda, float _Complex *b, const int *ldb,
int *info);
```

```
void ztrtrs_(const char *uplo, const char *trans, const char *diag, const int *n, const
int *nrhs, const double _Complex *a, const int *lda, double _Complex *b, const int
*ldb, int *info);
```

B. Fortran interface:

DTRTRS(UPLO, TRANS, DIAG, N, NRHS, A, LDA, B, LDB, INFO);

STRTRS(UPLO, TRANS, DIAG, N, NRHS, A, LDA, B, LDB, INFO);

CTRTRS(UPLO, TRANS, DIAG, N, NRHS, A, LDA, B, LDB, INFO);

ZTRTRS(UPLO, TRANS, DIAG, N, NRHS, A, LDA, B, LDB, INFO);

2.2.4.5.2 参数

参数名	类型	描述	输入/输出
UPLO	字符型	'U', A 是上三角矩阵。	输入
		'L', A 是下三角矩阵。	
TRANS	字符型	'N', $A * X = B$	输入
		'T', $A ** T * X = B$	
		'C', $A ** T * X = B$ (共轭转置)	
DIAG	字符型	'N', A 是非单位三角矩阵。	输入
		'U', A 是单位三角矩阵。	
N	整数型	矩阵 A 的维数, $N \geq 0$ 。	输入
NRHS	整数型	矩阵 B 的列数, $NRHS \geq 0$ 。	输入
A	在 strtrs 中为单精度浮点型数组。	三角矩阵 A, 大小为(LDA, N)。	输入
	在 dtrtrs 中为双精度浮点型数组。		
	在 ctrtrs 中为单精度复数型数组。		
	在 ztrtrs 中为双精度复数型数组。		
LDA	整数型	矩阵 A 的主维, $LDA \geq \max(1, N)$ 。	输入
B	在 strtrs 中为单精度浮点型数组。	输入时, 为右端矩阵 B, 大小为(LDB, NRHS)。	输入/输出
	在 dtrtrs 中为双精度浮点型数组。	输出时, 若 INFO = 0, 为方程的解矩阵 X。	

参数名	类型	描述	输入/输出
	在 <code>ctrtrs</code> 中为单精度复数型数组。		
	在 <code>ztrtrs</code> 中为双精度复数型数组。		
LDB	整数型	右端矩阵 B 的主维, $LDB \geq \max(1, N)$ 。	输入
INFO	整数型	等于 0: 成功。 小于 0: 第-INFO 个参数值不合法。 大于 0: 矩阵 A 的第 INFO 个对角元素为 0, 为奇异矩阵。	输出

2.2.4.5.3 依赖

```
#include "klapack.h"
```

2.2.4.5.4 示例

A. C interface:

```
const char uplo = 'L';
const char trans = 'N';
const char diag = 'N';
const int n = 4;
const int lda = n;
const int ldb = n;
const int nrhs = 1;

double *a = (double*)malloc(lda * n * sizeof(double));
double *b = (double*)malloc(ldb * nrhs * sizeof(double));
int info = 0;

// Generate Matrix A.
for (int i = 0; i < n; i++) {
```

```

    for (int j = i; j < lda; j++) {
        a[i * lda + j] = (rand() % 53 + 1) / 53.0;
    }
}

// Generate right hand side matrix B.
for (int i = 0; i < ldb * nrhs; i++) {
    b[i] = (rand() % 23 + 1) / 23.0;
}

// Call dtrtrs.
dtrtrs_(&uplo, &trans, &diag, &n, &nrhs, a, &lda, b, &ldb, &info);
if (info != 0) {
    printf("ERROR, info = %d\n", info);
}

```

2.2.5 特征值问题函数

2.2.5.1 函数说明

函数族	简介
?sy(he)evd	计算对称（Hermite）矩阵的全部特征值和特征向量（可选）。其中特征向量通过分治算法计算。即矩阵 A 分解为：$A = Z\Lambda Z^T$, 其中 Λ 为对角矩阵，对角线元素 λ_i 为特征值；Z 为正交矩阵，其每列向量 Z_i 为对应的特征向量。即 $Az_i = \lambda_i z_i$，其中 $i=1,2,\dots,n$。 具体分为以下四个函数： ssyevd: 单精度浮点型 dsyevd: 双精度浮点型 cheevd: 单精度复数型 zheevd: 双精度复数型
?sy(he)ev	计算对称（Hermite）矩阵的所有特征值与特征向量。 具体分为以下四个函数： ssyev: 单精度浮点型 dsyev: 双精度浮点型 cheev: 单精度复数型 zheev: 双精度复数型

2.2.5.2 ?sy(he)evd

计算对称（Hermite）矩阵的全部特征值和特征向量（可选）。其中特征向量通过分治算法计算。

即矩阵 A 分解为： $A = Z\Lambda Z^T$ ，其中 Λ 为对角矩阵，对角线元素 λ_i 为特征值； Z 为正交矩阵，其每列向量 Z_i 为对应的特征向量。即 $Az_i = \lambda_i z_i$ ，其中 $i = 1, 2, \dots, n$ 。

2.2.5.2.1 接口定义

A. C interface:

```
void dsyevd_(const char *jobz, const char *uplo, const int *n, double *a, const int *lda, double *w, double *work, const int *lwork, int *iwork, const int *liwork, int *info);
```

```
void ssyevd_(const char *jobz, const char *uplo, const int *n, float *a, const int *lda, float *w, float *work, const int *lwork, int *iwork, const int *liwork, int *info);
```

```
void cheevd_(const char *jobz, const char *uplo, const int *n, float _Complex *a, const int *lda, float *w, float _Complex *work, const int *lwork, float *rwork, const int *lrwork, int *iwork, const int *liwork, int *info);
```

```
void zheevd_(const char *jobz, const char *uplo, const int *n, double _Complex *a, const int *lda, double *w, double _Complex *work, const int *lwork, double *rwork, const int *lrwork, int *iwork, const int *liwork, int *info);
```

B. Fortran interface:

```
DSYEVD(JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, IWORK, LIWORK, INFO);
```

```
SSYEVD(JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, IWORK, LIWORK, INFO);
```

```
CHEEVD(JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, RWORK, LRWORK, IWORK, LIWORK, INFO);
```

```
ZHEEVD(JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, RWORK, LRWORK, IWORK, LIWORK, INFO);
```

2.2.5.2.2 参数

参数名	类型	描述	输入/输出
jobz	字符型	'N': 只计算特征向量。	输入

参数名	类型	描述	输入/输出
		'V': 同时计算特征向量和特征值。	
uplo	字符型	'U': A 中保存上三角矩阵。	输入
		'L': A 中保存下三角矩阵。	
n	整数型	对称矩阵 A 的行数或列数。	输入
a	在 ssyevd 中为单精度浮点型数组。	调用前保存待分解的对称矩阵。 调用后保存特征向量。	输入/输出
	在 dsyevd 中为双精度浮点型数组。		
	在 cheevd 中为单精度复数型数组。		
	在 zheevd 中为双精度复数型数组。		
lda	整数型	矩阵 A 的主维, $lda \geq \max(1, N)$ 。	输入
w	在 ssyevd/cheevd 中为单精度浮点型数组。	按升序排列的特征值, 长度为 n。	输出
	在 dsyevd/zheevd 中为双精度浮点型数组。		
work	在 ssyevd 中为单精度浮点型数组。	临时存储空间, 调用后 work[0] 为最优的 lwork 值。	输出
	在 dsyevd 中为双精度浮点型数组。		
	在 cheevd 中为单精度复数型数组。		
	在 zheevd 中为双精度复数型数组。		

参数名	类型	描述	输入/输出
lwork	整数型	work 数组的长度。 lwork=-1 时查询最优 work 大小，结果保存在 work[0]中，否则： 当 $n \leq 1$ 时， $lwork \geq 1$ 。 当 jobz='N' 且 $n > 1$ 时， $lwork \geq 2*n+1$ 。 当 jobz='V' 且 $n > 1$ 时， $lwork \geq 1+6*n+2*n*n$ 。	输入
rwork (复数类型特有)	在 cheevd 中为单精度实数。 在 zheevd 中为双精度实数。	临时存储空间，调用后 work[0] 为最优的 lwork 值。	输出
lrwork (复数类型特有)	在 cheevd 中为单精度复数型数组。 在 zheevd 中为双精度复数型数组。	rwork 数组的长度。 lwork=-1 时查询最优 work 大小，结果保存在 work[0]中，否则： 当 $n \leq 1$ 时， $lwork \geq 1$ 。 当 jobz='N' 且 $n > 1$ 时， $lwork \geq 2*n+1$ 。 当 jobz='V' 且 $n > 1$ 时， $lwork \geq 1+6*n+2*n*n$ 。	输入
iwork	整数型数组	临时存储空间，使用 lwork=-1 调用后 iwork[0]为最优的 liwork 值。	输出

参数名	类型	描述	输入/输出
liwork	整数型	iwork 数组的长度。 liwork=-1 时查询最优 iwork 大小，结果保存在 iwork[0]中，否则： 当 jobz='N'或 $n \leq 1$ 时，liwork ≥ 1 。 当 jobz='V'且 $n > 1$ 时，liwork $\geq 3+5*n$ 。	输入
info	整数型	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合法。 大于 0：jobz='N'算法无法收敛，无法计算第 info/(n+1)个特征值。	输出

2.2.5.2.3 依赖

#include "klapack.h"

2.2.5.2.4 示例

A. C interface:

```
char jobz = 'V';
char uplo = 'L';
int n = 5;
int lda = 5;
int info = 0;
double w[5];
double *work = NULL;
double qwork;
int lwork = -1;
int *iwork = NULL;
int qiwork;
```

```

int liwork = -1;

/*
 * Symmetric A (stored in column-major):
 *
 * 7.027  8.710  1.015  6.929  7.584
 *
 * 8.710  0.839  2.469  3.850  0.559
 *
 * 1.015  2.469  1.930  6.761  7.207
 *
 * 6.929  3.850  6.761  4.344  4.804
 *
 * 7.584  0.559  7.207  4.804  6.177
 */

double a[] = {7.027, 8.710, 1.015, 6.929, 7.584,
               8.710, 0.839, 2.469, 3.850, 0.559,
               1.015, 2.469, 1.930, 6.761, 7.207,
               6.929, 3.850, 6.761, 4.344, 4.804,
               7.584, 0.559, 7.207, 4.804, 6.177};

/* Query optimal work size */
dsyevd_(&jobz, &uplo, &n, a, &lda, w, &qwork, &lwork, &qwork,
&liwork, &info);

if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);

liwork = (int)qiwork;
iwork = (int *)malloc(sizeof(int) * liwork);

/* Calculate eigenvalues and eigenvectors */
dsyevd_(&jobz, &uplo, &n, a, &lda, w, work, &lwork, iwork, &liwork,
&info);

free(work);
free(iwork);

/*
 * Output:
 *
 * Eigenvalues (in w)

```

```
* -8.8422 -3.3411 1.1888 6.2050 25.1065
* Eigenvectors (in a, stored in column-major)
* 0.5405 -0.1616 -0.3059 -0.5235 -0.5604
* -0.4913 0.5975 0.2432 -0.4882 -0.3228
* 0.4886 0.4989 0.3454 0.5213 -0.3482
* -0.2410 -0.6062 0.5831 0.1038 -0.4729
* -0.4120 0.0219 -0.6228 0.4528 -0.4867
*/
```

2.2.5.3 ?sy(he)ev

计算对称（Hermite）矩阵的全部特征值和特征向量。

2.2.5.3.1 接口定义

A. C interface:

```
void ssyev_(const char *jobz, const char *uplo, const int *n, float *A, const int *lda,
float *w, float *work, const int *lwork, int * info);
```

```
void dsyev_(const char *jobz, const char *uplo, const int *n, double *A, const int
*lda, double *w, double *work, const int *lwork, int * info);
```

```
void cheev_(const char *jobz, const char *uplo, const int *n, float _Complex *A,
const int *lda, float *w, float _Complex *work, const int *lwork, float* rwork, int *
info);
```

```
void zheev_(const char *jobz, const char *uplo, const int *n, double _Complex *A,
const int *lda, double *w, double _Complex *work, const int *lwork, double* rwork,
int * info);
```

B. Fortran interface:

```
SSYEV(jobz, uplo, n, a, lda, w, work, lwork, info);
```

```
DSYEV(jobz, uplo, n, a, lda, w, work, lwork, info);
```

```
CHEEV(jobz, uplo, n, a, lda, w, work, lwork, rwork, info);
```

```
ZHEEV(jobz, uplo, n, a, lda, w, work, lwork, rwork, info);
```

2.2.5.3.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
jobz	字符型	'N': 只计算特征向量。	输入
		'V': 同时计算特征向量和特征值。	
uplo	字符型	'U': A 中保存上三角矩阵。	输入
		'L': A 中保存下三角矩阵。	
n	整数型	对称矩阵 A 的行数或列数。	输入
a	在 ssyev 中为单精度浮点型数组。	调用前保存待分解的对称矩阵。 调用后保存特征向量。	输入/输出
	在 dsyev 中为双精度浮点型数组。		
	在 cheev 中为单精度复数型数组。		
	在 zheev 中为双精度复数型数组。		
lda	整数型	矩阵 A 的主维, $lda \geq \max(1, n)$ 。	输入
w	在 ssyev/cheev 中为单精度浮点型数组。	Info=0 时, 以升序存储特征值, 长度为 N。	输出
	在 dsyev/zheev 中为双精度浮点型数组。		
work	在 ssyev 中为单精度浮点型数组。	Info=0 时, work[0]返回最优 lwork 大小。	输出
	在 dsyev 中为双精度浮点型数组。		
	在 cheev 中为单精度复数型数组。		
	在 zheev 中为双精度复数型数组。		
lwork	整数型	work 数组的长度。 lwork=-1 时查询最优 work 大小, 结果保存在 work[0]中, 且 $lwork \geq \max(1, 2*n-1)$ 。	输入
rwork (复数)	在 ssyevd/cheevd 中为单精度浮点型数组。	工作数组, 取值 $\max(1, 3*n-1)$ 。	输出

参数名	类型	描述	输入/输出
类型特 有)	在 dsyevd/zheevd 中为 双精度浮点型数组。		
info	整数型	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合 法。 大于 0：jobz='N'算法无法收 敛，无法计算第 info/(n+1)个特 征值。	输出

2.2.5.3.3 依赖

#include "klapack.h"

2.2.5.3.4 示例

A. C interface:

```
char jobz = 'V';
char uplo = 'L';
int n = 5;
int lda = 5;
int info = 0;
double w[5];
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * Symmetric A (stored in column-major):
 *   7.027  8.710  1.015  6.929  7.584
 *   8.710  0.839  2.469  3.850  0.559
 *   1.015  2.469  1.930  6.761  7.207
 *   6.929  3.850  6.761  4.344  4.804
 *   7.584  0.559  7.207  4.804  6.177
```

```

*/
double a[] = {7.027, 8.710, 1.015, 6.929, 7.584,
              8.710, 0.839, 2.469, 3.850, 0.559,
              1.015, 2.469, 1.930, 6.761, 7.207,
              6.929, 3.850, 6.761, 4.344, 4.804,
              7.584, 0.559, 7.207, 4.804, 6.177};

/* Query optimal work size */
dsyev_(&jobz, &uplo, &n, a, &lda, w, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate eigenvalues and eigenvectors */
dsyev_(&jobz, &uplo, &n, a, &lda, w, work, &lwork, &info);
free(work);
/*
 * Output:
 * Eigenvalues (in w)
 * -1.770722   -0.845606   -0.203058   -5.712058   3.044889
 * Eigenvectors (in a, stored in column-major)
 *  0.547211    -0.528441     0.339953    -0.172302
 * -0.531245
 *  0.050685    -16.554919    -5.702155     2.261598
 * 15.521155
 * -0.456110     0.988485     0.587808    -3.161022
 * -1.099502
 * -0.443428    -1.398420     0.424293     0.800628
 * 0.016423
 * -0.541596    -1.770722    -0.845606    -0.203058
 * -5.712058

```

*/

2.2.6 其他函数

2.2.6.1 函数说明

函数族	简介
?(or,un)glq	生成具有正交行的实/复矩阵 Q，且该矩阵定义为 K 个 N 阶基本反射器的乘积的前 M 行，即：$Q=H(k)...H(2)H(1)$，其中 H 为?gelqf 返回的。 具体分为以下四个函数： sorglq_：单精度浮点型 dorglq_：双精度浮点型 cunglq_：单精度复数型 zunglq_：双精度复数型
?(or,un)gqr	生成具有正交行的实/复矩阵 Q，且该矩阵定义为 K 个 N 阶基本反射器的乘积的前 M 行，即：$Q=H(k)...H(2)H(1)$，其中 H 为?geqrf 返回的。 sorgqr_：单精度浮点型 dorgqr_：双精度浮点型 cungqr_：单精度复数型 zungqr_：双精度复数型
?(or,un)grq	生成具有正交行的实/复矩阵 Q，且该矩阵定义为 K 个 N 阶基本反射器的乘积的前 M 行，即：$Q=H(1)H(2) \cdots H(k)$，其中 H 为?gerqf 返回的。 sorgrq_：单精度浮点型 dorgrq_：双精度浮点型 cungrq_：单精度复数型 zungrq_：双精度复数型
?(or,un)gql	生成具有正交列的实/复矩阵 Q，且该矩阵定义为 K 个 M 阶基本反射器的乘积的前 N 列，即：$Q=H(k)...H(2)H(1)$，其中 H 为?geqlf 返回的。 sorgql_：单精度浮点型 dorgql_：双精度浮点型 cungql_：单精度复数型 zungql_：双精度复数型
?(or,un)mlq	计算 $C = Q^* * C$ 或 $C = C * Q^*$，其中 Q 是由?gelqf 计算得到。当 Q 转置，实数时 Q^* 表示 Q^T，复数时 Q^* 表示 Q^H；当 Q 不转置，Q^* 表示 Q。 sorglq_：单精度浮点型 dorglq_：双精度浮点型 cunglq_：单精度复数型 zunglq_：双精度复数型

函数族	简介
?(or,un)mqr	计算 $C = Q^* * C$ 或 $C = C * Q^*$，其中 Q 是由?geqrf 计算得到。当 Q 转置，实数时 Q^* 表示 Q^T，复数时 Q^* 表示 Q^H；当 Q 不转置，Q^* 表示 Q。 sormqr_：单精度浮点型 dormqr_：双精度浮点型 cunmqr_：单精度复数型 zunmqr_：双精度复数型
?(or,un)mql	计算 $C = Q^* * C$ 或 $C = C * Q^*$，其中 Q 是由?geqlf 计算得到。当 Q 转置，实数时 Q^* 表示 Q^T，复数时 Q^* 表示 Q^H；当 Q 不转置，Q^* 表示 Q。 sormql_：单精度浮点型 dormql_：双精度浮点型 cunmql_：单精度复数型 zunmql_：双精度复数型
?(or,un)mrq	计算 $C = Q^* * C$ 或 $C = C * Q^*$，其中 Q 是由?gerqf 计算得到。当 Q 转置，实数时 Q^* 表示 Q^T，复数时 Q^* 表示 Q^H；当 Q 不转置，Q^* 表示 Q。 sormrq_：单精度浮点型 dormrq_：双精度浮点型 cunmrq_：单精度复数型 zunmrq_：双精度复数型
?sy(he)trd	将对称矩阵或 Hermite 矩阵通过相似变换成对称三对角 T，即 $Q^* * H * A * Q = T$。 ssytrd_：单精度浮点型 dormrq_：双精度浮点型 cunmrq_：单精度复数型 zunmrq_：双精度复数型
?lasr	对矩阵 A 做平面旋转操作。 slasr_：单精度浮点型 dlasr_：双精度浮点型 clasr_：单精度复数型 zlasr_：双精度复数型
?sy(he)trd_2stage	将对称矩阵或 Hermite 矩阵 A 转为对称或 Hermite 三对角矩阵 T，即 $A^* * H * A * Q = T$，其中 Q 为正交矩阵。 ssytrd_2stage：单精度浮点型 dsytrd_2stage：双精度浮点型 csytrd_2stage：单精度复数型 zsytrd_2stage：双精度复数型

函数族	简介
?laset	初始化 $m \times n$ 的矩阵，且将对角线元素设置为 β ，非对角线元素设置为 α 。 slaset: 单精度浮点型 dlaset: 双精度浮点型 claset: 单精度复数型 zlaset: 双精度复数型

2.2.6.2 ?(or,un)glq

生成具有正交行的实/复矩阵 Q ，且该矩阵定义为 K 个 N 阶基本反射器的乘积的前 M 行，即： $Q=H(k) \dots H(2)H(1)$ ，其中 H 为?gelqf 返回的。

2.2.6.2.1 接口定义

A. C interface:

```
void sorglq_(const int *m, const int *n, const int *k, float *a, const int *lda, const float *tau, float *work, const int *lwork, int *info);
```

```
void dorglq_(const int *m, const int *n, const int *k, double *a, const int *lda, const double *tau, double *work, const int *lwork, int *info);
```

```
void cunglq_(const int *m, const int *n, const int *k, float_Complex *a, const int *lda, const float_Complex *tau, float_Complex *work, const int *lwork, int *info);
```

```
void zunglq_(const int *m, const int *n, const int *k, double_Complex *a, const int *lda, const double_Complex *tau, double_Complex *work, const int *lwork, int *info);
```

B. Fortran interface:

```
SORGLQ(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);
```

```
DORGLQ(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);
```

```
CUNGLQ(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);
```

```
ZUNGLQ(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);
```

2.2.6.2.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
m	整数型	矩阵 Q 的行数, $m \geq 0$ 。	输入
n	整数型	矩阵 Q 的列数, $n \geq m$ 。	输入
k	整数型	构成矩阵 Q 的基本反射器的数量, $m \geq k \geq 0$ 。	输入
a	在 sorglq 中为单精度浮点型数组。 在 dorglq 中为双精度浮点型数组。 在 cunqlq 中为单精度复数型数组。 在 zunqlq 中为双精度复数型数组。	维度(lda, n)。 作为输入时, 第 i 行必须包含基本反射器 H(i)向量, 其中 H(i)由?gelqf 返回。 作为输出时, 其为 $m*n$ 的矩阵 Q。	输入/输出
lda	整数型	矩阵 A 的主维, $lda \geq \max(1, n)$ 。	输入
tau	在 sorglq 中为单精度浮点型数组。 在 dorglq 中为双精度浮点型数组。 在 cunqlq 中为单精度复数型数组。 在 zunqlq 中为双精度复数型数组。	tau(i)必须包含基本反射器 H(i)的常量因子, 其由?gelqf 返回的。	输出
work	在 sorglq 中为单精度浮点型数组。 在 dorglq 中为双精度浮点型数组。 在 cunqlq 中为单精度复数型数组。 在 zunqlq 中为双精度复数型数组。	维数(max(1,lwork))。若 info=0, work(1)返回最优 lwork 值。	输出

参数名	类型	描述	输入/输出
lwork	整数型	数组 work 的维数。lwork $\geq \max(1, m)$ ，对于最优性能： lwork $\geq m \cdot nb$ ，其中 nb 为最优块大小如果 lwork=-1，则该例程只计算 work 数组的最优大小，并以 work 数组的第一个值返回。	输入
info	整数型	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合法。	输出

2.2.6.2.3 依赖

```
#include "klapack.h"
```

2.2.6.2.4 示例

A. C interface:

```
int m = 4;
int n = 4;
int k = 4;
int lda = 4;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502  -0.120115  -0.425611
```



```

*   0.654120   0.468127   0.644302   -0.601906
*   -0.135860   0.502159   1.077637   -0.103204
*   -0.480109  -1.207226   0.162668   0.227483
*/

double a[] = {-1.036040, 0.654120, 0.135860, -0.480109,
              -0.892502, 0.468127, 0.502159, -1.207226,
              -0.120115, 0.644302, 1.077637, 0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};

/* Query optimal work size */
dorglq_(&m, &n, &k, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dorglq_(&m, &n, &k, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * A output (stored in column-major)
 *   -0.003949   0.089763   0.576860   0.811886
 *   0.896027  -0.205343   0.333180   -0.209669
 *   0.427292   0.639078  -0.551720   0.323428
 *   0.120589  -0.735769  -0.501826   0.438490
 */

```

2.2.6.3 ?(or,un)gqr

生成具有正交行的实/复矩阵 Q，且该矩阵定义为 K 个 N 阶基本反射器的乘积的前 M 行，即： $Q=H(k)...H(2)H(1)$ ，其中 H 为?geqrf 返回的。

2.2.6.3.1 接口定义

A. C interface:

```
void sorgqr_(const int *m, const int *n, const int *k, float *a, const int *lda, const
float *tau, float *work, const int *lwork, int *info);
```

```
void dorgqr_(const int *m, const int *n, const int *k, double *a, const int *lda, const
double *tau, double *work, const int *lwork, int *info);
```

```
void cungqr_(const int *m, const int *n, const int *k, float_Complex *a, const int *lda,
const float_Complex *tau, float_Complex *work, const int *lwork, int *info);
```

```
void zungqr_(const int *m, const int *n, const int *k, double_Complex *a, const int
*lda, const double_Complex *tau, double_Complex *work, const int *lwork, int
*info);
```

B. Fortran interface:

```
SORGQR(M, N, K, A, LDA, TAU, WORK, LWORK, INFO),
```

```
DORGQR(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);
```

```
CUNGQR(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);
```

```
ZUNGQR(M, N, K, A, LDA, TAU, WORK, LWORK, INFO);
```

2.2.6.3.2 参数

参数名	类型	描述	输入/输出
m	整数型	矩阵 Q 的行数， $m \geq 0$ 。	输入
n	整数型	矩阵 Q 的列数， $m \geq n \geq 0$ 。	输入
k	整数型	构成矩阵 Q 的基本反射器的数量， $n \geq k \geq 0$ 。	输入
a	在 sorgqr 中为单精度浮点型数组。 在 dorgqr 中为双精度浮点型数组。 在 cungqr 中为单精度复数型数组。 在 zungqr 中为双精度复数型数组。	维度(lda, n)。 作为输入时，第 i 行必须包含基本反射器 H(i)向量，其中 H(i)由?gelqf 返回。 作为输出时，其为 $m*n$ 的矩阵 Q。	输入/输出

参数名	类型	描述	输入/输出
lda	整数型	矩阵 A 的主维, $lda \geq \max(1, n)$ 。	输入
tau	在 sorgqr 中为单精度浮点型数组。 在 dorgqr 中为双精度浮点型数组。 在 cungqr 中为单精度复数型数组。 在 zungqr 中为双精度复数型数组。	tau(i)必须包含基本反射器 H(i)的常量因子, 其由?gelqf 返回的。	输出
work	在 sorgqr 中为单精度浮点型数组。 在 dorgqr 中为双精度浮点型数组。 在 cungqr 中为单精度复数型数组。 在 zungqr 中为双精度复数型数组。	维数(max(1,lwork))。若 info=0, work(1)返回最优 lwork 值。	输出
lwork	整数型	数组 work 的维数。lwork $\geq \max(1, m)$, 对于最优性能: lwork $\geq m*nb$, 其中 nb 为最优块大小如果 lwork=-1, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入
info	整数型	执行结果: 等于 0: 成功。 小于 0: 第-info 个参数值不合法。	输出

2.2.6.3.3 依赖

```
#include "klapack.h"
```

2.2.6.3.4 示例

A. C interface:

```
int m = 3;

int n = 3;

int k = 3;

int lda = 3;

int info = 0;

double tau[3] = {1.421559, 1.259761, 0.000000};

double *work = NULL;

double qwork;

int lwork = -1;

/*
 * tau:
 *   1.421559  1.259761  0.000000
 * A (3x3, stored in column-major):
 *   -1.053312  -0.004846  1.108093
 *   0.372543  -1.483477  0.560874
 *   0.517801  -0.766553  0.127926
 */
double a[] = {-1.053312, 0.372543, 0.517801,
               -0.004846, -1.483477, -0.766553,
               1.108093, 0.560874, 0.127926};

/* Query optimal work size */
dorgqr_(&m, &n, &k, a, &lda, tau, &qwork, &lwork, &info);

if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;

work = (double *)malloc(sizeof(double) * lwork);
```

```

/* Calculate Q */
dorgqr_(&m, &n, &k, a, &lda, tau, work, &lwork, &info);
free(work);
/*
 * Output:
 * A output (stored in column-major)
 *   -0.421559  -0.529592  -0.736085
 *   -0.573250  -0.473321  0.668844
 *   -0.702618  0.703918   -0.104055
 */

```

2.2.6.4 ?(or,un)grq

生成具有正交行的实/复矩阵 Q，且该矩阵定义为 K 个 N 阶基本反射器的乘积的前 M 行，即： $Q=H(1)H(2)\cdots H(k)$ ，其中 H 为 ?gerqf 返回的。

2.2.6.4.1 接口定义

A. C interface:

```
void sorgqr_(const int *M, const int *N, const int *K, float *A, const int *lda, float
*tau, float *work, const int *lwork, int *info);
```

```
void dorgqr_(const int *M, const int *N, const int *K, double *A, const int *lda,
double *tau, double *work, const int *lwork, int *info);
```

```
void cungrq_(const int *M, const int *N, const int *K, float _Complex *A, const int
*lda, float _Complex *tau, float _Complex *work, const int *lwork, int *info);
```

```
void zungrq_(const int *M, const int *N, const int *K, double _Complex *A, const int
*lda, double _Complex *tau, double _Complex *work, const int *lwork, int *info);
```

B. Fortran interface:

```
SORGRQ(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);
```

```
DORGRQ(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);
```

```
CUNGRQ(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);
```

```
ZUNGRQ(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);
```

2.2.6.4.2 参数

参数名	类型	描述	输入/输出
M	整数型	矩阵 Q 的行数, $M \geq 0$ 。	输入
N	整数型	矩阵 Q 的列数, $N \geq 0$ 。	输入
K	整数型	构成矩阵 Q 的基本反射器的数量, $M \geq K \geq 0$ 。	输入
A	在 sorgqr 中为单精度浮点型数组。 在 dorgqr 中为双精度浮点型数组。 在 cungrq 中为单精度复数型数组。 在 zungrq 中为双精度复数型数组。	维度(lda, n)。 作为输入时, 第 i 列必须包含基本反射器 H(i) 向量, 其中 H(i) 由?gerqf 返回。 作为输出时, 其为 $m*n$ 的矩阵 Q。	输入/输出
lda	整数型	矩阵 A 的主维, $lda \geq \max(1, M)$ 。	输入
tau	在 sorgqr 中为单精度浮点型数组。 在 dorgqr 中为双精度浮点型数组。 在 cungrq 中为单精度复数型数组。 在 zungrq 中为双精度复数型数组。	tau(i) 必须包含基本反射器 H(i) 的常量因子, 其由?gerqf 返回的。	输出
work	在 sorgqr 中为单精度浮点型数组。 在 dorgqr 中为双精度浮点型数组。 在 cungrq 中为单精度复数型数组。 在 zungrq 中为双精度复数型数组。	维数(max(1,lwork))。若 info=0, work(1) 返回最优 lwork 值。	输出

参数名	类型	描述	输入/输出
lwork	整数型	数组 work 的维数。lwork $\geq \max(1, M)$ ，对于最优性能： lwork $\geq M \cdot NB$ ，其中 NB 为最优块大小如果 lwork=-1，则该例程只计算 work 数组的最优大小，并以 work 数组的第一个值返回。	输入
info	整数型	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合法。	输出

2.2.6.4.3 依赖

```
#include "klapack.h"
```

2.2.6.4.4 示例

A. C interface:

```
int m = 4;
int n = 4;
int k = 4;
int lda = m;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502  -0.120115  -0.425611
```

```

*   0.654120   0.468127   0.644302   -0.601906
*   -0.135860   0.502159   1.077637   -0.103204
*   -0.480109  -1.207226   0.162668   0.227483

*

*/

double a[] = {-1.036040, 0.654120,  0.135860, -0.480109,
              -0.892502, 0.468127,  0.502159, -1.207226,
              -0.120115, 0.644302,  1.077637,  0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};

/* Query optimal work size */
dorghr_(&m, &n, &k, a, &lda, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dorghr_(&m, &n, &k, a, &lda, tau, work, &lwork, &info);
free(work);
/*
* Output:
* A output (stored in column-major)
*
*   -0.002365      -0.692243      -0.268856      -0.000000
*   0.001733       0.036633      -0.993734      -0.000000
*   -0.002338      0.322332      -0.978923      -0.000000
*   0.000000       0.000000      0.000000      1.000000
*/

```


2.2.6.5 ?(or,un)gql

生成具有正交列的实/复矩阵 Q ，且该矩阵定义为 K 个 M 阶基本反射器的乘积的前 N 列，即： $Q=H(k)...H(2)H(1)$ ，其中 H 为?geqlf 返回的。

2.2.6.5.1 接口定义

A. C interface:

```
void sorgql_(const int *m, const int *n, const int *k, float *a, const int *lda, float *tau,
float *work, const int *lwork, int *info);
```

```
void dorgql_(const int *m, const int *n, const int *k, double *a, const int *lda, double
*tau, double *work, const int *lwork, int *info);
```

```
void cungql_(const int *m, const int *n, const int *k, float _Complex *a, const int
*lda, float _Complex *tau, float _Complex *work, const int *lwork, int *info);
```

```
void zungql_(const int *m, const int *n, const int *k, double _Complex *a, const int
*lda, double _Complex *tau, double _Complex *work, const int *lwork, int *info);
```

B. Fortran interface:

```
SORGQL(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);
```

```
DORGQL(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);
```

```
CUNGQL(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);
```

```
ZUNGQL(M,N,K,A,LDA,TAU,WORK,LWORK,INFO);
```

2.2.6.5.2 参数

参数名	类型	描述	输入/输出
m	整数型	矩阵 Q 的行数， $m \geq 0$ 。	输入
n	整数型	矩阵 Q 的列数， $m \geq n \geq 0$ 。	输入
k	整数型	构成矩阵 Q 的基本反射器的数量， $n \geq k \geq 0$ 。	输入
a	在 sorgql 中为单精度浮点型数组。 在 dorgql 中为双精度浮点型数组。 在 cungql 中为单精度复数型数组。	维度(lda, n)。 作为输入时，第 i 列必须包含基本反射器 $H(i)$ 向量，其中 $H(i)$ 由?geqlf 返回。 作为输出时，其为 $m*n$ 的矩阵 Q 。	输入/输出

参数名	类型	描述	输入/输出
	在 zungql 中为双精度复数型数组。		
lda	整数型	矩阵 A 的主维, $lda \geq \max(1, M)$ 。	输入
tau	在 sorgql 中为单精度浮点型数组。	tau(i)必须包含基本反射器 H(i)的常量因子, 其由?geqlf返回的。	输出
	在 dorgql 中为双精度浮点型数组。		
	在 cungql 中为单精度复数型数组。		
	在 zungql 中为双精度复数型数组。		
work	在 sorgql 中为单精度浮点型数组。	维数(max(1,lwork))。若 info=0, work(1)返回最优 lwork 值。	输出
	在 dorgql 中为双精度浮点型数组。		
	在 cungql 中为单精度复数型数组。		
	在 zungql 中为双精度复数型数组。		
lwork	整数型	数组 work 的维数。lwork $\geq \max(1, M)$, 对于最优性能: lwork $\geq M \cdot NB$, 其中 NB 为最优块大小如果 lwork=-1, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入
info	整数型	执行结果: 等于 0: 成功。 小于 0: 第-info 个参数值不合法。	输出

2.2.6.5.3 依赖

```
#include "klapack.h"
```

2.2.6.5.4 示例

A. C interface:

```
int m = 4;

int n = 4;

int k = 4;

int lda = m;

int info = 0;

double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};

double *work = NULL;

double qwork;

int lwork = -1;

/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502  -0.120115  -0.425611
 *   0.654120   0.468127   0.644302  -0.601906
 *   -0.135860  0.502159   1.077637  -0.103204
 *   -0.480109 -1.207226   0.162668   0.227483
 *
 */

double a[] = {-1.036040, 0.654120, 0.135860, -0.480109,
               -0.892502, 0.468127, 0.502159, -1.207226,
               -0.120115, 0.644302, 1.077637, 0.162668,
               -0.425611, -0.601906, -0.103204, 0.227483};

/* Query optimal work size */

dorgql_(&m, &n, &k, a, &lda, tau, &qwork, &lwork, &info);
```

```

if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;

work = (double *)malloc(sizeof(double) * lwork);

/* Calculate Q */
dorgql_(&m, &n, &k, a, &lda, tau, work, &lwork, &info);

free(work);

/*
 * Output:
 * A output (stored in column-major)
 *
 *   -0.001005      -0.000771      0.004959      0.000000
 *   0.956417       0.131450       0.398383      0.000000
 *   0.237698      -1.275024      -0.978923      0.000000
 *   -0.000000      -0.000000      -0.000000      1.000000
 */

```

2.2.6.6 ?(or,un)mlq

计算 $C = Q^* * C$ 或 $C = C * Q^*$, 其中 Q 是由 ?gelqf 计算得到。当 Q 转置, 实数时 Q^* 表示 Q^T , 复数时 Q^* 表示 Q^H ; 当 Q 不转置, Q^* 表示 Q。

2.2.6.6.1 接口定义

A. C interface:

```
void sormlq_(const char *uplo, const char *trans, const int *m, const int *n, const int
*k, float *a, const int *lda, const float *tau, float *c, const int *ldc, float *work, const
int *lwork, int *info);
```

```
void dormlq_(const char *uplo, const char *trans, const int *m, const int *n, const int
*k, double *a, const int *lda, const double *tau, double *c, const int *ldc, double
*work, const int *lwork, int *info);
```

```
void cunmlq_(const char *uplo, const char *trans, const int *m, const int *n, const int
*k, float _Complex *a, const int *lda, const float _Complex *tau, float _Complex *c,
const int *ldc, float _Complex *work, const int *lwork, int *info);
```

```
void zunmlq_(const char *uplo, const char *trans, const int *m, const int *n, const int
*k, double _Complex *a, const int *lda, const double _Complex *tau, double
_Complex *c, const int *ldc, double _Complex *work, const int *lwork, int *info);
```

B. Fortran interface:

```
SORMLQ(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);
```

```
DORMLQ(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);
```

```
CUNMLQ(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);
```

```
ZUNMLQ(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);
```

2.2.6.6.2 参数

参数名	类型	描述	输入/输出
side	字符型	"L": Q*在左边。	输入
		"R": Q*在右边。	
trans	字符型	"N": Q 不转置。	输入
		"T": Q 转置。	
M	整数型	C 的行数, $M \geq 0$ 。	输入
N	整数型	C 的列数, $N \geq 0$ 。	输入
K	整数型	生成 Q 的元素反射器的个数。 side="L"时, $M \geq K \geq 0$ 。 side="R"时, $N \geq K \geq 0$ 。	输入
A	在 sormlq 中为单精度浮点型数组。	原始矩阵。 当 side="L"时, 维数为 (LDA, M)。 当 side="R" 时, 维数为 (LDA,N)	输入/输出
	在 dormlq 中为双精度浮点型数组。		
	在 cunmlq 中为单精度复数型数组。		
	在 zunmlq 中为双精度复数型数组。		
lda	整数型	矩阵 A 的主维, $lda \geq \max(1, K)$ 。	输入
tau	在 sormlq 中为单精度浮点型数组。	tau(i)必须包含基本反射器 H(i)的常量因子, 其由?gelqf 返回的。	输入
	在 dormlq 中为双精度浮点型数组。		

参数名	类型	描述	输入/输出
	在 cunmlq 中为单精度复数型数组。		
	在 zunmlq 中为双精度复数型数组。		
C	在 sormlq 中为单精度浮点型数组。	待计算矩阵，维数为 (LDC, N)。	输入/输出
	在 dormlq 中为双精度浮点型数组。		
	在 cunmlq 中为单精度复数型数组。		
	在 zunmlq 中为双精度复数型数组。		
ldc	整数型	矩阵 C 的主维， $ldc \geq \max(1, M)$ 。	输入
work	在 sormlq 中为单精度浮点型数组。	维数($\max(1, lwork)$)。若 info=0, work(1)返回最优 lwork 值。	输出
	在 dormlq 中为双精度浮点型数组。		
	在 cunmlq 中为单精度复数型数组。		
	在 zunmlq 中为双精度复数型数组。		
lwork	整数型	work 的大小。 当 side="L" 时， $lwork \geq \max(1, N)$ 。 当 side="R" 时， $lwork \geq \max(1, M)$ 。 如果 lwork=-1, 则该例程只计算 work 数组的最优大小，并以 work 数组的第一个值返回。	输入
info	整数型	执行结果： 等于 0: 成功。 小于 0: 第-info 个参数值不合法。	输出

2.2.6.6.3 依赖

```
#include "klapack.h"
```

2.2.6.6.4 示例

A. C interface:

```
char side = 'L';
char trans = 'N';
int m = 4;
int n = 4;
int k = 4;
int lda = 4;
int ldc = 4;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 * 1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 * -1.036040  -0.892502  -0.120115  -0.425611
 * 0.654120   0.468127   0.644302  -0.601906
 * -0.135860  0.502159   1.077637  -0.103204
 * -0.480109  -1.207226  0.162668  0.227483
 *
 * C (4x4, stored in column-major):
 * 2.229, 8.103, 7.810, 0.087,
 * 8.667, 8.859, 2.911, 4.091,
```

```

*   0.205, 1.273, 1.396, 2.553,
*   2.758, 4.317, 3.161, 8.791
*/

double a[] = {-1.036040, 0.654120, 0.135860, -0.480109,
              -0.892502, 0.468127, 0.502159, -1.207226,
              -0.120115, 0.644302, 1.077637, 0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};

double c[] = {2.229, 8.667, 0.205, 2.758,
              8.103, 8.859, 1.273, 4.317,
              7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791};

/* Query optimal work size */
dormlq_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork,
&lwork, &info);

if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);

/* Calculate Q */
dormlq_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork,
&lwork, &info);

free(work);

/*
* Output:
* C output (stored in column-major)
*   8.960251      0.032123      2.548981      0.974400
*   9.904029      0.730485      4.605351      6.675692
*   4.096503      1.096288      3.030637      7.364972
*   7.729487      2.907470     -4.718100      3.175610
*/

```


2.2.6.7 ?(or,un)mqr

计算 $C = Q^* * C$ 或 $C = C * Q^*$ ，其中 Q 是由?geqrf 计算得到。当 Q 转置，实数时 Q^* 表示 Q^T ，复数时 Q^* 表示 Q^H ；当 Q 不转置， Q^* 表示 Q 。

2.2.6.7.1 接口定义

A. C interface:

```
void sormqr_(const char *side, const char *trans, const int *m, const int *n, const int
*k, float *a, const int *lda, float *tau, float *c, const int *ldc, float *work, const int
*lwork, int *info);
```

```
void dormqr_(const char *side, const char *trans, const int *m, const int *n, const int
*k, double *a, const int *lda, double *tau, double *c, const int *ldc, double *work,
const int *lwork, int *info);
```

```
void cunmqr_(const char *side, const char *trans, const int *m, const int *n, const int
*k, float _Complex *a, const int *lda, float _Complex *tau, float _Complex *c, const
int *ldc, float _Complex *work, const int *lwork, int *info);
```

```
void zunmqr_(const char *side, const char *trans, const int *m, const int *n, const int
*k, double _Complex *a, const int *lda, double _Complex *tau, double _Complex *c,
const int *ldc, double _Complex *work, const int *lwork, int *info);
```

B. Fortran interface:

```
SORMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK,LWORK,INFO);
```

```
DORMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK,LWORK,INFO);
```

```
CUNMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK,LWORK,INFO);
```

```
ZUNMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK,LWORK,INFO);
```

2.2.6.7.2 参数

参数名	类型	描述	输入/输出
side	字符型	"L": Q^* 在左边。	输入
		"R": Q^* 在右边。	
trans	字符型	"N": Q 不转置。	输入
		"T": Q 转置。	
M	整数型	C 的行数， $M \geq 0$ 。	输入
N	整数型	C 的列数， $N \geq 0$ 。	输入

参数名	类型	描述	输入/输出
K	整数型	生成 Q 的元素反射器的个数。 side="L"时, $M \geq K \geq 0$ 。 side="R"时, $N \geq K \geq 0$ 。	输入
A	在 sormqr 中为单精度浮点型数组。	原始矩阵。	输入
	在 dormqr 中为双精度浮点型数组。		
	在 cunmqr 中为单精度复数型数组。		
	在 zunmqr 中为双精度复数型数组。		
lda	整数型	矩阵 A 的主维。 当 side="L" 时, $lda \geq \max(1, M)$ 。 当 side="R" 时, $lda \geq \max(1, N)$ 。	输入
tau	在 sormqr 中为单精度浮点型数组。	tau(i)必须包含基本反射器 H(i)的常量因子, 其由?geqrf返回的。	输入
	在 dormqr 中为双精度浮点型数组。		
	在 cunmqr 中为单精度复数型数组。		
	在 zunmqr 中为双精度复数型数组。		
C	在 sormqr 中为单精度浮点型数组。	待计算矩阵, 维数为 (ldc, N)。	输入/输出
	在 dormqr 中为双精度浮点型数组。		
	在 cunmqr 中为单精度复数型数组。		
	在 zunmqr 中为双精度复数型数组。		
ldc	整数型	矩阵 C 的主维, $ldc \geq \max(1, M)$ 。	输入

参数名	类型	描述	输入/输出
work	在 sormqr 中为单精度浮点型数组。	维数(max(1,lwork))。若 info=0, work(1)返回最优 lwork 值。	输出
	在 dormqr 中为双精度浮点型数组。		
	在 cunmqr 中为单精度复数型数组。		
	在 zunmqr 中为双精度复数型数组。		
lwork	整数型	work 的大小。 当 side="L"时, $lwork \geq \max(1,N)$ 。 当 side="R"时, $lwork \geq \max(1,M)$ 。 如果 lwork=-1, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入
info	整数型	执行结果: 等于 0: 成功。 小于 0: 第-info 个参数值不合法。	输出

2.2.6.7.3 依赖

```
#include "klapack.h"
```

2.2.6.7.4 示例

A. C interface:

```
char side = 'L';
char trans = 'N';
int m = 4;
int n = 4;
int k = 4;
int lda = 4;
```

```

int ldc = 4;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
 * A (4x4, stored in column-major):
 *   -1.036040  -0.892502  -0.120115  -0.425611
 *   0.654120   0.468127   0.644302  -0.601906
 *   -0.135860  0.502159   1.077637  -0.103204
 *   -0.480109  -1.207226   0.162668   0.227483
 *
 *
 *C (4x4, stored in column-major):
 *   2.229, 8.103, 7.810, 0.087,
 *   8.667, 8.859, 2.911, 4.091,
 *   0.205, 1.273, 1.396, 2.553,
 *   2.758, 4.317, 3.161, 8.791
 */
double a[] = {-1.036040, 0.654120, 0.135860, -0.480109,
              -0.892502, 0.468127, 0.502159, -1.207226,
              -0.120115, 0.644302, 1.077637, 0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};
double c[] = {2.229, 8.667, 0.205, 2.758,
              8.103, 8.859, 1.273, 4.317,
              7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791};

```

```

/* Query optimal work size */

dormqr_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork,
&lwork, &info);

if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);

/* Calculate Q */

dormqr_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork,
&lwork, &info);

free(work);

/*
 * Output:
 * C output (stored in column-major)
 *
 *   3.095833      3.557362      -3.821313      8.984414
 *   0.830861      0.623616      -5.370613     11.371951
 *  -2.390220     -2.230944      -3.001512      5.595986
 *  -9.592006      6.367668      -2.321534      1.764135
 */

```

2.2.6.8 ?(or,un)mql

计算 $C = Q^* * C$ 或 $C = C * Q^*$ ，其中 Q 是由 `geqlf` 计算得到。当 Q 转置，实数时 Q^* 表示 Q^T ，复数时 Q^* 表示 Q^H ；当 Q 不转置， Q^* 表示 Q 。

2.2.6.8.1 接口定义

A. C interface:

```
void sormql_(const char *uplo, const char *trans, const int *m, const int *n, const int
*k, float *a, const int *lda, const float *tau, float *c, const int *ldc, float *work, const
int *lwork, int *info);
```

```
void dormql_(const char *uplo, const char *trans, const int *m, const int *n, const int
*k, double *a, const int *lda, const double *tau, double *c, const int *ldc, double
*work, const int *lwork, int *info);
```

```
void cunmql_(const char *uplo, const char *trans, const int *m, const int *n, const int
*k, float _Complex *a, const int *lda, const float _Complex *tau, float _Complex *c,
const int *ldc, float _Complex *work, const int *lwork, int *info);
```

```
void zunmql_(const char *uplo, const char *trans, const int *m, const int *n, const int
*k, double _Complex *a, const int *lda, const double _Complex *tau, double
_Complex *c, const int *ldc, double _Complex *work, const int *lwork, int *info);
```

B. Fortran interface:

```
SORMQL(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);
```

```
DORMQL(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);
```

```
CUNMQL(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);
```

```
ZUNMQL(side,trans,m,n,k,a,lda,tau,c,ldc,work,lwork,info);
```

2.2.6.8.2 参数

参数名	类型	描述	输入/输出
side	字符型	"L": Q*在左边。	输入
		"R": Q*在右边。	
trans	字符型	"N": Q 不转置。	输入
		"T": Q 转置。	
M	整数型	C 的行数, $M \geq 0$ 。	输入
N	整数型	C 的列数, $N \geq 0$ 。	输入
K	整数型	生成 Q 的元素反射器的个数。 side="L"时, $M \geq K \geq 0$ 。 side="R"时, $N \geq K \geq 0$ 。	输入
A	在 sormql 中为单精度浮点型数组。	原始矩阵, 维数 (LDA,K)。	输入
	在 dormql 中为双精度浮点型数组。		
	在 cunmql 中为单精度复数型数组。		
	在 zunmql 中为双精度复数型数组。		

参数名	类型	描述	输入/输出
lda	整数型	矩阵 A 的主维。 当 side="L" 时 , $lda \geq \max(1,M)$ 。 当 side="R" 时 , $lda \geq \max(1,N)$ 。	输入
tau	在 sormql 中为单精度浮点型数组。 在 dormql 中为双精度浮点型数组。 在 cunmql 中为单精度复数型数组。 在 zunmql 中为双精度复数型数组。	tau(i)必须包含基本反射器 H(i)的常量因子, 其由?geqlf返回的。	输入
C	在 sormql 中为单精度浮点型数组。 在 dormql 中为双精度浮点型数组。 在 cunmql 中为单精度复数型数组。 在 zunmql 中为双精度复数型数组。	待计算矩阵, 维数为 (ldc, N)。	输入/输出
ldc	整数型	矩阵 C 的主维, $ldc \geq \max(1,M)$ 。	输入
work	在 sormql 中为单精度浮点型数组。 在 dormql 中为双精度浮点型数组。 在 cunmql 中为单精度复数型数组。 在 zunmql 中为双精度复数型数组。	维数(max(1,lwork))。若 info=0, work(1)返回最优 lwork 值。	输出

参数名	类型	描述	输入/输出
lwork	整数型	work 的大小。 当 side="L"时, $lwork \geq \max(1,N)$ 。 当 side="R"时, $lwork \geq \max(1,M)$ 。 如果 lwork=-1, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入
info	整数型	执行结果: 等于 0: 成功。 小于 0: 第-info 个参数值不合法。	输出

2.2.6.8.3 依赖

```
#include "klapack.h"
```

2.2.6.8.4 示例

A. C interface:

```
char side = 'L';
char trans = 'N';
int m = 4;
int n = 4;
int k = 4;
int lda = 4;
int ldc = 4;
int info = 0;
double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};
double *work = NULL;
double qwork;
int lwork = -1;
/*
```



```

* tau:

*   1.003949  1.125229  1.978923  0.000000

* A (4x4, stored in column-major):

*   -1.036040  -0.892502  -0.120115  -0.425611
*    0.654120   0.468127   0.644302  -0.601906
*   -0.135860   0.502159   1.077637  -0.103204
*   -0.480109  -1.207226   0.162668   0.227483

*

*C (4x4, stored in column-major):

*   2.229, 8.103, 7.810, 0.087,
*   8.667, 8.859, 2.911, 4.091,
*   0.205, 1.273, 1.396, 2.553,
*   2.758, 4.317, 3.161, 8.791

*/

double a[] = {-1.036040, 0.654120, 0.135860, -0.480109,
              -0.892502, 0.468127, 0.502159, -1.207226,
              -0.120115, 0.644302, 1.077637, 0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};

double c[] = {2.229, 8.667, 0.205, 2.758,
              8.103, 8.859, 1.273, 4.317,
              7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791};

/* Query optimal work size */

dormql_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork,
&lwork, &info);

if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;

```

```

work = (double *)malloc(sizeof(double) * lwork);

/* Calculate Q */

dormql_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &lvc, &qwork,
&lwork, &info);

free(work);

/*
 * Output:
 * C output (stored in column-major)
 *
 *      8.335757      0.876180      3.263161      2.758000
 *      8.767347     -0.464834      2.323291      4.317000
 *      3.108108     -1.403301     -0.168152      3.161000
 *      4.519460     -2.717441     -0.868974      8.791000
 */

```

2.2.6.9 ?(or,un)mrq

计算 $C = Q^* * C$ 或 $C = C * Q^*$, 其中 Q 是由 ?gerqf 计算得到。当 Q 转置, 实数时 Q^* 表示 Q^T , 复数时 Q^* 表示 Q^H ; 当 Q 不转置, Q^* 表示 Q 。

2.2.6.9.1 接口定义

A. C interface:

```
void sormrq(const char *side, const char *trans, const int *m, const int *n, const int
*k, float *a, const int *lda, float *tau, float *c, const int *lvc, float *work, const int
*lwork, int *info);
```

```
void dormrq_(const char *side, const char *trans, const int *m, const int *n, const int
*k, double *a, const int *lda, double *tau, double *c, const int *lvc, double *work,
const int *lwork, int *info);
```

```
void cunmrq_(const char *side, const char *trans, const int *m, const int *n, const int
*k, float _Complex *a, const int *lda, float _Complex *tau, float _Complex *c, const
int *lvc, float _Complex *work, const int *lwork, int *info);
```

```
void zunmrq_(const char *side, const char *trans, const int *m, const int *n, const int
*k, double _Complex *a, const int *lda, double _Complex *tau, double _Complex *c,
const int *lvc, double _Complex *work, const int *lwork, int *info);
```

B. Fortran interface:

SORMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);
DORMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);
CUNMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);
ZUNMQR(SIDE,TRANS,M,N,K,A,LDA,TAU,C,LDC,WORK, LWORK, INFO);

2.2.6.9.2 参数

参数名	类型	描述	输入/输出
side	字符型	"L": Q*在左边。	输入
		"R": Q*在右边。	
trans	字符型	"N": Q 不转置。	输入
		"T": Q 转置。	
M	整数型	C 的行数, $M \geq 0$ 。	输入
N	整数型	C 的列数, $N \geq 0$ 。	输入
K	整数型	生成 Q 的元素反射器的个数。 side="L"时, $M \geq K \geq 0$ 。 side="R"时, $N \geq K \geq 0$ 。	输入
A	在 sormrq 中为单精度浮点型数组。	原始矩阵, 当 side="L"时, 维数为 (lda, M)。 当 side="R" 时, 维数为 (lda,N)	输入
	在 dormrq 中为双精度浮点型数组。		
	在 cunmrq 中为单精度复数型数组。		
	在 zunmrq 中为双精度复数型数组。		
lda	整数型	矩阵 A 的主维。 $lda \geq \max(1,K)$ 。	输入
tau	在 sormrq 中为单精度浮点型数组。	tau(i)必须包含基本反射器 H(i)的常量因子, 其由?gerqf 返回的。	输入
	在 dormrq 中为双精度浮点型数组。		
	在 cunmrq 中为单精度复数型数组。		
	在 zunmrq 中为双精度复数型数组。		

参数名	类型	描述	输入/输出
C	在 sormrq 中为单精度浮点型数组。	待计算矩阵，维数为 (ldc, N)。	输入/输出
	在 dormrq 中为双精度浮点型数组。		
	在 cunmrq 中为单精度复数型数组。		
	在 zunmrq 中为双精度复数型数组。		
ldc	整数型	矩阵 C 的主维， $ldc \geq \max(1, M)$ 。	输入
work	在 sormrq 中为单精度浮点型数组。	维数($\max(1, lwork)$)。若 info=0，work(1)返回最优 lwork 值。	输出
	在 dormrq 中为双精度浮点型数组。		
	在 cunmrq 中为单精度复数型数组。		
	在 zunmrq 中为双精度复数型数组。		
lwork	整数型	work 的大小。 当 side="L" 时， $lwork \geq \max(1, N)$ 。 当 side="R" 时， $lwork \geq \max(1, M)$ 。 如果 lwork=-1，则该例程只计算 work 数组的最优大小，并以 work 数组的第一个值返回。	输入
info	整数型	执行结果： 等于 0：成功。 小于 0：第-info 个参数值不合法。	输出

2.2.6.9.3 依赖

```
#include "klapack.h"
```

2.2.6.9.4 示例

A. C interface:

```
char side = 'L';

char trans = 'N';

int m = 4;

int n = 4;

int k = 4;

int lda = 4;

int ldc = 4;

int info = 0;

double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};

double *work = NULL;

double qwork;

int lwork = -1;

/*
 * tau:
 *
 * 1.003949 1.125229 1.978923 0.000000
 *
 * A (4x4, stored in column-major):
 *
 * -1.036040 -0.892502 -0.120115 -0.425611
 *
 * 0.654120 0.468127 0.644302 -0.601906
 *
 * -0.135860 0.502159 1.077637 -0.103204
 *
 * -0.480109 -1.207226 0.162668 0.227483
 *
 *
 * C (4x4, stored in column-major):
 *
 * 2.229, 8.103, 7.810, 0.087,
 *
 * 8.667, 8.859, 2.911, 4.091,
 *
 * 0.205, 1.273, 1.396, 2.553,
 *
 * 2.758, 4.317, 3.161, 8.791
```

```

*/

double a[] = {-1.036040, 0.654120, 0.135860, -0.480109,
              -0.892502, 0.468127, 0.502159, -1.207226,
              -0.120115, 0.644302, 1.077637, 0.162668,
              -0.425611, -0.601906, -0.103204, 0.227483};

double c[] = {2.229, 8.667, 0.205, 2.758,
              8.103, 8.859, 1.273, 4.317,
              7.810, 2.911, 1.396, 3.161,
              0.087, 4.091, 2.553, 8.791};

/* Query optimal work size */

dormrq_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork,
&lwork, &info);

if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);

/* Calculate Q */

dormrq_(&side, &trans, &m, &n, &k, a, &lda, tau, c, &ldc, &qwork,
&lwork, &info);
free(work);

/*
 * Output:
 * C output (stored in column-major)
 *
 * 0.009265      -1.159433      -9.412653      2.758000
 * -0.006793     -4.874383     -12.228202     4.317000
 * -0.016693     -4.849802     -6.359105     3.161000
 * 0.000914       0.912554      -6.587947     8.791000
 */

```

2.2.6.10 ?sy(he)trd

将对称矩阵或 Hermite 矩阵通过相似变换成对称三对角 T ，即 $Q^* * H * A * Q = T$ 。

2.2.6.10.1 接口定义

A. C interface:

```
void ssytrd_(const char *uplo, const int *n, float *a, const int *lda, float *d, float *e,
float *tau, float *work, const int *lwork, int *info);
```

```
void dsytrd_(const char *uplo, const int *n, double *a, const int *lda, double *d,
double *e, double *tau, double *work, const int *lwork, int *info);
```

```
void chetrd_(const char *uplo, const int *n, float _Complex *a, const int *lda, float *d,
float *e, float _Complex *tau, float _Complex *work, const int *lwork, int *info);
```

```
void zhetrd_(const char *uplo, const int *n, double _Complex *a, const int *lda,
double *d, double *e, double _Complex *tau, double _Complex *work, const int
*lwork, int *info);
```

B. Fortran interface:

```
SSYTRD(UPLO,N,A,LDA,D,E,TAU,WORK,LWORK,INFO);
```

```
DSYTRD(UPLO,N,A,LDA,D,E,TAU,WORK,LWORK,INFO);
```

```
CHETRD(UPLO,N,A,LDA,D,E,TAU,WORK,LWORK,INFO);
```

```
ZHETRD(UPLO,N,A,LDA,D,E,TAU,WORK,LWORK,INFO);
```

2.2.6.10.2 参数

参数名	类型	描述	输入/输出
uplo	字符型	'U': A 中保存上三角矩阵。	输入
		'L': A 中保存下三角矩阵。	
N	整数型	矩阵 A 的维数， $N \geq 0$ 。	输入
A	在 ssytrd 中为单精度浮点型数组。	调用前保存待分解的对称矩阵。 调用后保存特征向量。	输入/输出
	在 dsytrd 中为双精度浮点型数组。		
	在 chetrd 中为单精度复数型数组。		

参数名	类型	描述	输入/输出
	在 zhetrd 中为双精度复数型数组。		
lda	整数型	矩阵 A 的主维。 $lda \geq \max(1, N)$ 。	输入
D	在 ssytrd/chetrd 中为单精度浮点型数组。 在 dsytrd/zhetrd 中为双精度浮点型数组。	三对角矩阵 T 的对角元素，即 $D(i)=A(i,i)$ 。	输出
E	在 ssytrd/chetrd 中为单精度浮点型数组。 在 dsytrd/zhetrd 中为双精度浮点型数组。	三对角矩阵 T 非对角元素。 当 UPLO="U" 时， $E(i)=A(i,i+1)$ 。 当 UPLO="L" 时， $E(i)=A(i+1,i)$ 。	输出
tau	在 ssytrd 中为单精度浮点型数组。 在 dsytrd 中为双精度浮点型数组。 在 chetrd 中为单精度复数型数组。 在 zhetrd 中为双精度复数型数组。	tau(i) 必须包含基本反射器的常量因子，维数为 N-1。	输出
work	在 ssytrd 中为单精度浮点型数组。 在 dsytrd 中为双精度浮点型数组。 在 chetrd 中为单精度复数型数组。 在 zhetrd 中为双精度复数型数组。	维数(max(1,lwork))。若 info=0，work(1) 返回最优 lwork 值。	输出

参数名	类型	描述	输入/输出
lwork	整数型	work 的大小。 当 side="L"时, $lwork \geq \max(1,N)$ 。 当 side="R"时, $lwork \geq \max(1,M)$ 。 如果 lwork=-1, 则该例程只计算 work 数组的最优大小, 并以 work 数组的第一个值返回。	输入
info	整数型	执行结果: 等于 0: 成功。 小于 0: 第-info 个参数值不合法。	输出

2.2.6.10.3 依赖

```
#include "klapack.h"
```

2.2.6.10.4 示例

A. C interface:

```
char uplo = 'L';
int n = 4;
int lda = n;
int info = 0;

double tau[4] = {1.003949, 1.125229, 1.978923, 0.000000};

double *work = NULL;

double qwork;

int lwork = -1;

double *d = (double*)malloc(n * sizeof(double));
double *e = (double*)malloc((n - 1) * sizeof(double));

/*
 * tau:
 *   1.003949  1.125229  1.978923  0.000000
```

```

* A (4x4, stored in column-major):
*   7.027  8.710  1.015  6.929
*   8.710  0.839  2.469  3.850
*   1.015  2.469  1.930  6.761
*   6.929  3.850  6.761  4.344
*
*/

double a[] = {7.027, 8.710, 1.015, 6.929,
              8.710, 0.839, 2.469, 3.850,
              1.015, 2.469, 1.930, 6.761,
              6.929, 3.850, 6.761, 4.344};

/* Query optimal work size */
dsytrd_(&uplo, &n, a, &lda, d, e, tau, &qwork, &lwork, &info);
if (info != 0) {
    return ERROR;
}

lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
/* Calculate Q */
dsytrd_(&uplo, &n, a, &lda, d, e, tau, work, &lwork, &info);
free(work);
/*
* Output:
* A output (stored in column-major)
*   7.027000      -11.176107      0.051041      0.348434
*   8.710000      7.026600      6.375997      0.224161
*   1.015000      2.469000      3.358516     -1.395021
*   6.929000      3.850000      6.761000     -3.272116
*
* D output (stored in column-major)

```

```

*   7.027000      7.026600      3.358516      -3.272116
*
* E output (stored in column-major)
*  -11.176107      6.375997      -1.395021
*/

```

2.2.6.11 ?lasr

对矩阵 A 做平面旋转操作。

2.2.6.11.1 接口定义

A. C interface:

```
void slasr_(const char *SIDE, const char *PIVOT, const char *DIRECT, const int *M,
const int *N, float *C, float *S, float *A, const int *LDA);
```

```
void dlasr_(const char *SIDE, const char *PIVOT, const char *DIRECT, const int *M,
const int *N, double *C, double *S, double *A, const int *LDA);
```

```
void clasr_(const char *SIDE, const char *PIVOT, const char *DIRECT, const int *M,
const int *N, float *C, float *S, float *A, const int *LDA);
```

```
void zlasr_(const char *SIDE, const char *PIVOT, const char *DIRECT, const int *M,
const int *N, double *C, double *S, double *A, const int *LDA);
```

B. Fortran interface:

```
SLASR(SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA);
```

```
DLASR(SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA);
```

```
CLASR(SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA);
```

```
ZLASR(SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA);
```

2.2.6.11.2 参数

参数名	类型	描述	输入/输出
SIDE	字符型	'L', $A := P * A$ 。	输入
		'R', $A := A * P^* * T$ 。	
PIVOT	字符型	'V', Variable pivot, plane (k, k+1)。	输入
		'T', Top pivot, plane(1, k+1)。	

参数名	类型	描述	输入/输出
		'B', Bottom pivot, plane (k, z)。	
DIRECT	字符型	'F' , Forward, $P=P(z-1)* \dots *P(2)*P(1)$ 。	输入
		'B', Backward, $P=P(1)*P(2)* \dots *P(z-1)$ 。	
lda	整数型	矩阵 A 的主维。 $lda \geq \max(1,N)$ 。	输入
M	整数型	矩阵 A 的行数。	输入
N	整数型	矩阵 A 的列数。	输入
C	在 slasr/clasr 中为单精度浮点型数组。	平面旋转的 cosine 值。	输入
	在 dlasr/zlasr 中为双精度浮点型数组。		
S	在 slasr/clasr 中为单精度浮点型数组。	平面旋转的 sine 值。	输入
	在 dlasr/zlasr 中为双精度浮点型数组。		
A	在 slasr 中为单精度浮点型数组。	矩阵 A, 维度为 (LDA,N)。	输入/输出
	在 dlasr 中为双精度浮点型数组。		
	在 clasr 中为单精度复数型数组。		
	在 zlasr 中为双精度复数型数组。		
LDA	整数型	矩阵 A 的主维。	输入

2.2.6.11.3 依赖

#include "klapack.h"

2.2.6.11.4 示例

A. C interface:

```
const char side = 'L';
```

```

const char pivot = 'V';
const char direct = 'F';

int m = 4;
int n = 4;
int lda = m;

double c[] = {0.000000, 0.900000, 1.100000};
double s[] = {0.181818, 0.545455, 0.000000};
double a[] = {0.600000,0.200000,0.900000,0.100000,
               0.200000,0.700000,0.000000,0.900000,
               0.300000,0.600000,0.000000,0.600000,
               0.200000,0.600000,0.100000,0.800000};

dlasr_(&side, &pivot, &direct, &m, &n, c, s, a, &lda);
/*
 * Output:
 * a output (stored in column-major)
 *   0.036364    0.392727    0.956455    0.110000
 *   0.127273   -0.032727    0.021818    0.990000
 *   0.109091   -0.049091    0.032727    0.660000
 *   0.109091    0.021818    0.120818    0.880000

```

2.2.6.12 ?sy(he)trd_2stage

将对称矩阵或 Hermite 矩阵 A 转为对称或 Hermite 三对角矩阵 T，即 $A^* * H * A * Q = T$ ，其中 Q 为正交矩阵。

2.2.6.12.1 接口定义

A. C interface:

```

void ssytrd_2stage(const char *VECT, const char *UPLO, const int *N, float *A,
const int *LDA, float *D, float *E, float *TAU, float *HOUS2, int *LHOUS2, float
*WORK, const int *LWORK, int *INFO);

```

```
void dsytrd_2stage(const char *VECT, const char *UPLO, const int *N, double *A,
const int *LDA, double *D, double *E, double *TAU, double *HOUS2, int
*LHOUS2, double *WORK, const int *LWORK, int *INFO);
```

```
void chetrd_2stage(const char *VECT, const char *UPLO, const int *N, float
_Complex *A, const int *LDA, float *D, float *E, float _Complex *TAU, float
_Complex *HOUS2, int *LHOUS2, float _Complex *WORK, const int *LWORK,
int *INFO);
```

```
void zhetrd_2stage(const char *VECT, const char *UPLO, const int *N, double
_Complex *A, const int *LDA, double *D, double *E, double _Complex *TAU,
double _Complex *HOUS2, int *LHOUS2, double _Complex *WORK, const int
*LWORK, int *INFO);
```

B. Fortran interface:

```
SSYTRD_2STAGE(VECT, UPLO, N, A, LDA, D, E, TAU, HOUS2, LHOUS2,
WORK, LWORK, INFO);
```

```
DSYTRD_2STAGE(VECT, UPLO, N, A, LDA, D, E, TAU, HOUS2, LHOUS2,
WORK, LWORK, INFO);
```

```
CHETRD_2STAGE(VECT, UPLO, N, A, LDA, D, E, TAU, HOUS2, LHOUS2,
WORK, LWORK, INFO);
```

```
ZHETRD_2STAGE(VECT, UPLO, N, A, LDA, D, E, TAU, HOUS2, LHOUS2,
WORK, LWORK, INFO);
```

2.2.6.12.2 参数

参数名	类型	描述	输入/输出
VECT	字符型	'N': 不需要 Householder 表达式。	输入
		'V': 需要 Householder 表达式。	
UPLO	字符型	'U': 存放 A 的上三角部分。	输入
		'L': 存放 A 的下三角部分。	
N	在 ssytrd_2stage 中为单精度浮点型数组。	矩阵 A 的列数, $N \geq 0$ 。	输入
	在 dsytrd_2stage 中为双精度浮点型数组。		

参数名	类型	描述	输入/输出
	在 chetrd_2stage 中为单精度复数型数组。		
	在 zhetrd_2stage 中为双精度复数型数组。		
A	在 ssytrd_2stage 中为单精度浮点型数组。	矩阵 A，维度为 (LDA, N)。	输入/输出
	在 dsytrd_2stage 中为双精度浮点型数组。		
	在 chetrd_2stage 中为单精度复数型数组。		
	在 zhetrd_2stage 中为双精度复数型数组。		
LDA	整数型	矩阵 A 的主维， $LDA \geq \max(1, N)$ 。	输入
D	在 ssytrd_2stage/chetrd_2stage 中为单精度浮点型数组。	对称三对角矩阵 T 的对角元素，大小为 N。	输出
	在 dsytrd_2stage/zhetrd_2stage 中为双精度浮点型数组。		
E	在 ssytrd_2stage/chetrd_2stage 中为单精度浮点型数组。	对称三对角矩阵 T 的非对角元素，大小为 N-1。	输出
	在 dsytrd_2stage/zhetrd_2stage 中为双精度浮点型数组。		
TAU	在 ssytrd_2stage 中为单精度浮点型数组。	基本反射器的标量因子。	输出
	在 dsytrd_2stage 中为双精度浮点型数组。		
	在 chetrd_2stage 中为单精度复数型数组。		

参数名	类型	描述	输入/输出
	在 zhetrd_2stage 中为双精度复数型数组。		
HOUS2	在 ssytrd_2stage 中为单精度浮点型数组。	存放第二阶段中 Householder 表达形式。	输出
	在 dsytrd_2stage 中为双精度浮点型数组。		
	在 chetrd_2stage 中为单精度复数型数组。		
	在 zhetrd_2stage 中为双精度复数型数组。		
LHOUS2	整数型	HOUS2 的长度。 VECT='N'时, LHOUS2=max(1,4*n)。 VECT='V'时, 不可用。	输入
WORK	在 ssytrd_2stage 中为单精度浮点型数组。	工作数组, 大小为 LWORK。	输出
	在 dsytrd_2stage 中为双精度浮点型数组。		
	在 chetrd_2stage 中为单精度复数型数组。		
	在 zhetrd_2stage 中为双精度复数型数组。		
LWORK	整数型	工作数组的长度, LWORK $\geq \max(1, \text{dimension})$ 。其中, dimension= $\max(\text{stage1}, \text{stage2}) +$ $(\text{KD}+1)*N = N*\text{KD} +$ $N*\max(\text{KD}+1, \text{FACTOPTNB})$ $+ \max(2*\text{KD}*\text{KD},$ $\text{KD}*N\text{THREADS}) +$ $(\text{KD}+1)*N$, 当 LWORK=-1 时, 返回最优 LWORK 值。	输入

参数名	类型	描述	输入/输出
INFO	整数型	等于 0: 成功退出。 小于 0: 表示第-INFO 个参数非法。	输出

2.2.6.12.3 依赖

```
#include "klapack.h"
```

2.2.6.12.4 示例

A. C interface:

```
char vect = 'N';
char uplo = 'L';
int n = 4;
int lda = n;
int info = 0;
double tau = (double*)malloc(n * sizeof(double));
double *work = NULL;
double *hous2 = NULL;
double qwork;
double qhous2;
int lwork = -1;
int lhous2 = -1;
double *d = (double*)malloc(n * sizeof(double));
double *e = (double*)malloc((n - 1) * sizeof(double));

double a[] = {0.521739, 0.043478, 0.304348, 0.130435,
              0.043478, 0.304348, 0.043478, 0.652174,
              0.304348, 0.043478, 0.086957, 0.521739,
              0.130435, 0.652174, 0.521739, 0.086957};
```

```

/* Query optimal work size and hous2 size */
dsytrd_2stage_(&vect, &uplo, &n, a, &lدا, d, e, tau, &qhous2, &lhous2,
&qwork, &lwork, &info)
if (info != 0) {
    return ERROR;
}
lwork = (int)qwork;
work = (double *)malloc(sizeof(double) * lwork);
lhous2 = (int)qhous2;
hous2 = (double*)malloc(lhous2 * sizeof(double));

/* Calculate */
dsytrd_2stage_(&vect, &uplo, &n, a, &lدا, d, e, tau, hous2, &lhous2,
work, &lwork, &info);
if (info != 0) {
    return ERROR;
}

free(work);
free(hous2);
free(tau);
}

/*
* Output:
* A output (stored in column-major)
*   0.521739      0.043478      0.304348      0.130435
*   0.043478      0.304348      0.043478      0.652174
*   0.304348      0.043478      0.086957      0.521739
*   0.130435      0.652174      0.521739      0.086957
*
* D output (stored in column-major)
*   0.521739      0.538688      0.325765      -0.386192

```

```

*
* E output (stored in column-major)
*   -0.333963      -0.507861      -0.471868
*/

```

2.2.6.13 ?laset

初始化 $m \times n$ 的矩阵，且将对角线元素设置为 β ，非对角线元素设置为 α 。

2.2.6.13.1 接口定义

A. C interface:

```
void slaset_(const char *uplo, const int *m, const int *n, const float *alpha, const float
*beta, float *a, const int *lda);
```

```
void dlaset_(const char *uplo, const int *m, const int *n, const double *alpha, const
double *beta, double *a, const int *lda);
```

```
void claset_(const char *uplo, const int *m, const int *n, const float _Complex *alpha,
const float _Complex *beta, float _Complex *a, const int *lda);
```

```
void zlaset_(const char *uplo, const int *m, const int *n, const double _Complex
*alpha, const double _Complex *beta, double _Complex *a, const int *lda);
```

B. Fortran interface:

```
SLASET(UPLO, M, N, ALPHA, BETA, A, LDA);
```

```
DLASET(UPLO, M, N, ALPHA, BETA, A, LDA);
```

```
CLASET(UPLO, M, N, ALPHA, BETA, A, LDA);
```

```
ZLASET(UPLO, M, N, ALPHA, BETA, A, LDA);
```

2.2.6.13.2 参数

参数名	类型	描述	输入/输出
UPLO	字符型	'U': 矩阵 A 的上三角部分被初始化。	输入
		'L': 矩阵 A 的下三角部分被初始化。	

参数名	类型	描述	输入/输出
		其他：整个矩阵 A 都将被初始化。	
M	整数型	矩阵 A 的行数。	输入
N	整数型	矩阵 A 的列数。	输入
ALPHA	在 slaset 中为单精度浮点型。 在 dlaset 中为双精度浮点型。 在 claset 中为单精度复数型。 在 zlaset 中为双精度复数型。	矩阵 A 的非对角线元素值。	输入
BETA	在 slaset 中为单精度浮点型。 在 dlaset 中为双精度浮点型。 在 claset 中为单精度复数型。 在 zlaset 中为双精度复数型。	矩阵 A 的对角线元素值。	输入
A	在 slaset 中为单精度浮点型数组。 在 dlaset 中为双精度浮点型数组。 在 claset 中为单精度复数型数组。 在 zlaset 中为双精度复数型数组。	矩阵 A，维度为 (LDA, N)。	输出
LDA	整数型	矩阵 A 的主维， $LDA \geq \max(1, N)$ 。	输入

2.2.6.13.3 依赖

```
#include "klapack.h"
```

2.2.6.13.4 示例

A. C interface:

```
const char uplo = 'U';
const int m = 4;
const int n = 4;
const int lda = m;

const double alpha = 2.0;
const double beta = 1.0;
double *a = (double*)malloc(lda * n * sizeof(double));
for (int i = 0; i < lda * n; i++) {
    a[i] = -1;
}

dlaset_(&uplo, &m, &n, &alpha, &beta, a, &lda);
/*
 * Output:
 * a output (stored in column-major)
 *
 * 1.000000      2.000000      2.000000      2.000000
 * -1.000000      1.000000      2.000000      2.000000
 * -1.000000     -1.000000      1.000000      2.000000
 * -1.000000     -1.000000     -1.000000      1.000000
```

2.2.7 Support 函数

获取 LAPACK 产品版本信息。

2.2.7.1 接口定义

```
int KLAPACKGetVersion(KLAPACKVersion *ver);
```

2.2.7.2 参数

参数名	类型	描述	输入/输出
ver	KLAPACKVersion 结构体 指针	存储版本信息，非空。	输入/输出

2.2.7.3 依赖

```
#include "service.h"
```

```
#include "klapack.h"
```

2.2.7.4 示例

```
int main()
{
    KLAPACKVersion *ver =
(KLAPACKVersion*)malloc(sizeof(KLAPACKVersion));

    int res = KLAPACKGetVersion(ver);
    if (res != 0) {
        printf("Get version error.\n");
        return 0;
    }

    printf("Product Name: %s\nProduct Version: %s\nComponent
Name: %s\nComponent Version: %s\nComponent AppendInfo: %s\nSoftware
Name: %s\nSoftware Version: %s\n",

        ver->productName, ver->productVersion,
        ver->componentName, ver->componentVersion, ver-
>componentAppendInfo,
        ver->softwareName, ver->softwareVersion);

    return 0;
}
```

运行结果:

```
Product Name: Kunpeng Boostkit
```

```
Product Version: 23.0.RC2
```

```
Component Name: BoostKit-kml
```

Component Version: 1.8.0

Component AppendInfo: gcc

Software Name: boostkit-kml-lapack

Software Version: 1.8.0

说明：版本号以实际运行结果为准，上述结果仅供参考。

2.3 FFT 接口说明

2.3.1 C2C 变换

2.3.1.1 函数说明

函数名	简介
fft(f)_plan_dft	建立单个连续数据序列 n 维 C2C 变换的 plan。
fft(f)_plan_dft_1d	建立单个连续数据序列 1 维 C2C 变换的 plan。
fft(f)_plan_dft_2d	建立单个连续数据序列 2 维 C2C 变换的 plan。
fft(f)_plan_dft_3d	建立单个连续数据序列 3 维 C2C 变换的 plan。
fft(f)_plan_guru_dft	建立多组数据序列 n 维 C2C 变换的 plan。
fft(f)_plan_guru_split_dft	建立多组数据序列 n 维 C2C 变换的 plan。
fft(f)_plan_guru64_dft	建立多组数据序列 n 维 C2C 变换的 plan。
fft(f)_plan_guru64_split_dft	建立多组数据序列 n 维 C2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。
fft(f)_plan_many_dft	建立 howmany 组数据序列 n 维 C2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

2.3.1.2 fft(f)_plan_dft

建立单个连续数据序列 n 维 C2C 变换的 plan。

2.3.1.2.1 接口定义

C interface:

```
fft_plan fft_plan_dft(int rank, const int *n, fft_complex *in, fft_complex *out, int sign,
unsigned flags);
```

```
fftf_plan fftf_plan_dft(int rank, const int *n, fftf_complex *in, fftf_complex *out, int
sign, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT(RANK, N, IN, OUT, SIGN, FLAGS);
```

```
RES = FFTF_PLAN_DFT(RANK, N, IN, OUT, SIGN, FLAGS);
```

2.3.1.2.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.1.2.3 参数

参数名	类型	描述	输入/输出
rank	int	FFT 变换的维度是 rank，约束： $1 \leq \text{rank} \leq 3$ 。	输入
n	const int*	n 是维度为 rank 的数组，包含 FFT 序列每一维度的大小，约束： $n[i] \geq 1$, for i in 0 to rank - 1。	输入
in	双精度: fft_complex*	输入待变换的数据。	输入
	单精度: fftf_complex*		
out	双精度: fft_complex*	输出快速傅里叶变换后的数据。	输出
	单精度: fftf_complex*		
sign	int	描述正向变换或反向变换。 -1(FFT_FORWARD): forward +1(FFT_BACKWARD): backward	输入
flags	unsigned int	planning 选项，未使用。	输入

2.3.1.2.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.1.2.5 示例

C interface:

```
int rank = 2;

int *n;

n = (int*)fft_malloc(sizeof(int) * rank);

n[0] = 2;

n[1] = 3;

double init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};

fft_complex *in;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * n[0] * n[1]);

for (int i = 0; i < n[0] * n[1]; i++) {

    in[i].r = init[i][0];

    in[i].i = init[i][1];

}

fft_complex *out;

out = (fft_complex*)fft_malloc(sizeof(fft_complex) * n[0] * n[1]);

fft_plan plan;

plan = fft_plan_dft(rank, n, in, out, FFT_FORWARD, FFT_ESTIMATE);

fft_execute_dft(plan, in, out);

fft_destroy_plan(plan);

fft_free(n);

fft_free(in);

fft_free(out);
```

```

/*
 * out = {{1.200000e+02, 4.800000e+01}, {1.338564e+02, -
1.385641e+01},
 *        {1.061436e+02, 1.385641e+01}, {1.360000e+02, -
3.200000e+01},
 *        {1.120000e+02, -8.000000e+00}, {1.120000e+02, -
8.000000e+00}}
 */

```

2.3.1.3 fft(f)_plan_dft_1d

建立单个连续数据序列 1 维 C2C 变换的 plan。

2.3.1.3.1 接口定义

C interface:

fft_plan fft_plan_dft_1d(int n, fft_complex *in, fft_complex *out, int sign, unsigned flags);

fftf_plan fftf_plan_dft_1d(int n, fftf_complex *in, fftf_complex *out, int sign, unsigned flags);

Fortran interface:

RES = FFT_PLAN_DFT_1D(N, IN, OUT, SIGN, FLAGS);

RES = FFTF_PLAN_DFT_1D(N, IN, OUT, SIGN, FLAGS);

2.3.1.3.2 返回值

函数返回一个 fft(f)_plan 类型的结构体指针。将该对象作为参数传入 fft(f)_execute 函数中使用，将对当前提供的输入 in 和输出 out 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 fft(f)_execute_dft 函数中以对新的输入 in 和输出 out 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.1.3.3 参数

参数名	类型	描述	输入/输出
n	int	输入 FFT 序列的大小，约束：n ≥ 1。	输入

参数名	类型	描述	输入/输出
in	双精度: fft_complex *	输入待变换的数据。	输入
	单精度: fftf_complex *		
out	双精度: fft_complex *	输出快速傅里叶变换后的数据。	输出
	单精度: fftf_complex *		
sign	int	描述正向变换或反向变换。 -1(FFT_FORWARD): forward +1(FFT_BACKWARD): backward	输入
flags	unsigned int	planning 选项, 未使用。	输入

2.3.1.3.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.1.3.5 示例

C interface:

```

int N = 16;
fft_complex *in;
fft_complex *out;
fft_plan plan;
in = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
out = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i].r = (double)i;
    in[i].i = (double)i;
}
plan = fft_plan_dft_1d(N, in, out, FFT_FORWARD, FFT_ESTIMATE);
fft_execute_dft(plan, in, out);

```

```

fft_destroy_plan(plan);

fft_free(in);

fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01,
3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01,
3.972846e+00},
 *        {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -
2.654571e+00},
 *        {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -
6.408701e+00},
 *        {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -
9.591299e+00},
 *        {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -
1.334543e+01},
 *        {0.000000e+00, -1.600000e+01}, {3.972846e+00, -
1.997285e+01},
 *        {1.131371e+01, -2.731371e+01}, {3.221872e+01, -
4.821872e+01}}
 */

```

2.3.1.4 fft(f)_plan_dft_2d

建立单个连续数据序列 2 维 C2C 变换的 plan。

2.3.1.4.1 接口定义

C interface:

```
fft_plan fft_plan_dft_2d(int n0, int n1, fft_complex *in, fft_complex *out, int sign,
unsigned flags);
```

```
fftf_plan fftf_plan_dft_2d(int n0, int n1, fftf_complex *in, fftf_complex *out, int sign,
unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_2D(N0, N1, IN, OUT, SIGN, FLAGS);
```

```
RES = FFTF_PLAN_DFT_2D(N0, N1, IN, OUT, SIGN, FLAGS);
```

2.3.1.4.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.1.4.3 参数

参数名	类型	描述	输入/输出
<code>n0</code>	<code>int</code>	输入 FFT 序列第一维的大小，约束： $n0 \geq 1$ 。	输入
<code>n1</code>	<code>int</code>	输入 FFT 序列第二维的大小，约束： $n1 \geq 1$ 。	输入
<code>in</code>	双精度： <code>fft_complex*</code> 单精度： <code>fftf_complex*</code>	输入待变换的数据。	输入
<code>out</code>	双精度： <code>fft_complex*</code> 单精度： <code>fftf_complex*</code>	输出快速傅里叶变换后的数据。	输出
<code>sign</code>	<code>int</code>	描述正向变换或反向变换。 -1(FFT_FORWARD): forward +1(FFT_BACKWARD): backward	输入
<code>flags</code>	<code>unsigned int</code>	planning 选项，未使用。	输入

2.3.1.4.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.1.4.5 示例

C interface:

```

int n0 = 2;

int n1 = 3;

double init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-
8, 8}};

fft_complex *in;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * n0 * n1);

for (int i = 0; i < n0 * n1; i++) {

    in[i].r = init[i][0];

    in[i].i = init[i][1];

}

fft_complex *out;

out = (fft_complex*)fft_malloc(sizeof(fft_complex) * n0 * n1);

fft_plan plan;

plan = fft_plan_dft_2d(n0, n1, in, out, FFT_FORWARD, FFT_ESTIMATE);

fft_execute_dft(plan, in, out);

fft_destroy_plan(plan);

fft_free(in);

fft_free(out);

/*

* out = {{1.200000e+02, 4.800000e+01}, {1.338564e+02, -
1.385641e+01},

*         {1.061436e+02, 1.385641e+01}, {1.360000e+02, -
3.200000e+01},

*         {1.120000e+02, -8.000000e+00}, {1.120000e+02, -
8.000000e+00}}

*/

```

2.3.1.5 fft(f)_plan_dft_3d

建立单个连续数据序列 3 维 C2C 变换的 plan。

2.3.1.5.1 接口定义

C interface:

```
fft_plan fft_plan_dft_3d(int n0, int n1, int n2, fft_complex *in, fft_complex *out, int
sign, unsigned flags);
```

```
fftf_plan fftf_plan_dft_3d(int n0, int n1, int n2, fftf_complex *in, fftf_complex *out,
int sign, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_3D(N0, N1, N2, IN, OUT, SIGN, FLAGS);
```

```
RES = FFTF_PLAN_DFT_3D(N0, N1, N2, IN, OUT, SIGN, FLAGS);
```

2.3.1.5.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.1.5.3 参数

参数名	类型	描述	输入/输出
n0	int	输入 FFT 序列第一维的大小， 约束： $n0 \geq 1$ 。	输入
n1	int	输入 FFT 序列第二维的大小， 约束： $n1 \geq 1$ 。	输入
n2	int	输入 FFT 序列第三维的大小， 约束： $n2 \geq 1$ 。	输入
in	双精度：fft_complex *	输入待变换的数据。	输入
	单精度：fftf_comple x*		

参数名	类型	描述	输入/输出
out	双精度: fft_complex *	输出快速傅里叶变换后的数据。	输出
	单精度: fftf_complex *		
sign	int	描述正向变换或反向变换。 -1(FFT_FORWARD): forward +1(FFT_BACKWARD): backward	输入
flags	unsigned int	planning 选项, 未使用。	输入

2.3.1.5.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.1.5.5 示例

C interface:

```

int n0 = 2;
int n1 = 3;
int n2 = 2;

double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16},
{-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8}};

fft_complex *in;
in = (fft_complex*)fft_malloc(sizeof(fft_complex) * n0 * n1 * n2);

for (int i = 0; i < n0 * n1 * n2; i++) {
    in[i].r = init[i][0];
    in[i].i = init[i][1];
}

fft_complex *out;
out = (fft_complex*)fft_malloc(sizeof(fft_complex) * n0 * n1 * n2);

fft_plan plan;

plan = fft_plan_dft_3d(n0, n1, n2, in, out, FFT_FORWARD,
FFT_ESTIMATE);

```



```

fft_execute_dft(plan, in, out);

fft_destroy_plan(plan);

fft_free(in);

fft_free(out);

/*
 * out = {{4.000000e+01, 4.000000e+01}, {7.200000e+01,
8.000000e+00},
 *        {1.272154e+02, -3.064102e+01}, {1.112154e+02, -
3.492820e+01},
 *        {1.687846e+02, 3.864102e+01}, {1.527846e+02, -
2.107180e+01},
 *        {2.000000e+02, 5.600000e+01}, {1.680000e+02, -
4.000000e+01},
 *        {1.229282e+02, -7.215390e+00}, {6.321539e+01,
4.078461e+01},
 *        {1.090718e+02, -4.878461e+01}, {1.047846e+02, -7.846097e-
01}}
 */

```

2.3.1.6 fft(f)_plan_guru_dft

建立多组数据序列 n 维 C2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

2.3.1.6.1 接口定义

C interface:

```

fft_plan fft_plan_guru_dft(int rank, const fft_iodim *dims, int howmany_rank, const
fft_iodim *howmany_dims, fft_complex *in, fft_complex *out, int sign, unsigned
flags);

```

```
fftf_plan fftf_plan_guru_dft(int rank, const fftf_iodim *dims, int howmany_rank,
const fftf_iodim *howmany_dims, fftf_complex *in, fftf_complex *out, int sign,
unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_GURU_DFT(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, IN, OUT, SIGN, FLAGS);
```

```
RES = FFTF_PLAN_GURU_DFT(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, IN, OUT, SIGN, FLAGS);
```

2.3.1.6.2 返回值

函数返回一个 `fftf_plan` 类型的结构体指针。将该对象作为参数传入 `fftf_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fftf_execute_dft` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.1.6.3 参数

参数名	类型	描述	输入/输出
rank	int	单个 FFT 序列的维度，约束：1 ≤ rank ≤ 3。	输入
dims	双精度：const fftf_iodim* 单精度：const fftf_io dim*	dims 是大小为 rank 的结构体数组，dims[i] 包含以下成员： int n - 第 i 维 FFT 的长度。 int is - 第 i 维 FFT 输入序列的相继元素之间的间隔。 int os - 第 i 维 FFT 输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入

参数名	类型	描述	输入/输出
howmany_rank	int	多个 rank 维 FFT 之间的内存排布用 howmany_rank 维的 howmany_dims 数组来描述, howmany_rank 表示每个要计算的 rank 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束: $0 \leq \text{howmany_rank} \leq 3$	输入
howmany_dims	双精度: const fft_iodim* 单精度: const fftf_iodim*	howmany_dims 是大小为 howmany_rank 的结构体数组, howmany_dims[i] 包含以下成员: int n - howmany_rank 维空间的第 i 维的待变换 FFT 的个数。 int is - 第 i 维的相继 FFT 输入序列之间的间隔。 int os - 第 i 维的相继 FFT 输出序列之间的间隔。	输入
in	双精度: fft_complex* 单精度: fftf_complex*	输入待变换的数据。	输入
out	双精度: fft_complex* 单精度: fftf_complex*	输出快速傅里叶变换后的数据。	输出
sign	int	描述正向变换或反向变换。 -1(FFT_FORWARD): forward +1(FFT_BACKWARD): backward	输入
flags	unsigned int	planning 选项, 未使用。	输入

2.3.1.6.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.1.6.5 示例

C interface:

```

int rank = 2;

fft_iodim *dims;

dims = (fft_iodim*)fft_malloc(sizeof(fft_iodim) * rank);

dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 3;

dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;

int howmany_rank = 1;

fft_iodim *howmany_dims;

howmany_dims = (fft_iodim*)fft_malloc(sizeof(fft_iodim) *
howmany_rank);

howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 3;

double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16},
{-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8}};

fft_complex *in;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * 12);

for (int i = 0; i < 12; i++) {
    in[i].r = init[i][0];
    in[i].i = init[i][1];
}

fft_complex *out;

out = (fft_complex*)fft_malloc(sizeof(fft_complex) * 12);

fft_plan plan;

```

```

    plan = fft_plan_guru_dft(rank, dims, howmany_rank, howmany_dims, in,
out, FFT_FORWARD, FFT_ESTIMATE);

    fft_execute_dft(plan, in, out);

    fft_destroy_plan(plan);
    fft_free(howmany_dims);
    fft_free(dims);
    fft_free(in);
    fft_free(out);

    /*
        * out = {{1.200000e+02, 4.800000e+01}, {1.338564e+02, -
1.385641e+01}},
        *         {1.061436e+02, 1.385641e+01}, {1.360000e+02, -
3.200000e+01}},
        *         {1.120000e+02, -8.000000e+00}, {1.120000e+02, -
8.000000e+00}},
        *         {-8.000000e+01, -8.000000e+00}, {4.878461e+01, 7.846097e-
01}},
        *         {7.215390e+00, -4.078461e+01}, {1.600000e+01,
2.400000e+01}},
        *         {-2.692820e+01, -2.264102e+01}, {-1.307180e+01,
4.664102e+01}}
        */

```

2.3.1.7 fft(f)_plan_guru_split_dft

建立多组数据序列 n 维 C2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。split 类接口的输入和输出分别存储在实部和虚部数组中。

2.3.1.7.1 接口定义

C interface:

```
fft_plan fft_plan_guru_split_dft(int rank, const fft_iodim *dims, int howmany_rank,
const fft_iodim *howmany_dims, double *ri, double *ii, double *ro, double *io,
unsigned flags);
```

```
fftf_plan fftf_plan_guru_split_dft(int rank, const fftf_iodim *dims, int howmany_rank,
const fftf_iodim *howmany_dims, float *ri, float *ii, float *ro, float *io, unsigned
flags);
```

Fortran interface:

```
RES = FFT_PLAN_GURU_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, RI, II, RO, IO, FLAGS);
```

```
RES = FFTF_PLAN_GURU_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, RI, II, RO, IO, FLAGS);
```

2.3.1.7.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `ri`, `ii` 和输出 `ro`, `io` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_split_dft` 函数中以对新的输入 `ri`, `ii` 和输出 `ro`, `io` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.1.7.3 参数

参数名	类型	描述	输入/输出
<code>rank</code>	<code>int</code>	单个 FFT 序列的维度。 约束： $1 \leq \text{rank} \leq 3$ 。	输入
<code>dims</code>	双精度： <code>const fft_iodim*</code> 单精度： <code>const fftf_iodim*</code>	<code>dims</code> 是大小为 <code>rank</code> 的结构体数组， <code>dims[i]</code> 包含以下成员： <code>int n</code> - 第 <code>i</code> 维 FFT 的长度。 <code>int is</code> - 第 <code>i</code> 维 FFT 输入序列的相继元素之间的间隔。 <code>int os</code> - 第 <code>i</code> 维 FFT 输出序列的相继元素之间的间隔。 约束： $\text{dims}[i].n \geq 1$, for <code>i</code> in 0 to <code>rank - 1</code> 。	输入

参数名	类型	描述	输入/输出
howmany_rank	int	多个 rank 维 FFT 之间的内存排布用 howmany_rank 维的 howmany_dims 数组来描述，howmany_rank 表示每个要计算的 rank 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束：0≤howmany_rank≤3	输入
howmany_dims	双精度：const fft_iodim* 单精度：const fftf_io dim*	howmany_dims 是大小为 howmany_rank 的结构体数组，howmany_dims[i] 包含以下成员： int n - howmany_rank 维空间的第 i 维的待变换 FFT 的个数。 int is - 第 i 维的相继 FFT 输入序列之间的间隔。 int os - 第 i 维的相继 FFT 输出序列之间的间隔。	输入
ri	双精度：double* 单精度：float*	输入待变换数据的实部。	输入
ii	双精度：double* 单精度：float*	输入待变换数据的虚部。	输入
ro	双精度：double* 单精度：float*	输出待变换数据的实部。	输出
io	双精度：double* 单精度：float*	输出待变换数据的虚部。	输出
flags	unsigned int	planning 选项，未使用。	输入

2.3.1.7.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.1.7.5 示例

C interface:

```

int rank = 2;

fft_iodim *dims;

dims = (fft_iodim*)fft_malloc(sizeof(fft_iodim) * rank);

dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 3;

dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;

int howmany_rank = 1;

fft_iodim *howmany_dims;

howmany_dims = (fft_iodim*)fft_malloc(sizeof(fft_iodim) *
howmany_rank);

howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 3;

double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16},
{-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8}};

double *ri;

ri = (double*)fft_malloc(sizeof(double) * 12);

double *ii;

ii = (double*)fft_malloc(sizeof(double) * 12);

for (int i = 0; i < 12; i++) {
    ri[i] = init[i][0];
    ii[i] = init[i][1];
}

double *ro;

ro = (double*)fft_malloc(sizeof(double) * 12);

```



```
double *io;

io = (double*)fft_malloc(sizeof(double) * 12);

fft_plan plan;

plan = fft_plan_guru_split_dft(rank, dims, howmany_rank,
howmany_dims, ri, ii, ro, io, FFT_ESTIMATE);

fft_execute_split_dft(plan, ri, ii, ro, io);

fft_destroy_plan(plan);

fft_free(howmany_dims);

fft_free(dims);

fft_free(ri);

fft_free(ii);

fft_free(ro);

fft_free(io);

/*
 * ro = {1.200000e+02, 1.338564e+02, 1.061436e+02, 1.360000e+02,
 *       1.120000e+02, 1.120000e+02, -8.000000e+01, 4.878461e+01,
 *       7.215390e+00, 1.600000e+01, -2.692820e+01, -1.307180e+01}
 */
/*
 * io = {4.800000e+01, -1.385641e+01, 1.385641e+01, -3.200000e+01,
 *       -8.000000e+00, -8.000000e+00, -8.000000e+00, 7.846097e-01,
 *       -4.078461e+01, 2.400000e+01, -2.264102e+01, 4.664102e+01}
 */
```

2.3.1.8 fft(f)_plan_guru64_dft

建立多组数据序列 n 维 C2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。与 `fft(f)_plan_guru_dft` 不同的是，`fft(f)_plan_guru64_dft` 允许部分参数是 64 位的整型。

2.3.1.8.1 接口定义

C interface:

```
fft_plan fft_plan_guru64_dft(int rank, const fft_iodim64 *dims, int howmany_rank,
const fft_iodim64 *howmany_dims, fft_complex *in, fft_complex *out, int sign,
unsigned flags);
```

```
fftf_plan fftf_plan_guru64_dft(int rank, const fftf_iodim64 *dims, int howmany_rank,
const fftf_iodim64 *howmany_dims, fftf_complex *in, fftf_complex *out, int sign,
unsigned flags);
```

Fortran interface:

```
WJX B K Y d U Q F S d L Z W Z ; 9 d I K Y - W F S P I N R X I M T \ R F S ^ d W F S P I M T \ R F S ^ d I N R X I M S I
T Z Y I K N L S I K Q F L X . I
```

```
WJX B K Y K d U Q F S d L Z W Z ; 9 d I K Y - W F S P I N R X I M T \ R F S ^ d W F S P I M T \ R F S ^ d I N R X I M S I
T Z Y I K N L S I K Q F L X . I
```

2.3.1.8.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.1.8.3 参数

参数名	类型	描述	输入/输出
rank	int	单个 FFT 序列的维度。 约束： $1 \leq \text{rank} \leq 3$	输入
dims	双精度：const fft_iodim64*	dims 是大小为 rank 的结构体数	输入

参数名	类型	描述	输入/输出
	单精度: <code>const fftf_io dim64*</code>	组, <code>dims[i]</code> 包含以下成员: <code>ptrdiff_t n</code> - 第 <code>i</code> 维 FFT 的长度。 <code>ptrdiff_t is</code> - 第 <code>i</code> 维 FFT 输入序列的相继元素之间的间隔。 <code>ptrdiff_t os</code> - 第 <code>i</code> 维 FFT 输出序列的相继元素之间的间隔。 约束: <code>dims[i].n ≥ 1, for i in 0 to rank - 1</code>	
<code>howmany_rank</code>	<code>int</code>	多个 <code>rank</code> 维 FFT 之间的内存排布用 <code>howmany_rank</code> 维的 <code>howmany_dims</code> 数组来描述, <code>howmany_rank</code> 表示每个要计算的 <code>rank</code> 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束: <code>0 ≤ howmany_rank ≤ 3</code>	输入
<code>howmany_dims</code>	双精度: <code>const fftf_ioidim64*</code> 单精度: <code>const fftf_ioidim64*</code>	<code>howmany_dims</code> 是大小为 <code>howmany_rank</code> 的结构体数组, <code>howmany_dims[i]</code> 包含以下成员: <code>ptrdiff_t n</code> - <code>howmany_rank</code> 维空间的第 <code>i</code> 维的待变换 FFT 的个数。 <code>ptrdiff_t is</code> - 第 <code>i</code> 维的相继 FFT 输入序列之间的间隔。 <code>ptrdiff_t os</code> - 第 <code>i</code> 维的相继 FFT 输出序列之间的间隔。	输入
<code>in</code>	双精度: <code>fftf_complex*</code> 单精度: <code>fftf_complex*</code>	输入待变换的数据。	输入
<code>out</code>	双精度: <code>fftf_complex*</code> 单精度: <code>fftf_complex*</code>	输出快速傅里叶变换后的数据。	输出

参数名	类型	描述	输入/输出
sign	int	描述正向变换或反向变换。 -1(FFT_FORWARD): forward +1(FFT_BACKWARD): backward	输入
flags	unsigned int	planning 选项, 未使用。	输入

2.3.1.8.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.1.8.5 示例

C interface:

```

int rank = 2;

fft_iodim64 *dims;

dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) * rank);

dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 3;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;

int howmany_rank = 1;

fft_iodim64 *howmany_dims;

howmany_dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) *
howmany_rank);

howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 3;

double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16},
{-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8}};

fft_complex *in;

```

```

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * 12);
for (int i = 0; i < 12; i++) {
    in[i].r = init[i][0];
    in[i].i = init[i][1];
}

fft_complex *out;
out = (fft_complex*)fft_malloc(sizeof(fft_complex) * 12);

fft_plan plan;

plan = fft_plan_guru64_dft(rank, dims, howmany_rank, howmany_dims,
in, out, FFT_FORWARD, FFT_ESTIMATE);

fft_execute_dft(plan, in, out);

fft_destroy_plan(plan);
fft_free(howmany_dims);
fft_free(dims);
fft_free(in);
fft_free(out);

/*
 * out = {{1.200000e+02, 4.800000e+01}, {1.338564e+02, -
1.385641e+01},
 *         {1.061436e+02, 1.385641e+01}, {1.360000e+02, -
3.200000e+01},
 *         {1.120000e+02, -8.000000e+00}, {1.120000e+02, -
8.000000e+00},
 *         {-8.000000e+01, -8.000000e+00}, {4.878461e+01, 7.846097e-
01},
 *         {7.215390e+00, -4.078461e+01}, {1.600000e+01,
2.400000e+01},
 *         {-2.692820e+01, -2.264102e+01}, {-1.307180e+01,
4.664102e+01}}
 */

```

2.3.1.9 fft(f)_plan_guru64_split_dft

建立多组数据序列 n 维 C2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。split 类接口的输入和输出分别存储在实部和虚部数组中。与 `fft(f)_plan_guru_split_dft` 不同的是，`fft(f)_plan_guru64_split_dft` 允许部分参数是 64 位的整型。

2.3.1.9.1 接口定义

C interface:

```
fft_plan fft_plan_guru64_split_dft(int rank, const fft_iodim64 *dims, int
howmany_rank, const fft_iodim64 *howmany_dims, double *ri, double *ii, double
*ro, double *io, unsigned flags);
```

```
fftf_plan fftf_plan_guru64_split_dft(int rank, const fftf_iodim64 *dims, int
howmany_rank, const fftf_iodim64 *howmany_dims, float *ri, float *ii, float *ro,
float *io, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_GURU64_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, RI, II, RO, IO, FLAGS);
```

```
RES = FFTF_PLAN_GURU64_SPLIT_DFT(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, RI, II, RO, IO, FLAGS);
```

2.3.1.9.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `ri`, `ii` 和输出 `ro`, `io` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_split_dft` 函数中以对新的输入 `ri`, `ii` 和输出 `ro`, `io` 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.1.9.3 参数

参数名	类型	描述	输入/输出
rank	int	单个 FFT 序列的维度。 约束： $1 \leq \text{rank} \leq 3$ 。	输入

参数名	类型	描述	输入/输出
dims	双精度: const fft_iodim64*	dims 是大小为 rank 的结构体数组, dims[i] 包含以下成员: ptrdiff_t n - 第 i 维 FFT 的长度。 ptrdiff_t is - 第 i 维 FFT 输入序列的相继元素之间的间隔。 ptrdiff_t os - 第 i 维 FFT 输出序列的相继元素之间的间隔。 约束: $\text{dims}[i].n \geq 1$, for i in 0 to rank - 1	输入
	单精度: const fftf_io dim64*		
howmany_rank	int	多个 rank 维 FFT 之间的内存排布用 howmany_rank 维的 howmany_dims 数组来描述, howmany_rank 表示每个要计算的 rank 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束: $0 \leq \text{howmany_rank} \leq 3$	输入
howmany_dims	双精度: const fft_iodim64*	howmany_dims 是大小为 howmany_rank 的结构体数组, howmany_dims[i] 包含以下成员: ptrdiff_t n - howmany_rank 维空间的第 i 维的待变换 FFT 的个数。 ptrdiff_t is - 第 i 维的相继 FFT 输入序列之间的间隔。 ptrdiff_t os - 第 i 维的相继 FFT 输出序列之间的间隔。	输入
	单精度: const fftf_io dim64*		
ri	双精度: double*	输入待变换数据的实部。	输入
	单精度: float*		
ii	双精度: double*	输入待变换数据的虚部。	输入
	单精度: float*		
ro	双精度: double*	输出待变换数据的实部。	输出
	单精度: float*		

参数名	类型	描述	输入/输出
io	双精度: double*	输出待变换数据的虚部。	输出
	单精度: float*		
flags	unsigned int	planning 选项, 未使用。	输入

2.3.1.9.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.1.9.5 示例

C interface:

```

int rank = 2;
fft_iodim64 *dims;
dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) * rank);
dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 3;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
fft_iodim64 *howmany_dims;
howmany_dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) *
howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 3;
double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16},
{-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8}};
double *ri;
ri = (double*)fft_malloc(sizeof(double) * 12);
double *ii;

```



```

ii = (double*)fft_malloc(sizeof(double) * 12);
for (int i = 0; i < 12; i++) {
    ri[i] = init[i][0];
    ii[i] = init[i][1];
}

double *ro;
ro = (double*)fft_malloc(sizeof(double) * 12);

double *io;
io = (double*)fft_malloc(sizeof(double) * 12);

fft_plan plan;

plan = fft_plan_guru64_split_dft(rank, dims, howmany_rank,
howmany_dims, ri, ii, ro, io, FFT_ESTIMATE);

fft_execute_split_dft(plan, ri, ii, ro, io);

fft_destroy_plan(plan);
fft_free(howmany_dims);
fft_free(dims);
fft_free(ri);
fft_free(ii);
fft_free(ro);
fft_free(io);

/*
 * ro = {1.200000e+02, 1.338564e+02, 1.061436e+02, 1.360000e+02,
 *       1.120000e+02, 1.120000e+02, -8.000000e+01, 4.878461e+01,
 *       7.215390e+00, 1.600000e+01, -2.692820e+01, -1.307180e+01}
 */

/*
 * io = {4.800000e+01, -1.385641e+01, 1.385641e+01, -3.200000e+01,
 *       -8.000000e+00, -8.000000e+00, -8.000000e+00, 7.846097e-01,

```

```
*      -4.078461e+01, 2.400000e+01, -2.264102e+01, 4.664102e+01}  
*/
```

2.3.1.10 fft(f)_plan_many_dft

建立 `howmany` 组数据序列 `n` 维 C2C 变换的 `plan`。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

2.3.1.10.1 接口定义

C interface:

`fft_plan` `fft_plan_many_dft`(`int rank`, `const int *n`, `int howmany`, `fft_complex *in`, `const int *inembed`, `int istride`, `int idist`, `fft_complex *out`, `const int *onembed`, `int ostride`, `int odist`, `int sign`, `unsigned flags`);

`fftf_plan` `fftf_plan_many_dft`(`int rank`, `const int *n`, `int howmany`, `fftf_complex *in`, `const int *inembed`, `int istride`, `int idist`, `fftf_complex *out`, `const int *onembed`, `int ostride`, `int odist`, `int sign`, `unsigned flags`);

Fortran interface:

`RES = FFT_PLAN_MANY_DFT(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, SIGN, FLAGS);`

`RES = FFTF_PLAN_MANY_DFT(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, SIGN, FLAGS);`

2.3.1.10.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.1.10.3 参数

参数名	类型	描述	输入/输出
<code>rank</code>	<code>int</code>	FFT 变换的维度是 <code>rank</code> ，约束： $1 \leq \text{rank} \leq 3$ 。	输入

参数名	类型	描述	输入/输出
n	const int*	n 是维度为 rank 的数组，包含 FFT 序列每一维度的大小，约束： $n[i] \geq 1$, for i in $0, \text{rank} - 1$ 。	输入
howmany	int	howmany 表示要多少个多维 FFT 变换。	输入
in	双精度: fft_complex*	输入待变换的数据。	输入
	单精度: fftf_complex*		
inembed	const int*	inembed 是大小为 rank 的数组或者 NULL，该数组表示 FFT 数据 in 存储的更大空间的每一维度的大小。 约束: $\text{inembed}[i] \geq n[i]$ for i in $0, \text{rank}-1$ 或者如果 $\text{inembed} == \text{NULL}$ ，则可以认为 inembed 与 n 相等。	输入
istride	int	FFT 输入序列的相继元素之间的间隔。	输入
idist	int	idist 表示输入的每个 FFT 序列的间隔。	输入
out	双精度: fft_complex*	输出快速傅里叶变换后的数据。	输出
	单精度: fftf_complex*		
onembed	const int*	onembed 是大小为 rank 的数组或者 NULL，该数组表示 FFT 数据 out 存储的更大空间的每一维度的大小。 约束: $\text{onembed}[i] \geq n[i]$ for i in $0, \text{rank}-1$ 或者如果 $\text{onembed} == \text{NULL}$ ，则可以认为 onembed 与 n 相等。	输入
ostride	int	FFT 输出序列的相继元素之间的间隔。	输入
odist	int	odist 表示输出的每个 FFT 序列的间隔。	输入

参数名	类型	描述	输入/输出
sign	int	描述正向变换或反向变换。 -1(FFT_FORWARD): forward +1(FFT_BACKWARD): backward	输入
flags	unsigned int	planning 选项, 未使用。	输入

2.3.1.10.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.1.10.5 示例

C interface:

```

int rank = 1;

int *n;

n = (int*)fft_malloc(sizeof(int) * rank);

n[0] = 2;

int howmany = 3;

int istride = 1;

int ostride = 1;

int idist = 2;

int odist = 2;

double init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};

fft_complex *in;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * 6);

for (int i = 0; i < 6; i++) {

    in[i].r = init[i][0];

    in[i].i = init[i][1];

}

fft_complex *out;

out = (fft_complex*)fft_malloc(sizeof(fft_complex) * 6);

```

```

fft_plan plan;

plan = fft_plan_many_dft(rank, n, howmany, in, NULL, istride, idist,
out, NULL, ostride, odist, FFT_FORWARD, FFT_ESTIMATE);

fft_execute_dft(plan, in, out);

fft_destroy_plan(plan);

fft_free(n);

fft_free(in);

fft_free(out);

/*
 * out = {{1.280000e+02, 8.000000e+00}, {1.120000e+02, -
8.000000e+00},
 *        {0.000000e+00, 1.600000e+01}, {0.000000e+00, -
1.600000e+01},
 *        {-8.000000e+00, 2.400000e+01}, {8.000000e+00,
8.000000e+00}}
 */

```

2.3.2 R2C 变换

2.3.2.1 函数说明

函数名	简介
fft(f)_plan_dft_r2c	建立单个连续数据序列 n 维 R2C 变换的 plan。
fft(f)_plan_dft_r2c_1d	建立单个连续数据序列 1 维 R2C 变换的 plan。
fft(f)_plan_dft_r2c_2d	建立单个连续数据序列 2 维 R2C 变换的 plan。
fft(f)_plan_dft_r2c_3d	建立单个连续数据序列 3 维 R2C 变换的 plan。
fft(f)_plan_guru_dft_r2c	建立多组数据序列 n 维 R2C 变换的 plan。
fft(f)_plan_guru_split_dft_r2c	建立多组数据序列 n 维 R2C 变换的 plan。

函数名	简介
<code>fft(f)_plan_guru64_dft_r2c</code>	建立多组数据序列 n 维 R2C 变换的 plan。
<code>fft(f)_plan_guru64_split_dft_r2c</code>	建立多组数据序列 n 维 R2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。
<code>fft(f)_plan_many_dft_r2c</code>	建立 howmany 组数据序列 n 维 R2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

2.3.2.2 `fft(f)_plan_dft_r2c`

建立单个连续数据序列 n 维 R2C 变换的 plan。

2.3.2.2.1 接口定义

C interface:

```
fft_plan fft_plan_dft_r2c(int rank, const int *n, double *in, fft_complex *out,
unsigned flags);
```

```
fftf_plan fftf_plan_dft_r2c(int rank, const int *n, float *in, fftf_complex *out,
unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_R2C(RANK, N, IN, OUT, FLAGS);
```

```
RES = FFTF_PLAN_DFT_R2C(RANK, N, IN, OUT, FLAGS);
```

2.3.2.2.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_r2c` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.2.2.3 参数

参数名	类型	描述	输入/输出
rank	int	FFT 变换的维度是 rank，约束： $1 \leq \text{rank} \leq 3$ 。	输入
n	const int*	n 是维度为 rank 的数组，包含 FFT 序列每一维度的大小，约束： $n[i] \geq 1$, for i in 0 to rank - 1。	输入
in	双精度: double*	输入待变换的数据。	输入
	单精度: float*		
out	双精度: fft_complex*	输出快速傅里叶变换后的数据。	输出
	单精度: fftf_complex*		
flags	unsigned int	planning 选项，未使用。	输入

2.3.2.2.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.2.2.5 示例

C interface:

```
int rank = 2;
int *n;
n = (int*)fft_malloc(sizeof(int) * rank);
n[0] = 2;
n[1] = 4;
double init[8] = {1, 2, 3, 4, 5, 6, 7, 8};
double *in;
in = (double*)fft_malloc(sizeof(double) * n[0] * n[1]);
for (int i = 0; i < n[0] * n[1]; i++) {
    in[i] = init[i];
}
fftf_complex *out;
```

```

    out = (fft_complex*)fft_malloc(sizeof(fft_complex) * n[0] * (n[1] /
2 + 1));

    fft_plan plan;

    plan = fft_plan_dft_r2c(rank, n, in, out, FFT_ESTIMATE);

    fft_execute_dft_r2c(plan, in, out);

    fft_destroy_plan(plan);

    fft_free(n);

    fft_free(in);

    fft_free(out);

    /*
     * out = {{3.600000e+01, 0.000000e+00}, {-4.000000e+00,
4.000000e+00},
     *        {-4.000000e+00, 0.000000e+00}, {-1.600000e+01,
0.000000e+00},
     *        {0.000000e+00, 0.000000e+00}, {0.000000e+00,
0.000000e+00}}
     */

```

2.3.2.3 fft(f)_plan_dft_r2c_1d

建立单个连续数据序列 1 维 R2C 变换的 plan。

2.3.2.3.1 接口定义

C interface:

```
fft_plan fft_plan_dft_r2c_1d(int n, double *in, fft_complex *out, unsigned flags);
```

```
fftf_plan fftf_plan_dft_r2c_1d(int n, float *in, fftf_complex *out, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_R2C_1D(N, IN, OUT, FLAGS);
```

```
RES = FFTF_PLAN_DFT_R2C_1D(N, IN, OUT, FLAGS);
```


2.3.2.3.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_r2c` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.2.3.3 参数

参数名	类型	描述	输入/输出
<code>n</code>	<code>int</code>	输入 FFT 序列的大小，约束： $n \geq 1$ 。	输入
<code>in</code>	双精度： <code>double*</code>	输入待变换的数据。	输入
	单精度： <code>float*</code>		
<code>out</code>	双精度： <code>fft_complex*</code>	输出快速傅里叶变换后的数据。	输出
	单精度： <code>fftf_complex*</code>		
<code>flags</code>	<code>unsigned int</code>	<code>planning</code> 选项，未使用。	输入

2.3.2.3.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.2.3.5 示例

C interface:

```
int n0 = 4;

double *in;

double init[4] = {1, 2, 3, 4};

in = (double*)fft_malloc(sizeof(double) * n0);

for (int i = 0; i < n0; i++) {
    in[i] = init[i];
}
```

```

fft_complex *out;

out = (fft_complex*)fft_malloc(sizeof(fft_complex) * (n0 / 2 + 1));

fft_plan plan;

plan = fft_plan_dft_r2c_1d(n0, in, out, FFT_ESTIMATE);

fft_execute_dft_r2c(plan, in, out);


fft_destroy_plan(plan);

fft_free(in);

fft_free(out);


/*
 * out = {{1.000000e+01, 0.000000e+00}, {-2.000000e+00,
2.000000e+00},
 *        {-2.000000e+00, 0.000000e+00}}
 */

```

2.3.2.4 fft(f)_plan_dft_r2c_2d

建立单个连续数据序列 2 维 R2C 变换的 plan。

2.3.2.4.1 接口定义

C interface:

```
fft_plan fft_plan_dft_r2c_2d(int n0, int n1, double *in, fft_complex *out, unsigned flags);
```

```
fftf_plan fftf_plan_dft_r2c_2d(int n0, int n1, float *in, fftf_complex *out, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_R2C_2D(N0, N1, IN, OUT, FLAGS);
```

```
RES = FFTF_PLAN_DFT_R2C_2D(N0, N1, IN, OUT, FLAGS);
```

2.3.2.4.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入

fft(f)_execute 函数中使用，将对当前提供的输入 in 和输出 out 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 fft(f)_execute_dft_r2c 函数中以对新的输入 in 和输出 out 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.2.4.3 参数

参数名	类型	描述	输入/输出
n0	int	输入 FFT 序列第一维的大小，约束： $n0 \geq 1$ 。	输入
n1	int	输入 FFT 序列第二维的大小，约束： $n1 \geq 1$ 。	输入
in	双精度：double*	输入待变换的数据。	输入
	单精度：float*		
out	双精度：fft_complex*	输出快速傅里叶变换后的数据。	输出
	单精度：fftf_complex*		
flags	unsigned int	planning 选项，未使用。	输入

2.3.2.4.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.2.4.5 示例

C interface:

```
int n0 = 2;
int n1 = 4;

double init[8] = {1, 2, 3, 4, 5, 6, 7, 8};

double *in;

in = (double*)fft_malloc(sizeof(double) * n0 * n1);

for (int i = 0; i < n0 * n1; i++) {
    in[i] = init[i];
}

fft_complex *out;
```

```

    out = (fft_complex*)fft_malloc(sizeof(fft_complex) * n0 * (n1 / 2 +
1));

    fft_plan plan;

    plan = fft_plan_dft_r2c_2d(n0, n1, in, out, FFT_ESTIMATE);

    fft_execute_dft_r2c(plan, in, out);

    fft_destroy_plan(plan);

    fft_free(in);

    fft_free(out);

    /*
     * out = {{3.600000e+01, 0.000000e+00}, {-4.000000e+00,
4.000000e+00},
     *        {-4.000000e+00, 0.000000e+00}, {-1.600000e+01,
0.000000e+00},
     *        {0.000000e+00, 0.000000e+00}, {0.000000e+00,
0.000000e+00}}
     */

```

2.3.2.5 fft(f)_plan_dft_r2c_3d

建立单个连续数据序列 3 维 R2C 变换的 plan。

2.3.2.5.1 接口定义

C interface:

```
fft_plan fft_plan_dft_r2c_3d(int n0, int n1, int n2, double *in, fft_complex *out,
unsigned flags);
```

```
fftf_plan fftf_plan_dft_r2c_3d(int n0, int n1, int n2, float *in, fftf_complex *out,
unsigned flags);
```

Fortran interface:

```
RES = fft_plan_dft_r2c_3d(N0, N1, N2, IN, OUT, FLAGS);
```

```
RES = fftf_plan_dft_r2c_3d(N0, N1, N2, IN, OUT, FLAGS);
```

2.3.2.5.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_r2c` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.2.5.3 参数

参数名	类型	描述	输入/输出
<code>n0</code>	<code>int</code>	输入 FFT 序列第一维的大小，约束： $n0 \geq 1$ 。	输入
<code>n1</code>	<code>int</code>	输入 FFT 序列第二维的大小，约束： $n1 \geq 1$ 。	输入
<code>n2</code>	<code>int</code>	输入 FFT 序列第三维的大小，约束： $n2 \geq 1$ 。	输入
<code>in</code>	双精度： <code>double*</code>	输入待变换的数据。	输入
	单精度： <code>float*</code>		
<code>out</code>	双精度： <code>fft_complex*</code>	输出快速傅里叶变换后的数据。	输出
	单精度： <code>fftf_complex*</code>		
<code>flags</code>	<code>unsigned int</code>	<code>planning</code> 选项，未使用。	输入

2.3.2.5.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.2.5.5 示例

C interface:

```
int n0 = 2;
int n1 = 3;
int n2 = 2;

double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};

double *in;
```

```

in = (double*)fft_malloc(sizeof(double) * n0 * n1 * n2);
for (int i = 0; i < n0 * n1 * n2; i++) {
    in[i] = init[i];
}

fft_complex *out;

out = (fft_complex*)fft_malloc(sizeof(fft_complex) * n0 * n1 * (n2
/ 2 + 1));

fft_plan plan;

plan = fft_plan_dft_r2c_3d(n0, n1, n2, in, out, FFT_ESTIMATE);

fft_execute_dft_r2c(plan, in, out);

fft_destroy_plan(plan);

fft_free(in);

fft_free(out);

/*
 * out = {{4.000000e+01, 0.000000e+00}, {7.200000e+01,
0.000000e+00},
 *        {1.480000e+02, -3.464102e+01}, {1.320000e+02, -
6.928203e+00},
 *        {1.480000e+02, 3.464102e+01}, {1.320000e+02,
6.928203e+00},
 *        {2.000000e+02, 0.000000e+00}, {1.680000e+02,
0.000000e+00},
 *        {1.160000e+02, 2.078461e+01}, {8.400000e+01,
2.078461e+01},
 *        {1.160000e+02, -2.078461e+01}, {8.400000e+01, -
2.078461e+01}}
 */

```

2.3.2.6 fft(f)_plan_guru_dft_r2c

建立多组数据序列 n 维 R2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

2.3.2.6.1 接口定义

C interface:

```
fft_plan fft_plan_guru_dft_r2c(int rank, const fft_iodim *dims, int howmany_rank,
const fft_iodim *howmany_dims,
```

```
double *in, fft_complex *out, unsigned flags);
```

```
fftf_plan fftf_plan_guru_dft_r2c(int rank, const fftf_iodim *dims, int howmany_rank,
const fftf_iodim *howmany_dims, float *in, fftf_complex *out, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_GURU_DFT_R2C(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, IN, OUT, FLAGS);
```

```
RES = FFTF_PLAN_GURU_DFT_R2C(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, IN, OUT, FLAGS);
```

2.3.2.6.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_r2c` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.2.6.3 参数

参数名	类型	描述	输入/输出
rank	int	单个 FFT 序列的维度 约束: $1 \leq \text{rank} \leq 3$	输入
dims	双精度: <code>const fft_iodim*</code>	dims 是大小为 rank 的结构体数	输入

参数名	类型	描述	输入/输出
	单精度: <code>const fftf_io dim*</code>	组, <code>dims[i]</code> 包含以下成员: <code>int n</code> - 第 <code>i</code> 维 FFT 的长度。 <code>int is</code> - 第 <code>i</code> 维 FFT 输入序列的相继元素之间的间隔。 <code>int os</code> - 第 <code>i</code> 维 FFT 输出序列的相继元素之间的间隔。 约束: <code>dims[i].n ≥ 1, for i in 0 to rank - 1</code> 。	
<code>howmany_rank</code>	<code>int</code>	多个 <code>rank</code> 维 FFT 之间的内存排布用 <code>howmany_rank</code> 维的 <code>howmany_dims</code> 数组来描述, <code>howmany_rank</code> 表示每个要计算的 <code>rank</code> 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束: <code>0 ≤ howmany_rank ≤ 3</code>	输入
<code>howmany_dims</code>	双精度: <code>const fft_iodim*</code> 单精度: <code>const fftf_io dim*</code>	<code>howmany_dims</code> 是大小为 <code>howmany_rank</code> 的结构体数组, <code>howmany_dims[i]</code> 包含以下成员: <code>int n</code> - <code>howmany_rank</code> 维空间的第 <code>i</code> 维的待变换 FFT 的个数。 <code>int is</code> - 第 <code>i</code> 维的相继 FFT 输入序列之间的间隔。 <code>int os</code> - 第 <code>i</code> 维的相继 FFT 输出序列之间的间隔。	输入
<code>in</code>	双精度: <code>double*</code> 单精度: <code>float*</code>	输入待变换的数据。	输入
<code>out</code>	双精度: <code>fft_complex*</code> 单精度: <code>fftf_complex*</code>	输出快速傅里叶变换后的数据。	输出
<code>flags</code>	<code>unsigned int</code>	<code>planning</code> 选项, 未使用。	输入

2.3.2.6.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.2.6.5 示例

C interface:

```

int rank = 2;
fft_iodim *dims;
dims = (fft_iodim*)fft_malloc(sizeof(fft_iodim) * rank);
dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 2;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
fft_iodim *howmany_dims;
howmany_dims = (fft_iodim*)fft_malloc(sizeof(fft_iodim) *
howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 2;
double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
double *in;
in = (double*)fft_malloc(sizeof(double) * 12);
for (int i = 0; i < 12; i++) {
    in[i] = init[i];
}
fft_complex *out;
out = (fft_complex*)fft_malloc(sizeof(fft_complex) * 8);
fft_plan plan;
plan = fft_plan_guru_dft_r2c(rank, dims, howmany_rank, howmany_dims,
in, out, FFT_ESTIMATE);
fft_execute_dft_r2c(plan, in, out);

```

```

fft_destroy_plan(plan);
fft_free(howmany_dims);
fft_free(dims);
fft_free(in);
fft_free(out);

/*
 * out = {{1.200000e+02, 0.000000e+00}, {1.200000e+02, -
1.385641e+01},
 *        {1.360000e+02, 0.000000e+00}, {1.120000e+02,
0.000000e+00},
 *        {-8.000000e+01, 0.000000e+00}, {2.800000e+01, -
2.078461e+01},
 *        {1.600000e+01, 0.000000e+00}, {-2.000000e+01, -
3.464102e+01}}
 */

```

2.3.2.7 fft(f) plan_guru_split_dft_r2c

建立多组数据序列 n 维 R2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。split 类接口的输入和输出分别存储在实部和虚部数组中。

2.3.2.7.1 接口定义

C interface:

```

fft_plan fft_plan_guru_split_dft_r2c(int rank, const fft_iodim *dims, int
howmany_rank, const fft_iodim *howmany_dims, double *in, double *ro, double *io,
unsigned flags);

```

```

fftf_plan fftf_plan_guru_split_dft_r2c(int rank, const fftf_iodim *dims, int
howmany_rank, const fftf_iodim *howmany_dims, float *in, float *ro, float *io,
unsigned flags);

```

Fortran interface:

```
RES = FFT_PLAN_GURU_SPLIT_DFT_R2C(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, IN, RO, IO, FLAGS);
```

```
RES = FFTF_PLAN_GURU_SPLIT_DFT_R2C(RANK, DIMS,
HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);
```

2.3.2.7.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `ro`, `io` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_split_dft_r2c` 函数中以对新的输入 `in` 和输出 `ro`, `io` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.2.7.3 参数

参数名	类型	描述	输入/输出
<code>rank</code>	<code>int</code>	单个 FFT 序列的维度。 约束： $1 \leq \text{rank} \leq 3$ 。	输入
<code>dims</code>	双精度： <code>const fft_iodim*</code> 单精度： <code>const fftf_io dim*</code>	<code>dims</code> 是大小为 <code>rank</code> 的结构体数组， <code>dims[i]</code> 包含以下成员： <code>int n</code> - 第 <code>i</code> 维 FFT 的长度。 <code>int is</code> - 第 <code>i</code> 维 FFT 输入序列的相继元素之间的间隔。 <code>int os</code> - 第 <code>i</code> 维 FFT 输出序列的相继元素之间的间隔。 约束： $\text{dims}[i].n \geq 1$, for <code>i</code> in 0 to <code>rank - 1</code> 。	输入
<code>howmany_rank</code>	<code>int</code>	多个 <code>rank</code> 维 FFT 之间的内存排布用 <code>howmany_rank</code> 维的 <code>howmany_dims</code> 数组来描述， <code>howmany_rank</code> 表示每个要计算的 <code>rank</code> 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束： $0 \leq \text{howmany_rank} \leq 3$	输入
<code>howmany_dim</code>	双精度： <code>const fft_iodim*</code>	<code>howmany_dims</code> 是大小为 <code>howmany_rank</code> 的结构体数组，	输入

参数名	类型	描述	输入/输出
s	单精度: <code>const fftf_io dim*</code>	<code>howmany_dims[i]</code>包含以下成员: <code>int n</code> - <code>howmany_rank</code> 维空间的第 <code>i</code> 维的待变换 FFT 的个数。 <code>int is</code> - 第 <code>i</code> 维的相继 FFT 输入序列之间的间隔。 <code>int os</code> - 第 <code>i</code> 维的相继 FFT 输出序列之间的间隔。	
in	双精度: <code>double*</code>	输入待变换数据。	输入
	单精度: <code>float*</code>		
ro	双精度: <code>double*</code>	输出待变换数据的实部。	输出
	单精度: <code>float*</code>		
io	双精度: <code>double*</code>	输出待变换数据的虚部。	输出
	单精度: <code>float*</code>		
flags	<code>unsigned int</code>	<code>planning</code> 选项, 未使用。	输入

2.3.2.7.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.2.7.5 示例

C interface:

```
int rank = 2;

fftf_iodim *dims;

dims = (fftf_iodim*)fftf_malloc(sizeof(fftf_iodim) * rank);

dims[0].n = 2;

dims[0].is = 3;

dims[0].os = 2;

dims[1].n = 3;

dims[1].is = 1;

dims[1].os = 1;
```

```

int howmany_rank = 1;

fft_iodim *howmany_dims;

howmany_dims = (fft_iodim*)fft_malloc(sizeof(fft_iodim) *
howmany_rank);

howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 2;

double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};

double *in;

in = (double*)fft_malloc(sizeof(double) * 12);
for (int i = 0; i < 12; i++) {
    in[i] = init[i];
}

fft_plan plan;

double *ro;

double *io;

ro = (double*)fft_malloc(sizeof(double) * 8);
io = (double*)fft_malloc(sizeof(double) * 8);

plan = fft_plan_guru_split_dft_r2c(rank, dims, howmany_rank,
howmany_dims, in, ro, io, FFT_ESTIMATE);

fft_execute_split_dft_r2c(plan, in, ro, io);

fft_destroy_plan(plan);
fft_free(howmany_dims);
fft_free(dims);
fft_free(in);
fft_free(ro);
fft_free(io);

/*
 * ro = {1.200000e+02, 1.200000e+02, 1.360000e+02, 1.120000e+02,
 *       -8.000000e+01, 2.800000e+01, 1.600000e+01, -2.000000e+01}

```

```

*/

/*
 * io = {0.000000e+00, -1.385641e+01, 0.000000e+00, 0.000000e+00,
 *       0.000000e+00, 2.078461e+01, 0.000000e+00, -3.464102e+01}
 */

```

2.3.2.8 fft(f)_plan_guru64_dft_r2c

建立多组数据序列 n 维 R2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。与 `fft(f)_plan_guru_dft` 不同的是，`fft(f)_plan_guru64_dft` 允许部分参数是 64 位的整型。

2.3.2.8.1 接口定义

C interface:

```
fft_plan fft_plan_guru64_dft_r2c(int rank, const fft_iodim64 *dims, int
howmany_rank, const fft_iodim64 *howmany_dims, double *in, fft_complex *out,
unsigned flags);
```

```
fftf_plan fftf_plan_guru64_dft_r2c(int rank, const fftf_iodim64 *dims, int
howmany_rank, const fftf_iodim64 *howmany_dims, float *in, fftf_complex *out,
unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS,
HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);
RES = FFTF_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS,
HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);
```

2.3.2.8.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_r2c` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.2.8.3 参数

参数名	类型	描述	输入/输出
rank	int	单个 FFT 序列的维度。 约束: $1 \leq \text{rank} \leq 3$	输入
dims	双精度: const fft_iodim64* 单精度: const fftf_io dim64*	dims 是大小为 rank 的结构体数组, dims[i] 包含以下成员: ptrdiff_t n - 第 i 维 FFT 的长度。 ptrdiff_t is - 第 i 维 FFT 输入序列的相继元素之间的间隔。 ptrdiff_t os - 第 i 维 FFT 输出序列的相继元素之间的间隔。 约束: $\text{dims}[i].n \geq 1$, for i in 0 to rank - 1	输入
howmany_rank	int	多个 rank 维 FFT 之间的内存排布用 howmany_rank 维的 howmany_dims 数组来描述, howmany_rank 表示每个要计算的 rank 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束: $0 \leq \text{howmany_rank} \leq 3$	输入
howmany_dims	双精度: const fft_iodim64* 单精度: const fftf_io dim64*	howmany_dims 是大小为 howmany_rank 的结构体数组, howmany_dims[i] 包含以下成员: ptrdiff_t n - howmany_rank 维空间的第 i 维的待变换 FFT 的个数。 ptrdiff_t is - 第 i 维的相继 FFT 输入序列之间的间隔。 ptrdiff_t os - 第 i 维的相继 FFT 输出序列之间的间隔。	输入
in	双精度: double* 单精度: float*	输入待变换的数据。	输入
out	双精度: fft_complex* 单精度: fftf_complex*	输出快速傅里叶变换后的数据。	输出

参数名	类型	描述	输入/输出
flags	unsigned int	planning 选项，未使用。	输入

2.3.2.8.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.2.8.5 示例

C interface:

```
int rank = 2;
fft_iodim64 *dims;
dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) * rank);
dims[0].n = 2;
dims[0].is = 3;
dims[0].os = 2;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
fft_iodim64 *howmany_dims;
howmany_dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) *
howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 3;
howmany_dims[0].os = 2 * 2;
double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};
double *in;
in = (double*)fft_malloc(sizeof(double) * 12);
for (int i = 0; i < 12; i++) {
    in[i] = init[i];
}
```



```

fft_complex *out;

out = (fft_complex*)fft_malloc(sizeof(fft_complex) * 8);

fft_plan plan;

plan = fft_plan_guru64_dft_r2c(rank, dims, howmany_rank,
howmany_dims, in, out, FFT_ESTIMATE);

fft_execute_dft_r2c(plan, in, out);


fft_destroy_plan(plan);

fft_free(howmany_dims);

fft_free(dims);

fft_free(in);

fft_free(out);


/*
 * out = {{1.200000e+02, 0.000000e+00}, {1.200000e+02, -
1.385641e+01},
 *        {1.360000e+02, 0.000000e+00}, {1.120000e+02,
0.000000e+00},
 *        {-8.000000e+01, 0.000000e+00}, {2.800000e+01,
2.078461e+01},
 *        {1.600000e+01, 0.000000e+00}, {-2.000000e+01, -
3.464102e+01}}
 */

```

2.3.2.9 fft(f)_plan_guru64_split_dft_r2c

建立多组数据序列 n 维 R2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。split 类接口的输入和输出分别存储在实部和虚部数组中。与 `fft(f)_plan_guru_split_dft_r2c` 不同的是，`fft(f)_plan_guru64_split_dft_r2c` 允许部分参数是 64 位的整型。

2.3.2.9.1 接口定义

C interface:

```
fft_plan fft_plan_guru64_split_dft_r2c(int rank, const fft_iodim64 *dims, int
howmany_rank, const fft_iodim64 *howmany_dims, double *in, double *ro, double
*io, unsigned flags);
```

```
fftf_plan fftf_plan_guru64_split_dft_r2c(int rank, const fftf_iodim64 *dims, int
howmany_rank, const fftf_iodim64 *howmany_dims, float *in, float *ro, float *io,
unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS,
HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);
```

```
RES = FFTF_PLAN_GURU64_SPLIT_DFT_R2C(RANK, DIMS,
HOWMANY_RANK, HOWMANY_DIMS, IN, RO, IO, FLAGS);
```

2.3.2.9.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `ro`, `io` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_split_dft_r2c` 函数中以对新的输入 `in` 和输出 `ro`, `io` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.2.9.3 参数

参数名	类型	描述	输入/输出
<code>rank</code>	<code>int</code>	单个 FFT 序列的维度。 约束： $1 \leq \text{rank} \leq 3$ 。	输入
<code>dims</code>	单精度：const <code>fft_iodim64*</code> 双精度：const <code>fftf_io dim64*</code>	<code>dims</code> 是大小为 <code>rank</code> 的结构体数组， <code>dims[i]</code> 包含以下成员： <code>ptrdiff_t n</code> - 第 <code>i</code> 维 FFT 的长度。 <code>ptrdiff_t is</code> - 第 <code>i</code> 维 FFT 输入序列的相继元素之间的间隔。 <code>ptrdiff_t os</code> - 第 <code>i</code> 维 FFT 输出序列的相继元素之间的间隔。 约束： $\text{dims}[i].n \geq 1$, for <code>i</code> in 0 to <code>rank - 1</code>	输入

参数名	类型	描述	输入/输出
howmany_rank	int	多个 rank 维 FFT 之间的内存排布用 howmany_rank 维的 howmany_dims 数组来描述，howmany_rank 表示每个要计算的 rank 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束：0≤howmany_rank≤3	输入
howmany_dims	单精度：const fft_iodim64* 双精度：const fftf_io dim64*	howmany_dims 是大小为 howmany_rank 的结构体数组，howmany_dims[i] 包含以下成员： ptrdiff_t n - howmany_rank 维空间的第 i 维的待变换 FFT 的个数。 ptrdiff_t is - 第 i 维的相继 FFT 输入序列之间的间隔。 ptrdiff_t os - 第 i 维的相继 FFT 输出序列之间的间隔。	输入
in	双精度：double* 单精度：float*	输入待变换数据。	输入
ro	双精度：double* 单精度：float*	输出待变换数据的实部。	输出
io	双精度：double* 单精度：float*	输出待变换数据的虚部。	输出
flags	unsigned int	planning 选项，未使用。	输入

2.3.2.9.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.2.9.5 示例

C interface:

```
int rank = 2;

fft_iodim64 *dims;

dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) * rank);

dims[0].n = 2;

dims[0].is = 3;

dims[0].os = 2;

dims[1].n = 3;

dims[1].is = 1;

dims[1].os = 1;

int howmany_rank = 1;

fft_iodim64 *howmany_dims;

howmany_dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) *
howmany_rank);

howmany_dims[0].n = 2;

howmany_dims[0].is = 2 * 3;

howmany_dims[0].os = 2 * 2;

double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};

double *in;

in = (double*)fft_malloc(sizeof(double) * 12);

for (int i = 0; i < 12; i++) {
    in[i] = init[i];
}

fft_plan plan;

double *ro;

double *io;

ro = (double*)fft_malloc(sizeof(double) * 8);

io = (double*)fft_malloc(sizeof(double) * 8);

plan = fft_plan_guru64_split_dft_r2c(rank, dims, howmany_rank,
howmany_dims, in, ro, io, FFT_ESTIMATE);

fft_execute_split_dft_r2c(plan, in, ro, io);

fft_destroy_plan(plan);
```

```

fft_free(howmany_dims);
fft_free(dims);
fft_free(in);
fft_free(ro);
fft_free(io);

/*
 * ro = {1.200000e+02, 1.200000e+02, 1.360000e+02, 1.120000e+02,
 *       -8.000000e+01, 2.800000e+01, 1.600000e+01, -2.000000e+01}
 */

/*
 * io = {0.000000e+00, -1.385641e+01, 0.000000e+00, 0.000000e+00,
 *       0.000000e+00, 2.078461e+01, 0.000000e+00, -3.464102e+01}
 */

```

2.3.2.10 fft(f)_plan_many_dft_r2c

建立 howmany 组数据序列 n 维 R2C 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

2.3.2.10.1 接口定义

C interface:

```
fft_plan fft_plan_many_dft_r2c(int rank, const int *n, int howmany, double *in, const
int *inembed, int istride, int idist, fft_complex *out, const int *onembed, int ostride,
int odist, unsigned flags);
```

```
fftf_plan fftf_plan_many_dft_r2c(int rank, const int *n, int howmany, float *in, const
int *inembed, int istride, int idist, fftf_complex *out, const int *onembed, int ostride,
int odist, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_MANY_DFT_R2C(RANK, N, HOWMANY, IN, INEMBED,
ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, FLAGS);
```

RES = FFTF_PLAN_MANY_DFT_R2C(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, FLAGS);

2.3.2.10.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_r2c` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.2.10.3 参数

参数名	类型	描述	输入/输出
rank	int	FFT 变换的维度是 rank，约束： $1 \leq \text{rank} \leq 3$ 。	输入
n	const int*	n 是维度为 rank 的数组，包含 FFT 序列每一维度的大小，约束： $n[i] \geq 1$, for i in 0 to rank - 1。	输入
howmany	int	howmany 表示要多少个多维 FFT 变换。	输入
in	双精度: double*	输入待变换的数据。	输入
	单精度: float*		
inembed	const int*	inembed 是大小为 rank 的数组或者 NULL，该数组表示 FFT 数据 in 存储的更大空间的每一维度的大小。 约束: $\text{inembed}[i] \geq n[i]$ for i in 0, rank-1 或者如果 $\text{inembed} == \text{NULL}$ ，则可以认为 inembed 与 n 相等。	输入
istride	int	FFT 输入序列的相继元素之间的间隔。	输入
idist	int	idist 表示输入的每个 FFT 序列的间隔。	输入
out	双精度: fft_complex*	输出快速傅里叶变换后的数据。	输出

参数名	类型	描述	输入/输出
	单精度: <code>fftf_complex*</code>		
<code>onembed</code>	<code>const int*</code>	<code>onembed</code> 是大小为 <code>rank</code> 的数组或者 <code>NULL</code> ，该数组表示 FFT 数据 <code>out</code> 存储的更大空间的每一维度的大小。 约束: <code>onembed[i] ≥ n[i]</code> for <code>i</code> in <code>0, rank-1</code> 或者如果 <code>onembed == NULL</code> ，则可以认为 <code>onembed</code> 与 <code>n</code> 相等。	输入
<code>ostride</code>	<code>int</code>	FFT 输出序列的相继元素之间的间隔。	输入
<code>odist</code>	<code>int</code>	<code>odist</code> 表示输出的每个 FFT 序列的间隔。	输入
<code>flags</code>	<code>unsigned int</code>	<code>planning</code> 选项，未使用。	输入

2.3.2.10.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.2.10.5 示例

C interface:

```
int rank = 1;
int *n;

n = (int*)fft_malloc(sizeof(int) * rank);
n[0] = 4;

int howmany = 3;
int istride = 1;
int ostride = 1;
int idist = 4;
int odist = n[0] / 2 + 1;

double init[12] = {120, 8, 0, 0, 0, -8, -8, -8, -16, 0, -40, -8};

double *in;
```

```

in = (double*)fft_malloc(sizeof(double) * 12);
for (int i = 0; i < 12; i++) {
    in[i] = init[i];
}

fft_complex *out;

out = (fft_complex*)fft_malloc(sizeof(fft_complex) * (n[0] / 2 + 1)
* howmany);

fft_plan plan;

plan = fft_plan_many_dft_r2c(rank, n, howmany, in, NULL, istride,
idist, out, NULL, ostride, odist, FFT_ESTIMATE);

fft_execute_dft_r2c(plan, in, out);

fft_destroy_plan(plan);
fft_free(n);
fft_free(in);
fft_free(out);

/*
 * out = {{1.280000e+02, 0.000000e+00}, {1.200000e+02, -
8.000000e+00},
 *        {1.120000e+02, 0.000000e+00}, {-2.400000e+01,
0.000000e+00},
 *        {8.000000e+00, -0.000000e+00}, {8.000000e+00,
0.000000e+00},
 *        {-6.400000e+01, 0.000000e+00}, {2.400000e+01, -
8.000000e+00},
 *        {-4.800000e+01, 0.000000e+00}}
 */

```


2.3.3 C2R 变换

2.3.3.1 函数说明

函数名	简介
<code>fft(f)_plan_dft_c2r</code>	建立单个连续数据序列 n 维 C2R 变换的 plan。
<code>fft(f)_plan_dft_c2r_1d</code>	建立单个连续数据序列 1 维 C2R 变换的 plan。
<code>fft(f)_plan_dft_c2r_2d</code>	建立单个连续数据序列 2 维 C2R 变换的 plan。
<code>fft(f)_plan_dft_c2r_3d</code>	建立单个连续数据序列 3 维 C2R 变换的 plan。
<code>fft(f)_plan_guru_dft_c2r</code>	建立多组数据序列 n 维 C2R 变换的 plan。
<code>fft(f)_plan_guru_split_dft_c2r</code>	建立多组数据序列 n 维 C2R 变换的 plan。
<code>fft(f)_plan_guru64_dft_c2r</code>	建立多组数据序列 n 维 C2R 变换的 plan。
<code>fft(f)_plan_guru64_split_dft_c2r</code>	建立多组数据序列 n 维 C2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。
<code>fft(f)_plan_many_dft_c2r</code>	建立 <code>howmany</code> 组数据序列 n 维 C2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

2.3.3.2 `fft(f)_plan_dft_c2r`

建立单个连续数据序列 n 维 C2R 变换的 plan。

2.3.3.2.1 接口定义

C interface:

```
fft_plan fft_plan_dft_c2r(int rank, const int *n, fft_complex *in, double *out ,
unsigned flags);
```

```
fftf_plan fftf_plan_dft_c2r(int rank, const int *n, fftf_complex *in, float *out,
unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_C2R(RANK, N, IN, OUT, FLAGS);
```

```
RES = FFTF_PLAN_DFT_C2R(RANK, N, IN, OUT, FLAGS);
```

2.3.3.2.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_c2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.3.2.3 参数

参数名	类型	描述	输入/输出
rank	int	FFT 变换的维度是 <code>rank</code> ，约束： $1 \leq \text{rank} \leq 3$ 。	输入
n	const int*	<code>n</code> 是维度为 <code>rank</code> 的数组，包含 FFT 序列每一维度的大小，约束： $n[i] \geq 1$, for <code>i</code> in 0 to <code>rank</code> - 1。	输入
in	双精度: <code>fft_complex*</code>	输入待变换的数据。	输入
	单精度: <code>fftf_complex*</code>		
out	双精度: <code>double*</code>	输出快速傅里叶变换后的数据。	输出
	单精度: <code>float*</code>		
flags	unsigned int	planning 选项，未使用。	输入

2.3.3.2.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.3.2.5 示例

C interface:

```
int rank = 2;

int *n;
```

```

n = (int*)fft_malloc(sizeof(int) * rank);
n[0] = 2;
n[1] = 4;
double init[6][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-
8, 8}};
fft_complex *in;
in = (fft_complex*)fft_malloc(sizeof(fft_complex) * n[0] * (n[1] /
2 + 1));
for (int i = 0; i < n[0] * (n[1] / 2 + 1); i++) {
    in[i].r = init[i][0];
    in[i].i = init[i][1];
}
double *out;
out = (double*)fft_malloc(sizeof(double) * n[0] * n[1]);
fft_plan plan;
plan = fft_plan_dft_c2r(rank, n, in, out, FFT_ESTIMATE);
fft_execute_dft_c2r(plan, in, out);

fft_destroy_plan(plan);
fft_free(n);
fft_free(in);
fft_free(out);

/*
 * out = {1.280000e+02, 8.000000e+01, 9.600000e+01, 1.760000e+02,
 *         1.440000e+02, 1.280000e+02, 1.120000e+02, 9.600000e+01}
 */

```

2.3.3.3 fft(f)_plan_dft_c2r_1d

建立单个连续数据序列 1 维 C2R 变换的 plan。

2.3.3.3.1 接口定义

C interface:

```
fft_plan fft_plan_dft_c2r_1d(int n, fft_complex *in, double *out, unsigned flags);
fftf_plan fftf_plan_dft_c2r_1d(int n, fftf_complex *in, float *out, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_C2R_1D(N, IN, OUT, FLAGS);
RES = FFTF_PLAN_DFT_C2R_1D(N, IN, OUT, FLAGS);
```

2.3.3.3.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_c2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.3.3.3 参数

参数名	类型	描述	输入/输出
<code>n</code>	<code>int</code>	输入 FFT 序列的大小，约束： $n \geq 1$ 。	输入
<code>in</code>	双精度： <code>fft_complex *</code>	输入待变换的数据。	输入
	单精度： <code>fftf_complex *</code>		
<code>out</code>	双精度： <code>double *</code>	输出快速傅里叶变换后的数据。	输出
	单精度： <code>float *</code>		
<code>flags</code>	<code>unsigned int</code>	<code>planning</code> 选项，未使用。	输入

2.3.3.3.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.3.3.5 示例

C interface:

```
int n0 = 4;

double init[3][2] = {{120, 0}, {8, 8}, {0, 0}};

fft_complex *in;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * (n0 / 2 + 1));

for (int i = 0; i < (n0 / 2 + 1); i++) {

    in[i].r = init[i][0];

    in[i].i = init[i][1];

}

double *out;

out = (double*)fft_malloc(sizeof(double) * n0);

fft_plan plan;

plan = fft_plan_dft_c2r_1d(n0, in, out, FFT_ESTIMATE);

fft_execute_dft_c2r(plan, in, out);


fft_destroy_plan(plan);

fft_free(in);

fft_free(out);

/*

* out = {1.360000e+02, 1.040000e+02, 1.040000e+02, 1.360000e+02}

*/
```

2.3.3.4 fft(f)_plan_dft_c2r_2d

建立单个连续数据序列 2 维 C2R 变换的 plan。

2.3.3.4.1 接口定义

C interface:

```
fft_plan fft_plan_dft_c2r_2d(int n0, int n1, fft_complex *in, double *out, unsigned
flags);
```

```
fftf_plan fftf_plan_dft_c2r_2d(int n0, int n1, fftf_complex *in, float *out, unsigned
flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_C2R_2D(N0, N1, IN, OUT, FLAGS);
```

```
RES = FFTF_PLAN_DFT_C2R_2D(N0, N1, IN, OUT, FLAGS);
```

2.3.3.4.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_c2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.3.4.3 参数

参数名	类型	描述	输入/输出
<code>n0</code>	<code>int</code>	输入 FFT 序列第一维的大小，约束： $n0 \geq 1$ 。	输入
<code>n1</code>	<code>int</code>	输入 FFT 序列第二维的大小，约束： $n1 \geq 1$ 。	输入
<code>in</code>	双精度： <code>fft_complex *</code> 单精度： <code>fftf_complex *</code>	输入待变换的数据。	输入
<code>out</code>	双精度： <code>double *</code> 单精度： <code>float *</code>	输出快速傅里叶变换后的数据。	输出
<code>flags</code>	<code>unsigned int</code>	<code>planning</code> 选项，未使用。	输入

2.3.3.4.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.3.4.5 示例

C interface:

```

int n0 = 4;

int n1 = 4;

double init[12][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16},
{-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8}};

fft_complex *in;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * n0 * (n1 / 2 +
1));

for (int i = 0; i < n0 * (n1 / 2 + 1); i++){
    in[i].r = init[i][0];
    in[i].i = init[i][1];
}

double *out;

out = (double*)fft_malloc(sizeof(double) * n0 * n1);

fft_plan plan;

plan = fft_plan_dft_c2r_2d(n0, n1, in , out , FFT_ESTIMATE);

fft_execute_dft_c2r(plan, in, out);

fft_destroy_plan(plan);

fft_free(in);

fft_free(out);

/*
* out = {0.000000e+00, 6.400000e+01, 1.600000e+02, 2.240000e+02,
*        1.120000e+02, 1.600000e+01, 8.000000e+01, 1.760000e+02,
*        1.920000e+02, 1.280000e+02, 3.200000e+01, 9.600000e+01,
*        2.400000e+02, 2.080000e+02, 1.440000e+02, 4.800000e+01}
*/

```

2.3.3.5 fft(f)_plan_dft_c2r_3d

建立单个连续数据序列 3 维 C2R 变换的 plan。

2.3.3.5.1 接口定义

C interface:

```
fft_plan fft_plan_dft_c2r_3d(int n0, int n1, int n2, fft_complex *in, double *out,
unsigned flags);
```

```
fftf_plan fftf_plan_dft_c2r_3d(int n0, int n1, int n2, fftf_complex *in, float *out,
unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_C2R_3D(N0, N1, N2, IN, OUT, FLAGS);
```

```
RES = FFTF_PLAN_DFT_C2R_3D(N0, N1, N2, IN, OUT, FLAGS);
```

2.3.3.5.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_c2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.3.5.3 参数

参数名	类型	描述	输入/输出
n0	int	输入 FFT 序列第一维的大小，约束： $n0 \geq 1$ 。	输入
n1	int	输入 FFT 序列第二维的大小，约束： $n1 \geq 1$ 。	输入
n2	int	输入 FFT 序列第三维的大小，约束： $n2 \geq 1$ 。	输入
in	双精度：fft_complex *	输入待变换的数据。	输入
	单精度：fftf_complex *		
out	双精度：double *	输出快速傅里叶变换后的数据。	输出

参数名	类型	描述	输入/输出
	单精度: float*		
flags	unsigned int	planning 选项, 未使用。	输入

2.3.3.5.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.3.5.5 示例

C interface:

```

int n0 = 2;
int n1 = 3;
int n2 = 4;

double init[18][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16},
{-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}, {0, -16}, {-40, 8}, {-8, -8},
{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}};

fft_complex *in;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * n0 * n1 * (n2 /
2 + 1));

for (int i = 0; i < n0 * n1 * (n2 / 2 + 1); i++) {
    in[i].r = init[i][0];
    in[i].i = init[i][1];
}

double *out;

out = (double*)fft_malloc(sizeof(double) * n0 * n1 * n2);

fft_plan plan;

plan = fft_plan_dft_c2r_3d(n0, n1, n2, in, out, FFT_ESTIMATE);

fft_execute_dft_c2r(plan, in, out);

fft_free(in);

fft_free(out);

```

```

/*
 * out = {1.280000e+02, 1.440000e+02, 2.560000e+02, 4.000000e+02,
 *        8.000000e+00, 4.428719e+01, 1.360000e+02, 6.771281e+01,
 *        8.000000e+00, 9.971281e+01, 1.360000e+02, 1.228719e+01,
 *        4.800000e+01, 0.000000e+00, -8.000000e+01, 0.000000e+00,
 *        2.427180e+02, 1.541436e+02, 7.414359e+01, 1.541436e+02,
 *        3.812820e+02, 1.818564e+02, 1.018564e+02, 1.818564e+02}
 */

```

2.3.3.6 fft(f)_plan_guru_dft_c2r

建立多组数据序列 n 维 C2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

2.3.3.6.1 接口定义

C interface:

```
fft_plan fft_plan_guru_dft_c2r(int rank, const fft_iodim *dims, int howmany_rank,
const fft_iodim *howmany_dims, fft_complex *in, double *out, unsigned flags);
```

```
fftf_plan fftf_plan_guru_dft_c2r(int rank, const fftf_iodim *dims, int howmany_rank,
const fftf_iodim *howmany_dims, fftf_complex *in, float *out, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_GURU_DFT_C2R(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, IN, OUT, FLAGS);
```

```
RES = FFTF_PLAN_GURU_DFT_C2R(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, IN, OUT, FLAGS);
```

2.3.3.6.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_c2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.3.6.3 参数

参数名	类型	描述	输入/输出
rank	int	单个 FFT 序列的维度，约束：1 ≤ rank ≤ 3。	输入
dims	双精度：const fft_iodim*	dims 是大小为 rank 的结构体数组，dims[i] 包含以下成员： int n - 第 i 维 FFT 的长度。 int is - 第 i 维 FFT 输入序列的相继元素之间的间隔。 int os - 第 i 维 FFT 输出序列的相继元素之间的间隔。 约束：dims[i].n ≥ 1, for i in 0 to rank - 1。	输入
	单精度：const fftf_io dim*		
howmany_rank	int	多个 rank 维 FFT 之间的内存排布用 howmany_rank 维的 howmany_dims 数组来描述，howmany_rank 表示每个要计算的 rank 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束：0 ≤ howmany_rank ≤ 3	输入
howmany_dims	双精度：const fft_iodim*	howmany_dims 是大小为 howmany_rank 的结构体数组，howmany_dims[i] 包含以下成员： int n - howmany_rank 维空间的第 i 维的待变换 FFT 的个数。 int is - 第 i 维的相继 FFT 输入序列之间的间隔。 int os - 第 i 维的相继 FFT 输出序列之间的间隔。	输入
	单精度：const fftf_io dim*		
in	双精度：fft_complex*	输入待变换的数据。	输入
	单精度：fftf_complex*		
out	双精度：double*	输出快速傅里叶变换后的数据。	输出
	单精度：float*		
flags	unsigned int	planning 选项，未使用。	输入

2.3.3.6.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.3.6.5 示例

C interface:

```
int rank = 2;
fft_iodim *dims;
dims = (fft_iodim*)fft_malloc(sizeof(fft_iodim) * rank);
dims[0].n = 2;
dims[0].is = 2;
dims[0].os = 3;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
fft_iodim *howmany_dims;
howmany_dims = (fft_iodim*)fft_malloc(sizeof(fft_iodim) *
howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 2;
howmany_dims[0].os = 2 * 3;
double init[8][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-
8, 8}, {-8, 0}, {-8, 8}};
fft_complex *in;
in = (fft_complex*)fft_malloc(sizeof(fft_complex) * 8);
for (int i = 0; i < 8; i++) {
    in[i].r = init[i][0];
    in[i].i = init[i][1];
}
```

```

double *out;

out = (double*)fft_malloc(sizeof(double) * 12);

fft_plan plan;

plan = fft_plan_guru_dft_c2r(rank, dims, howmany_rank, howmany_dims,
in, out, FFT_ESTIMATE);

fft_execute_dft_c2r(plan, in, out);


fft_destroy_plan(plan);

fft_free(howmany_dims);

fft_free(dims);

fft_free(in);

fft_free(out);


/*
 * out = {1.360000e+02, 7.043078e+01, 1.535692e+02, 1.360000e+02,
 *        1.258564e+02, 9.814359e+01, -4.000000e+01, -1.971281e+01,
 *        3.571281e+01, 8.000000e+00, 8.000000e+00, 8.000000e+00}
 */

```

2.3.3.7 fft(f)_plan_guru_split_dft_c2r

建立多组数据序列 n 维 C2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。split 类接口的输入和输出分别存储在实部和虚部数组中。

2.3.3.7.1 接口定义

C interface:

```

fft_plan fft_plan_guru_split_dft_c2r(int rank, const fft_iodim *dims, int
howmany_rank, const fft_iodim *howmany_dims, double *ri, double *ii, double *out,
unsigned flags);

```

```

fftf_plan fftf_plan_guru_split_dft_c2r(int rank, const fftf_iodim *dims, int
howmany_rank, const fftf_iodim *howmany_dims, float *ri, float *ii, float *out,
unsigned flags);

```

Fortran interface:

```
RES = FFT_PLAN_GURU_SPLIT_DFT_C2R(RANK, DIMS, HOWMANY_RANK,
HOWMANY_DIMS, RI, II, OUT, FLAGS);
```

```
RES = FFTF_PLAN_GURU_SPLIT_DFT_C2R(RANK, DIMS,
HOWMANY_RANK, HOWMANY_DIMS, RI, II, OUT, FLAGS);
```

2.3.3.7.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `ri`, `ii` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_split_dft_c2r` 函数中以对新的输入 `ri`, `ii` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.3.7.3 参数

参数名	类型	描述	输入/输出
<code>rank</code>	<code>int</code>	单个 FFT 序列的维度。 约束: $1 \leq \text{rank} \leq 3$ 。	输入
<code>dims</code>	双精度: <code>const fft_iodim*</code> 单精度: <code>const fftf_io dim*</code>	<code>dims</code> 是大小为 <code>rank</code> 的结构体数组, <code>dims[i]</code> 包含以下成员: <code>int n</code> - 第 <code>i</code> 维 FFT 的长度。 <code>int is</code> - 第 <code>i</code> 维 FFT 输入序列的相继元素之间的间隔。 <code>int os</code> - 第 <code>i</code> 维 FFT 输出序列的相继元素之间的间隔。 约束: <code>dims[i].n ≥ 1</code> , for <code>i</code> in 0 to <code>rank - 1</code> 。	输入
<code>howmany_rank</code>	<code>int</code>	多个 <code>rank</code> 维 FFT 之间的内存排布用 <code>howmany_rank</code> 维的 <code>howmany_dims</code> 数组来描述, <code>howmany_rank</code> 表示每个要计算的 <code>rank</code> 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束: $0 \leq \text{howmany_rank} \leq 3$	输入
<code>howmany_dim</code>	双精度: <code>const fft_iodim*</code>	<code>howmany_dims</code> 是大小为 <code>howmany_rank</code> 的结构体数组,	输入

参数名	类型	描述	输入/输出
s	单精度: <code>const fftf_io dim*</code>	<code>howmany_dims[i]</code>包含以下成员: <code>int n</code> - <code>howmany_rank</code> 维空间的第 <code>i</code> 维的待变换 FFT 的个数。 <code>int is</code> - 第 <code>i</code> 维的相继 FFT 输入序列之间的间隔。 <code>int os</code> - 第 <code>i</code> 维的相继 FFT 输出序列之间的间隔。	
ri	双精度: <code>double*</code>	输入待变换数据的实部。	输入
	单精度: <code>float*</code>		
ii	双精度: <code>double*</code>	输入待变换数据的虚部。	输入
	单精度: <code>float*</code>		
out	双精度: <code>double*</code>	输出快速傅里叶变换后的数据。	输出
	单精度: <code>float*</code>		
flags	<code>unsigned int</code>	<code>planning</code> 选项, 未使用。	输入

2.3.3.7.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.3.7.5 示例

C interface:

```
int rank = 2;

fftf_iodim *dims;

dims = (fftf_iodim*)fftf_malloc(sizeof(fftf_iodim) * rank);

dims[0].n = 2;

dims[0].is = 2;

dims[0].os = 3;

dims[1].n = 3;

dims[1].is = 1;
```

```
dims[1].os = 1;
int howmany_rank = 1;
fft_iodim *howmany_dims;
howmany_dims = (fft_iodim*)fft_malloc(sizeof(fft_iodim) *
howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 2;
howmany_dims[0].os = 2 * 3;
double init[8][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-
8, 8}, {-8, 0}, {-8, 8}};
double *ri;
ri = (double*)fft_malloc(sizeof(double) * 8);
double *ii;
ii = (double*)fft_malloc(sizeof(double) * 8);
for (int i = 0; i < 8; i++) {
    ri[i] = init[i][0];
    ii[i] = init[i][1];
}
double *out;
out = (double*)fft_malloc(sizeof(double) * 12);
fft_plan plan;
plan = fft_plan_guru_split_dft_c2r(rank, dims, howmany_rank,
howmany_dims, ri, ii, out, FFT_ESTIMATE);
fft_execute_split_dft_c2r(plan, ri, ii, out);

fft_destroy_plan(plan);
fft_free(howmany_dims);
fft_free(dims);
fft_free(ri);
fft_free(ii);
fft_free(out);
```



```
/*
 * out = {1.360000e+02, 7.043078e+01, 1.535692e+02, 1.360000e+02,
 *        1.258564e+02, 9.814359e+01, -4.000000e+01, -1.971281e+01,
 *        3.571281e+01, 8.000000e+00, 8.000000e+00, 8.000000e+00}
 */
```

2.3.3.8 fft(f)_plan_guru64_dft_c2r

建立多组数据序列 n 维 C2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。与 `fft(f)_plan_guru_dft` 不同的是，`fft(f)_plan_guru64_dft` 允许部分参数是 64 位的整型。

2.3.3.8.1 接口定义

C interface:

```
fft_plan fft_plan_guru64_dft_c2r(int rank, const fft_iodim64 *dims, int
howmany_rank, const fft_iodim64 *howmany_dims, fft_complex *in, double *out,
unsigned flags);
```

```
fftf_plan fftf_plan_guru64_dft_c2r(int rank, const fftf_iodim64 *dims, int
howmany_rank, const fftf_iodim64 *howmany_dims, fftf_complex *in, float *out,
unsigned flags);
```

Fortran interface:

```
WJX B K Y K d U Q F S d L Z W Z ; 9 d I K Y d H 7 W - W F S P B I N R X I M T \ R F S ^ d W F S P B I M T \ R F S ^ d I N R X I B
N S B Z Y B K Q F L X . &
```

```
WJX B K Y K d U Q F S d L Z W Z ; 9 d I K Y d H 7 W - W F S P B I N R X I M T \ R F S ^ d W F S P B
M T \ R F S ^ d I N R X I B N S B Z Y B K Q F L X . &
```

2.3.3.8.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_c2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.3.8.3 参数

参数名	类型	描述	输入/输出
rank	int	单个 FFT 序列的维度。 约束: $1 \leq \text{rank} \leq 3$	输入
dims	双精度: const fft_iodim64* 单精度: const fftf_io dim64*	dims 是大小为 rank 的结构体数组, dims[i] 包含以下成员: ptrdiff_t n - 第 i 维 FFT 的长度。 ptrdiff_t is - 第 i 维 FFT 输入序列的相继元素之间的间隔。 ptrdiff_t os - 第 i 维 FFT 输出序列的相继元素之间的间隔。 约束: $\text{dims}[i].n \geq 1$, for i in 0 to rank - 1	输入
howmany_rank	int	多个 rank 维 FFT 之间的内存排布用 howmany_rank 维的 howmany_dims 数组来描述, howmany_rank 表示每个要计算的 rank 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束: $0 \leq \text{howmany_rank} \leq 3$	输入
howmany_dims	双精度: const fft_iodim64* 单精度: const fftf_io dim64*	howmany_dims 是大小为 howmany_rank 的结构体数组, howmany_dims[i] 包含以下成员: ptrdiff_t n - howmany_rank 维空间的第 i 维的待变换 FFT 的个数。 ptrdiff_t is - 第 i 维的相继 FFT 输入序列之间的间隔。 ptrdiff_t os - 第 i 维的相继 FFT 输出序列之间的间隔。	输入
in	双精度: fft_complex* 单精度: fftf_complex*	输入待变换的数据。	输入
out	双精度: double* 单精度: float*	输出快速傅里叶变换后的数据。	输出

参数名	类型	描述	输入/输出
flags	unsigned int	planning 选项，未使用。	输入

2.3.3.8.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.3.8.5 示例

C interface:

```
int rank = 2;
fft_iodim64 *dims;
dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) * rank);
dims[0].n = 2;
dims[0].is = 2;
dims[0].os = 3;
dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;
int howmany_rank = 1;
fft_iodim64 *howmany_dims;
howmany_dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) *
howmany_rank);
howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 2;
howmany_dims[0].os = 2 * 3;
double init[8][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-
8, 8}, {-8, 0}, {-8, 8}};
fft_complex *in;
in = (fft_complex*)fft_malloc(sizeof(fft_complex) * 8);
for (int i = 0; i < 8; i++) {
    in[i].r = init[i][0];
    in[i].i = init[i][1];
}
```

```

}

double *out;

out = (double*)fft_malloc(sizeof(double) * 12);

fft_plan plan;

plan = fft_plan_guru64_dft_c2r(rank, dims, howmany_rank,
howmany_dims, in, out, FFT_ESTIMATE);

fft_execute_dft_c2r(plan, in, out);

fft_destroy_plan(plan);
fft_free(howmany_dims);
fft_free(dims);
fft_free(in);
fft_free(out);

/*
 * out = {1.360000e+02, 7.043078e+01, 1.535692e+02, 1.360000e+02,
 *        1.258564e+02, 9.814359e+01, -4.000000e+01, -1.971281e+01,
 *        3.571281e+01, 8.000000e+00, 8.000000e+00, 8.000000e+00}
 */

```

2.3.3.9 `fft(f)_plan_guru64_split_dft_c2r`

建立多组数据序列 n 维 C2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。`split` 类接口的输入和输出分别存储在实部和虚部数组中。与 `fft(f)_plan_guru_split_dft_c2r` 不同的是，`fft(f)_plan_guru64_split_dft_c2r` 允许部分参数是 64 位的整型。

2.3.3.9.1 接口定义

C interface:

```
fft_plan fft_plan_guru64_split_dft_c2r(int rank, const fft_iodim64 *dims, int
howmany_rank, const fft_iodim64 *howmany_dims, double *ri, double *ii, double
*out, unsigned flags);
```

```
fftf_plan fftf_plan_guru64_split_dft_c2r(int rank, const fftf_iodim64 *dims, int
howmany_rank, const fftf_iodim64 *howmany_dims, float *ri, float *ii, float *out,
unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_GURU64_SPLIT_DFT_C2R(RANK, DIMS,
HOWMANY_RANK, HOWMANY_DIMS, RI, II, OUT, FLAGS);
```

```
RES = FFTF_PLAN_GURU64_SPLIT_DFT_C2R(RANK, DIMS,
HOWMANY_RANK, HOWMANY_DIMS, RI, II, OUT, FLAGS);
```

2.3.3.9.2 返回值

函数返回一个 `fftf_plan` 类型的结构体指针。将该对象作为参数传入 `fftf_execute` 函数中使用，将对当前提供的输入 `ri`, `ii` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fftf_execute_split_dft_c2r` 函数中以对新的输入 `ri`, `ii` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.3.9.3 参数

参数名	类型	描述	输入/输出
rank	int	单个 FFT 序列的维度。 约束: $1 \leq \text{rank} \leq 3$ 。	输入
dims	双精度: <code>const fftf_iodim64*</code> 单精度: <code>const fftf_iodim64*</code>	<code>dims</code> 是大小为 <code>rank</code> 的结构体数组, <code>dims[i]</code> 包含以下成员: <code>ptrdiff_t n</code> - 第 <code>i</code> 维 FFT 的长度。 <code>ptrdiff_t is</code> - 第 <code>i</code> 维 FFT 输入序列的相继元素之间的间隔。 <code>ptrdiff_t os</code> - 第 <code>i</code> 维 FFT 输出序列的相继元素之间的间隔。 约束: $\text{dims}[i].n \geq 1$, for <code>i</code> in 0 to <code>rank - 1</code>	输入

参数名	类型	描述	输入/输出
howmany_rank	int	多个 rank 维 FFT 之间的内存排布用 howmany_rank 维的 howmany_dims 数组来描述，howmany_rank 表示每个要计算的 rank 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束：0≤howmany_rank≤3	输入
howmany_dims	双精度：const fft_iodim64* 单精度：const fftf_io dim64*	howmany_dims 是大小为 howmany_rank 的结构体数组，howmany_dims[i] 包含以下成员： ptrdiff_t n - howmany_rank 维空间的第 i 维的待变换 FFT 的个数。 ptrdiff_t is - 第 i 维的相继 FFT 输入序列之间的间隔。 ptrdiff_t os - 第 i 维的相继 FFT 输出序列之间的间隔。	输入
ri	双精度：double* 单精度：float*	输入待变换数据的实部。	输入
ii	双精度：double* 单精度：float*	输入待变换数据的虚部。	输入
out	双精度：double* 单精度：float*	输出快速傅里叶变换后的数据。	输出
flags	unsigned int	planning 选项，未使用。	输入

2.3.3.9.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.3.9.5 示例

C interface:

```
int rank = 2;

fft_iodim64 *dims;

dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) * rank);

dims[0].n = 2;
dims[0].is = 2;
dims[0].os = 3;

dims[1].n = 3;
dims[1].is = 1;
dims[1].os = 1;

int howmany_rank = 1;

fft_iodim64 *howmany_dims;

howmany_dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) *
howmany_rank);

howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 2;
howmany_dims[0].os = 2 * 3;

double init[8][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-
8, 8}, {-8, 0}, {-8, 8}};

double *ri;

ri = (double*)fft_malloc(sizeof(double) * 8);

double *ii;

ii = (double*)fft_malloc(sizeof(double) * 8);

for (int i = 0; i < 8; i++) {
    ri[i] = init[i][0];
    ii[i] = init[i][1];
}

double *out;

out = (double*)fft_malloc(sizeof(double) * 12);
```

```

fft_plan plan;

plan = fft_plan_guru64_split_dft_c2r(rank, dims, howmany_rank,
howmany_dims, ri, ii, out, FFT_ESTIMATE);

fft_execute_split_dft_c2r(plan, ri, ii, out);

fft_destroy_plan(plan);
fft_free(howmany_dims);
fft_free(dims);
fft_free(ri);
fft_free(ii);
fft_free(out);

/*
 * out = {1.360000e+02, 7.043078e+01, 1.535692e+02, 1.360000e+02,
 *         1.258564e+02, 9.814359e+01, -4.000000e+01, -1.971281e+01,
 *         3.571281e+01, 8.000000e+00, 8.000000e+00, 8.000000e+00}
 */

```

2.3.3.10 `fft(f)_plan_many_dft_c2r`

建立 `howmany` 组数据序列 `n` 维 C2R 变换的 `plan`。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

2.3.3.10.1 接口定义

C interface:

```

fft_plan fft_plan_many_dft_c2r(int rank, const int *n, int howmany, fft_complex *in,
const int *inembed, int istride, int idist, double *out, const int *onembed, int ostride,
int odist, unsigned flags);

```

```

fftf_plan fftf_plan_many_dft_c2r(int rank, const int *n, int howmany, fftf_complex
*in, const int *inembed, int istride, int idist, float *out, const int *onembed, int ostride,
int odist, unsigned flags);

```

Fortran interface:


```
RES = FFT_PLAN_MANY_DFT_C2R(RANK, N, HOWMANY, IN, INEMBED,
ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, FLAGS);
```

```
RES = FFTF_PLAN_MANY_DFT_C2R(RANK, N, HOWMANY, IN, INEMBED,
ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, FLAGS);
```

2.3.3.10.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_c2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.3.10.3 参数

参数名	类型	描述	输入/输出
<code>rank</code>	<code>int</code>	FFT 变换的维度是 <code>rank</code> ，约束： $1 \leq \text{rank} \leq 3$ 。	输入
<code>n</code>	<code>const int*</code>	<code>n</code> 是维度为 <code>rank</code> 的数组，包含 FFT 序列每一维度的大小，约束： $n[i] \geq 1$, for i in 0 to <code>rank</code> - 1。	输入
<code>howmany</code>	<code>int</code>	<code>howmany</code> 表示要多少个多维 FFT 变换。	输入
<code>in</code>	双精度: <code>fft_complex*</code> 单精度: <code>fftf_complex*</code>	输入待变换的数据。	输入
<code>inembed</code>	<code>const int*</code>	<code>inembed</code> 是大小为 <code>rank</code> 的数组或者 NULL，该数组表示 FFT 数据 <code>in</code> 存储的更大空间的每一维度的大小。 约束: $\text{inembed}[i] \geq n[i]$ for i in 0, <code>rank</code> -1 或者如果 <code>inembed == NULL</code> ，则可以认为 <code>inembed</code> 与 <code>n</code> 相等。	输入
<code>istride</code>	<code>int</code>	FFT 输入序列的相继元素之间的间隔。	输入
<code>idist</code>	<code>int</code>	<code>idist</code> 表示输入的每个 FFT 序列的间隔。	输入

参数名	类型	描述	输入/输出
out	双精度: double*	输出快速傅里叶变换后的数据。	输出
	单精度: float*		
onembed	const int*	onembed 是大小为 rank 的数组或者 NULL, 该数组表示 FFT 数据 out 存储的更大空间的每一维度的大小。 约束: $\text{onembed}[i] \geq n[i]$ for i in $0, \text{rank}-1$ 或者如果 $\text{onembed} == \text{NULL}$, 则可以认为 onembed 与 n 相等。	输入
ostride	int	FFT 输出序列的相继元素之间的间隔。	输入
odist	int	odist 表示输出的每个 FFT 序列的间隔。	输入
flags	unsigned int	planning 选项, 未使用。	输入

2.3.3.10.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.3.10.5 示例

C interface:

```
int rank = 1;
int *n;
n = (int*)fft_malloc(sizeof(int) * rank);
n[0] = 4;
int howmany = 3;
int istride = 1;
int ostride = 1;
int idist = n[0] / 2 + 1;
int odist = 4;

double init[9][2] = {{120, 0}, {8, 8}, {0, 0}, {0, 16}, {0, 16}, {-8, 8}, {-8, 0}, {-8, 8}, {-16, 0}};
```

```

    fft_complex *in;

    in = (fft_complex*)fft_malloc(sizeof(fft_complex) * (n[0] / 2 + 1)
* howmany);

    for (int i = 0; i < (n[0] / 2 + 1) * howmany; i++){
        in[i].r = init[i][0];
        in[i].i = init[i][1];
    }

    double *out;

    out = (double*)fft_malloc(sizeof(double) * n[0] * howmany);

    fft_plan plan;

    plan = fft_plan_many_dft_c2r(rank, n, howmany, in, NULL, istride,
idist, out, NULL, ostride, odist, FFT_ESTIMATE);

    fft_execute_dft_c2r(plan, in, out);

    fft_destroy_plan(plan);

    fft_free(n);

    fft_free(in);

    fft_free(out);

    /*
    * out = {1.360000e+02, 1.040000e+02, 1.040000e+02, 1.360000e+02,
    *        -8.000000e+00, -2.400000e+01, -8.000000e+00, 4.000000e+01,
    *        -4.000000e+01, -8.000000e+00, -8.000000e+00, 2.400000e+01}
    */

```

2.3.4 R2R 变换

2.3.4.1 函数说明

函数名	简介
fft(f)_plan_r2r	建立单个连续数据序列 n 维 R2R 变换的 plan。

函数名	简介
fft(f)_plan_r2r_1d	建立单个连续数据序列 1 维 R2R 变换的 plan。
fft(f)_plan_r2r_2d	建立单个连续数据序列 2 维 R2R 变换的 plan。
fft(f)_plan_r2r_3d	建立单个连续数据序列 3 维 R2R 变换的 plan。
fft(f)_plan_guru_r2r	建立多组数据序列 n 维 R2R 变换的 plan。
fft(f)_plan_guru64_r2r	建立多组数据序列 n 维 R2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。
fft(f)_plan_many_r2r	建立 howmany 组数据序列 n 维 R2R 变换的 plan。

2.3.4.2 fft(f)_plan_dft_r2r

建立单个连续数据序列 n 维 R2R 变换的 plan。

2.3.4.2.1 接口定义

C interface:

```
fft_plan fft_plan_dft_r2r(int rank, const int *n, double *in, double *out, const
fft_r2r_kind *kind, unsigned flags);
```

```
fftf_plan fftf_plan_dft_c2r(int rank, const int *n, float *in, float *out, const
fft_r2r_kind *kind, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_R2R(RANK, N, IN, OUT, KIND, FLAGS);
```

```
RES = FFTF_PLAN_DFT_R2R(RANK, N, IN, OUT, KIND, FLAGS);
```

FFT_REDFT11 和 FFT_ROODFT11 只支持长度为 4 的整数倍大小的序列，其他变换类型只支持长度为 2 的整数倍大小的序列。

2.3.4.2.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_r2r` 函数中以对新的

输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.4.2.3 参数

参数名	类型	描述	输入/输出
<code>rank</code>	<code>int</code>	FFT 变换的维度是 <code>rank</code> ，约束： $1 \leq \text{rank} \leq 3$ 。	输入
<code>n</code>	<code>const int*</code>	<code>n</code> 是维度为 <code>rank</code> 的数组，包含 FFT 序列每一维度的大小，约束： $n[i] \geq 1$, for <code>i</code> in 0 to <code>rank - 1</code> 。	输入
<code>in</code>	双精度: <code>double*</code>	输入待变换的数据。	输入
	单精度: <code>float*</code>		
<code>out</code>	双精度: <code>double*</code>	输出快速傅里叶变换后的数据。	输出
	单精度: <code>float*</code>		
<code>kind</code>	双精度: <code>const fft_r2r_kind*</code>	<code>kind</code> 是大小为 <code>rank</code> 的数组，包含 FFT 序列每一维度的 R2R 变换类型， <code>kind[i]</code> (for <code>i</code> in 0 to <code>rank - 1</code>) 有以下可选值： FFT_R2HC FFT_HC2R FFT_DHT FFT_REDFT00 FFT_REDFT01 FFT_REDFT10 FFT_REDFT11 FFT_RODFT00 FFT_RODFT01 FFT_RODFT10 FFT_RODFT11	输入
	单精度: <code>const fftf_r2r_kind*</code>		
<code>flags</code>	<code>unsigned int</code>	<code>planning</code> 选项，未使用。	输入

2.3.4.2.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.4.2.5 示例

C interface:

```
int rank = 2;

int *n;

n = (int*)fft_malloc(sizeof(int) * rank);

n[0] = 2;

n[1] = 4;

double init[8] = {1, 2, 3, 4, 5, 6, 7, 8};

double *in;

in = (double*)fft_malloc(sizeof(double) * n[0] * n[1]);

for (int i = 0; i < n[0] * n[1]; i++) {
    in[i] = init[i];
}

double *out;

out = (double*)fft_malloc(sizeof(double) * n[0] * n[1]);

fft_r2r_kind *kind;

kind = (fft_r2r_kind*)fft_malloc(sizeof(fft_r2r_kind) * rank);

kind[0] = FFT_DHT;

kind[1] = FFT_DHT;

fft_plan plan;

plan = fft_plan_r2r(rank, n, in, out, kind, FFT_ESTIMATE);

fft_execute_r2r(plan, in, out);

fft_destroy_plan(plan);

fft_free(n);

fft_free(kind);

fft_free(in);

fft_free(out);

/*
```

```
* out = {3.600000e+01, -8.000000e+00, -4.000000e+00, 0.000000e+00,
*        -1.600000e+01, 0.000000e+00, 0.000000e+00, 0.000000e+00}
*/
```

2.3.4.3 fft(f)_plan_dft_r2r_1d

建立单个连续数据序列 1 维 R2R 变换的 plan。

2.3.4.3.1 接口定义

C interface:

```
fft_plan fft_plan_dft_r2r_1d(int n, double *in, double *out, fft_r2r_kind kind,
unsigned flags);
```

```
fftf_plan fftf_plan_dft_r2r_1d(int n, float *in, float *out, fft_r2r_kind kind, unsigned
flags);
```

Fortran interface:

```
RES = FFT_PLAN_DFT_R2R_1D(N, IN, OUT, KIND, FLAGS);
```

```
RES = FFTF_PLAN_DFT_R2R_1D(N, IN, OUT, KIND, FLAGS);
```

FFT_REDFT11 和 FFT_ROODFT11 只支持长度为 4 的整数倍大小的序列，其他变换类型只支持长度为 2 的整数倍大小的序列。

2.3.4.3.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；

另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_r2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 plan 执行成功，否则表示执行失败。

2.3.4.3.3 参数

参数名	类型	描述	输入/输出
n	int	输入 FFT 序列的大小，约束： $n \geq 1$ 。	输入
in	双精度：double*	输入待变换的数据。	输入
	单精度：float*		

参数名	类型	描述	输入/输出
out	双精度: double*	输出快速傅里叶变换后的数据。	输出
	单精度: float*		
kind	双精度: fft_r2r_kind*	Kind 表示 FFT 序列的 R2R 变换类型，有以下可选值： FFT_R2HC FFT_HC2R FFT_DHT FFT_REDFT00 FFT_REDFT01 FFT_REDFT10 FFT_REDFT11 FFT_RODFT00 FFT_RODFT01 FFT_RODFT10 FFT_RODFT11	输入
	单精度: fftf_r2r_kind*		
flags	unsigned int	planning 选项，未使用。	输入

2.3.4.3.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.4.3.5 示例

C interface:

```
int N = 16;

double *in;
double *out;

fft_plan plan;

in = (double*)fft_malloc(sizeof(double) * N);
out = (double*)fft_malloc(sizeof(double) * N);

for (int i = 0; i < N; i++){
    in[i] = (double)i;
}
```



```

}

plan = fft_plan_r2r_1d(N, in , out, FFT_REDFT10, FFT_ESTIMATE);
fft_execute_r2r(plan, in, out);

fft_destroy_plan(plan);
fft_free(in);
fft_free(out);

/*
 * out = {2.400000e+02, -1.035857e+02, 0.000000e+00, -1.135629e+01,
 *         0.000000e+00, -3.968777e+00, 0.000000e+00, -1.920738e+00,
 *         0.000000e+00, -1.061666e+00, 0.000000e+00, -6.060758e-01,
 *         0.000000e+00, -3.169964e-01, 0.000000e+00, -9.896796e-02}
 */

```

2.3.4.4 fft(f)_plan_dft_r2r_2d

建立单个连续数据序列 2 维 R2R 变换的 plan。

2.3.4.4.1 接口定义

C interface:

```
fft_plan fft_plan_r2r_2d(int n0, int n1, double *in, double *out, fft_r2r_kind kind0,
fft_r2r_kind kind1, unsigned flags);
```

```
fftf_plan fftf_plan_r2r_2d(int n0, int n1, float *in, float *out, fftf_r2r_kind kind0,
fftf_r2r_kind kind1, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_R2R_2D(N0, N1, IN, OUT, KIND0, KIND1, FLAGS);
```

```
RES = FFTF_PLAN_R2R_2D(N0, N1, IN, OUT, KIND0, KIND1, FLAGS);
```

FFT_REDFT11 和 FFT_ROODFT11 只支持长度为 4 的整数倍大小的序列，其他变换类型只支持长度为 2 的整数倍大小的序列。

2.3.4.4.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_r2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.4.4.3 参数

参数名	类型	描述	输入/输出
<code>n0</code>	<code>int</code>	输入 FFT 序列第一维的大小，约束： $n0 \geq 1$ 。	输入
<code>n1</code>	<code>int</code>	输入 FFT 序列第二维的大小，约束： $n1 \geq 1$ 。	输入
<code>in</code>	双精度： <code>double*</code>	输入待变换的数据。	输入
	单精度： <code>float*</code>		
<code>out</code>	双精度： <code>double*</code>	输出快速傅里叶变换后的数据。	输出
	单精度： <code>float*</code>		
<code>kind0</code>	双精度： <code>fft_r2r_kind*</code>	<code>kind0</code> 表示 FFT 序列的 R2R 变换类型，有以下可选值： FFT_R2HC FFT_HC2R FFT_DHT FFT_REDFFT00 FFT_REDFFT01 FFT_REDFFT10 FFT_REDFFT11 FFT_RODFT00 FFT_RODFT01 FFT_RODFT10 FFT_RODFT11	输入
	单精度： <code>fftf_r2r_kind*</code>		
<code>kind1</code>	双精度： <code>fft_r2r_kind*</code>	<code>kind1</code> 表示 FFT 序列的 R2R 变换类型，有以下可选值：	输入

参数名	类型	描述	输入/输出
	单精度: fftf_r2r_kind*	FFT_R2HC FFT_HC2R FFT_DHT FFT_REDFT00 FFT_REDFT01 FFT_REDFT10 FFT_REDFT11 FFT_RODFT00 FFT_RODFT01 FFT_RODFT10 FFT_RODFT11	
flags	unsigned int	planning 选项, 未使用。	输入

2.3.4.4.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.4.4.5 示例

C interface:

```

int n0 = 2;
int n1 = 4;
double init[8] = {120, 0, 8, 8, 0, 0, 1, 1};
double *in;

in = (double*)fft_malloc(sizeof(double) * n0 * n1);
for (int i = 0; i < n0 * n1; i++) {
    in[i] = init[i];
}

double *out;

out = (double*)fft_malloc(sizeof(double) * n0 * n1);

fftf_r2r_kind kind0;
fftf_r2r_kind kind1;

kind0 = FFT_DHT;

```

```

kind1 = FFT_DHT;
fft_plan plan;
plan = fft_plan_r2r_2d(n0, n1, in, out, kind0, kind1, FFT_ESTIMATE);
fft_execute_r2r(plan, in, out);

fft_destroy_plan(plan);
fft_free(in);
fft_free(out);

/*
 * out = {1.380000e+02, 1.020000e+02, 1.200000e+02, 1.200000e+02,
 *         1.340000e+02, 1.060000e+02, 1.200000e+02, 1.200000e+02}
 */

```

2.3.4.5 fft(f)_plan_dft_r2r_3d

建立单个连续数据序列 3 维 R2R 变换的 plan。

2.3.4.5.1 接口定义

C interface:

```
fft_plan fft_plan_r2r_3d(int n0, int n1, int n2, double *in, double *out, fft_r2r_kind
kind0, fft_r2r_kind kind1, fft_r2r_kind kind2, unsigned flags);
```

```
fftf_plan fftf_plan_r2r_3d(int n0, int n1, int n2, float *in, float *out, fftf_r2r_kind
kind0, fftf_r2r_kind kind1, fft_r2r_kind kind1, unsigned flags);
```

Fortran interface:

```
RES = FFT_PLAN_R2R_3D(N0, N1, N2, IN, OUT, KIND0, KIND1, KIND2,
FLAGS);
```

```
RES = FFTF_PLAN_R2R_3D(N0, N1, N2, IN, OUT, KIND0, KIND1, KIND2,
FLAGS);
```

FFT_REDFT11 和 FFT_ROODFT11 只支持长度为 4 的整数倍大小的序列，其他变换类型只支持长度为 2 的整数倍大小的序列。

2.3.4.5.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_r2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.4.5.3 参数

参数名	类型	描述	输入/输出
<code>n0</code>	<code>int</code>	输入 FFT 序列第一维的大小，约束： $n0 \geq 1$ 。	输入
<code>n1</code>	<code>int</code>	输入 FFT 序列第二维的大小，约束： $n1 \geq 1$ 。	输入
<code>n2</code>	<code>int</code>	输入 FFT 序列第三维的大小，约束： $n2 \geq 1$ 。	输入
<code>in</code>	双精度： <code>double*</code>	输入待变换的数据。	输入
	单精度： <code>float*</code>		
<code>out</code>	双精度： <code>double*</code>	输出快速傅里叶变换后的数据。	输出
	单精度： <code>float*</code>		
<code>kind0</code>	双精度： <code>fft_r2r_kind*</code>	<code>kind0</code> 表示 FFT 序列的 R2R 变换类型，有以下可选值： FFT_R2HC FFT_HC2R FFT_DHT	输入
	单精度： <code>fftf_r2r_kind*</code>	FFT_REDFT00 FFT_REDFT01 FFT_REDFT10 FFT_REDFT11 FFT_RODFT00 FFT_RODFT01 FFT_RODFT10 FFT_RODFT11	

参数名	类型	描述	输入/输出
kind1	双精度: fft_r2r_kind*	kind1 表示 FFT 序列的 R2R 变换类型，有以下可选值： FFT_R2HC FFT_HC2R FFT_DHT	输入
	单精度: fftf_r2r_kind*	FFT_REDFT00 FFT_REDFT01 FFT_REDFT10 FFT_REDFT11 FFT_RODFT00 FFT_RODFT01 FFT_RODFT10 FFT_RODFT11	
kind2	双精度: fft_r2r_kind*	kind2 表示 FFT 序列的 R2R 变换类型，有以下可选值： FFT_R2HC FFT_HC2R FFT_DHT	输入
	单精度: fftf_r2r_kind*	FFT_REDFT00 FFT_REDFT01 FFT_REDFT10 FFT_REDFT11 FFT_RODFT00 FFT_RODFT01 FFT_RODFT10 FFT_RODFT11	
flags	unsigned int	planning 选项，未使用。	输入

2.3.4.5.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.4.5.5 示例

C interface:

```
int n0 = 2;
```

```
int n1 = 2;
int n2 = 2;
double init[8] = {120, 0, 8, 8, 0, 0, 0, 16};
double *in;
in = (double*)fft_malloc(sizeof(double) * n0 * n1 * n2);
for (int i = 0; i < n0 * n1 * n2; i++) {
    in[i] = init[i];
}
double *out;
out = (double*)fft_malloc(sizeof(double) * n0 * n1 * n2);
fft_r2r_kind kind0 = FFT_RODFT00;
fft_r2r_kind kind1 = FFT_RODFT01;
fft_r2r_kind kind2 = FFT_RODFT10;
fft_plan plan;
plan = fft_plan_r2r_3d(n0, n1, n2, in, out, kind0, kind1, kind2,
FFT_ESTIMATE);
fft_execute_r2r(plan, in, out);

fft_destroy_plan(plan);
fft_free(in);
fft_free(out);

/*
 * out = {4.940759e+02, 5.324519e+02, 3.373085e+02, 6.433032e+02,
 *         4.156922e+02, 6.433032e+02, 4.156922e+02, 5.324519e+02}
 */
```

2.3.4.6 fft(f)_plan_guru_r2r

2.3.4.6.1 guru r2r 矩阵转置功能

通过 guru 类 r2r 接口的特殊用法，可以实现矩阵转置功能。使用 FFT guru 类接口创建一个 rank 为 0，howmany_rank 为 2 的转换。

示例：

```
fft_plan plan_transpose(int rows, int cols, double *in, double *out)
{
    const unsigned flags = FFT_ESTIMATE;
    fft_iodim howmany_dims[2];

    howmany_dims[0].n = rows;
    howmany_dims[0].is = cols;
    howmany_dims[0].os = 1;

    howmany_dims[1].n = cols;
    howmany_dims[1].is = 1;
    howmany_dims[1].os = rows;

    return fft_plan_guru_r2r(/*rank=*/ 0, /*dims=*/ NULL,
                             /*howmany_rank=*/ 2, howmany_dims,
                             in, out, /*kind=*/ NULL, flags);
}

int main() {
    int len = 12;
    int rows = 3;
    int cols = 4;
    double *in = (double *)malloc(len * sizeof(double));
    for(int i = 0; i < len; i++) {
        in[i] = i;
    }
}
```



```

    }

/*
 * 0 1 2 3
 * 4 5 6 7
 * 8 9 10 11
 */

    fft_plan p = plan_transpose(rows, cols, in, in);
    fft_execute(p);

/*
 * 0 4 8
 * 1 5 9
 * 2 6 10
 * 3 7 11
 */

    free(in);
    fft_destroy_plan(p);
}

```

2.3.4.7 `fft(f)_plan_guru64_r2r`

建立多组数据序列 n 维 R2R 变换的 plan。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。与 `fft(f)_plan_guru_dft` 不同的是，`fft(f)_plan_guru64_dft` 允许部分参数是 64 位的整型。

2.3.4.7.1 接口定义

C interface:

```
fft_plan fft_plan_guru64_r2r(int rank, const fft_iodim64 *dims, int howmany_rank,
const fft_iodim64 *howmany_dims, double *in, double *out, const fft_r2r_kind *kind,
unsigned flags);
```

```
fftf_plan fftf_plan_guru64_r2r(int rank, const fftf_iodim64 *dims, int howmany_rank,
const fftf_iodim64 *howmany_dims, float *in, float *out, const fftf_r2r_kind *kind,
unsigned flags);
```

Fortran interface:

WJX33KKYdUQFS dLZWZ ; 9dIKYdW7W-WFSP dNR X dMT \ R FS ^ dWFSP dMT \ R FS ^ dINR X d
NS dZY dPNS I dKQFLX .d
WJX33KKYKdUQFS dLZWZ ; 9dIKYdW7W-WFSP dNR X dMT \ R FS ^ dWFSP d
MT \ R FS ^ dINR X dNS dZY dPNS I dKQFLX .d
KKYdWJIKY66 和 KKYdWTTIKY66 只支持长度为 9 的整数倍大小的序列，其他变换类型只
支持长度为 7 的整数倍大小的序列。%

2.3.4.7.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_c2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。
如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.4.7.3 参数

参数名	类型	描述	输入/输出
rank	int	单个 FFT 序列的维度。 约束：1≤rank≤3	输入
dims	双精度：const fft_iod im64* 单精度：const fftf_io dim64*	<code>dims</code> 是大小为 <code>rank</code> 的结构体数组， <code>dims[i]</code> 包含以下成员： <code>ptrdiff_t n</code> - 第 <code>i</code> 维 FFT 的长度。 <code>ptrdiff_t is</code> - 第 <code>i</code> 维 FFT 输入序列的相继元素之间的间隔。 <code>ptrdiff_t os</code> - 第 <code>i</code> 维 FFT 输出序列的相继元素之间的间隔。 约束： <code>dims[i].n≥1, for i in 0 to rank - 1</code>	输入
howmany_rank	int	多个 <code>rank</code> 维 FFT 之间的内存排布用 <code>howmany_rank</code> 维的 <code>howmany_dims</code> 数组来描述， <code>howmany_rank</code> 表示每个要计算的 <code>rank</code> 维 FFT 变换的起始地址的内存访问模式所需的维数。 约束：0≤howmany_rank≤3	输入

参数名	类型	描述	输入/输出
howmany_dims	双精度: const fft_iodim64*	howmany_dims 是大小为 howmany_rank 的结构体数组, howmany_dims[i] 包含以下成员: ptrdiff_t n - howmany_rank 维空间的第 i 维的待变换 FFT 的个数。 ptrdiff_t is - 第 i 维的相继 FFT 输入序列之间的间隔。 ptrdiff_t os - 第 i 维的相继 FFT 输出序列之间的间隔。	输入
	单精度: const fftf_io dim64*		
in	双精度: double*	输入待变换的数据。	输入
	单精度: float*		
out	双精度: double*	输出快速傅里叶变换后的数据。	输出
	单精度: float*		
kind	双精度: const fft_r2r_kind*	kind 是大小为 rank 的数组, 包含 FFT 序列每一维度的 R2R 变换类型, kind[i] (for i in 0 to rank - 1) 有以下可选值: FFT_R2HC FFT_HC2R FFT_DHT FFT_REDFT00 FFT_REDFT01 FFT_REDFT10 FFT_REDFT11 FFT_RODFT00 FFT_RODFT01 FFT_RODFT10 FFT_RODFT11	输入
	单精度: const fftf_r2r_kind*		
flags	unsigned int	planning 选项, 未使用。	输入

2.3.4.7.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.4.7.5 示例

C interface:

```

int rank = 2;

fft_iodim64 *dims;

dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) * rank);

dims[0].n = 2;
dims[0].is = 2;
dims[0].os = 2;

dims[1].n = 2;
dims[1].is = 1;
dims[1].os = 1;

int howmany_rank = 1;

fft_iodim64 *howmany_dims;

howmany_dims = (fft_iodim64*)fft_malloc(sizeof(fft_iodim64) *
howmany_rank);

howmany_dims[0].n = 2;
howmany_dims[0].is = 2 * 2;
howmany_dims[0].os = 2 * 2;

double init[8] = {120, 0, 8, 8, 0, 0, 0, 16};
double *in;

in = (double*)fft_malloc(sizeof(double) * 8);

for (int i = 0; i < 8; i++) {
    in[i] = init[i];
}

double *out;

out = (double*)fft_malloc(sizeof(double) * 8);

fft_r2r_kind *kind;

kind = (fft_r2r_kind*)fft_malloc(sizeof(fft_r2r_kind) * rank);

kind[0] = FFT_DHT;

kind[1] = FFT_REDFT01;

```

```

fft_plan plan;

plan = fft_plan_guru64_r2r(rank, dims, howmany_rank, howmany_dims,
in, out, kind, FFT_ESTIMATE);

fft_execute_r2r(plan, in, out);


fft_destroy_plan(plan);
fft_free(howmany_dims);
fft_free(dims);
fft_free(kind);
fft_free(in);
fft_free(out);


/*
 * out = {1.393137e+02, 1.166863e+02, 1.006863e+02, 1.233137e+02,
 *        2.262742e+01, -2.262742e+01, -2.262742e+01, 2.262742e+01}
 */

```

2.3.4.8 fft(f) plan_many_r2r

建立 `howmany` 组数据序列 `n` 维 R2R 变换的 `plan`。其中，单个 FFT 的数据序列不需要是连续的，可以以跨步的形式提供。

2.3.4.8.1 接口定义

C interface:

```

fft_plan fft_plan_many_r2r(int rank, const int *n, int howmany, double *in, const int
*inembed, int istride, int idist, double *out, const int *onembed, int ostride, int odist,
const fft_r2r_kind *kind, unsigned flags);

```

```

fftf_plan fftf_plan_many_r2r(int rank, const int *n, int howmany, float *in, const int
*inembed, int istride, int idist, float *out, const int *onembed, int ostride, int odist,
const fftf_r2r_kind *kind, unsigned flags);

```

Fortran interface:

RES = FFT_PLAN_MANY_DFT_R2R(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, KIND, FLAGS);

RES = FFTF_PLAN_MANY_DFT_R2R(RANK, N, HOWMANY, IN, INEMBED, ISTRIDE, IDIST, OUT, ONEMBED, OSTRIDE, ODIST, KIND, FLAGS);

FFT_REDFFT11 和 FFT_ROODFT11 只支持长度为 4 的整数倍大小的序列，其他变换类型只支持长度为 2 的整数倍大小的序列。

2.3.4.8.2 返回值

函数返回一个 `fft(f)_plan` 类型的结构体指针。将该对象作为参数传入 `fft(f)_execute` 函数中使用，将对当前提供的输入 `in` 和输出 `out` 执行 FFT 变换；另外，也可以通过将该对象作为参数传入 `fft(f)_execute_dft_r2r` 函数中以对新的输入 `in` 和输出 `out` 执行 FFT 变换。

如果函数返回非空指针，则表示 `plan` 执行成功，否则表示执行失败。

2.3.4.8.3 参数

参数名	类型	描述	输入/输出
rank	int	FFT 变换的维度是 rank，约束： $1 \leq \text{rank} \leq 3$ 。	输入
n	const int*	n 是维度为 rank 的数组，包含 FFT 序列每一维度的大小，约束： $n[i] \geq 1$, for i in 0 to rank - 1。	输入
howmany	int	howmany 表示要多少个多维 FFT 变换。	输入
in	双精度: double*	输入待变换的数据。	输入
	单精度: float*		
inembed	const int*	inembed 是大小为 rank 的数组或者 NULL，该数组表示 FFT 数据 in 存储的更大空间的每一维度的大小。 约束: $\text{inembed}[i] \geq n[i]$ for i in 0, rank-1 或者如果 $\text{inembed} == \text{NULL}$ ，则可以认为 inembed 与 n 相等。	输入
istride	int	FFT 输入序列的相继元素之间的间隔。	输入

参数名	类型	描述	输入/输出
idist	int	idist 表示输入的每个 FFT 序列的间隔。	输入
out	双精度: double*	输出快速傅里叶变换后的数据。	输出
	单精度: float*		
onembed	const int*	onembed 是大小为 rank 的数组或者 NULL, 该数组表示 FFT 数据 out 存储的更大空间的每一维度的大小。 约束: onembed[i] ≥ n[i] for i in 0, rank-1 或者如果 onembed == NULL, 则可以认为 onembed 与 n 相等。	输入
ostride	int	FFT 输出序列的相继元素之间的间隔。	输入
odist	int	odist 表示输出的每个 FFT 序列的间隔。	输入
kind	双精度: const fft_r2r_kind* 单精度: const fftf_r2r_kind*	kind 是大小为 rank 的数组, 包含 FFT 序列每一维度的 R2R 变换类型, kind[i] (for i in 0 to rank - 1)有以下可选值: FFT_R2HC FFT_HC2R FFT_DHT FFT_REDFT00 FFT_REDFT01 FFT_REDFT10 FFT_REDFT11 FFT_RODFT00 FFT_RODFT01 FFT_RODFT10 FFT_RODFT11	输入
flags	unsigned int	planning 选项, 未使用。	输入

2.3.4.8.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.4.8.5 示例

C interface:

```
int rank = 1;

int *n;

n = (int*)fft_malloc(sizeof(int) * rank);

n[0] = 2;

int howmany = 3;

int istride = 1;

int ostride = 1;

int idist = 2;

int odist = 2;

double init[6] = {120, 1, 8, 8, 120, 1};

double *in;

in = (double*)fft_malloc(sizeof(double) * 6);

for (int i = 0; i < 6; i++) {
    in[i] = init[i];
}

double *out;

out = (double*)fft_malloc(sizeof(double) * 6);

fft_r2r_kind *kind;

kind = (fft_r2r_kind*)fft_malloc(sizeof(fft_r2r_kind) * rank);

kind[0] = FFT_DHT;

fft_plan plan;

plan = fft_plan_many_r2r(rank, n, howmany, in, NULL, istride, idist,
out, NULL, ostride, odist, kind, FFT_ESTIMATE);

fft_execute_r2r(plan, in, out);

fft_destroy_plan(plan);

fft_free(n);

fft_free(kind);
```



```

fft_free(in);
fft_free(out);

/*
 * out = {1.210000e+02, 1.190000e+02, 1.600000e+01, 0.000000e+00,
 *        1.210000e+02, 1.190000e+02}
 */

```

2.3.5 变换执行函数

2.3.5.1 函数说明

函数名	简介
fft(f)_execute	执行之前建立的 FFT 变换 plan。
fft(f)_execute_dft	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_dft_r2c	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_dft_c2r	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_r2r	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_split_dft	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_split_dft_r2c	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。
fft(f)_execute_split_dft_c2r	执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。

2.3.5.2 fft(f)_execute

fft(f)_execute 执行之前建立的 FFT 变换 plan。

2.3.5.2.1 接口定义

C interface:

```
void fft_execute(const fft_plan p);  
void fftf_execute(const fftf_plan p);
```

Fortran interface:

```
CALL FFT_EXECUTE(P);  
CALL FFTF_EXECUTE(P);
```

2.3.5.2.2 参数

参数名	类型	描述	输入/输出
p	双精度: const fft_plan	通过调用 plan 函数得到的 FFT 变换 plan。	输入
	单精度: const fftf_plan		

2.3.5.2.3 依赖

C: "kfft.h"
Fortran: "kfft.f03"

2.3.5.2.4 示例

C interface:

```
int N = 16;  
fft_complex *in;  
fft_complex *out;  
fft_plan plan;  
  
in = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);  
out = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
```

```

for (int i = 0; i < N; i++){
    in[i].r = (double)i;
    in[i].i = (double)i;
}

plan = fft_plan_dft_1d(N, in , out, FFT_FORWARD, FFT_ESTIMATE);
fft_execute(plan);

fft_destroy_plan(plan);
fft_free(in);
fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01,
3.221872e+01},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01,
3.972846e+00},
 *        {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -
2.654571e+00},
 *        {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -
6.408701e+00},
 *        {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -
9.591299e+00},
 *        {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -
1.334543e+01},
 *        {0.000000e+00, -1.600000e+01}, {3.972846e+00, -
1.997285e+01},
 *        {1.131371e+01, -2.731371e+01}, {3.221872e+01, -
4.821872e+01}}
 */

```

2.3.5.3 fft(f)_execute_dft

fft(f)_execute_dft 执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。

2.3.5.3.1 接口定义

C interface:

```
void fft_execute_dft(const fft_plan p, fft_complex *in, fft_complex *out);
```

```
void fftf_execute_dft(const fftf_plan p, fftf_complex *in, fftf_complex *out);
```

Fortran interface:

```
CALL FFT_EXECUTE_DFT(P, IN, OUT);
```

```
CALL FFTF_EXECUTE_DFT(P, IN, OUT);
```

2.3.5.3.2 参数

参数名	类型	描述	输入/输出
p	双精度: const fft_plan	通过调用 plan 函数得到的 FFT 变换 plan。	输入
	单精度: const fftf_plan		
in	双精度: fft_complex *	输入待变换的数据。	输入
	单精度: fftf_complex *		
out	双精度: fft_complex *	输出快速傅里叶变换后的数据。	输出
	单精度: fftf_complex *		

2.3.5.3.3 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.5.3.4 示例

见 C2C 变换的相关示例。

2.3.5.4 fft(f)_execute_dft_r2c

fft(f)_execute_dft_r2c 执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。

2.3.5.4.1 接口定义

C interface:

```
void fft_execute_dft_r2c(const fft_plan p, double *in, fft_complex *out);
void fftf_execute_dft_r2c(const fftf_plan p, float *in, fftf_complex *out);
```

Fortran interface:

```
CALL FFT_EXECUTE_DFT_R2C(P, IN, OUT);
CALL FFTF_EXECUTE_DFT_R2C(P, IN, OUT);
```

2.3.5.4.2 参数

参数名	类型	描述	输入/输出
p	双精度: const fft_pla n	通过调用 plan 函数得到的 FFT 变换 plan。	输入
	单精度: const fftf_pl an		
in	双精度: double*	输入待变换的数据。	输入
	单精度: float*		
out	双精度: fft_complex *	输出快速傅里叶变换后的数据。	输出
	单精度: fftf_comple x*		

2.3.5.4.3 依赖

```
C: "kfft.h"
Fortran: "kfft.f03"
```

2.3.5.4.4 示例

见 R2C 变换的相关示例。

2.3.5.5 fft(f)_execute_dft_c2r

fft(f)_execute_dft_c2r 执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。

2.3.5.5.1 接口定义

C interface:

```
void fft_execute_dft_c2r(const fft_plan p, fft_complex *in, double *out);
```

```
void fftf_execute_dft_c2r(const fftf_plan p, fftf_complex *in, float *out);
```

Fortran interface:

```
CALL FFT_EXECUTE_DFT_C2R(P, IN, OUT);
```

```
CALL FFTF_EXECUTE_DFT_C2R(P, IN, OUT);
```

2.3.5.5.2 参数

参数名	类型	描述	输入/输出
p	双精度: const fft_plan	通过调用 plan 函数得到的 FFT 变换 plan。	输入
	单精度: const fftf_plan		
in	双精度: fft_complex*	输入待变换的数据。	输入
	单精度: fftf_complex*		
out	双精度: double*	输出快速傅里叶变换后的数据。	输出
	单精度: float*		

2.3.5.5.3 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.5.5.4 示例

见 C2R 变换的相关示例。

2.3.5.6 fft(f)_execute_r2r

fft(f)_execute_dft_r2r 执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。

2.3.5.6.1 接口定义

C interface:

```
void fft_execute_r2r(const fft_plan p, double *in, double *out);
```

```
void fftf_execute_r2r(const fftf_plan p, float *in, float *out);
```

Fortran interface:

```
CALL FFT_EXECUTE_R2R(P, IN, OUT);.
```

```
CALL FFTF_EXECUTE_R2R(P, IN, OUT);
```

2.3.5.6.2 参数

参数名	类型	描述	输入/输出
p	双精度: const fft_plan	通过调用 plan 函数得到的 FFT 变换 plan。	输入
	单精度: const fftf_plan		
in	双精度: double*	输入待变换的数据。	输入
	单精度: float*		
out	双精度: double*	输出快速傅里叶变换后的数据。	输出
	单精度: float*		

2.3.5.6.3 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.5.6.4 示例

见 R2R 变换的相关示例。

2.3.5.7 fft(f)_execute_splite_dft

fft(f)_execute_splite_dft 执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。

2.3.5.7.1 接口定义

C interface:

```
void fft_execute_split_dft(const fft_plan p, double *ri, double *ii, double *ro, double *io);
```

```
void fftf_execute_split_dft(const fftf_plan p, float *ri, float *ii, float *ro, float *io);
```

Fortran interface:

```
CALL FFT_EXECUTE_SPLIT_DFT(P, RI, II, RO, IO);
```

```
CALL FFTF_EXECUTE_SPLIT_DFT(P, RI, II, RO, IO);
```

2.3.5.7.2 参数

参数名	类型	描述	输入/输出
p	双精度: const fft_plan	通过调用 plan 函数得到的 FFT 变换 plan。	输入
	单精度: const fftf_plan		
ri	双精度: double*	输入待变换数据的实部。	输入
	单精度: float*		
ii	双精度: double*	输入待变换数据的虚部	输入
	单精度: float*		
ro	双精度: double*	输出待变换数据的实部。	输出
	单精度: float*		
io	双精度: double*	输出待变换数据的虚部	输出
	单精度: float*		

2.3.5.7.3 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.5.7.4 示例

见 C2C 变换的相关示例。

2.3.5.8 fft(f)_execute_splite_dft_r2c

fft(f)_execute_splite_dft_r2c 执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。

2.3.5.8.1 接口定义

C interface:

```
void fft_execute_split_dft_r2c(const fft_plan p, double *in, double *ro, double *io);
```

```
void fftf_execute_split_dft_r2c(const fftf_plan p, float *in, float *ro, float *io);
```

Fortran interface:

```
CALL FFT_EXECUTE_SPLIT_DFT_R2C(P, IN, RO, IO);
```

```
CALL FFTF_EXECUTE_SPLIT_DFT_R2C(P, IN, RO, IO);
```

2.3.5.8.2 参数

参数名	类型	描述	输入/输出
p	双精度: const fft_plan	通过调用 plan 函数得到的 FFT 变换 plan。	输入
	单精度: const fftf_plan		
in	双精度: double*	输入待变换的数据。	输入
	单精度: float*		
ro	双精度: double*	输出待变换数据的实部。	输出
	单精度: float*		
io	双精度: double*	输出待变换数据的虚部	输出
	单精度: float*		

2.3.5.8.3 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.5.8.4 示例

见 R2C 变换的相关示例。

2.3.5.9 fft(f)_execute_splite_dft_c2r

fft(f)_execute_splite_dft_c2r 执行之前建立的 FFT 变换 plan，但是可以接受与 plan 不同的新的输入输出数据作为参数。

2.3.5.9.1 接口定义

C interface:

```
void fft_execute_split_dft_c2r(const fft_plan p, double *ri, double *ii, double *out);
```

```
void fftf_execute_split_dft_c2r(const fftf_plan p, float *ri, float *ii, float *out);
```

Fortran interface:

```
CALL FFT_EXECUTE_SPLIT_DFT_C2R(P, RI, II, OUT);
```

```
CALL FFTF_EXECUTE_SPLIT_DFT_C2R(P, RI, II, OUT);
```

2.3.5.9.2 参数

参数名	类型	描述	输入/输出
p	双精度: const fft_plan	通过调用 plan 函数得到的 FFT 变换 plan。	输入
	单精度: const fftf_plan		
ri	双精度: double*	输入待变换数据的实部。	输入
	单精度: float*		
ii	双精度: double*	输入待变换数据的虚部	输入
	单精度: float*		
out	双精度: double*	输出待变换的数据。	输出
	单精度: float*		

2.3.5.9.3 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.5.9.4 示例

见 C2R 变换的相关示例。

2.3.6 内存函数

2.3.6.1 函数说明

内存函数的调用约束：`plan` 使用完毕后需要通过调用 `fft(f)_destroy_plan` 函数来释放；`fft(f)_malloc` 函数所申请的内存空间只能通过调用 `fft(f)_free` 函数释放。

函数名	简介
<code>fft(f)_malloc</code>	<code>fft(f)_malloc</code> 用于分配所需的内存空间。所申请的内存空间只能通过调用 <code>fft(f)_free</code> 函数释放。
<code>fft(f)_free</code>	<code>fft(f)_free</code> 释放由 <code>fft(f)_malloc</code> 函数申请的内存。
<code>fft(f)_destroy_plan</code>	<code>fft(f)_destroy_plan</code> 释放 FFT 变换 <code>plan</code> 的所有内存。调用此函数之后，此 <code>plan</code> 不再可用。

2.3.6.2 `fft(f)_malloc`

`fft(f)_malloc` 用于分配所需的内存空间。所申请的内存空间只能通过调用 `fft(f)_free` 函数释放。

2.3.6.2.1 接口定义

C interface:

```
void* fft_malloc(size_t n);
void* fftf_malloc(size_t n);
```

Fortran interface:

```
RES = FFT_MALLOC(N);
RES = FFTF_MALLOC(N);
```

2.3.6.2.2 返回值

函数返回一个 `void*` 类型的指针，指向已分配的内存块。

2.3.6.2.3 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
n	size_t	需要申请的内存大小（以 bytes 为单位）。	输入

2.3.6.2.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.6.2.5 示例

C interface:

```
int N = 16;
fft_complex *in;
fft_complex *out;
fft_plan plan;
in = (fft_complex*) fft_malloc(sizeof(fft_complex) * N);
out = (fft_complex*) fft_malloc(sizeof(fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i].r = (double) i;
    in[i].i = (double) i;
}
plan = fft_plan_dft_1d(N, in , out, FFT_FORWARD, FFT_ESTIMATE);
fft_execute_dft(plan, in, out);

fft_destroy_plan(plan);
fft_free(in);
fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01,
3.221872e+01},
```

```

*      {-2.731371e+01, 1.131371e+01}, {-1.997285e+01,
3.972846e+00},
*      {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -
2.654571e+00},
*      {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -
6.408701e+00},
*      {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -
9.591299e+00},
*      {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -
1.334543e+01},
*      {0.000000e+00, -1.600000e+01}, {3.972846e+00, -
1.997285e+01},
*      {1.131371e+01, -2.731371e+01}, {3.221872e+01, -
4.821872e+01}}
*/

```

2.3.6.3 fft(f)_free

fft(f)_free 释放由 fft(f)_malloc 函数申请的内存。

2.3.6.3.1 接口定义

C interface:

```
void fft_free(void *p);
```

```
void fftf_free(void *p);
```

Fortran interface:

```
CALL FFT_FREE(P);.
```

```
CALL FFTF_FREE(P);
```

2.3.6.3.2 返回值

函数返回一个 void*类型的指针，指向已分配的内存块。

2.3.6.3.3 参数

参数名	类型	描述	输入/输出
p	void*	P 是一个指针，指向之前申请的内存块。	输入

2.3.6.3.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.6.3.5 示例

C interface:

```
int N = 16;
fft_complex *in;
fft_complex *out;
fft_plan plan;
in = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
out = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i].r = (double)i;
    in[i].i = (double)i;
}

plan = fft_plan_dft_1d(N, in , out, FFT_FORWARD, FFT_ESTIMATE);
fft_execute(plan);

fft_destroy_plan(plan);
fft_free(in);
fft_free(out);

/*
```

```
* out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01,
3.221872e+01},
*      {-2.731371e+01, 1.131371e+01}, {-1.997285e+01,
3.972846e+00},
*      {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -
2.654571e+00},
*      {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -
6.408701e+00},
*      {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -
9.591299e+00},
*      {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -
1.334543e+01},
*      {0.000000e+00, -1.600000e+01}, {3.972846e+00, -
1.997285e+01},
*      {1.131371e+01, -2.731371e+01}, {3.221872e+01, -
4.821872e+01}}
*/
```

2.3.6.4 fft(f)_destroy_plan

fft(f)_destroy_plan 释放 FFT 变换 plan 的所有内存。调用此函数之后，此 plan 不再可用。

2.3.6.4.1 接口定义

C interface:

```
void fft_destroy_plan(fft_plan p);
void fftf_destroy_plan(fftf_plan p);
```

Fortran interface:

```
CALL FFT_DESTROY_PLAN(P);
CALL FFTF_DESTROY_PLAN(P);
```

2.3.6.4.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
p	双精度: const fft_plan	待释放内存的 FFT 变换 plan。	输入
	单精度: const fftf_plan		

2.3.6.4.3 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.6.4.4 示例

C interface:

```

int N = 16;
fft_complex *in;
fft_complex *out;
fft_plan plan;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
out = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i].r = (double)i;
    in[i].i = (double)i;
}

plan = fft_plan_dft_1d(N, in , out, FFT_FORWARD, FFT_ESTIMATE);
fft_execute(plan);

fft_destroy_plan(plan);
fft_free(in);
fft_free(out);

/*

```



```

* out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01,
3.221872e+01},
*      {-2.731371e+01, 1.131371e+01}, {-1.997285e+01,
3.972846e+00},
*      {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -
2.654571e+00},
*      {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -
6.408701e+00},
*      {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -
9.591299e+00},
*      {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -
1.334543e+01},
*      {0.000000e+00, -1.600000e+01}, {3.972846e+00, -
1.997285e+01},
*      {1.131371e+01, -2.731371e+01}, {3.221872e+01, -
4.821872e+01}}
*/

```

2.3.7 线程函数

2.3.7.1 函数说明

线程函数的调用顺序约束：

1. 首先调用 `fft(f)_init_threads` 初始化线程的使用。
2. 然后调用 `fft(f)_plan_with_nthreads` 指定线程数。
3. 最后调用 `fft(f)_cleanup_threads` 释放多线程框架相关的资源。

函数名	简介
<code>fft(f)_init_threads</code>	<code>fft(f)_init_threads</code> 用于初始化线程的使用。（此函数不可重入。）
<code>fft(f)_plan_with_nthreads</code>	<code>fft(f)_plan_with_nthreads</code> 用于指定 FFT 库接口函数执行的线程数。（此函数不可重入。）

函数名	简介
fft(f)_cleanup_threads	fft(f)_cleanup_threads 用于释放多线程框架相关的资源，与 fft(f)_init_threads 相对应，在整个系统退出时调用。调用后不能再使用多线程功能。（此函数不可重入。）

2.3.7.2 fft(f)_init_threads

fft(f)_init_threads 用于初始化线程的使用。（此函数不可重入。）

2.3.7.2.1 接口定义

C interface:

int fft_init_threads(void);
int fftf_init_threads(void);

Fortran interface:

RES = FFT_INIT_THREADS();
RES = FFTF_INIT_THREADS();

2.3.7.2.2 返回值

Kml fft(f)_init_threads 函数执行成功返回 1，否则返回 0。

2.3.7.2.3 参数

无

2.3.7.2.4 依赖

C: "kfft.h"
Fortran: "kfft.f03"

2.3.7.2.5 示例

C interface:

```
fft_init_threads();

int nthreads = 4;

fft_plan_with_nthreads(nthreads);

int N = 16;

fft_complex *in;

fft_complex *out;

fft_plan plan;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);

out = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);

for (int i = 0; i < N; i++){

    in[i].r = (double)i;

    in[i].i = (double)i;

}

plan = fft_plan_dft_1d(N, in , out, FFT_FORWARD, FFT_ESTIMATE);

fft_execute(plan);

fft_cleanup_threads();

fft_destroy_plan(plan);

fft_free(in);

fft_free(out);

/*

    * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01,

3.221872e+01},

    *         {-2.731371e+01, 1.131371e+01}, {-1.997285e+01,

3.972846e+00},

    *         {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -

2.654571e+00},

    *         {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -

6.408701e+00},
```

```

      *      {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -
9.591299e+00},
      *      {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -
1.334543e+01},
      *      {0.000000e+00, -1.600000e+01}, {3.972846e+00, -
1.997285e+01},
      *      {1.131371e+01, -2.731371e+01}, {3.221872e+01, -
4.821872e+01}}
      */

```

2.3.7.3 fft(f)_free

fft(f)_plan_with_nthreads 用于指定 FFT 库接口函数执行的线程数。（此函数不可重入。）

2.3.7.3.1 接口定义

C interface:

void fft_plan_with_nthreads(int nthreads);
void fftf_plan_with_nthreads(int nthreads);

Fortran interface:

CALL FFT_PLAN_WITH_NTHREADS(NTHREADS);
CALL FFTF_PLAN_WITH_NTHREADS(NTHREADS);

2.3.7.3.2 返回值

函数返回一个 void*类型的指针，指向已分配的内存块。

2.3.7.3.3 参数

参数名	类型	描述	输入/输出
nthreads	int	执行后续 FFT 变换 plan 所使用的线程数。	输入

2.3.7.3.4 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.7.3.5 示例

C interface:

```
fft_init_threads();

int nthreads = 4;

fft_plan_with_nthreads(nthreads);

int N = 16;

fft_complex *in;

fft_complex *out;

fft_plan plan;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
out = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i].r = (double)i;
    in[i].i = (double)i;
}

plan = fft_plan_dft_1d(N, in , out, FFT_FORWARD, FFT_ESTIMATE);
fft_execute(plan);
fft_cleanup_threads();

fft_destroy_plan(plan);
fft_free(in);
fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01,
3.221872e+01}},
 *        {-2.731371e+01, 1.131371e+01}, {-1.997285e+01,
3.972846e+00}},
```

```

      *      {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -
2.654571e+00},
      *      {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -
6.408701e+00},
      *      {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -
9.591299e+00},
      *      {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -
1.334543e+01},
      *      {0.000000e+00, -1.600000e+01}, {3.972846e+00, -
1.997285e+01},
      *      {1.131371e+01, -2.731371e+01}, {3.221872e+01, -
4.821872e+01}}
      */

```

2.3.7.4 fft(f)_cleanup_threads

fft(f)_cleanup_threads 用于释放多线程框架相关的资源，与 fft(f)_init_threads 相对应，在整个系统退出时调用。调用后不能再使用多线程功能。（此函数不可重入。）

2.3.7.4.1 接口定义

C interface:

```

void fft_cleanup_threads(void);
void fftf_cleanup_threads(void);

```

Fortran interface:

```

CALL FFT_CLEANUP_THREADS();
CALL FFTF_CLEANUP_THREADS();

```

2.3.7.4.2 参数

无

2.3.7.4.3 依赖

C: "kfft.h"

Fortran: "kfft.f03"

2.3.7.4.4 示例

C interface:

```
fft_init_threads();

int nthreads = 4;

fft_plan_with_nthreads(nthreads);

int N = 16;

fft_complex *in;

fft_complex *out;

fft_plan plan;

in = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
out = (fft_complex*)fft_malloc(sizeof(fft_complex) * N);
for (int i = 0; i < N; i++){
    in[i].r = (double)i;
    in[i].i = (double)i;
}

plan = fft_plan_dft_1d(N, in , out, FFT_FORWARD, FFT_ESTIMATE);
fft_execute(plan);
fft_cleanup_threads();

fft_destroy_plan(plan);
fft_free(in);
fft_free(out);

/*
 * out = {{1.200000e+02, 1.200000e+02}, {-4.821872e+01,
3.221872e+01}},
 *       {-2.731371e+01, 1.131371e+01}, {-1.997285e+01,
3.972846e+00}},
```

```

*      {-1.600000e+01, 0.000000e+00}, {-1.334543e+01, -
2.654571e+00},
*      {-1.131371e+01, -4.686292e+00}, {-9.591299e+00, -
6.408701e+00},
*      {-8.000000e+00, -8.000000e+00}, {-6.408701e+00, -
9.591299e+00},
*      {-4.686292e+00, -1.131371e+01}, {-2.654571e+00, -
1.334543e+01},
*      {0.000000e+00, -1.600000e+01}, {3.972846e+00, -
1.997285e+01},
*      {1.131371e+01, -2.731371e+01}, {3.221872e+01, -
4.821872e+01}}
*/

```

2.3.8 获取 FFT 产品版本信息

2.3.8.1 函数说明

函数名	简介
FFTGetVersion	获取 FFT 产品版本信息

2.3.8.2 FFTGetVersion

获取 FFT 产品版本信息。

2.3.8.2.1 接口定义

```
int KFFTGetVersion(KFFTVersion*ver)
```

2.3.8.2.2 返回值

成功：返回 1。

失败：返回 0。

2.3.8.2.3 参数

参数名	描述	取值范围	输入/输出
-----	----	------	-------

参数名	描述	取值范围	输入/输出
ver	产品信息结构体	非空	输出

2.3.8.2.4 示例

C interface:

```
#include <stdio.h>

#include "kfft.h"

int main()
{
    KFFTVersion ver;
    int ret = KFFTGetVersion(&ver);
    if (ret == 1) {
        printf("\nFFT\n");
        printf("Product Name:      %s\n", ver.productName);
        printf("Product Version:      %s\n", ver.productVersion);
        printf("Component Name:       %s\n", ver.componentName);
        printf("Component Version:    %s\n", ver.componentVersion);
        printf("Component AppendInfo: %s\n",
ver.componentAppendInfo);
        printf("Software Name:        %s\n", ver.softwareName);
        printf("Software Version:     %s\n", ver.softwareVersion);
    }

    return 0;
}
```

2.4 SPARSE 接口说明

2.4.1 Level 1

2.4.1.1 函数说明

函数名	简介
<code>sparse_?axpyi</code>	标量-向量乘，并加到另一向量上：$y = a * x + y$。其中，a 是缩放系数，x 是稀疏向量，y 是稠密向量。具体分为以下四个函数： <code>sparse_saxpyi</code>: 单精度浮点类型标量-向量乘 <code>sparse_daxpyi</code>: 双精度浮点类型标量-向量乘 <code>sparse_caxpyi</code>: 单精度复数类型标量-向量乘 <code>sparse_zaxpyi</code>: 双精度复数类型标量-向量乘
<code>sparse_?doti</code>	实数域的点积（欧几里得空间）。 $res = x[0] * y[indx[0]] + x[1] * y[indx[1]] + ... + x[nz-1] * y[indx[nz-1]]$ 具体分为以下两个函数： <code>sparse_sdoti</code>: 单精度浮点类型点积 <code>sparse_ddoti</code>: 双精度浮点类型点积
<code>sparse_?dotci_sub</code>	共轭点积（希尔伯特空间）；$x \cdot y = x^H * y$，即 x 向量中的元素取其共轭复数后与 y 向量中对应元素相乘然后累加（复数 $a + bi$ 的共轭为 $a - bi$），计算公式表示如下： $conjg(x[0]) * y[indx[0]] + ... + conjg(x[nz-1]) * y[indx[nz-1]]$ 具体分为以下两个函数： <code>sparse_cdotci_sub</code>: 单精度浮点类型共轭点积 <code>sparse_zdotci_sub</code>: 双精度浮点类型共轭点积

函数名	简介
<code>sparse_?dotui_sub</code>	复数域的非共轭点积（欧几里得空间）：$x \cdot y = x^T * y$，即对应元素相乘然后累加。 $res = x[0] * y[indx[0]] + x[1] * y[indx[1]] + \dots + x[nz - 1] * y[indx[nz - 1]]$ 具体分为以下两个函数： <code>sparse_cdotui_sub</code>：单精度浮点类型非共轭点积 <code>sparse_zdotui_sub</code>：双精度浮点类型非共轭点积
<code>sparse_?gthr</code>	将 full-storage 格式的稀疏向量 y 中指定的元素加载到 compressed 格式的向量中。即 $x[i]=y[indx[i]]$，$i=0,1,\dots,nz-1$。 具体分为以下四个函数： <code>sparse_sgthr</code>：单精度浮点类型元素加载 <code>sparse_dgthr</code>：双精度浮点类型元素加载 <code>sparse_cgthr</code>：单精度复数类型元素加载 <code>sparse_zgthr</code>：双精度复数类型元素加载
<code>sparse_?gthrz</code>	将 full-storage 格式的稀疏向量 y 中指定的元素加载到 compressed 格式的向量中。即 $x[i]=y[indx[i]]$，$i=0,1,\dots,nz-1$，并将 y 中对应元素清零。 具体分为以下四个函数： <code>sparse_sgthrz</code>：单精度浮点类型元素加载并清零 <code>sparse_dgthrz</code>：双精度浮点类型元素加载并清零 <code>sparse_cgthrz</code>：单精度复数类型元素加载并清零 <code>sparse_zgthrz</code>：双精度复数类型元素加载并清零
<code>sparse_?roti</code>	对 2 个实数类型的向量进行旋转操作。 $x[i] = c * x[i] + s * y[indx[i]]$ $y[indx[i]] = c * y[indx[i]] - s * x[i]$ 具体分为以下两个函数： <code>sparse_sroti</code>：单精度浮点类型向量旋转 <code>sparse_droti</code>：双精度浮点类型向量旋转

函数名	简介
<code>sparse_?sctr</code>	将 compressed 格式的向量写入 full-storage 格式的稀疏向量 y 指定位置。即 $y[\text{indx}[i]] = x[i]$, $i=0,1,\dots,nz-1$。具体分为以下四个函数： <code>sparse_ssctr</code>: 单精度浮点类型向量写入 <code>sparse_dsctr</code>: 双精度浮点类型向量写入 <code>sparse_csctr</code>: 单精度复数类型向量写入 <code>sparse_zsctr</code>: 双精度复数类型向量写入

2.4.1.2 sparse_?axpyi

标量-向量乘，并加到另一向量上： $y = a * x + y$ 。

其中，a 是缩放系数，x 是稀疏向量，y 是稠密向量。

2.4.1.2.1 接口定义

A. C interface:

```
sparse_status_t sparse_saxpyi(const INT nz, const float a, const float *x, const INT
*indx, float *y);
```

```
sparse_status_t sparse_daxpyi(const INT nz, const double a, const double *x, const
INT *indx, double *y);
```

```
sparse_status_t sparse_caxpyi(const INT nz, const Complex8 a, const Complex8 *x,
const INT *indx, Complex8 *y);
```

```
sparse_status_t sparse_zaxpyi(const INT nz, const Complex16 a, const Complex16 *x,
const INT *indx, Complex16 *y);
```

B. Fortran interface:

```
RES = SPARSE_SAXPYI(NZ, A, X, INDX, Y);
```

```
RES = SPARSE_DAXPYI(NZ, A, X, INDX, Y);
```

```
RES = SPARSE_CAXPYI(NZ, A, X, INDX, Y);
```

```
RES = SPARSE_ZAXPYI(NZ, A, X, INDX, Y);
```

2.4.1.2.2 参数

参数名	类型	描述	输入/输出
nz	整型数	x 及 indx 数组中元素的个数。	输入
a	在 saxpyi 中, a 是单精度浮点类型。	标量 a。	输入
	在 daxpyi 中, a 是双精度浮点类型。		
	在 caxpyi 中, a 是单精度复数。		
	在 zaxpyi 中, a 是双精度复数。		
x	在 saxpyi 中, x 是单精度浮点类型数组。	存储非零元素的数组 x, 大于或等于 nz。	输入
	在 daxpyi 中, x 是双精度浮点类型数组。		
	在 caxpyi 中, x 是单精度复数数组。		
	在 zaxpyi 中, x 是双精度复数数组。		
indx	整型数组	吉文斯旋转参数 s。	输入
y	在 saxpyi 中, y 是单精度浮点类型数组。	数组 y, 大于或等于 max(indx[i])。	输入/输出
	在 daxpyi 中, y 是双精度浮点类型数组。		
	在 caxpyi 中, y 是单精度复数数组。		
	在 zaxpyi 中, y 是双精度复数数组。		

2.4.1.2.3 返回值

函数执行状态, 枚举类型 sparse_status_t。

2.4.1.2.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.1.2.5 示例

A. C interface:

```
INT nz = 2;

float alpha = 1;

INT indx[2] = {1, 2};

float x[2] = {1, 3};

float y[4] = {-1, 1, 5, 3};

sparse_status_t status = sparse_saxpyi(nz, alpha, x, indx, y);

/*
 * Output Y:
 *      -1.000000    2.000000    8.000000    3.000000
 *
 */
```

2.4.1.3 sparse_?doti

实数域的点积（欧几里得空间）。

$\text{res} = x[0] * y[\text{indx}[0]] + x[1] * y[\text{indx}[1]] + \dots + x[\text{nz}-1] * y[\text{indx}[\text{nz}-1]]$ 。

2.4.1.3.1 接口定义

A. C interface:

```
sparse_status_t sparse_sdotti(const INT nz, const float *x, const INT *indx, const float *y, float *doti);
```

```
sparse_status_t sparse_ddotti(const INT nz, const double *x, const INT *indx, const double *y, double *doti);
```

B. Fortran interface:

RES = SPARSE_SDOTI(NZ, X, INDX, Y, DOTI);

RES = SPARSE_DDOTI(NZ, X, INDX, Y, DOTI);

2.4.1.3.2 参数

参数名	类型	描述	输入/输出
nz	整型数	x 及 indx 数组中元素的个数。	输入
x	在 sdoti 中, x 是单精度浮点类型数组。	存储非零元素的数组 x, 大于或等于 nz。	输入
	在 ddoti 中, x 是双精度浮点类型数组。		
indx	整型数组	indx[i]表示 x 数组中第 i 个元素在稠密向量中的序列号, 数组大于或等于 nz。	输入
y	在 sdoti 中, y 是单精度浮点类型数组。	数组 y, 大于或等于 max(indx[i])。	输入
	在 ddoti 中, y 是双精度浮点类型数组。		
doti	在 sdoti 中, doti 是单精度浮点类型。	x 与 y 的点积。	输入/输出
	在 ddoti 中, doti 是双精度浮点类型。		

2.4.1.3.3 返回值

函数执行状态, 枚举类型 sparse_status_t。

2.4.1.3.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.1.3.5 示例

A. C interface:

```
INT nz = 2;
```

```

float doti = 0;

INT indx[2] = {1, 2};

float x[2] = {1, 3};

float y[4] = {-1, 1, 5, 3};

sparse_status_t status = sparse_sdoti(nz, x, indx, y, &doti);

/*
 *   Output doti:
 *       16.000000
 */

```

2.4.1.4 sparse_?dotci_sub

共轭点积（希尔伯特空间）： $x \cdot y = x^H * y$ ，即 x 向量中的元素取其共轭复数后与 y 向量中对应元素相乘然后累加（复数 $a + bi$ 的共轭为 $a - bi$ ），计算公式表示如下：

$\text{conjg}(x[0]) * y[\text{indx}[0]] + \dots + \text{conjg}(x[\text{nz}-1]) * y[\text{indx}[\text{nz}-1]]$ 。

2.4.1.4.1 接口定义

A. C interface:

```

sparse_status_t sparse_cdotci_sub(const INT nz, const Complex8 *x, const INT *indx,
const Complex8 *y, Complex8 *dotci);

```

```

sparse_status_t sparse_zdotci_sub(const INT nz, const Complex16 *x, const INT
*indx, const Complex16 *y, Complex16 *dotci);

```

B. Fortran interface:

```

RES = SPARSE_CDOTCI_SUB(NZ, X, INDX, Y, DOTCI);

```

```

RES = SPARSE_ZDOTCI_SUB(NZ, X, INDX, Y, DOTCI);

```

2.4.1.4.2 参数

参数名	类型	描述	输入/输出
nz	整型数	x 及 indx 数组中元素的个数。	输入

参数名	类型	描述	输入/输出
x	在 cdotci 中，x 是单精度浮点类型数组。	存储非零元素的数组 x，大于或等于 nz。	输入
	在 zdotci 中，x 是双精度浮点类型数组。		
indx	整型数组	indx[i]表示 x 数组中第 i 个元素在稠密向量中的序列号，数组大于或等于 nz。	输入
y	在 cdotci 中，y 是单精度浮点类型数组。	数组 y，大于或等于 max(indx[i])。	输入
	在 zdotci 中，y 是双精度浮点类型数组。		
dotci	在 cdotci 中，dotci 是单精度浮点类型。	返回共轭 x 与 y 的点积。	输入/输出
	在 zdotci 中，dotci 是双精度浮点类型。		

2.4.1.4.3 返回值

函数执行状态，枚举类型 `sparse_status_t`。

2.4.1.4.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.1.4.5 示例

A. C interface:

```
INT nz = 2;

Complex8 dotci = {0, 0};

INT indx[2] = {1, 2};

Complex8 x[2] = {{1, 2}, {3, 4}};

Complex8 y[4] = {{-1, 1}, {5, 3}, {-2, -3}, {4, 1}};

sparse_status_t status = sparse_cdotci_sub(nz, x, indx, y, &dotci);
```

```

/*
 *   Output dotci:
 *
 *   {-7.000000, -8.000000}
 */

```

2.4.1.5 sparse_?dotui_sub

复数域的非共轭点积（欧几里得空间）： $x \cdot y = x^T \cdot y$ ，即对应元素相乘然后累加。

$res = x[0] * y[indx[0]] + x[1] * y[indx[1]] + \dots + x[nz - 1] * y[indx[nz - 1]]$ 。

2.4.1.5.1 接口定义

A. C interface:

`sparse_status_t sparse_cdotui_sub(const INT nz, const Complex8 *x, const INT *indx, const Complex8 *y, Complex8 *dotui);`

`sparse_status_t sparse_zdotui_sub(const INT nz, const Complex16 *x, const INT *indx, const Complex16 *y, Complex16 *dotui);`

B. Fortran interface:

`RES = SPARSE_CDOTUI_SUB(NZ, X, INDX, Y, DOTUI);`

`RES = SPARSE_ZDOTUI_SUB(NZ, X, INDX, Y, DOTUI);`

2.4.1.5.2 参数

参数名	类型	描述	输入/输出
<code>nz</code>	整型数	<code>x</code> 及 <code>indx</code> 数组中元素的个数。	输入
<code>x</code>	在 <code>cdotui</code> 中， <code>x</code> 是单精度浮点类型数组。 在 <code>zdotui</code> 中， <code>x</code> 是双精度浮点类型数组。	存储非零元素的数组 <code>x</code> ，大于或等于 <code>nz</code> 。	输入
<code>indx</code>	整型数组	<code>indx[i]</code> 表示 <code>x</code> 数组中第 <code>i</code> 个元素在稠密向量中的序列号，数组大于或等于 <code>nz</code> 。	输入

参数名	类型	描述	输入/输出
y	在 cdotui 中，y 是单精度浮点类型数组。	数组 y，大于或等于 max(indx[i])。	输入
	在 zdotui 中，y 是双精度浮点类型数组。		
dotui	在 cdotui 中，dotui 是单精度浮点类型。	x 与 y 的点积。	输入/输出
	在 zdotui 中，dotui 是双精度浮点类型。		

2.4.1.5.3 返回值

函数执行状态，枚举类型 `sparse_status_t`。

2.4.1.5.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.1.5.5 示例

A. C interface:

```
INT nz = 2;
Complex8 dotui = {0,0};
INT indx[2] = {1, 2};
Complex8 x[2] = {{1, 2}, {3, 4}};
Complex8 y[4] = {{-1, 1}, {5, 3}, {-2, -3}, {4, 1}};
sparse_status_t status = sparse_cdotui_sub(nz, x, indx, y, &dotui);

/*
 * Output dotui:
 *      {5.000000, -4.000000}
 */
```

2.4.1.6 sparse_?gthr

将 full-storage 格式的稀疏向量 y 中指定的元素加载到 compressed 格式的向量中。
即 $x[i]=y[indx[i]]$, $i=0,1,\dots,nz-1$ 。

2.4.1.6.1 接口定义

A. C interface:

```
sparse_status_t sparse_sgthr(const INT nz, const float *y, float *x, const INT *indx);
```

```
sparse_status_t sparse_dgthr(const INT nz, const double *y, double *x, const INT *indx);
```

```
sparse_status_t sparse_cgthr(const INT nz, const Complex8 *y, Complex16 *x, const INT *indx);
```

```
sparse_status_t sparse_zgthr(const INT nz, const Complex16 *y, Complex16 *x, const INT *indx);
```

B. Fortran interface:

```
RES = SPARSE_SGTHR(NZ, Y, X, INDX);
```

```
RES = SPARSE_DGTHR(NZ, Y, X, INDX);
```

```
RES = SPARSE_CGTHR(NZ, Y, X, INDX);
```

```
RES = SPARSE_ZGTHR(NZ, Y, X, INDX);
```

2.4.1.6.2 参数

参数名	类型	描述	输入/输出
nz	整型数	x 及 $indx$ 数组中元素的个数。	输入
y	在 sgthr 中, y 是单精度浮点类型数组。	数组 y , 大于或等于 $\max(indx[i])$ 。	输入
	在 dgthr 中, y 是双精度浮点类型数组。		
	在 cgthr 中, y 是单精度复数数组。		
	在 zgthr 中, y 是双精度复数数组。		

参数名	类型	描述	输入/输出
x	在 sgthr 中, x 是单精度浮点类型数组。	存储非零元素的数组 x, 大于或等于 nz。	输入/输出
	在 dgthr 中, x 是双精度浮点类型数组。		
	在 cgthr 中, x 是单精度复数数组。		
	在 zgthr 中, x 是双精度复数数组。		
indx	整型数组	indx[i]表示 x 数组中第 i 个元素在稠密向量中的序列号, 数组大小大于或等于 nz。	输入

2.4.1.6.3 返回值

函数执行状态, 枚举类型 `sparse_status_t`。

2.4.1.6.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.1.6.5 示例

A. C interface:

```

    INT nz = 2;

    INT indx[2] = {1,2};

    float x[2] = {0,0};

    float y[4] = {-1, 5, -2 ,3};

    sparse_status_t status = sparse_sgthr(nz,y,x,indx);

    /*
     *   Output x:
     *
     *      5.000000   -2.000000
  
```

*/

2.4.1.7 sparse_gthrz

将 full-storage 格式的稀疏向量 y 中指定的元素加载到 compressed 格式的向量中。即 $x[i]=y[indx[i]]$, $i=0,1,\dots,nz-1$, 并将 y 中对应元素清零。

2.4.1.7.1 接口定义

A. C interface:

```
sparse_status_t sparse_sgthrz(const INT nz, float *y, float *x, const INT *indx);
sparse_status_t sparse_dgthrz(const INT nz, double *y, double *x, const INT *indx);
sparse_status_t sparse_cgthrz(const INT nz, Complex8 *y, Complex8 *x, const INT *indx);
sparse_status_t sparse_zgthrz(const INT nz, Complex16 *y, Complex16 *x, const INT *indx);
```

B. Fortran interface:

```
RES = SPARSE_SGTHRZ(NZ, Y, X, INDX);
RES = SPARSE_DGTHRZ(NZ, Y, X, INDX);
RES = SPARSE_CGTHRZ(NZ, Y, X, INDX);
RES = SPARSE_ZGTHRZ(NZ, Y, X, INDX);
```

2.4.1.7.2 参数

参数名	类型	描述	输入/输出
nz	整型数	x 及 indx 数组中元素的个数。	输入
y	在 sgthrz 中, y 是单精度浮点类型数组。	数组 y, 大于或等于 max(indx[i])。	输入/输出
	在 dgthrz 中, y 是双精度浮点类型数组。		
	在 cgthrz 中, y 是单精度复数数组。		
	在 zgthrz 中, y 是双精度复数数组。		

参数名	类型	描述	输入/输出
x	在 sgthrz 中，x 是单精度浮点类型数组。	存储非零元素的数组 x，大于或等于 nz。	输入/输出
	在 dgthrz 中，x 是双精度浮点类型数组。		
	在 cgthrz 中，x 是单精度复数数组。		
	在 zgthrz 中，x 是双精度复数数组。		
indx	整型数组	indx[i]表示 x 数组中第 i 个元素在稠密向量中的序列号，数组大小大于或等于 nz。	输入

2.4.1.7.3 返回值

函数执行状态，枚举类型 sparse_status_t。

2.4.1.7.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.1.7.5 示例

A. C interface:

```
INT nz = 2;

INT indx[2] = {1, 2};

float x[2] = {0, 0};

float y[4] = {-1, 5, -2 ,3};

sparse_status_t status = sparse_sgthrz(nz, y, x, indx);

/*
 * Output x:
 *      5.000000      -2.000000
 * Output y:
```

```

*      -1.000000      0.000000      0.000000      3.000000
*/

```

2.4.1.8 sparse_roti

对 2 个实数类型的向量进行旋转操作。

$$x[i] = c * x[i] + s * y[indx[i]]$$

$$y[indx[i]] = c * y[indx[i]] - s * x[i]。$$

2.4.1.8.1 接口定义

A. C interface:

```

sparse_status_t sparse_sroti(const INT nz, float *x, const INT *indx, float *y, const
float c, const float s);

```

```

sparse_status_t sparse_droti(const INT nz, double *x, const INT *indx, double *y,
const double c, const double s);

```

B. Fortran interface:

```

RES = SPARSE_SROTI(NZ, X, INDX, Y, C, S);

```

```

RES = SPARSE_DROTI(NZ, X, INDX, Y, C, S);

```

2.4.1.8.2 参数

参数名	类型	描述	输入/输出
nz	整型数	x 及 indx 数组中元素的个数。	输入
x	在 sroti 中, x 是单精度浮点类型数组。	compressed 格式的向量 x, 大于或等于 nz。	输入/输出
	在 droti 中, x 是双精度浮点类型数组。		
indx	整型数组	存储要被加载到数组 x 中非零元素下标索引, 大于或等于 nz。	输入
y	在 sroti 中, y 是单精度浮点类型数组。	full-storage 格式的向量 y, 大于或等于 max(indx[i])。	输入/输出

参数名	类型	描述	输入/输出
	在 droti 中, y 是双精度浮点类型数组。		
c	在 sroti 中, c 是单精度浮点类型数组。	标量。	输入
	在 droti 中, c 是双精度浮点类型数组。		
s	在 sroti 中, z 是单精度浮点类型数组。	标量。	输入
	在 droti 中, z 是双精度浮点类型数组。		

2.4.1.8.3 返回值

函数执行状态, 枚举类型 `sparse_status_t`。

2.4.1.8.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.1.8.5 示例

A. C interface:

```

INT nz = 2;
INT indx[2] = {1, 2};
float x[2] = {1, 3};
float y[4] = {-1, 5, -2, 4};
float c = 1;
float s = 2;

sparse_status_t status = sparse_sroti(nz, x, indx, y, c, s);

/*
 * Output x:
 *      11.000000    -1.000000

```

```

*   Output y:

*       -1.000000      3.000000      -8.000000      4.000000

*/

```

2.4.1.9 sparse_?sctr

将 compressed 格式的向量写入 full-storage 格式的稀疏向量 y 指定位置。即 $y[\text{indx}[i]] = x[i]$, $i=0,1,\dots,nz-1$ 。

2.4.1.9.1 接口定义

A. C interface:

```
sparse_status_t sparse_ssctr(const INT nz, const float *x, const INT *indx, float *y);
```

```
sparse_status_t sparse_dsctr(const INT nz, const double *x, const INT *indx, double *y);
```

```
sparse_status_t sparse_csctr(const INT nz, const Complex8 *x, const INT *indx, Complex8 *y);
```

```
sparse_status_t sparse_zsctr(const INT nz, const Complex16 *x, const INT *indx, Complex16 *y);
```

B. Fortran interface:

```
RES = SPARSE_SSCTR(NZ, X, INDX, Y);
```

```
RES = SPARSE_DSCTR(NZ, X, INDX, Y);
```

```
RES = SPARSE_CSCTR(NZ, X, INDX, Y);
```

```
RES = SPARSE_ZSCTR(NZ, X, INDX, Y);
```

2.4.1.9.2 参数

参数名	类型	描述	输入/输出
nz	整型数	x 及 indx 数组中元素的个数。	输入
y	在 ssctr 中, y 是单精度浮点类型数组。 在 dsctr 中, y 是双精度浮点类型数组。	数组 y, 大于或等于 max(indx[i])。	输入/输出

参数名	类型	描述	输入/输出
	在 csctr 中，y 是单精度复数数组。		
	在 zsctr 中，y 是双精度复数数组。		
x	在 ssctr 中，x 是单精度浮点类型数组。	存储非零元素的数组 x，大于或等于 nz。	输入
	在 dsctr 中，x 是双精度浮点类型数组。		
	在 csctr 中，x 是单精度复数数组。		
	在 zsctr 中，x 是双精度复数数组。		
indx	整型数组	indx[i]表示 x 数组中第 i 个元素在稠密向量中的序列号，数组大小大于或等于 nz。	输入

2.4.1.9.3 返回值

函数执行状态，枚举类型 sparse_status_t。

2.4.1.9.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.1.9.5 示例

A. C interface:

```
INT nz = 2;

INT indx[2] = {1, 2};

float x[2] = {1, 1};

float y[4] = {-1, 5, -2 ,3};

sparse_status_t status = sparse_ssctr(nz, x, indx, y);
```

```
/*
 * Output sctr:
 *      -1.000000      1.000000      1.000000      3.000000
 */
```

2.4.2 Level 2 和 Level 3

2.4.2.1 函数说明

函数名	简介
sparse_?csrghmv	矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： $y = A * x$ 或 $y = A^T * x$ 或 $y = A^H * x$， 其中，x 和 y 为向量，A 是的稀疏矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 $indx$ 从 1 开始的矩阵。具体分为以下四个函数： sparse_scsrghmv: 单精度浮点类型矩阵与向量计算 sparse_dcsrghmv: 双精度浮点类型矩阵与向量计算 sparse_ccsrghmv: 单精度复数类型矩阵与向量计算 sparse_zcsrghmv: 双精度复数类型矩阵与向量计算
sparse_?csrsymv	矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： $y = A * x$ 其中，x 和 y 为向量，A 是的上三角或者下三角的对称矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 $indx$ 从 1 开始的矩阵。 具体分为以下四个函数： sparse_scsrsymv: 单精度浮点类型矩阵与向量计算 sparse_dcsrsymv: 双精度浮点类型矩阵与向量计算 sparse_ccsrsymv: 单精度复数类型矩阵与向量计算 sparse_zcsrsymv: 双精度复数类型矩阵与向量计算

函数名	简介
csparse_?csrgev	矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： $y = A * x \text{ 或 } y = A^T * x \text{ 或 } y = A^H * x,$ 其中，x 和 y 为向量，A 是的稀疏矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 indx 从 0 开始的矩阵。具体分为以下四个函数： csparse_scsrgev: 单精度浮点类型矩阵与向量计算 csparse_dcsrgev: 双精度浮点类型矩阵与向量计算 csparse_ccsrgev: 单精度复数类型矩阵与向量计算 csparse_zcsrgev: 双精度复数类型矩阵与向量计算
csparse_?csrsymv	矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： $y = A * x$ 其中，x 和 y 为向量，A 是的上三角或者下三角的对称矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 indx 从 0 开始的矩阵。具体分为以下四个函数： csparse_scsrcsymv: 单精度浮点类型矩阵与向量计算 csparse_dcsrcsymv: 双精度浮点类型矩阵与向量计算 csparse_ccsrcsymv: 单精度复数类型矩阵与向量计算 csparse_zcsrcsymv: 双精度复数类型矩阵与向量计算

函数名	简介
<code>sparse_?csrsv</code>	矩阵与向量乘积，矩阵是 CSR 格式的稀疏矩阵。 $y = \alpha * A * x + \beta * y \text{ 或 } y = \alpha * A^T * x + \beta * y \text{ 或 } y = \alpha * A^H * x + \beta * y$ 其中，α 和 β 是缩放系数，x 和 y 是向量，A 是 CSR 格式的稀疏向量。具体分为以下四个函数： <code>sparse_scsrsv</code>: 单精度浮点类型矩阵与向量乘积 <code>sparse_dcsrsv</code>: 双精度浮点类型矩阵与向量乘积 <code>sparse_ccsrsv</code>: 单精度复数类型矩阵与向量乘积 <code>sparse_zcsrsv</code>: 双精度复数类型矩阵与向量乘积
<code>sparse_?csrsv</code>	求解三角矩阵方程组计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： $A * y = \alpha * x \text{ 或 } A^T * y = \alpha * x \text{ 或 } A^H * y = \alpha * x$ 其中，x 和 y 为向量，A 是 $m * m$ 的稀疏矩阵，采用 CSR 格式存储。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。 具体分为以下四个函数： <code>sparse_scsrsv</code>: 单精度浮点类型求解三角矩阵方程组 <code>sparse_dcsrsv</code>: 双精度浮点类型求解三角矩阵方程组 <code>sparse_ccsrsv</code>: 单精度复数类型求解三角矩阵方程组 <code>sparse_zcsrsv</code>: 双精度复数类型求解三角矩阵方程组

函数名	简介
<code>sparse_?csrmm</code>	矩阵与矩阵计算，其中一个稀疏矩阵采用 CSR 格式存储，具体执行操作如下： ● $C = \alpha * A * B + \beta * C$ ● $C = \alpha * A^T * B + \beta * C$ ● $C = \alpha * A^H * B + \beta * C$ 其中，B 和 C 为稠密矩阵，A 是采用 CSR 格式存储的 <code>mxk</code> 的稀疏矩阵。 具体分为以下四个函数： <code>sparse_scsrmm</code>：单精度浮点类型矩阵与矩阵计算 <code>sparse_dcsrmm</code>：双精度浮点类型矩阵与矩阵计算 <code>sparse_ccsrmm</code>：单精度复数类型矩阵与矩阵计算 <code>sparse_zcsrmm</code>：双精度复数类型矩阵与矩阵计算
<code>sparse_?csradd</code>	计算 2 个 CSR 格式的稀疏矩阵之和。 $C = A + \beta * \text{op}(B)$ A、B、C 为 3 数组 CSR 格式稀疏矩阵，仅支持基 1 索引。 <code>op(B)</code>可能的操作为： ● $\text{op}(B) = B$ ● $\text{op}(B) = B^T$ ● $\text{op}(B) = B^H$ 具体分为以下四个函数： <code>sparse_scsradd</code>：单精度浮点类型稀疏矩阵之和 <code>sparse_dcsradd</code>：双精度浮点类型稀疏矩阵之和 <code>sparse_ccsradd</code>：单精度复数类型稀疏矩阵之和 <code>sparse_zcsradd</code>：双精度复数类型稀疏矩阵之和

函数名	简介
sparse_?csrmultcsr	矩阵与矩阵计算，其中 3 个稀疏矩阵采用 3-arrays CSR 格式存储，为基 1 索引，具体执行操作为：$C = \text{op}(A) * B$。 其中，A、B、C 是采用 CSR 格式存储的稀疏矩阵。 op(A)根据参数是以下 3 种情况之一： ● $\text{op}(A) = A$ ● $\text{op}(A) = A^T$ ● $\text{op}(A) = A^H$ 具体分为以下四个函数： sparse_scsrmultcsr: 单精度浮点类型矩阵与矩阵计算 sparse_dcscrmultcsr: 双精度浮点类型矩阵与矩阵计算 sparse_ccscrmultcsr: 单精度复数类型矩阵与矩阵计算 sparse_zcscrmultcsr: 双精度复数类型矩阵与矩阵计算
sparse_?csrmultd	矩阵与矩阵计算，其中 2 个稀疏矩阵采用 CSR 格式存储，为基 1 索引，具体执行操作为：$C := \text{op}(A) * B$。 其中，C 为稠密矩阵，A 和 B 是采用 CSR 格式存储的稀疏矩阵。 op(A)根据参数是以下 3 种情况之一： ● $\text{op}(A) = A$ ● $\text{op}(A) = A^T$ ● $\text{op}(A) = A^H$ 具体分为以下四个函数： sparse_scsrmultd: 单精度浮点类型矩阵与矩阵计算 sparse_dcscrmultd: 双精度浮点类型矩阵与矩阵计算 sparse_ccscrmultd: 单精度复数类型矩阵与矩阵计算 sparse_zcscrmultd: 双精度复数类型矩阵与矩阵计算

函数名	简介
<code>sparse_?csrtrsv</code>	求解稀疏三角线性方程组计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： ● $x = A * y$ ● $x = A^T * y$ ● $x = A^H * y$ A 为 3 数组 CSR 格式稀疏矩阵，x、y 为向量，仅支持 One-based indexing。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。 具体分为以下四个函数： <code>sparse_scsrtrsv</code>：单精度浮点类型求解稀疏三角线性方程组 <code>sparse_dcsrtrsv</code>：双精度浮点类型求解稀疏三角线性方程组 <code>sparse_ccsrtrsv</code>：单精度复数类型求解稀疏三角线性方程组 <code>sparse_zcsrtrsv</code>：双精度复数类型求解稀疏三角线性方程组
<code>csparse_?csrtrsv</code>	求解稀疏三角线性方程组计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下： ● $x = A * y$ ● $x = A^T * y$ ● $x = A^H * y$ A 为 3 数组 CSR 格式稀疏矩阵，x、y 为向量，仅支持 ze-based indexing。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。 具体分为以下四个函数： <code>csparse_scsrtrsv</code>：单精度浮点类型求解稀疏三角线性方程组 <code>csparse_dcsrtrsv</code>：双精度浮点类型求解稀疏三角线性方程组 <code>csparse_ccsrtrsv</code>：单精度复数类型求解稀疏三角线性方程组 <code>csparse_zcsrtrsv</code>：双精度复数类型求解稀疏三角线性方程组

函数名	简介
<code>sparse_?cscsv</code>	求解三角矩阵方程组计算，稀疏矩阵采用 CSC 格式存储，具体执行操作如下： ● $A * y = \alpha * x$ ● $A^T * y = \alpha * x$ ● $A^H * y = \alpha * x$ 其中，x 和 y 为向量，A 是 $m * m$ 的稀疏矩阵，采用 CSC 格式存储。矩阵 A 中的给定列的非零元素的存储顺序必须与列中显示的顺序相同（从上到下）。 具体分为以下四个函数： <code>sparse_scscsv</code>：单精度浮点类型求解三角矩阵方程组 <code>sparse_dcscsv</code>：双精度浮点类型求解三角矩阵方程组 <code>sparse_ccscsv</code>：单精度复数类型求解三角矩阵方程组 <code>sparse_zcscsv</code>：双精度复数类型求解三角矩阵方程组
<code>sparse_?cscmv</code>	矩阵与向量乘积，矩阵是 CSC 格式的稀疏矩阵。 ● $y = \alpha * A * x + \beta * y$ ● $y = \alpha * A^T * x + \beta * y$ ● $y = \alpha * A^H * x + \beta * y$ 其中，α 和 β 是缩放系数，x 和 y 是向量，A 是 CSC 格式的稀疏向量。具体分为以下四个函数： <code>sparse_scscmv</code>：单精度浮点类型矩阵与向量乘积 <code>sparse_dcscmv</code>：双精度浮点类型矩阵与向量乘积 <code>sparse_ccscmv</code>：单精度复数类型矩阵与向量乘积 <code>sparse_zcscmv</code>：双精度复数类型矩阵与向量乘积

函数名	简介
<code>sparse_?cscmm</code>	矩阵与矩阵计算，其中一个稀疏矩阵采用 CSC 格式存储。具体执行操作如下： $C = \alpha * A * B + \beta * C$ $C = \alpha * A^T * B + \beta * C$ $C = \alpha * A^H * B + \beta * C$ 其中，B 和 C 为稠密矩阵，A 是采用 CSC 格式存储的 <code>mxk</code> 的稀疏矩阵。具体分为以下四个函数： <code>sparse_scscmm</code>：单精度浮点类型矩阵与矩阵计算 <code>sparse_dcscmm</code>：双精度浮点类型矩阵与矩阵计算 <code>sparse_ccscmm</code>：单精度复数类型矩阵与矩阵计算 <code>sparse_zcscmm</code>：双精度复数类型矩阵与矩阵计算
<code>sparse_?cscsm</code>	求解三角矩阵方程组计算，其中一个稀疏矩阵采用 CSC 格式存储，具体执行操作如下： ● $A * y = \alpha * x$ ● $A^T * y = \alpha * x$ ● $A^H * y = \alpha * x$ 其中，x 和 y 为 <code>m*n</code> 的稠密矩阵，A 是采用 CSC 格式存储的 <code>m*m</code> 的稀疏矩阵。矩阵 A 中的给定列的非零元素的存储顺序必须与列中显示的顺序相同（从上到下）。 具体分为以下四个函数： <code>sparse_scscsm</code>：单精度浮点类型求解三角矩阵方程组 <code>sparse_dcscsm</code>：双精度浮点类型求解三角矩阵方程组 <code>sparse_ccscsm</code>：单精度复数类型求解三角矩阵方程组 <code>sparse_zcscsm</code>：双精度复数类型求解三角矩阵方程组

函数名	简介
<code>sparse_?csrsm</code>	求解三角矩阵方程组计算，其中一个稀疏矩阵采用 CSR 格式存储，具体执行操作如下： ● $A * y = \alpha * x$ ● $A^T * y = \alpha * x$ ● $A^H * y = \alpha * x$ 其中，x 和 y 为 $m \times n$ 的稠密矩阵，A 是采用 CSR 格式存储的 $m \times m$ 的稀疏矩阵。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。 具体分为以下四个函数： <code>sparse_scsrsm</code>：单精度浮点类型求解三角矩阵方程组 <code>sparse_dcsrsm</code>：双精度浮点类型求解三角矩阵方程组 <code>sparse_ccsrsm</code>：单精度复数类型求解三角矩阵方程组 <code>sparse_zcsrsm</code>：双精度复数类型求解三角矩阵方程组

2.4.2.2 `sparse_?csrgemv`

矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下：

$y = A * x$ 或 $y = A^T * x$ 或 $y = A^H * x$ ，

其中， x 和 y 为向量， A 是的稀疏矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 `indx` 从 1 开始的矩阵。

2.4.2.2.1 接口定义

A. C interface:

```
sparse_status_t sparse_scsrgemv(const sparse_operation_t opt, const INT m, const float *a, const INT *ia, const INT *ja, const float *x, float *y);
```

```
sparse_status_t sparse_dcsrgemv(const sparse_operation_t opt, const INT m, const double *a, const INT *ia, const INT *ja, const double *x, double *y);
```

```
sparse_status_t sparse_ccsrgemv(const sparse_operation_t opt, const INT m, const Complex8 *a, const INT *ia, const INT *ja, const Complex8 *x, Complex8 *y);
```

```
sparse_status_t sparse_zcsrgemv(const sparse_operation_t opt, const INT m, const
Complex16 *a ,const INT *ia, const INT *ja, const Complex16 *x, Complex16 *y);
```

B. Fortran interface:

```
RES = SPARSE_SCSRGEMV(OPT, M, A, IA, JA, X, Y);
```

```
RES = SPARSE_DCSRGEMV(OPT, M, A, IA, JA, X, Y);
```

```
RES = SPARSE_CCSRGEMV(OPT, M, A, IA, JA, X, Y);
```

```
RES = SPARSE_ZCSRGEMV(OPT, M, A, IA, JA, X, Y);
```

2.4.2.2.2 参数

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作， 如果 opt=SPARSE_OPERATION_NO N_TRANSPOSE，则 $y = A * x$ 。 如果 opt=SPARSE_OPERATION_TR ANSPOSE，则 $y = A^T * x$ 。 如果 opt=SPARSE_OPERATION_CO NJUGATE_TRANSPOSE，则 $y = A^H * x$ 。	输入
m	整型数	表示 A 矩阵的行数和列数，取 值范围为[1, MAX_INT]。	输入
a	在 scsrgemv 中，a 是 单精度浮点数组。 在 dcsrgemv 中，a 是 双精度浮点数组。 在 ccsrgemv 中，a 是 单精度复数数组。 在 zcsrgemv 中，a 是 双精度复数数组。	A 矩阵中的 values 数组，存储 非零元素。	输入

参数名	类型	描述	输入/输出
ia	整型数组	长度为 m+1 的向量。 ia[i]-1 表示 A 矩阵第 i 行第一个非零元素在 val 数组中的索引。	输入
ja	整型数组。	ja[i]表示 val 数组中第 i 个元素在矩阵 A 中的列序号。	输入
x	在 scsrgemv 中，x 是单精度浮点数组。	向量 x 的数组。	输入
	在 dcsrgemv 中，x 是双精度浮点数组。		
	在 ccsrgemv 中，x 是单精度复数数组。		
	在 zcsrgemv 中，x 是双精度复数数组。		
y	在 scsrgemv 中，y 是单精度浮点数组。	向量 y 的数组，更新后输出。	输入/输出
	在 dcsrgemv 中，y 是双精度浮点数组。		
	在 ccsrgemv 中，y 是单精度复数数组。		
	在 zcsrgemv 中，y 是双精度复数数组。		
注意： 函数内部不对参数做完整校验，接口调用者需保证 ia 和 ja 内部元素不超出矩阵最大索引值。			

2.4.2.2.3 返回值

函数执行状态，枚举类型 sparse_status_t。

2.4.2.2.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.2.5 示例

A. C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;

INT m = 4;

float a[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};

INT ja[9] = {1, 2, 4, 3, 4, 1, 3, 4, 1};

INT ia[5] = {1, 4, 6, 9, 10};

float x[4] = {1, 3, -2, 5};

float y[4] = {-1, 1, 5, 3};

sparse_status_t status = sparse_scsrgemv(opt, m, a, ia, ja, x, y);

/*
 * Output Y:
 *      28.000000   -32.000000   41.000000    9.000000
 *
 */
```

2.4.2.3 sparse_?csrsymv

矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下：

$$y = A * x$$

其中，x 和 y 为向量，A 是的上三角或者下三角的对称矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 indx 从 1 开始的矩阵。

2.4.2.3.1 接口定义

A. C interface:

```
sparse_status_t sparse_scsrsymv(const sparse_fill_mode_t uplo, const INT m, const
float *a, const INT *ia, const INT *ja, const float *x, float *y);
```

```
sparse_status_t sparse_dcsrsymv(const sparse_fill_mode_t uplo, const INT m, const
double *a, const INT *ia, const INT *ja, const double *x, double *y);
```

```
sparse_status_t sparse_ccsrsymv(const sparse_fill_mode_t uplo, const INT m, const
Complex8 *a, const INT *ia, const INT *ja, const Complex8 *x, Complex8 *y);
```

```
sparse_status_t sparse_zcsrsymv(const sparse_fill_mode_t uplo, const INT m, const
Complex16 *a, const INT *ia, const INT *ja, const Complex16 *x, Complex16 *y);
```

B. Fortran interface:

```
RES = SPARSE_SCSRSYMV(UPLO, M, A, IA, JA, X, Y);
```

```
RES = SPARSE_DCSRSYMV(UPLO, M, A, IA, JA, X, Y);
```

```
RES = SPARSE_CCSRSYMV(UPLO, M, A, IA, JA, X, Y);
```

```
RES = SPARSE_ZCSRSYMV(UPLO, M, A, IA, JA, X, Y);
```

2.4.2.3.2 参数

参数名	类型	描述	输入/输出
uplo	枚举类型 sparse_fill_mode_t uplo	指示矩阵 A 的上三角或者下三角被使用： U: 上三角 L: 下三角	输入
m	整型数	表示 A 矩阵的行数和列数，取值范围为 [1, MAX_INT]。	输入
a	在 scsrsymv 中，val 是单精度浮点数组。 在 dcsrsymv 中，val 是双精度浮点数组。 在 ccsrsymv 中，val 是单精度复数数组。 在 zcsrsymv 中，val 是双精度复数数组。	A 矩阵中的非零元素。	输入
ia	整型数组	长度为 m+1 的向量。 ia[i]-1 表示 A 矩阵第 i 行第一个非零元素在 val 数组中的索引。	输入
ja	整型数组。	ja[i]表示 val 数组中第 i 个元素在矩阵 A 中的列序号。	输入

参数名	类型	描述	输入/输出
x	在 <code>scsrsvmv</code> 中, x 是单精度浮点数组。	向量 x 的数组。	输入
	在 <code>dcsrsvmv</code> 中, x 是双精度浮点数组。		
	在 <code>ccsrsvmv</code> 中, x 是单精度复数数组。		
	在 <code>zcsrsvmv</code> 中, x 是双精度复数数组。		
y	在 <code>scsrsvmv</code> 中, y 是单精度浮点数组。	向量 y 的数组, 更新后输出。	输入/输出
	在 <code>dcsrsvmv</code> 中, y 是双精度浮点数组。		
	在 <code>ccsrsvmv</code> 中, y 是单精度复数数组。		
	在 <code>zcsrsvmv</code> 中, y 是双精度复数数组。		
注意: 函数内部不对参数做完整校验, 接口调用者需保证 <code>ia</code> 和 <code>ja</code> 内部元素不超出矩阵最大索引值。			

2.4.2.3.3 返回值

函数执行状态, 枚举类型 `sparse_status_t`。

2.4.2.3.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.3.5 示例

A. C interface:

```
sparse_fill_mode_t uplo = SPARSE_FILL_MODE_LOWER;

INT m = 4;

float a[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
```

```

INT ja[9] = {1, 2, 4, 3, 4, 1, 3, 4, 1};
INT ia[5] = {1, 4, 6, 9, 10};
float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};

sparse_status_t status = sparse_scsrsymv(uplo, m, a, ia, ja, x, y);

/*
 * Output Y:
 *      31.000000      0.000000      16.000000      9.000000
 *
 */

```

2.4.2.4 csparse_?csrgev

矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下：

$$y = A * x \text{ or } y = A^T * x \text{ or } y = A^H * x$$

其中，x 和 y 为向量，A 是稀疏矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 indx 从 0 开始的矩阵。

2.4.2.4.1 接口定义

A. C interface:

```
sparse_status_t csparse_scsrgemv(const sparse_operation_t opt, const INT m, const
float *a, const INT *ia, const INT *ja, const float *x, float *y);
```

```
sparse_status_t csparse_dcsrgemv(const sparse_operation_t opt, const INT m, const
double *a, const INT *ia, const INT *ja, const double *x, double *y);
```

```
sparse_status_t csparse_ccsrgemv(const sparse_operation_t opt, const INT m, const
Complex8 *a, const INT *ia, const INT *ja, const Complex8 *x, Complex8 *y);
```

```
sparse_status_t csparse_zcsrgemv(const sparse_operation_t opt, const INT m, const
Complex16 *a, const INT *ia, const INT *ja, const Complex16 *x, Complex16 *y);
```

B. Fortran interface:

```
RES = CSPARSE_SCSRGEV(OPT, M, A, IA, JA, X, Y);
```

```
RES = CSPARSE_DCSRGEV(OPT, M, A, IA, JA, X, Y);
```

```
RES = CSPARSE_CCSRGEV(OPT, M, A, IA, JA, X, Y);
```

RES = CSPARSE_ZCSRGMV(OPT, M, A, IA, JA, X, Y);

2.4.2.4.2 参数

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作， 如果 opt=SPARSE_OPERATION_NO N_TRANSPOSE，则 $y = A * x$ 。 如果 opt=SPARSE_OPERATION_TR ANSPOSE，则 $y = A^T * x$ 。 如果 opt=SPARSE_OPERATION_CO NJUGATE_TRANSPOSE，则 $y = A^H * x$ 。	输入
m	整型数	表示 A 矩阵的行数和列数，取 值范围为 [1, MAX_INT]。	输入
a	在 c_scsrgemv 中，val 是单精度浮点数组。 在 c_dcsrgemv 中，val 是双精度浮点数组。 在 c_ccsrgemv 中，val 是单精度复数数组。 在 c_zcsrgemv 中，val 是双精度复数数组。	A 矩阵中的非零元素。	输入
ia	整型数组	长度为 m+1 的向量。 ia[i]-1 表示 A 矩阵第 i 行第一个 非零元素在 val 数组中的索引。	输入
ja	整型数组。	ja[i]表示 val 数组中第 i 个元素 在矩阵 A 中的列序号。	输入
x	在 c_scsrgemv 中，x 是单精度浮点数组。	向量 x。	输入

参数名	类型	描述	输入/输出
	在 <code>c_dcsrgemv</code> 中， <code>x</code> 是双精度浮点数组。		
	在 <code>c_ccsrgemv</code> 中， <code>x</code> 是单精度复数数组。		
	在 <code>c_zcsrgemv</code> 中， <code>x</code> 是双精度复数数组。		
y	在 <code>c_scsrgemv</code> 中， <code>y</code> 是单精度浮点数组。	向量 y，更新后输出。	输入/输出
	在 <code>c_dcsrgemv</code> 中， <code>y</code> 是双精度浮点数组。		
	在 <code>c_ccsrgemv</code> 中， <code>y</code> 是单精度复数数组。		
	在 <code>c_zcsrgemv</code> 中， <code>y</code> 是双精度复数数组。		

注意:

函数内部不对参数做完整校验，接口调用者需保证 ia 和 ja 内部元素不超出矩阵最大索引值。

2.4.2.4.3 返回值

函数执行状态，枚举类型 `sparse_status_t`。

2.4.2.4.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.4.5 示例

A. C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;

INT m = 4;

float a[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};

INT ja[9] = {0, 1, 3, 2, 3, 0, 2, 3, 0};

INT ia[5] = {0, 3, 5, 8, 9};
```

```

float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};

sparse_status_t status = csparse_scsrgemv(opt, m, a, ia, ja, x, y);

/*
 * Output Y:
 *      28.000000   -32.000000   41.000000   9.000000
 *
 */

```

2.4.2.5 csparse_?csrsymv

矩阵与向量计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下：

$$y = A * x$$

其中，x 和 y 为向量，A 是上三角或者下三角的对称矩阵，采用 CSR 格式 3 数组存储。仅支持矩阵 indx 从 0 开始的矩阵。

2.4.2.5.1 接口定义

A. C interface:

```
sparse_status_t csparse_scsrsymv(const sparse_fill_mode_t uplo, const INT m, const float *a, const INT *ia, const INT *ja, const float *x, float *y);
```

```
sparse_status_t csparse_dcsrsymv(const sparse_fill_mode_t uplo, const INT m, const double *a, const INT *ia, const INT *ja, const double *x, double *y);
```

```
sparse_status_t csparse_ccsrsymv(const sparse_fill_mode_t uplo, const INT m, const Complex8 *a, const INT *ia, const INT *ja, const Complex8 *x, Complex8 *y);
```

```
sparse_status_t csparse_zcsrsymv(const sparse_fill_mode_t uplo, const INT m, const Complex16 *a, const INT *ia, const INT *ja, const Complex16 *x, Complex16 *y);
```

B. Fortran interface:

```
RES = CSPARSE_SCRSYMV(UPLO, M, A, IA, JA, X, Y);
```

```
RES = CSPARSE_DCSYMV(UPLO, M, A, IA, JA, X, Y);
```

```
RES = CSPARSE_CCYSMV(UPLO, M, A, IA, JA, X, Y);
```

```
RES = CSPARSE_ZCSYMV(UPLO, M, A, IA, JA, X, Y);
```

2.4.2.5.2 参数

参数名	类型	描述	输入/输出
uplo	枚举类型 sparse_fill_mode_t uplo	指示矩阵 A 的上三角或者下三角被使用： U: 上三角 L: 下三角	输入
m	整型数	表示 A 矩阵的行数和列数，取值范围为 [1, MAX_INT]。	输入
a	在 c_scsrsymv 中，val 是单精度浮点数组。 在 c_dcsrsymv 中，val 是双精度浮点数组。 在 c_ccsrsymv 中，val 是单精度复数数组。 在 c_zcsrsymv 中，val 是双精度复数数组。	A 矩阵中的非零元素。	输入
ia	整型数组	长度为 m+1 的向量。 ia[i]-1 表示 A 矩阵第 i 行第一个非零元素在 val 数组中的索引。	输入
ja	整型数组。	ja[i]表示 val 数组中第 i 个元素在矩阵 A 中的列序号。	输入
x	在 c_scsrsymv 中，x 是单精度浮点数组。 在 c_dcsrsymv 中，x 是双精度浮点数组。 在 c_ccsrsymv 中，x 是单精度复数数组。 在 c_zcsrsymv 中，x 是双精度复数数组。	向量 x。	输入
y	在 c_scsrsymv 中，y 是单精度浮点数组。	向量 y，更新后输出。	输入/输出

参数名	类型	描述	输入/输出
	在 <code>c_dcsrsvmv</code> 中， <code>y</code> 是双精度浮点数组。		
	在 <code>c_ccsrsvmv</code> 中， <code>y</code> 是单精度复数数组。		
	在 <code>c_zcsrsvmv</code> 中， <code>y</code> 是双精度复数数组。		
注意： 函数内部不对参数做完整校验，接口调用者需保证 <code>ia</code> 和 <code>ja</code> 内部元素不超出矩阵最大索引值。			

2.4.2.5.3 返回值

函数执行状态，枚举类型 `sparse_status_t`。

2.4.2.5.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.5.5 示例

A. C interface:

```
sparse_fill_mode_t uplo = SPARSE_FILL_MODE_LOWER;
INT m = 4;
float a[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
INT ja[9] = {0, 1, 3, 2, 3, 0, 2, 3, 0};
INT ia[5] = {0, 3, 5, 8, 9};
float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};
sparse_status_t status = csparse_ccsrsvmv(uplo, m, a, ia, ja, x, y);

/*
 * Output Y:
 *
 *      31.000000      0.000000      16.000000      9.000000
```

```
*
*/
```

2.4.2.6 sparse_csr_mv

矩阵与向量计算，矩阵采用 CSR 格式存储，具体执行操作如下：

$$y = \alpha * A * x + \beta y \text{ or } y = \alpha * A^T * x + \beta y \text{ or } y = \alpha * A^H * x + \beta y$$

其中，alpha 和 beta 是缩放系数，x 和 y 是向量，A 是 CSR 格式的稀疏向量。

2.4.2.6.1 接口定义

A. C interface:

```
sparse_status_t sparse_scsr_mv(const sparse_operation_t opt, const INT m, const INT
k, const float alpha, const char *matdescra, const float *val, const INT *indx, const
INT *pntrb, const INT *pntre, const float *x, const float beta, float *y);
```

```
sparse_status_t sparse_dcsr_mv(const sparse_operation_t opt, const INT m, const INT
k, const double alpha, const char *matdescra, const double *val, const INT *indx,
const INT *pntrb, const INT *pntre, const double *x, const double beta, double *y);
```

```
sparse_status_t sparse_ccsr_mv(const sparse_operation_t opt, const INT m, const INT
k, const Complex8 alpha, const char *matdescra, const Complex8 *val, const INT
*indx, const INT *pntrb, const INT *pntre, const Complex8 *x, const Complex8 beta,
Complex8 *y);
```

```
sparse_status_t sparse_zcsr_mv(const sparse_operation_t opt, const INT m, const INT
k, const Complex16 alpha, const char *matdescra, const Complex16 *val, const INT
*indx, const INT *pntrb, const INT *pntre, const Complex16 *x, const Complex16
beta, Complex16 *y);
```

B. Fortran interface:

```
RES = SPARSE_SCSR_MV(OPT, M, K, ALPHA, MATDESCRA, VAL, INDX,
PNTRB, PNTRE, X, BETA, Y);
```

```
RES = SPARSE_DCSR_MV(OPT, M, K, ALPHA, MATDESCRA, VAL, INDX,
PNTRB, PNTRE, X, BETA, Y);
```

```
RES = SPARSE_CCSP_MV(OPT, M, K, ALPHA, MATDESCRA, VAL, INDX,
PNTRB, PNTRE, X, BETA, Y);
```

```
RES = SPARSE_ZCSR_MV(OPT, M, K, ALPHA, MATDESCRA, VAL, INDX,
PNTRB, PNTRE, X, BETA, Y);
```


2.4.2.6.2 参数

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作。 如果 opt=SPARSE_OPERATION_NO N_TRANSPOSE, 则 $y = \alpha * A * x + \beta * y$ 。 如果 opt=SPARSE_OPERATION_TR ANSPOSE, 则 $y = \alpha * A^T * x + \beta * y$ 。 如果 opt=SPARSE_OPERATION_CO NJUGATE_TRANSPOSE, 则 $y = \alpha * A^H * x + \beta * y$ 。	输入
m	整型数	表示 A 矩阵的行数, 取值范围 为[1, MAX_INT]。	输入
k	整型数	表示 A 矩阵的列数, 取值范围 为[1, MAX_INT]。	输入
alpha	在 scsrnv 中, alpha 是 单精度浮点类型。 在 dcsrmv 中, alpha 是双精度浮点类型。 在 ccsrmv 中, alpha 是 单精度复数。 在 zcsrmv 中, alpha 是 双精度复数。	表示系数。	输入
matdescra	char 指针	矩阵的操作属性, 具体参考 matdescra 说明。	输入
val	在 scsrnv 中, val 是单 精度浮点数组。 在 dcsrmv 中, val 是 双精度浮点数组。	A 矩阵中的非零元素。	输入

参数名	类型	描述	输入/输出
	在 <code>ccsrmv</code> 中, <code>val</code> 是单精度复数数组。		
	在 <code>zcsrmv</code> 中, <code>val</code> 是双精度复数数组。		
<code>indx</code>	整型数组	<code>indx[i]</code> 表示 <code>val</code> 数组中第 i 个元素在矩阵 <code>A</code> 中的列序号。	输入
<code>pntrb</code>	整型数组	长度为 m 的数组, 包含矩阵 <code>A</code> 的行索引, <code>pntrb[i] - pntrb[0]</code> 表示第 i 行第一个非零元素在 <code>val</code> 和 <code>indx</code> 数组内的下标。	输入
<code>pntre</code>	整型数组	长度为 m 的数组, 包含矩阵 <code>A</code> 的行索引, <code>pntre[i] - pntrb[0]-1</code> 表示第 i 行最后一个非零元素在 <code>val</code> 和 <code>indx</code> 数组内的下标。	输入
<code>x</code>	在 <code>scsrmv</code> 中, <code>x</code> 是单精度浮点数组。	向量 <code>x</code> 。	输入
	在 <code>dcscrmv</code> 中, <code>x</code> 是双精度浮点数组。		
	在 <code>ccsrmv</code> 中, <code>x</code> 是单精度复数数组。		
	在 <code>zcsrmv</code> 中, <code>x</code> 是双精度复数数组。		
<code>beta</code>	在 <code>scsrmv</code> 中, <code>beta</code> 是单精度浮点类型。	表示系数。	输入
	在 <code>dcscrmv</code> 中, <code>beta</code> 是双精度浮点类型。		
	在 <code>ccsrmv</code> 中, <code>beta</code> 是单精度复数。		
	在 <code>zcsrmv</code> 中, <code>beta</code> 是双精度复数。		
<code>y</code>	在 <code>scsrmv</code> 中, <code>y</code> 是单精度浮点数组。	向量 <code>y</code> , 更新后输出。	输入/输出
	在 <code>dcscrmv</code> 中, <code>y</code> 是双精度浮点数组。		

参数名	类型	描述	输入/输出
	在 <code>ccsrmv</code> 中, <code>y</code> 是单精度复数数组。		
	在 <code>zcsrmv</code> 中, <code>y</code> 是双精度复数数组。		
注意: 函数内部不对参数做完整校验, 接口调用者需保证 <code>pntrb</code> 和 <code>pntre</code> 内部元素不超出矩阵最大索引值。			

2.4.2.6.3 返回值

函数执行状态, 枚举类型 `sparse_status_t`。

2.4.2.6.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.6.5 示例

A. C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;

INT m = 4;
INT k = 4;
float alpha = 0.5;
float beta = 1.2;
char *matdescra = "G00F"; //一般矩阵, 基 1 索引
float val[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
INT indx[9] = {1, 2, 4, 3, 4, 1, 3, 4, 1};
INT pntrb[4] = {1, 4, 6, 9};
INT pntre[4] = {4, 6, 9, 10};
float x[4] = {1, 3, -2, 5};
float y[4] = {-1, 1, 5, 3};

sparse_status_t status = sparse_scsrmv(opt, m, k, alpha, matdescra,
val, indx, pntrb, pntre, x, beta, y);
```

```

/*
 *   Output Y:
 *       12.80  -14.80  26.50   8.10
 *
 */

```

2.4.2.7 sparse_?csrsv

求解三角矩阵方程组计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下：

$$A * y = \alpha * x \text{ or } A^T * y = \alpha * x \text{ or } A^H * y = \alpha * x$$

其中，x 和 y 为向量，A 是 $m * m$ 的稀疏矩阵，采用 CSR 格式存储。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。

2.4.2.7.1 接口定义

A. C interface:

sparse_status_t sparse_scsrsv(const sparse_operation_t opt, const INT m, const float alpha, const char *matdescra, const float *val, const INT *indx, const INT *pntrb, const INT *pntre, const float *x, float *y);

sparse_status_t sparse_dcsrsv(const sparse_operation_t opt, const INT m, const double alpha, const char *matdescra, const double *val, const INT *indx, const INT *pntrb, const INT *pntre, const double *x, double *y);

sparse_status_t sparse_ccsrsv(const sparse_operation_t opt, const INT m, const Complex8 alpha, const char *matdescra, const Complex8 *val, const INT *indx, const INT *pntrb, const INT *pntre, const Complex8 *x, Complex8 *y);

sparse_status_t sparse_zcsrsv(const sparse_operation_t opt, const INT m, const Complex16 alpha, const char *matdescra, const Complex16 *val, const INT *indx, const INT *pntrb, const INT *pntre, const Complex16 *x, Complex16 *y);

B. Fortran interface:

RES = SPARSE_SCSRVS(OPT, M, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, Y);

RES = SPARSE_DCSRVS(OPT, M, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, Y);

RES = SPARSE_CCRRVS(OPT, M, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, Y);

RES = SPARSE_ZCSRVS(OPT, M, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, X, Y);

2.4.2.7.2 参数

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作。 如果 opt=SPARSE_OPERATION_NO N_TRANSPOSE, 则 $A * y = \alpha * x$ 。 如果 opt=SPARSE_OPERATION_TR ANSPOSE, 则 $A^T * y = \alpha * x$ 。 如果 opt=SPARSE_OPERATION_CO NJUGATE_TRANSPOSE, 则 $A^H * y = \alpha * x$ 。	输入
m	整型数	表示 A 矩阵的行数, 取值范围 为[1, MAX_INT]。	输入
alpha	在 scsrsv 中, alpha 是 单精度浮点类型。 在 dcsrsv 中, alpha 是 双精度浮点类型。 在 ccsrsv 中, alpha 是 单精度复数。 在 zcsrsv 中, alpha 是 双精度复数。	表示系数。	输入
matdescra	char 指针	矩阵的操作属性, 具体参考 matdescra 说明。	输入
val	在 scsrsv 中, val 是单 精度浮点数组。 在 dcsrsv 中, val 是双 精度浮点数组。 在 ccsrsv 中, val 是单 精度复数数组。	A 矩阵中的非零元素。	输入

参数名	类型	描述	输入/输出
	在 <code>zcscsv</code> 中, <code>val</code> 是双精度复数数组。		
<code>indx</code>	整型数组	<code>indx[i]</code> 表示 <code>val</code> 数组中第 <code>i</code> 个元素在矩阵 <code>A</code> 中的列序号。	输入
<code>pntrb</code>	整型数组	长度为 <code>m</code> 的数组, 包含矩阵 <code>A</code> 的行索引, <code>pntrb[i] - pntrb[0]</code> 表示第 <code>i</code> 行第一个非零元素在 <code>val</code> 和 <code>indx</code> 数组内的下标。	输入
<code>pntrc</code>	整型数组	长度为 <code>m</code> 的数组, 包含矩阵 <code>A</code> 的行索引, <code>pntrc[i] - pntrb[0]-1</code> 表示第 <code>i</code> 行最后一个非零元素在 <code>val</code> 和 <code>indx</code> 数组内的下标。	输入
<code>x</code>	在 <code>scscsv</code> 中, <code>x</code> 是单精度浮点数组。	向量 <code>x</code> 。	输入
	在 <code>dcscsv</code> 中, <code>x</code> 是双精度浮点数组。		
	在 <code>ccscsv</code> 中, <code>x</code> 是单精度复数数组。		
	在 <code>zcscsv</code> 中, <code>x</code> 是双精度复数数组。		
<code>y</code>	在 <code>scscsv</code> 中, <code>y</code> 是单精度浮点数组。	向量 <code>y</code> , 更新后输出。	输入/输出
	在 <code>dcscsv</code> 中, <code>y</code> 是双精度浮点数组。		
	在 <code>ccscsv</code> 中, <code>y</code> 是单精度复数数组。		
	在 <code>zcscsv</code> 中, <code>y</code> 是双精度复数数组。		
注意: 函数内部不对参数做完整校验, 接口调用者需保证 <code>pntrb</code> 和 <code>pntrc</code> 内部元素不超出矩阵最大索引值。			

2.4.2.7.3 返回值

函数执行状态, 枚举类型 `sparse_status_t`。

2.4.2.7.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.7.5 示例

A. C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;

INT m = 4;

float alpha = 1.0;

char *matdescra = "TLNF"; //一般矩阵，基 1 索引

float val[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};

INT indx[9] = {1, 2, 4, 2, 4, 1, 3, 1, 4};

INT pntreb[4] = {1, 4, 6, 8};

INT pntre[4] = {4, 6, 8, 10};

float x[4] = {1, 3, -2, 5};

float y[4] = {0, 0, 0, 0};

sparse_status_t status = sparse_scsrsv(opt, m, alpha, matdescra,
val, indx, pntreb, pntre, x, y);

/*
 * Output Y:
 *      0.500000   3.000000   1.500000   0.277778
 *
 */
```

2.4.2.8 sparse_?csrmm

矩阵与矩阵计算，其中一个稀疏矩阵采用 HXW 格式存储，具体执行操作如下：

$$C = \alpha * A * B + \beta * C \text{ or } C = \alpha * A^T * x + \beta * C \text{ or } C = \alpha * A^H * x + \beta * C$$

其中，B 和 C 为稠密矩阵，A 是采用 CSR 格式存储的 mxk 的稀疏矩阵。

layout 与 sparse matrix indexing 的关系如下表所示。

Sparse matrix indexing	Dense matrix layout
------------------------	---------------------

Sparse matrix indexing	Dense matrix layout
SPARSE_INDEX_BASE_ZERO	SPARSE_LAYOUT_ROW_MAJOR
SPARSE_INDEX_BASE_ONE	SPARSE_LAYOUT_COLUMN_MAJOR

2.4.2.8.1 接口定义

A. C interface:

`sparse_status_t sparse_scsrmm(const sparse_operation_t opt, const INT m, const INT n, const INT k, const float alpha, const char *matdescra, const float *val, const INT *indx, const INT *pntrb, const INT *pntre, const float *b, const INT ldb, const float beta, float *c, const INT ldc);`

`sparse_status_t sparse_dcsrmm(const sparse_operation_t opt, const INT m, const INT n, const INT k, const double alpha, const char *matdescra, const double *val, const INT *indx, const INT *pntrb, const INT *pntre, const double *b, const INT ldb, const double beta, double *c, const INT ldc);`

`sparse_status_t sparse_ccsrmm(const sparse_operation_t opt, const INT m, const INT n, const INT k, const Complex8 alpha, const char *matdescra, const Complex8 *val, const INT *indx, const INT *pntrb, const INT *pntre, const Complex8 *b, const INT ldb, const Complex8 beta, Complex8 *c, const INT ldc);`

`sparse_status_t sparse_zcsrmm(const sparse_operation_t opt, const INT m, const INT n, const INT k, const Complex16 alpha, const char *matdescra, const Complex16 *val, const INT *indx, const INT *pntrb, const INT *pntre, const Complex16 *b, const INT ldb, const Complex16 beta, Complex16 *c, const INT ldc);`

B. Fortran interface:

`RES = SPARSE_SCSRMM(OPT, M, N K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, B, LDB, BETA, C, LDC);`

`RES = SPARSE_DCSRMM(OPT, M, N K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, B, LDB, BETA, C, LDC);`

`RES = SPARSE_CCSRMM(OPT, M, N K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, B, LDB, BETA, C, LDC);`

`RES = SPARSE_ZCSRMM(OPT, M, N K, ALPHA, MATDESCRA, VAL, INDX, PNTRB, PNTRE, B, LDB, BETA, C, LDC);`

2.4.2.8.2 参数

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示是否转置： SPARSE_OPERATION_NON_TRANSPOSE, $C = \alpha * A * B + \beta * C$ 。 SPARSE_OPERATION_TRANSPOSE, $C = \alpha * A^T * x + \beta * C$ 。 SPARSE_OPERATION_CONJUGATE_TRANSPOSE, $C = \alpha * A^H * x + \beta * C$	输入
m	整型数	表示 A 矩阵的行数，取值范围为[1, MAX_INT]。	输入
n	整型数	矩阵 C 的列数，取值范围为[1, MAX_INT]。	输入
k	整型数	矩阵 A 的列数，取值范围为[1, MAX_INT]。	输入
alpha	在 scsrmm 中，alpha 是单精度浮点类型。	标量 alpha。	输入
	在 dcsrmm 中，alpha 是双精度浮点类型。		
	在 ccsrmm 中，alpha 是单精度复数。		
	在 zcsrmm 中，alpha 是双精度复数。		
matdescra	char 指针	矩阵的操作属性，具体参考 matdescra 说明。	输入
val	在 scsrmm 中，val 是单精度浮点数组。	CSR 格式中 values 数组，存储矩阵 A 的非零元素，长度为 $\text{pntre}[m-1] - \text{pntrb}[0]$ 。	输入
	在 dcsrmm 中，val 是双精度浮点数组。		
	在 ccsrmm 中，val 是单精度复数数组。		

参数名	类型	描述	输入/输出
	在 <code>zcsrmm</code> 中, <code>val</code> 是双精度复数数组。		
<code>indx</code>	整型数组	CSR 格式中 <code>columns</code> 数组, 用于表示矩阵 <code>A</code> 中非零元素所在的列索引。	输入
<code>pntrb</code>	整型数组	长度为 <code>m</code> 的数组, 包含矩阵 <code>A</code> 的行索引, <code>pntrb[i] - pntrb[0]</code> 表示第 <code>i</code> 行第一个非零元素在 <code>val</code> 和 <code>indx</code> 数组内的下标。	输入
<code>pntrc</code>	整型数组	长度为 <code>m</code> 的数组, 包含矩阵 <code>A</code> 的行索引, <code>pntrc[i] - pntrb[0]-1</code> 表示第 <code>i</code> 行最后一个非零元素在 <code>val</code> 和 <code>indx</code> 数组内的下标。	输入
<code>b</code>	在 <code>scsrmm</code> 中, <code>b</code> 是单精度浮点数组。	矩阵 <code>B</code> 的 <code>value</code> 数组。	输入
	在 <code>dcscrmm</code> 中, <code>b</code> 是双精度浮点数组。		
	在 <code>ccsrmm</code> 中, <code>b</code> 是单精度复数数组。		
	在 <code>zcsrmm</code> 中, <code>b</code> 是双精度复数数组。		
<code>ldb</code>	整型数	矩阵 <code>indx</code> 从 1 开始时, 矩阵 <code>B</code> 的主维度大小。 矩阵 <code>indx</code> 从 0 开始时, 矩阵 <code>B</code> 的第二维度大小。	输入
<code>beta</code>	在 <code>scsrmm</code> 中, <code>beta</code> 是单精度浮点类型。	标量 <code>beta</code> 。	输入
	在 <code>dcscrmm</code> 中, <code>beta</code> 是双精度浮点类型。		
	在 <code>ccsrmm</code> 中, <code>beta</code> 是单精度复数。		
	在 <code>zcsrmm</code> 中, <code>beta</code> 是双精度复数。		

参数名	类型	描述	输入/输出
c	在 scsrmm 中, c 是单精度浮点数组。	矩阵 C 的 value 数组。	输入/输出
	在 dcsrmm 中, c 是双精度浮点数组。		
	在 ccsrmm 中, c 是单精度复数数组。		
	在 zcsrmm 中, c 是双精度复数数组。		
ldc	整型数	矩阵 indx 从 1 开始时, 矩阵 C 的主维度大小。 矩阵 indx 从 0 开始时, 矩阵 C 的第二维度大小。	输入
注意: 函数内部不对参数做完整校验, 接口调用者需保证 pntrb 和 pntrc 内部元素不超出矩阵最大索引值。			

B 矩阵参数约束如下所示

opt	B 矩阵规模	B 矩阵数据排布	参数范围
op(A) = A	$k * n$	行主序	$k * ldbMAX_INT$
op(A) = A	$k * n$	列主序	$ldb * nMAX_INT$
$OP(A) = A^T \text{ or } A^H$	$m * n$	行主序	$m * ldbMAX_INT$
$OP(A) = A^T \text{ or } A^H$	$m * n$	列主序	$ldb * nMAX_INT$

C 矩阵参数约束如下所示。

opt	C 矩阵规模	C 矩阵数据排布	参数范围
op(A) = A	$k * n$	行主序	$k * ldcMAX_INT$
op(A) = A	$k * n$	列主序	$ldc * nMAX_INT$
$OP(A) = A^T \text{ or } A^H$	$m * n$	行主序	$m * ldcMAX_INT$
$OP(A) = A^T \text{ or } A^H$	$m * n$	列主序	$ldc * nMAX_INT$

2.4.2.8.3 返回值

函数执行状态, 枚举类型 sparse_status_t。

2.4.2.8.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.8.5 示例

A. C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;

INT m = 3;
INT n = 3;
INT k = 3;
float alpha = 1.0;
float beta = 1.0;
char *matdescra = "G00C"; //一般矩阵, 基0索引
float val[4] = {9, 8, 5, 2};
INT indx[4] = {2, 1, 2, 2};
INT pntbrb[3] = {0, 2, 3};
INT pntre[3] = {2, 3, 4};
float b[9] = {1, 5, 7, 4, 7, 7, 3, 3, 7};
float c[9] = {0, 8, 8, 0, 2, 8, 3, 5, 6};
INT ldb = 3;
INT ldc = 3;

sparse_status_t status = sparse_scsrmm(opt, m, n, k, alpha,
matdescra, val, indx, pntbrb, pntre, b, ldb, beta, c, ldc);

/*
 * Output c:
 *      59.00  91.00 127.00 15.00 17.00 43.00 9.00 11.00
20.00
 *
 */
```

2.4.2.9 sparse_csradd

计算 2 个 CSR 格式的稀疏矩阵之和。 $C = A + \text{beta} * \text{op}(B)$

A、B、C 为 3 数组 CSR 格式稀疏矩阵，仅支持基 1 索引。

op(B)可能的操作为：

$$\text{op}(B) = B \text{ or } \text{op}(B) = B^T \text{ or } \text{op}(B) = B^H$$

2.4.2.9.1 接口定义

A. C interface:

```
sparse_status_t sparse_scsradd(const sparse_operation_t opt, const INT request, const
INT sort, const INT m, const INT n, const float *a, const INT *ja, INT *ia, const float
beta, const float *b, const INT *jb, const INT *ib, float *c, INT *jc, INT *ic, const
INT nzmax);
```

```
sparse_status_t sparse_dcsradd(const sparse_operation_t opt, const INT request, const
INT sort, const INT m, const INT n, const double *a, const INT *ja, const INT *ia,
const double beta, const double *b, const INT *jb, const INT *ib, double *c, INT *jc,
INT *ic, const INT nzmax);
```

```
sparse_status_t sparse_ccsradd(const sparse_operation_t opt, const INT request, const
INT sort, const INT m, const INT n, const Complex8 *a, const INT *ja, const INT *ia,
const Complex8 beta, const Complex8 *b, const INT *jb, const INT *ib, Complex8
*c, INT *jc, INT *ic, const INT nzmax);
```

```
sparse_status_t sparse_zcsradd(const sparse_operation_t opt, const INT request, const
INT sort, const INT m, const INT n, const Complex16 *a, const INT *ja, const INT
*ia, const Complex16 beta, const Complex16 *b, const INT *jb, const INT *ib,
Complex16 *c, INT *jc, INT *ic, const INT nzmax);
```

B. Fortran interface:

```
RES = SPARSE_SCSRADD(OPT, REQUEST, SORT, M, N, A, JA, IA, BETA, B, JB,
IB, C, JC, IC, NZMAX);
```

```
RES = SPARSE_DCSRADD(OPT, REQUEST, SORT, M, N, A, JA, IA, BETA, B, JB,
IB, C, JC, IC, NZMAX);
```

```
RES = SPARSE_CCSRADD(OPT, REQUEST, SORT, M, N, A, JA, IA, BETA, B, JB,
IB, C, JC, IC, NZMAX);
```

```
RES = SPARSE_ZCSRADD(OPT, REQUEST, SORT, M, N, A, JA, IA, BETA, B, JB,
IB, C, JC, IC, NZMAX);
```

2.4.2.9.2 参数

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示是否转置： SPARSE_OPERATION_NON_TRANSPOSE, $C = A + \beta * B$ 。 SPARSE_OPERATION_TRANSPOSE, $C = A + \beta * B^T$ 。 SPARSE_OPERATION_CONJUGATE_TRANSPOSE, $C = A + \beta * B^H$	输入
request	整型数	0: 执行加法操作, ic, jc, c 的内存需要提前申请好。 1: 仅仅计算 ic 数组的值, 长度为 m+1, ic 的内存需要提前申请好。 2: 仅计算 jc、c 的值, 长度为 ic[m]-1, 为 request=1 的后续步骤, 需要在前一步骤计算好 ic。	输入
sort	整型数	1: 需要对 ja 内的列索引排序。 2: 需要对 jb 内的列索引排序。 3: 需要对 ja 和 jb 内的列索引排序。 其它值: 不对 ja 和 jb 内的列索引排序。此时默认 ja 和 jb 内的列索引是有序的。	输入
m	整型数	矩阵 A 的行数, 取值范围为[1, MAX_INT]。	输入
n	整型数	矩阵 A 的列数。	输入
a	在 scsradd 中, a 是单精度浮点数组。	矩阵 A 的非零元素数组。	输入
	在 dcsradd 中, a 是双精度浮点数组。		
	在 ccsradd 中, a 是单精度复数数组。		

参数名	类型	描述	输入/输出
	在 zcsradd 中, a 是双精度复数数组。		
ja	整型数组	CSR 格式的列索引数组。	输入
ia	整型数组	长度为 m+1 的数组, 包含矩阵 A 的行索引, ia[i] - ia[0]表示第 i 行第一个非零元素在 val 和 indx 数组内的下标。	输入
beta	在 scsradd 中, beta 是单精度浮点类型。	标量 beta。	输入
	在 dcsradd 中, beta 是双精度浮点类型。		
	在 ccsradd 中, beta 是单精度复数。		
	在 zcsradd 中, beta 是双精度复数。		
jb	整型数组	CSR 格式的列索引数组。	输入
ib	整型数组	CSR 格式稀疏矩阵 B 的三数组形式的行索引数组, 非转置时大小为 m+1, 转置时大小为 n+1, ib[i] - ib[0]表示第 i 行第一个非零元素在 val 和 indx 数组内的下标。	输入
c	在 scsradd 中, c 是单精度浮点数组	矩阵 C 的非零元素数组。	输入/输出
	在 dcsradd 中, c 是双精度浮点数组		
	在 ccsradd 中, c 是单精度复数数组		
	在 zcsradd 中, c 是双精度复数数组		
jc	整型数组	CSR 格式的列索引数组。	输入/输出

参数名	类型	描述	输入/输出
ic	整型数组	长度为 $m+1$ 的数组，包含矩阵 C 的行索引， $ic[i] - ic[0]$ 表示第 i 行第一个非零元素在 val 和 $indx$ 数组内的下标。	输入/输出
nzmax	整型数	矩阵 C 内的非零元素最大个数，当超过 $nzmax$ 时，计算过程停止。该参数只有 $request=0$ 或 2 时有效。	输入
注意： 函数内部不对参数做完整校验，接口调用者需保证 ia 和 ja 内部元素不超出矩阵最大索引值。			

2.4.2.9.3 返回值

函数执行状态，枚举类型 `sparse_status_t`。

2.4.2.9.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.9.5 示例

A-C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;

INT m = 3;

INT n = 3;

INT request = 0; //同时计算 c, jc, ic 三个数组

INT sort = 0; //a,b 每行元素的列索引输入前已排序

float beta = 1.0;

float a[4] = {9, 4, 2, 2};

INT ja[4] = {1, 3, 2, 2};

INT ia[4] = {1, 3, 4, 5};

float b[4] = {2, 5, 9, 8};
```



```

INT jb[4] = {2, 3, 2, 1};
INT ib[4] = {1, 3, 4, 5};
float c[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
INT jc[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
INT ic[4] = {0, 0, 0, 0};
INT nzmax = 9;

sparse_status_t status = sparse_scsradd(opt, request, sort, m, n, a,
ja, ia, beta, b, jb, ib, c, jc, ic, nzmax);

/*
 * Output c:
 *      9.00  2.00  9.00  11.00  8.00  2.00
 * Output jc:
 *      1  2  3  2  1  2
 * Output ic:
 *      1  4  5  7
 *
 */

```

2.4.2.10 sparse_csrmultcsr

矩阵与矩阵计算，其中 3 个稀疏矩阵采用 3-arrays CSR 格式存储，为基 1 索引，具体执行操作为： $C = op(A) * B$ 。

其中，A、B、C 是采用 CSR 格式存储的稀疏矩阵。

$op(A)$ 根据参数是以下 3 种情况之一：

$$op(A) = A \text{ or } op(A) = A^T \text{ or } op(A) = A^H$$

2.4.2.10.1 接口定义

A. C interface:

```

sparse_status_t sparse_csrmultcsr(const sparse_operation_t opt, const INT request,
const INT sort, const INT m, const INT n, const INT k, const float *a, const INT *ja,
const INT *ia, const float *b, const INT *jb, const INT *ib, float *c, INT *jc, INT *ic,
const INT nzmax);

```

```

sparse_status_t sparse_dcsmultcsr(const sparse_operation_t opt, const INT request,
const INT sort, const INT m, const INT n, const INT k, const double *a, const INT *ja,

```

```
const INT *ia, const double *b, const INT *jb, const INT *ib, double *c, INT *jc, INT *ic, const INT nzmax);
```

```
sparse_status_t sparse_ccsrmultcsr(const sparse_operation_t opt, const INT request, const INT sort, const INT m, const INT n, const INT k, const Complex8 *a, const INT *ja, const INT *ia, const Complex8 *b, const INT *jb, const INT *ib, Complex8 *c, INT *jc, INT *ic, const INT nzmax);
```

```
sparse_status_t sparse_zcsrmultcsr(const sparse_operation_t opt, const INT request, const INT sort, const INT m, const INT n, const INT k, const Complex16 *a, const INT *ja, const INT *ia, const Complex16 *b, const INT *jb, const INT *ib, Complex16 *c, INT *jc, INT *ic, const INT nzmax);
```

B. Fortran interface:

```
RES = SPARSE_SCSRMULTCSR(OPT, REQUEST, SORT, M, N, K, A, JA, IA, B, JB, IB, C, JC, IC, NZMAX);
```

```
RES = SPARSE_DCSRMULTCSR(OPT, REQUEST, SORT, M, N, K, A, JA, IA, B, JB, IB, C, JC, IC, NZMAX);
```

```
RES = SPARSE_CCSRMULTCSR(OPT, REQUEST, SORT, M, N, K, A, JA, IA, B, JB, IB, C, JC, IC, NZMAX);
```

```
RES = SPARSE_ZCSRMULTCSR(OPT, REQUEST, SORT, M, N, K, A, JA, IA, B, JB, IB, C, JC, IC, NZMAX);
```

2.4.2.10.2 参数

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示是否转置： opt = 0, $C = \alpha * A * B + \beta * C$ 。 opt = 1, $C = \alpha * A^T * B + \beta * C$ 。 opt = 2, $C = \alpha * A^H * B + \beta * C$ 。	输入
request	整型数	request=0, ic、jc、c 的内存由调用者申请。 request=1, 仅更新 ic, ic 内存由调用者申请。 request=2, 结合 request=1 使用, 更新 jc、c。	输入

参数名	类型	描述	输入/输出
sort	整型数	当前 sort 参数未定义。	输入
m	整型数	矩阵 A 的行数，取值范围为[1, MAX_INT]。	输入
n	整型数	矩阵 A 的列数，取值范围为[1, MAX_INT]。	输入
k	整型数	矩阵 B 的列数，取值范围为[1, MAX_INT]。	输入
a	在 scsrmultcsr 中，a 是单精度浮点数组。 在 dcsrmultcsr 中，a 是双精度浮点数组。 在 ccsrmultcsr 中，a 是单精度复数数组。 在 zcsrmultcsr 中，a 是双精度复数数组。	CSR 格式中 values 数组，存储矩阵 A 的非零元素。	输入
ja	整型数组	CSR 格式中 columns 数组，用于表示矩阵 A 中非零元素所在的列索引。	输入
ia	整型数组	长度为 m+1 的数组，包含矩阵 A 的行索引，ia[i] - ia[0]表示第 i 行第一个非零元素在 val 和 indx 数组内的下标。	输入
b	在 scsrmultcsr 中，b 是单精度浮点数组。 在 dcsrmultcsr 中，b 是双精度浮点数组。 在 ccsrmultcsr 中，b 是单精度复数数组。 在 zcsrmultcsr 中，b 是双精度复数数组。	CSR 格式中 values 数组，存储矩阵 B 非零元素。	输入
jb	整型数组	CSR 格式中 columns 数组，用于表示矩阵 B 非零元素所在的列索引。	输入

参数名	类型	描述	输入/输出
ib	整型数组	长度为 $m+1$ 的数组，包含矩阵 B 的行索引， $ib[i] - ib[0]$ 表示第 i 行第一个非零元素在 val 和 $indx$ 数组内的下标。	输入
c	在 <code>scsrmultcsr</code> 中， c 是单精度浮点数组。	CSR 格式中 <code>values</code> 数组，存储矩阵 C 非零元素。	输入/输出
	在 <code>dcsrcmultcsr</code> 中， c 是双精度浮点数组。		
	在 <code>ccsrcmultcsr</code> 中， c 是单精度复数数组。		
	在 <code>zcsrcmultcsr</code> 中， c 是双精度复数数组。		
jc	整型数组	CSR 格式的列索引数组。	输入/输出
ic	整型数组	长度为 $m+1$ 的数组，包含矩阵 C 的行索引， $ic[i] - ic[0]$ 表示第 i 行第一个非零元素在 val 和 $indx$ 数组内的下标。	输入/输出
nzmax	整型数	矩阵 C 内的非零元素最大个数，当超过 <code>nzmax</code> 时，计算过程停止。该参数只有 <code>request=0</code> 或 2 时有效。	输入
注意： 函数内部不对参数做完整校验，接口调用者需保证 <code>ia</code> 、 <code>ja</code> 、 <code>ib</code> 、 <code>jb</code> 、 <code>ic</code> 、 <code>jc</code> 内部元素不超出矩阵最大索引值。			

2.4.2.10.3 返回值

函数执行状态，枚举类型 `sparse_status_t`。

2.4.2.10.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.10.5 示例

A. C interface:

```

sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;

INT m = 3;
INT n = 3;
INT k = 3;
INT request = 0; //同时计算 c, jc, ic 三个数组
INT sort = 3; //a,b 每行元素的列索引输入前未排序
float a[4] = {4, 5, 4, 5};
INT ja[4] = {3, 2, 1, 3};
INT ia[4] = {1, 2, 4, 5};
float b[4] = {7, 7, 6, 2};
INT jb[4] = {3, 2, 3, 2};
INT ib[4] = {1, 2, 3, 5};
float c[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
INT jc[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
INT ic[4] = {0, 0, 0, 0};
INT nzmax = 9;
sparse_status_t status = sparse_scsrmultcsr(opt, request, sort, m,
n, k, a, ja, ia, b, jb, ib, c, jc, ic, nzmax);
/*
 * Output c:
 *      8.00  24.00  35.00  28.00  10.00  30.00
 * Output jc:
 *      2  3  2  3  2  3
 * Output ic:
 *      1  3  5  7
 *
 */

```

2.4.2.11 sparse_csrmultd

矩阵与矩阵计算，其中 2 个稀疏矩阵采用 CSR 格式存储，为基 1 索引，具体执行操作为： $C := \text{op}(A) * B$ 。

其中，C 为稠密矩阵，A 和 B 是采用 CSR 格式存储的稀疏矩阵。

$\text{op}(A)$ 根据参数是以下 3 种情况之一：

$$\text{op}(A) = A \text{ or } \text{op}(A) = A^T \text{ or } \text{op}(A) = A^H$$

2.4.2.11.1 接口定义

A. C interface:

```
sparse_status_t sparse_csrmultd(const sparse_operation_t opt, const INT m, const
INT n, const INT k, const float *a, const INT *ja, const INT *ia, const float *b, const
INT *jb, const INT *ib, float *c, const INT ldc);
```

```
sparse_status_t sparse_dcsmultd(const sparse_operation_t opt, const INT m, const
INT n, const INT k, const double *a, const INT *ja, const INT *ia, const double *b,
const INT *jb, const INT *ib, double *c, const INT ldc);
```

```
sparse_status_t sparse_ccsmultd(const sparse_operation_t opt, const INT m, const
INT n, const INT k, const Complex8 *a, const INT *ja, const INT *ia, const
Complex8 *b, const INT *jb, const INT *ib, Complex8 *c, const INT ldc);
```

```
sparse_status_t sparse_zcsmultd(const sparse_operation_t opt, const INT m, const
INT n, const INT k, const Complex16 *a, const INT *ja, const INT *ia, const
Complex16 *b, const INT *jb, const INT *ib, Complex16 *c, const INT ldc);
```

B. Fortran interface:

```
RES = SPARSE_SC SRMULTD(OPT, M, N, K, A, JA, IA, B, JB, IB, C, LDC);
```

```
RES = SPARSE_DCSRMULTD(OPT, M, N, K, A, JA, IA, B, JB, IB, C, LDC);
```

```
RES = SPARSE_CC SRMULTD(OPT, M, N, K, A, JA, IA, B, JB, IB, C, LDC);
```

```
RES = SPARSE_ZCSRMULTD(OPT, M, N, K, A, JA, IA, B, JB, IB, C, LDC);
```

2.4.2.11.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示是否转置： opt = 0, $C = \alpha * A * B + \beta * C$ 。 opt = 1, $C = \alpha * A^T * B + \beta * C$ 。 opt = 2, $C = \alpha * A^H * B + \beta * C$ 。	输入
m	整型数	矩阵 A 的行数，取值范围为[1, MAX_INT]。	输入
n	整型数	矩阵 A 的列数，取值范围为[1, MAX_INT]。	输入
k	整型数	矩阵 B 的列数，取值范围为[1, MAX_INT]。	输入
a	在 scsrmultd 中，a 是单精度浮点数组。 在 dcsrmultd 中，a 是双精度浮点数组。 在 ccsrmultd 中，a 是单精度复数数组。 在 zcsrmultd 中，a 是双精度复数数组。	CSR 格式中 values 数组，存储矩阵 A 的非零元素。	输入
ja	整型数组	CSR 格式中 columns 数组，用于表示矩阵 A 中非零元素所在的列索引。	输入
ia	整型数组	长度为 m+1 的数组，包含矩阵 A 的行索引，ia[i] - ia[0]表示第 i 行第一个非零元素在 val 和 indx 数组内的下标。	输入
b	在 scsrmultd 中，b 是单精度浮点数组。 在 dcsrmultd 中，b 是双精度浮点数组。 在 ccsrmultd 中，b 是单精度复数数组。	CSR 格式中 values 数组，存储矩阵 B 非零元素。	输入

参数名	类型	描述	输入/输出
	在 <code>zcsrmultd</code> 中, <code>b</code> 是双精度复数数组。		
<code>jb</code>	整型数组	CSR 格式中 <code>columns</code> 数组, 用于表示矩阵 <code>B</code> 非零元素所在的列索引。	输入
<code>ib</code>	整型数组	长度为 <code>m+1</code> 的数组, 包含矩阵 <code>B</code> 的行索引, <code>ib[i] - ib[0]</code> 表示第 <code>i</code> 行第一个非零元素在 <code>val</code> 和 <code>indx</code> 数组内的下标。	输入
<code>c</code>	在 <code>scsrmultd</code> 中, <code>c</code> 是单精度浮点数组。	CSR 格式中 <code>values</code> 数组, 存储矩阵 <code>C</code> 非零元素。	输入/输出
	在 <code>dcsrmultd</code> 中, <code>c</code> 是双精度浮点数组。		
	在 <code>ccsrmultd</code> 中, <code>c</code> 是单精度复数数组。		
	在 <code>zcsrmultd</code> 中, <code>c</code> 是双精度复数数组。		
<code>ldc</code>	整型数	稠密矩阵 <code>C</code> 的主维度, <code>ldc*kMAX_INT</code> 。	输入
注意: 函数内部不对参数做完整校验, 接口调用者需保证 <code>ia</code> 、 <code>ja</code> 、 <code>ib</code> 、 <code>jb</code> 内部元素不超出矩阵最大索引值。			

2.4.2.11.3 返回值

函数执行状态, 枚举类型 `sparse_status_t`。

2.4.2.11.4 依赖

C: `"kspblas.h"`

Fortran: `"kspblas.f03"`

2.4.2.11.5 示例

A. C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;
```



```

INT m = 3;
INT n = 3;
INT k = 3;
float a[4] = {1, 5, 4, 5};
INT ja[4] = {3, 1, 1, 1};
INT ia[4] = {1, 3, 4, 5};
float b[4] = {9, 4, 2, 8};
INT jb[4] = {2, 3, 1, 1};
INT ib[4] = {1, 3, 4, 5};
float c[9] = {0, 0, 0, 0, 0, 0, 0, 0, 0};
INT ldc = 3;

sparse_status_t status = sparse_scsrmultd(opt, m, n, k, a, ja, ia,
b, jb, ib, c, ldc);

/*
 * Output c:
 *      8.00  0.00  0.00  45.00 36.00  45.00  20.00  16.00  20.00
 *
 */

```

2.4.2.12 sparse_?csrtrsv

求解稀疏三角线性方程组计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下：

$$x = A * y \text{ or } x = A^T * y \text{ or } x = A^H * y$$

A 为 3 数组 CSR 格式稀疏矩阵，x、y 为向量，仅支持 One-based indexing。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。

2.4.2.12.1 接口定义

A. C interface:

```

sparse_status_t sparse_scsrtrsv(const char uplo, const sparse_operation_t opt, const
char diag, const INT m, const float *a, const INT *ia, const INT *ja, const float *x,
float *y);

```

```
sparse_status_t sparse_dcsrtrsv(const char uplo, const sparse_operation_t opt, const
char diag, const INT m, const double *a, const INT *ia, const INT *ja, const double
*x, double *y);
```

```
sparse_status_t sparse_ccsrtrsv(const char uplo, const sparse_operation_t opt, const
char *diag, const INT m, const Complex8 *a, const INT *ia, const INT *ja, const
Complex8 *x, Complex8 *y);
```

```
sparse_status_t sparse_zcsrtrsv(const char uplo, const sparse_operation_t opt, const
char *diag, const INT m, const Complex16 *a, const INT *ia, const INT *ja, const
Complex16 *x, Complex16 *y);
```

2.4.2.12.2 参数

参数名	类型	描述	输入/输出
uplo	枚举类型 sparse_fill_mode_t uplo	指示矩阵 A 的上三角或者下三角被使用： U: 上三角 L: 下三角	输入
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作， 如果 opt=SPARSE_OPERATION_NO N_TRANSPOSE, 则 $x = A * y$ 。 如果 opt=SPARSE_OPERATION_TR ANSPOSE, 则 $x = A^T * y$ 。 如果 opt=SPARSE_OPERATION_CO NJUGATE_TRANSPOSE, 则 $x = A^H * y$ 。	输入
diag	枚举类型 sparse_diag_type_t	对角元素是否是单位元素。 U: 单位元素 N: 非单位元素	输入
m	整型数	表示 A 矩阵的行数和列数，取值范围为[1, MAX_INT]。	输入
a	在 scsrtrsv 中，val 是单精度浮点数组。	CSR 格式中 values 数组，存储矩阵 A 的非零元素。	输入

参数名	类型	描述	输入/输出
	在 dcsrtrsv 中, val 是双精度浮点数组。		
	在 ccsrtrsv 中, val 是单精度复数数组。		
	在 zcsrtrsv 中, val 是双精度复数数组。		
ja	整型数组	CSR 格式中 columns 数组, 用于表示矩阵 A 中非零元素所在的列索引。	输入
ia	整型数组	长度为 m+1 的数组, 包含矩阵 A 的行索引, ia[i] - ia[0]表示第 i 行第一个非零元素在 val 和 indx 数组内的下标。	输入
x	在 scsrtrsv 中, x 是单精度浮点数组。	向量 x。	输入
	在 dcsrtrsv 中, x 是双精度浮点数组。		
	在 ccsrtrsv 中, x 是单精度复数数组。		
	在 zcsrtrsv 中, x 是双精度复数数组。		
y	在 scsrtrsv 中, y 是单精度浮点数组。	向量 y, 更新后输出。	输入/输出
	在 dcsrtrsv 中, y 是双精度浮点数组。		
	在 ccsrtrsv 中, y 是单精度复数数组。		
	在 zcsrtrsv 中, y 是双精度复数数组。		
注意: 函数内部不对参数做完整校验, 接口调用者需保证 ia、 ja 内部元素不超出矩阵最大索引值。			

2.4.2.12.3 返回值

函数执行状态，枚举类型 `sparse_status_t`。

2.4.2.12.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.12.5 示例

A. C interface:

```
sparse_fill_mode_t uplo = SPARSE_FILL_MODE_LOWER;
sparse_diag_type_t diag = SPARSE_DIAG_NON_UNIT;
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;
INT m = 4;
float val[9] = {2, -2, 1, 5, 3, 8, 6, 2, 2};
INT ja[9] = {4, 1, 4, 2, 3, 3, 4, 4, 2};
INT ia[5] = {1, 3, 6, 8, 10};
float x[4] = {6, 6, 4, 6};
float y[4] = {1, 1, 6, 1};
sparse_status_t status = sparse_scsrtrsv(uplo, opt, diag, m, val, ia,
ja, x, y);
/*
 * Output Y:
 *
 * -3.000000 1.200000 0.500000 1.800000
 *
 */
```

2.4.2.13 `csparse_?csrtrsv`

求解稀疏三角线性方程组计算，稀疏矩阵采用 CSR 格式存储，具体执行操作如下：

$$x = A * y \text{ or } x = A^T * y \text{ or } x = A^H * y$$

A 为 3 数组 CSR 格式稀疏矩阵，x、y 为向量，仅支持 zero-based indexing。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。

2.4.2.13.1 接口定义

A. C interface:

sparse_status_t csparse_scsrtrsv(const char uplo, const sparse_operation_t opt, const char diag, const INT m, const float *a, const INT *ia, const INT *ja, const float *x, float *y);

sparse_status_t csparse_dcsrtrsv(const char uplo, const sparse_operation_t opt, const char diag, const INT m, const double *a, const INT *ia, const INT *ja, const double *x, double *y);

sparse_status_t csparse_ccsrtrsv(const char uplo, const sparse_operation_t opt, const char *diag, const INT m, const Complex8 *a, const INT *ia, const INT *ja, const Complex8 *x, Complex8 *y);

sparse_status_t csparse_zcsrtrsv(const char uplo, const sparse_operation_t opt, const char *diag, const INT m, const Complex16 *a, const INT *ia, const INT *ja, const Complex16 *x, Complex16 *y);

2.4.2.13.2 参数

参数名	类型	描述	输入/输出
uplo	枚举类型 sparse_fill_mode_t uplo	指示矩阵 A 的上三角或者下三角被使用： U: 上三角 L: 下三角	输入

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作， 如果 opt=SPARSE_OPERATION_NO N_TRANSPOSE，则 $x = A * y$ 。 如果 opt=SPARSE_OPERATION_TR ANSPOSE，则 $x = A^T * y$ 。 如果 opt=SPARSE_OPERATION_CO NJUGATE_TRANSPOSE，则 $x = A^H * y$ 。	输入
diag	枚举类型 sparse_diag_type_t	对角元素是否是单位元素。 U: 单位元素 N: 非单位元素	输入
m	整型数	表示 A 矩阵的行数和列数，取 值范围为[1, MAX_INT]。	输入
a	在 c_scsrtrsv 中，val 是单精度浮点数组。 在 c_dcsrtrsv 中，val 是双精度浮点数组。 在 c_ccsrtrsv 中，val 是单精度复数数组。 在 c_zcsrtrsv 中，val 是双精度复数数组。	CSR 格式中 values 数组，存储 矩阵 A 的非零元素。	输入
ja	整型数组	CSR 格式中 columns 数组，用 于表示矩阵 A 中非零元素所在 的列索引。	输入
ia	整型数组	长度为 m+1 的数组，包含矩阵 A 的行索引，ia[i] - ia[0]表示第 i 行第一个非零元素在 val 和 indx 数组内的下标。	输入
x	在 c_scsrtrsv 中，x 是 单精度浮点数组。	向量 x。	输入

参数名	类型	描述	输入/输出
	在 <code>c_dcsrtrsv</code> 中, <code>x</code> 是双精度浮点数组。		
	在 <code>c_ccsrtrsv</code> 中, <code>x</code> 是单精度复数数组。		
	在 <code>c_zcsrtrsv</code> 中, <code>x</code> 是双精度复数数组。		
y	在 <code>c_scsrtrsv</code> 中, <code>y</code> 是单精度浮点数组。	向量 <code>y</code> , 更新后输出。	输入/输出
	在 <code>c_dcsrtrsv</code> 中, <code>y</code> 是双精度浮点数组。		
	在 <code>c_ccsrtrsv</code> 中, <code>y</code> 是单精度复数数组。		
	在 <code>c_zcsrtrsv</code> 中, <code>y</code> 是双精度复数数组。		

注意:

函数内部不对参数做完整校验, 接口调用者需保证 `ia`、`ja` 内部元素不超出矩阵最大索引值。

2.4.2.13.3 返回值

函数执行状态, 枚举类型 `sparse_status_t`。

2.4.2.13.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.13.5 示例

A. C interface:

```
sparse_fill_mode_t uplo = SPARSE_FILL_MODE_LOWER;
sparse_diag_type_t diag = SPARSE_DIAG_NON_UNIT;
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;
INT m = 4;
float val[9] = {2, -2, 1, 5, 3, 8, 6, 2, 2};
```

```

INT ja[9] = {3, 0, 3, 1, 2, 2, 3, 3, 1};
INT ia[5] = {0, 2, 5, 7, 9};
float x[4] = {6, 6, 4, 6};
float y[4] = {1, 1, 6, 1};

sparse_status_t status = csparse_scsrtrsv(uplo, opt, diag, m, val, ia,
ja, x, y);

/*
*   Output Y:
*
*   -3.000000  1.200000  0.500000  1.800000
*
* */

```

2.4.2.14 sparse_?cscsv

求解三角矩阵方程组计算，稀疏矩阵采用 CSC 格式存储，具体执行操作如下：

$$A * y = \alpha * x \text{ or } A^T * y = \alpha * x \text{ or } A^H * y = \alpha * x$$

其中，x 和 y 为向量，A 是 m * m 的稀疏矩阵，采用 CSC 格式存储。矩阵 A 中的给定列的非零元素的存储顺序必须与列中显示的顺序相同（从上到下）。

2.4.2.14.1 接口定义

A. C interface:

```

sparse_status_t sparse_scscsv(const sparse_operation_t opt, const INT m, const float
alpha, const char *matdescra, const float *val, const INT *indx, const INT *pntrb,
const INT *pntrc, const float *x, const float beta, float *y);

```

```

sparse_status_t sparse_dcscsv(const sparse_operation_t opt, const INT m, const
double alpha, const char *matdescra, const double *val, const INT *indx, const INT
*pntrb, const INT *pntrc, const double *x, const double beta, double *y);

```

```

sparse_status_t sparse_ccscsv(const sparse_operation_t opt, const INT m, const
Complex8 alpha, const char *matdescra, const Complex8 *val, const INT *indx,
const INT *pntrb, const INT *pntrc, const Complex8 *x, const Complex8 beta,
Complex8 *y);

```

```

sparse_status_t sparse_zcscsv(const sparse_operation_t opt, const INT m, const
Complex16 alpha, const char *matdescra, const Complex16 *val, const INT *indx,
const INT *pntrb, const INT *pntrc, const Complex16 *x, const Complex16 beta,
Complex16 *y);

```


2.4.2.14.2 参数

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作， 如果 opt=SPARSE_OPERATION_NO N_TRANSPOSE，则 $A * y = \alpha * x$ 。 如果 opt=SPARSE_OPERATION_TR ANSPOSE，则 $A^T * y = \alpha * x$ 。 如果 opt=SPARSE_OPERATION_CO NJUGATE_TRANSPOSE，则 $A^H * y = \alpha * x$	输入
m	整型数	表示 A 矩阵的行数和列数，取 值范围为[1, MAX_INT]。	输入
alpha	在 scscsv 中，alpha 是 单精度浮点类型。 在 dcscsv 中，alpha 是 双精度浮点类型。 在 ccscsv 中，alpha 是 单精度复数。 在 zcscsv 中，alpha 是 双精度复数。	表示系数。	输入
matdescra	char 指针	矩阵的操作属性，具体参考 matdescra 说明。	输入
val	在 scscsv 中，val 是单 精度浮点数组。 在 dcscsv 中，val 是双 精度浮点数组。 在 ccscsv 中，val 是单 精度复数数组。 在 zcscsv 中，val 是双 精度复数数组。	A 矩阵中的非零元素。	输入

参数名	类型	描述	输入/输出
indx	整型数组	indx[i]表示 val 数组中第 i 个元素在矩阵 A 中的行序号。	输入
pntrb	整型数组	长度为 m 的数组，包含矩阵 A 的列索引，pntrb[i] - pntrb[0]表示第 i 列第一个非零元素在 val 和 indx 数组内的下标。	输入
pntrc	整型数组	长度为 m 的数组，包含矩阵 A 的列索引，pntrc[i] - pntrb[0]-1表示第 i 列最后一个非零元素在 val 和 indx 数组内的下标。	输入
x	在 scscsv 中，x 是单精度浮点数组。	向量 x。	输入
	在 dcscsv 中，x 是双精度浮点数组。		
	在 ccscsv 中，x 是单精度复数数组。		
	在 zcscsv 中，x 是双精度复数数组。		
y	在 scscsv 中，y 是单精度浮点数组。	向量 y，更新后输出。	输入/输出

参数名	类型	描述	输入/输出
	在 <code>dcscsv</code> 中, <code>y</code> 是双精度浮点数组。		
	在 <code>ccscsv</code> 中, <code>y</code> 是单精度复数数组。		
	在 <code>zcscsv</code> 中, <code>y</code> 是双精度复数数组。		
注意: 函数内部不对参数做完整校验, 接口调用者需保证 <code>pntrb</code> 和 <code>pntre</code> 内部元素不超出矩阵最大索引值。			

2.4.2.14.3 返回值

函数执行状态, 枚举类型 `sparse_status_t`。

2.4.2.14.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.14.5 示例

A. C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;
char *matdescra = "TLNC";
INT m = 4;
float alpha = 1.0;
float val[9] = {2, 3, 1, 5, 7, 8, 3, 6, 7};
INT indx[9] = {0, 1, 2, 1, 1, 2, 3, 0, 3};
INT pntbrb[4] = {0, 3, 4, 7};
INT pntre[4] = {3, 4, 7, 9};
float x[4] = {6, 6, 4, 6};
float y[4] = {1, 1, 6, 1};
sparse_status_t status = sparse_scscsv(opt, m, alpha, matdescra, val,
indx, pntbrb, pntre, x, y);
```

```

/*
 * Output Y:
 *      3.000000 -0.600000 0.125000 0.803571
 *
 * */

```

2.4.2.15 sparse_?cscmv

矩阵与向量乘积，矩阵是 CSC 格式的稀疏矩阵：

$$y = \alpha * A * x + \beta * y \text{ or } y = \alpha * A^T * x + \beta * y \text{ or } y = \alpha * A^H * x + \beta * y$$

其中，alpha 和 beta 是缩放系数，x 和 y 是向量，A 是 CSC 格式的稀疏向量。

2.4.2.15.1 接口定义

A. C interface:

sparse_status_t sparse_scscmv(const sparse_operation_t opt, const INT m, const INT k, const float alpha, const char *matdescra, const float *val, const INT *indx, const INT *pntrb, const INT *pntrc, const float *x, const float beta, float *y);

sparse_status_t sparse_dcscmv(const sparse_operation_t opt, const INT m, const INT k, const double alpha, const char *matdescra, const double *val, const INT *indx, const INT *pntrb, const INT *pntrc, const double *x, const double beta, double *y);

sparse_status_t sparse_ccscmv(const sparse_operation_t opt, const INT m, const INT k, const Complex8 alpha, const char *matdescra, const Complex8 *val, const INT *indx, const INT *pntrb, const INT *pntrc, const Complex8 *x, const Complex8 beta, Complex8 *y);

sparse_status_t sparse_zcscmv(const sparse_operation_t opt, const INT m, const INT k, const Complex16 alpha, const char *matdescra, const Complex16 *val, const INT *indx, const INT *pntrb, const INT *pntrc, const Complex16 *x, const Complex16 beta, Complex16 *y);

2.4.2.15.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作， 如果 opt=SPARSE_OPERATION_NO N_TRANSPOSE，则 $y = \alpha * A * x + \beta * y$ 。 如果 opt=SPARSE_OPERATION_TR ANSPOSE，则 $y = \alpha * A^T * x + \beta * y$ 。 如果 opt=SPARSE_OPERATION_CO NJUGATE_TRANSPOSE，则 $y = \alpha * A^H * x + \beta * y$	输入
m	整型数	表示 A 矩阵的行数，取值范围 为[1, MAX_INT]。	输入
k	整型数	表示 A 矩阵的列数，取值范围 为[1, MAX_INT]。	输入
alpha	在 scscmv 中，alpha 是单精度浮点类型。 在 dcscmv 中，alpha 是双精度浮点类型。 在 ccscmv 中，alpha 是单精度复数。 在 zcscmv 中，alpha 是双精度复数。	表示系数。	输入
matdescra	char 指针	矩阵的操作属性，具体参考 matdescra 说明。	输入
val	在 scscmv 中，val 是 单精度浮点数组。 在 dcscmv 中，val 是 双精度浮点数组。 在 ccscmv 中，val 是 单精度复数数组。 在 zcscmv 中，val 是 双精度复数数组。	A 矩阵中的非零元素。	输入

参数名	类型	描述	输入/输出
indx	整型数组	indx[i]表示 val 数组中第 i 个元素在矩阵 A 中的行序号。	输入
pntrb	整型数组	长度为 m 的数组，包含矩阵 A 的列索引，pntrb[i] - pntrb[0]表示第 i 列第一个非零元素在 val 和 indx 数组内的下标。	输入
pntrb	整型数组	长度为 m 的数组，包含矩阵 A 的列索引，pntrb[i] - pntrb[0]-1表示第 i 列最后一个非零元素在 val 和 indx 数组内的下标。	输入
x	在 scscmv 中，x 是单精度浮点数组。 在 dcscmv 中，x 是双精度浮点数组。 在 ccscmv 中，x 是单精度复数数组。 在 zcscmv 中，x 是双精度复数数组。	向量 x。	输入
beta	在 scscmv 中，beta 是单精度浮点类型。	表示系数。	输入

参数名	类型	描述	输入/输出
	在 <code>dcscmv</code> 中, <code>beta</code> 是双精度浮点类型。		
	在 <code>ccscmv</code> 中, <code>beta</code> 是单精度复数。		
	在 <code>zcscmv</code> 中, <code>beta</code> 是双精度复数。		
<code>y</code>	在 <code>scscmv</code> 中, <code>y</code> 是单精度浮点数组。	向量 <code>y</code> , 更新后输出。	输入/输出
	在 <code>dcscmv</code> 中, <code>y</code> 是双精度浮点数组。		
	在 <code>ccscmv</code> 中, <code>y</code> 是单精度复数数组。		
	在 <code>zcscmv</code> 中, <code>y</code> 是双精度复数数组。		

注意:

函数内部不对参数做完整校验, 接口调用者需保证 `pntrb` 和 `pntre` 内部元素不超出矩阵最大索引值。

2.4.2.15.3 返回值

函数执行状态, 枚举类型 `sparse_status_t`。

2.4.2.15.4 依赖

C: `"kspblas.h"`

Fortran: `"kspblas.f03"`

2.4.2.15.5 示例

A. C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;  
  
char *matdescra = "G00C";  
  
INT m = 4;  
  
INT k = 4;  
  
float alpha = 1.0;
```

```

float beta = 1.0;

float val[9] = {2, 3, 1, 5, 7, 8, 3, 6, 7};

INT indx[9] = {0, 1, 2, 1, 1, 2, 3, 0, 3};

INT pntrb[4] = {0, 3, 4, 7};

INT pntre[4] = {3, 4, 7, 9};

float x[4] = {6, 6, 4, 6};

float y[4] = {1, 1, 6, 1};

sparse_status_t status = sparse_scscmv(opt, m, k, alpha, matdescra,
val, indx, pntrb, pntre, x, beta, y);

/*
 * Output Y:
 *      49.000000 77.000000 44.000000 55.000000
 *
 * */

```

2.4.2.16 sparse_?cscmm

矩阵与矩阵计算，其中一个稀疏矩阵采用 CSC 格式存储。具体执行操作如下：

$$C = \alpha * A * B + \beta * C \text{ or } C = \alpha * A^T * B + \beta * C \text{ or } C = \alpha * A^H * B + \beta * C$$

其中，B 和 C 为稠密矩阵，A 是采用 CSC 格式存储的 mxk 的稀疏矩阵。

layout 与 sparse matrix indexing 的关系如下所示。

Sparse matrix indexing	Dense matrix layout
SPARSE_INDEX_BASE_ZERO	SPARSE_LAYOUT_ROW_MAJOR
SPARSE_INDEX_BASE_ONE	SPARSE_LAYOUT_COLUMN_MAJOR

2.4.2.16.1 接口定义

A. C interface:

```

sparse_status_t sparse_scscmm(const sparse_operation_t opt, const INT m, const INT
n, const INT k, const float alpha, const char *matdescra, const float *val, const INT

```


*indx, const INT *pntrb, const INT *pntre, const float *b, const INT ldb, const float beta, float *c, const INT ldc);

sparse_status_t sparse_dcscmm(const sparse_operation_t opt, const INT m, const INT n, const INT k, const double alpha, const char *matdescra, const double *val, const INT *indx, const INT *pntrb, const INT *pntre, const double *b, const INT ldb, const double beta, double *c, const INT ldc);

sparse_status_t sparse_ccscmm(const sparse_operation_t opt, const INT m, const INT n, const INT k, const Complex8 alpha, const char *matdescra, const Complex8 *val, const INT *indx, const INT *pntrb, const INT *pntre, const Complex8 *b, const INT ldb, const Complex8 beta, Complex8 *c, const INT ldc);

sparse_status_t sparse_zcscmm(const sparse_operation_t opt, const INT m, const INT n, const INT k, const Complex16 alpha, const char *matdescra, const Complex16 *val, const INT *indx, const INT *pntrb, const INT *pntre, const Complex16 *b, const INT ldb, const Complex16 beta, Complex16 *c, const INT ldc);

2.4.2.16.2 参数

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作， 如果 opt=SPARSE_OPERATION_NO_N_TRANSPOSE，则 $C = \alpha * A * B + \beta * C$ 。 如果 opt=SPARSE_OPERATION_TRANSPOSE，则 $C = \alpha * A^T * B + \beta * C$ 。 如果 opt=SPARSE_OPERATION_CONJUGATE_TRANSPOSE，则 $C = \alpha * A^H * B + \beta * C$	输入
m	整型数	表示 A 矩阵的行数，取值范围为[1, MAX_INT]。	输入
n	整型数	矩阵 C 的列数，取值范围为[1, MAX_INT]。	输入
k	整型数	表示 A 矩阵的列数，取值范围为[1, MAX_INT]。	输入

参数名	类型	描述	输入/输出
alpha	在 scscmm 中，alpha 是单精度浮点类型。	标量 alpha。	输入
	在 dcscmm 中，alpha 是双精度浮点类型。		
	在 ccscmm 中，alpha 是单精度复数。		
	在 zcscmm 中，alpha 是双精度复数。		
matdescra	char 指针	矩阵的操作属性，具体参考 matdescra 说明。	输入
val	在 scscmm 中，val 是单精度浮点数组。	CSC 格式中 values 数组，存储矩阵 A 的非零元素，长度为 $\text{pntrb}[m-1] - \text{pntrb}[0]$ 。	输入
	在 dcscmm 中，val 是双精度浮点数组。		
	在 ccscmm 中，val 是单精度复数数组。		
	在 zcscmm 中，val 是双精度复数数组。		
indx	整型数组	CSC 格式中 columns 数组，用于表示矩阵 A 中非零元素所在的行索引。	输入
pntrb	整型数组	长度为 m 的数组，包含矩阵 A 的列索引， $\text{pntrb}[i] - \text{pntrb}[0]$ 表示第 i 列第一个非零元素在 val 和 indx 数组内的下标。	输入

参数名	类型	描述	输入/输出
pntrc	整型数组	长度为 m 的数组，包含矩阵 A 的列索引， $pntrc[i] - pntrb[0]-1$ 表示第 i 列最后一个非零元素在 val 和 $indx$ 数组内的下标。	输入
b	在 scscmm 中， b 是单精度浮点数组。	矩阵 B 的 value 数组。	输入
	在 dcscmm 中， b 是双精度浮点数组。		
	在 ccscmm 中， b 是单精度复数数组。		
	在 zcscmm 中， b 是双精度复数数组。		
ldb	整型数组	矩阵 $indx$ 从 1 开始时，矩阵 B 的主维度大小。 矩阵 $indx$ 从 0 开始时，矩阵 B 的第二维度大小。	输入
beta	在 scscmm 中， β 是单精度浮点类型。	标量 β 。	输入
	在 dcscmm 中， β 是双精度浮点类型。		
	在 ccscmm 中， β 是单精度复数。		
	在 zcscmm 中， β 是双精度复数。		
c	在 scscmm 中， c 是单精度浮点数组。	矩阵 C 的 value 数组。	输入/输出
	在 dcscmm 中， c 是双精度浮点数组。		
	在 ccscmm 中， c 是单精度复数数组。		

参数名	类型	描述	输入/输出
	在 <code>zcscmm</code> 中， <code>c</code> 是双精度复数数组。		
<code>ldc</code>	矩阵 <code>C</code> 的 <code>value</code> 数组。	矩阵 <code>indx</code> 从 1 开始时，矩阵 <code>C</code> 的主维度大小。 矩阵 <code>indx</code> 从 0 开始时，矩阵 <code>C</code> 的第二维度大小。	输入
注意： 函数内部不对参数做完整校验，接口调用者需保证 <code>pntbr</code> 和 <code>pntre</code> 内部元素不超出矩阵最大索引值。			

B 矩阵参数约束如下所示

opt	B 矩阵规模	B 矩阵数据排布	参数范围
$\text{op}(A) = A$	$k * n$	行主序	$k * \text{ldbMAX_INT}$
$\text{op}(A) = A$	$k * n$	列主序	$\text{ldb} * n\text{MAX_INT}$
$OP(A) = A^T \text{ or } A^H$	$m * n$	行主序	$m * \text{ldbMAX_INT}$
$OP(A) = A^T \text{ or } A^H$	$m * n$	列主序	$\text{ldb} * n\text{MAX_INT}$

C 矩阵参数约束如下所示。

opt	C 矩阵规模	C 矩阵数据排布	参数范围
$\text{op}(A) = A$	$k * n$	行主序	$k * \text{ldcMAX_INT}$
$\text{op}(A) = A$	$k * n$	列主序	$\text{ldc} * n\text{MAX_INT}$
$OP(A) = A^T \text{ or } A^H$	$m * n$	行主序	$m * \text{ldcMAX_INT}$
$OP(A) = A^T \text{ or } A^H$	$m * n$	列主序	$\text{ldc} * n\text{MAX_INT}$

2.4.2.16.3 返回值

函数执行状态，枚举类型 `sparse_status_t`。

2.4.2.16.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.16.5 示例

A. C interface:

```

sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;

INT m = 3;
INT n = 3;
INT k = 3;
float alpha = 1.0;
float beta = 1.0;
char *matdescra = "G00C"; // 一般矩阵, 基0索引
float val[4] = {8, 5, 2, 9};
INT indx[4] = {0, 1, 2, 0};
INT pntrb[3] = {0, 0, 1};
INT pntre[3] = {0, 1, 4};
float b[9] = {1, 5, 7, 4, 7, 7, 3, 3, 7};
float c[9] = {0, 8, 8, 0, 2, 8, 3, 5, 6};
INT ldb = 3;
INT ldc = 3;
sparse_status_t status = sparse_scscmm(opt, m, n, k, alpha,
matdescra, val, indx, pntrb, pntre, b, ldb, beta, c, ldc);
/*
 * Output c:
 *      59.00  91.00 127.00
15.00  17.00  43.00
9.00   11.00  20.00
 *
 * */

```

2.4.2.17 sparse_?cscsm

求解三角矩阵方程组计算，其中一个稀疏矩阵采用 CSC 格式存储，具体执行操作如下：

$$A * y = \alpha * x \text{ or } A^T * y = \alpha * x \text{ or } A^H * y = \alpha * x$$

其中，x 和 y 为 m*n 的稠密矩阵，A 是采用 CSC 格式存储的 m*m 的稀疏矩阵。矩阵 A 中的给定列的非零元素的存储顺序必须与列中显示的顺序相同（从上到下）。

layout 与 sparse matrix indexing 的关系如表 1 所示。

Sparse matrix indexing	Dense matrix layout
SPARSE_INDEX_BASE_ZERO	SPARSE_LAYOUT_ROW_MAJOR
SPARSE_INDEX_BASE_ONE	SPARSE_LAYOUT_COLUMN_MAJOR

2.4.2.17.1 接口定义

A. C interface:

```
sparse_status_t sparse_scscsm(const sparse_operation_t opt, const INT m, const INT n, const float alpha, const char *matdescra, const float *val, const INT *indx, const INT *pntrb, const INT *pntre, const float *x, const INT ldx, float *y, const INT ldy);
```

```
sparse_status_t sparse_dcscsm(const sparse_operation_t opt, const INT m, const INT n, const double alpha, const char *matdescra, const double *val, const INT *indx, const INT *pntrb, const INT *pntre, const double *x, const INT ldx, double *y, const INT ldy);
```

```
sparse_status_t sparse_ccscsm(const sparse_operation_t opt, const INT m, const INT n, const Complex8 alpha, const char *matdescra, const Complex8 *val, const INT *indx, const INT *pntrb, const INT *pntre, const Complex8 *x, const INT ldx, Complex8 *y, const INT ldy);
```

```
sparse_status_t sparse_zcscsm(const sparse_operation_t opt, const INT m, const INT n, const Complex16 alpha, const char *matdescra, const Complex16 *val, const INT *indx, const INT *pntrb, const INT *pntre, const Complex16 *x, const INT ldx, Complex16 *y, const INT ldy);
```

2.4.2.17.2 参数

参数名	类型	描述	输入/输出
-----	----	----	-------

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作， 如果 opt=SPARSE_OPERATION_NO N_TRANSPOSE，则 $A * y = \alpha * x$ 。 如果 opt=SPARSE_OPERATION_TR ANSPOSE，则 $A^T * y = \alpha * x$ 。 如果 opt=SPARSE_OPERATION_CO NJUGATE_TRANSPOSE，则 $A^H * y = \alpha * x$	输入
m	整型数	表示 A 矩阵的行数，取值范围 为[1, MAX_INT]。	输入
n	整型数	矩阵 C 的列数，取值范围为[1, MAX_INT]。	输入
alpha	在 scscsm 中，alpha 是 单精度浮点类型。 在 dcscsm 中，alpha 是双精度浮点类型。 在 ccscsm 中，alpha 是 单精度复数。 在 zcscsm 中，alpha 是 双精度复数。	标量 alpha。	输入
matdescra	char 指针	矩阵的操作属性，具体参考 matdescra 说明。	输入
val	在 scscsm 中，val 是单 精度浮点数组。 在 dcscsm 中，val 是 双精度浮点数组。 在 ccscsm 中，val 是 单精度复数数组。 在 zcscsm 中，val 是 双精度复数数组。	CSC 格式中 values 数组，存储 矩阵 A 的非零元素，长度为 pntre[m-1] - pntrb[0]。	输入

参数名	类型	描述	输入/输出
indx	整型数组	CSC 格式中 columns 数组，用于表示矩阵 A 中非零元素所在的行索引。	输入
pntrb	整型数组	长度为 m 的数组，包含矩阵 A 的列索引，pntrb[i] - pntrb[0] 表示第 i 列第一个非零元素在 val 和 indx 数组内的下标。	输入
pntrc	整型数组	长度为 m 的数组，包含矩阵 A 的列索引，pntrc[i] - pntrb[0]-1 表示第 i 列最后一个非零元素在 val 和 indx 数组内的下标。	输入
x	在 scscsm 中，x 是单精度浮点数组。	矩阵 x 的 value 数组。	输入
	在 dcscsm 中，x 是双精度浮点数组。		
	在 ccscsm 中，x 是单精度复数数组。		
	在 zcscsm 中，x 是双精度复数数组。		

参数名	类型	描述	输入/输出
ldx	整型数组	矩阵 indx 从 1 开始时, 矩阵 B 的主维度大小。 矩阵 indx 从 0 开始时, 矩阵 B 的第二维度大小。	输入
y	在 scscsm 中, y 是单精度浮点数组。	矩阵 y 的 value 数组。	输入/输出
	在 dcscsm 中, y 是双精度浮点数组。		
	在 ccscsm 中, y 是单精度复数数组。		
	在 zcscsm 中, y 是双精度复数数组。		
ldy	整型数组	矩阵 indy 从 1 开始时, 矩阵 C 的主维度大小。 矩阵 indy 从 0 开始时, 矩阵 C 的第二维度大小。	输入
注意: 函数内部不对参数做完整校验, 接口调用者需保证 pntrb 和 pntre 内部元素不超出矩阵最大索引值。			

B 矩阵参数约束如下所示

opt	B 矩阵规模	B 矩阵数据排布	参数范围
op(A) = A	$k * n$	行主序	$k * ldbMAX_INT$
op(A) = A	$k * n$	列主序	$ldb * nMAX_INT$
$OP(A) = A^T \text{ or } A^H$	$m * n$	行主序	$m * ldbMAX_INT$
$OP(A) = A^T \text{ or } A^H$	$m * n$	列主序	$ldb * nMAX_INT$

C 矩阵参数约束如下所示。

opt	C 矩阵规模	C 矩阵数据排布	参数范围
op(A) = A	$k * n$	行主序	$k * ldcMAX_INT$
op(A) = A	$k * n$	列主序	$ldc * nMAX_INT$
$OP(A) = A^T \text{ or } A^H$	$m * n$	行主序	$m * ldcMAX_INT$
$OP(A) = A^T \text{ or } A^H$	$m * n$	列主序	$ldc * nMAX_INT$

2.4.2.17.3 返回值

函数执行状态，枚举类型 `sparse_status_t`。

2.4.2.17.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.17.5 示例

A. C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;

INT m = 4;
INT n = 2;
float alpha = 1.0;
char *matdescra = "TLNC"; //三角矩阵, 基0索引
float val[9] = {2, 3, 1, 5, 7, 8, 3, 6, 7};
INT indx[9] = {0, 1, 2, 1, 1, 2, 3, 0, 3};
INT pntrb[4] = {0, 3, 4, 7};
INT pntre[4] = {3, 4, 7, 9};
float x[8] = {6, 7, 6, -3, 4, 6, 6, 8};
float y[8] = {0, 0, 0, 0, 0, 0, 0, 0};
INT ldx = 2;
INT ldy = 2;

sparse_status_t status = sparse_scscsm(opt, m, n, alpha, matdescra,
val, indx, pntrb, pntre, x, ldx, y, ldy);

/*
 * Output Y:
 *
 *      3.000000  3.500000
-0.600000 -2.700000
0.125000  0.312500
0.803571  1.008929
```

```

*
* */

```

2.4.2.18 sparse_?csrsm

求解三角矩阵方程组计算，其中一个稀疏矩阵采用 CSR 格式存储，具体执行操作如下：

$$A * y = \alpha * x \text{ or } A^T * y = \alpha * x \text{ or } A^H * y = \alpha * x$$

其中，x 和 y 为 mxn 的稠密矩阵，A 是采用 CSR 格式存储的 mxm 的稀疏矩阵。矩阵 A 中的给定行的非零元素的存储顺序必须与行中显示的顺序相同（从左到右）。

layout 与 sparse matrix indexing 的关系如表 1 所示。

Sparse matrix indexing	Dense matrix layout
SPARSE_INDEX_BASE_ZERO	SPARSE_LAYOUT_ROW_MAJOR
SPARSE_INDEX_BASE_ONE	SPARSE_LAYOUT_COLUMN_MAJOR

2.4.2.18.1 接口定义

A. C interface:

```

sparse_status_t sparse_scsrsm(const sparse_operation_t opt, const INT m, const INT n,
const float alpha, const char *matdescra, const float *val, const INT *indx, const INT
*pntrb, const INT *pntrc, const float *x, const INT ldx, float *y, const INT ldy);

```

```

sparse_status_t sparse_dcsrsm(const sparse_operation_t opt, const INT m, const INT
n, const double alpha, const char *matdescra, const double *val, const INT *indx,
const INT *pntrb, const INT *pntrc, const double *x, const INT ldx, double *y, const
INT ldy);

```

```

sparse_status_t sparse_ccsrsm(const sparse_operation_t opt, const INT m, const INT
n, const Complex8 alpha, const char *matdescra, const Complex8 *val, const INT
*indx, const INT *pntrb, const INT *pntrc, const Complex8 *x, const INT ldx,
Complex8 *y, const INT ldy);

```

```

sparse_status_t sparse_zcsrsm(const sparse_operation_t opt, const INT m, const INT
n, const Complex16 alpha, const char *matdescra, const Complex16 *val, const INT
*indx, const INT *pntrb, const INT *pntrc, const Complex16 *x, const INT ldx,
Complex16 *y, const INT ldy);

```

2.4.2.18.2 参数

参数名	类型	描述	输入/输出
opt	枚举类型 sparse_operation_t	表示矩阵 A 的操作， 如果 opt=SPARSE_OPERATION_NO N_TRANSPOSE，则 $A * y = \alpha * x$ 。 如果 opt=SPARSE_OPERATION_TR ANSPOSE，则 $A^T * y = \alpha * x$ 。 如果 opt=SPARSE_OPERATION_CO NJUGATE_TRANSPOSE，则 $A^H * y = \alpha * x$	输入
m	整型数	表示 A 矩阵的行数，取值范围 为[1, MAX_INT]。	输入
n	整型数	矩阵 C 的列数，取值范围为[1, MAX_INT]。	输入
alpha	在 scsrsm 中，alpha 是 单精度浮点类型。 在 dcsrsm 中，alpha 是 双精度浮点类型。 在 ccsrsm 中，alpha 是 单精度复数。 在 zcsrsm 中，alpha 是 双精度复数。	标量 alpha。	输入
matdescra	char 指针	矩阵的操作属性，具体参考 matdescra 说明。	输入
val	在 scsrsm 中，val 是单 精度浮点数组。 在 dcsrsm 中，val 是双 精度浮点数组。 在 ccsrsm 中，val 是单 精度复数数组。	CSC 格式中 values 数组，存储 矩阵 A 的非零元素，长度为 pntre[m-1] - pntrb[0]。	输入

参数名	类型	描述	输入/输出
	在 <code>zcsrsm</code> 中, <code>val</code> 是双精度复数数组。		
<code>indx</code>	整型数组	CSC 格式中 <code>columns</code> 数组, 用于表示矩阵 <code>A</code> 中非零元素所在的行索引。	输入
<code>pntrb</code>	整型数组	长度为 <code>m</code> 的数组, 包含矩阵 <code>A</code> 的列索引, <code>pntrb[i] - pntrb[0]</code> 表示第 <code>i</code> 列第一个非零元素在 <code>val</code> 和 <code>indx</code> 数组内的下标。	输入
<code>pntre</code>	整型数组	长度为 <code>m</code> 的数组, 包含矩阵 <code>A</code> 的列索引, <code>pntre[i] - pntrb[0]-1</code> 表示第 <code>i</code> 列最后一个非零元素在 <code>val</code> 和 <code>indx</code> 数组内的下标。	输入
<code>x</code>	在 <code>scsrsm</code> 中, <code>x</code> 是单精度浮点数组。	矩阵 <code>x</code> 的 <code>value</code> 数组。	输入
	在 <code>dcsrcsm</code> 中, <code>x</code> 是双精度浮点数组。		
	在 <code>ccsrcsm</code> 中, <code>x</code> 是单精度复数数组。		
	在 <code>zccsrcsm</code> 中, <code>x</code> 是双精度复数数组。		

参数名	类型	描述	输入/输出
ldx	整型数组	矩阵 indx 从 1 开始时, 矩阵 B 的主维度大小。 矩阵 indx 从 0 开始时, 矩阵 B 的第二维度大小。	输入
y	在 scsrsm 中, y 是单精度浮点数组。	矩阵 y 的 value 数组。	输入/输出
	在 dcsrsm 中, y 是双精度浮点数组。		
	在 ccsrsm 中, y 是单精度复数数组。		
	在 zcsrsm 中, y 是双精度复数数组。		
ldy	整型数组	矩阵 indy 从 1 开始时, 矩阵 C 的主维度大小。 矩阵 indy 从 0 开始时, 矩阵 C 的第二维度大小。	输入
注意: 函数内部不对参数做完整校验, 接口调用者需保证 pntrb 和 pntre 内部元素不超出矩阵最大索引值。			

B 矩阵参数约束如下所示

opt	B 矩阵规模	B 矩阵数据排布	参数范围
op(A) = A	k * n	行主序	k * ldbMAX_INT
op(A) = A	k * n	列主序	ldb * nMAX_INT
$OP(A) = A^T \text{ or } A^H$	m * n	行主序	m * ldbMAX_INT
$OP(A) = A^T \text{ or } A^H$	m * n	列主序	ldb * nMAX_INT

C 矩阵参数约束如下所示。

opt	C 矩阵规模	C 矩阵数据排布	参数范围
op(A) = A	k * n	行主序	k * ldcMAX_INT
op(A) = A	k * n	列主序	ldc * nMAX_INT
$OP(A) = A^T \text{ or } A^H$	m * n	行主序	m * ldcMAX_INT
$OP(A) = A^T \text{ or } A^H$	m * n	列主序	ldc * nMAX_INT

2.4.2.18.3 返回值

函数执行状态，枚举类型 `sparse_status_t`。

2.4.2.18.4 依赖

C: "kspblas.h"

Fortran: "kspblas.f03"

2.4.2.18.5 示例

A. C interface:

```
sparse_operation_t opt = SPARSE_OPERATION_NON_TRANSPOSE;

INT m = 4;
INT n = 2;
float alpha = 1.0;
char *matdescra = "TLNF"; //三角矩阵, 基1 索引
float val[9] = {2, -3, 7, 1, -6, 8, -4, 5, 9};
INT indx[9] = {1, 2, 4, 2, 4, 1, 3, 1, 4};
INT pntrb[4] = {1, 4, 6, 8};
INT pntre[4] = {4, 6, 8, 10};
float x[8] = {1, 3, -2, 5, 7, -3, 6, 8};
float y[8] = {0, 0, 0, 0, 0, 0, 0, 0};
INT ldx = 4;
INT ldy = 4;

sparse_status_t status = sparse_scsrsm(opt, m, n, alpha, matdescra,
val, indx, pntrb, pntre, x, ldx, y, ldy);

/*
 * Output Y:
 *
 *      0.500000  3.000000  1.500000  0.277778
3.500000 -3.000000  5.500000 -1.055556
 *
 * */
```

2.4.3 Support 函数

2.4.3.1 set_thread_num

设置 Sparse BLAS 库接口函数执行的线程数（此函数不可重入）。

2.4.3.1.1 接口定义

A. C interface:

```
sparse_status_t set_thread_num(const int thread_num);
```

B. Fortran interface:

```
RES = SET_THREAD_NUM(THREAD_NUM);
```

2.4.3.1.2 参数

参数名	类型	描述	输入/输出
thread_num	整型	线程数，取值范围为 1~256。	输入

2.4.3.1.3 返回值

函数执行状态，枚举类型 sparse_status_t。

2.4.3.2 KSPBLASGetVersion

2.4.3.2.1 获取 SPBLAS 产品版本信息

```
int KSPBLASGetVersion(KBLASVersion *ver)。
```

2.4.3.2.2 参数

参数名	描述	取值范围	输入/输出
ver	产品信息结构体。	非空	输出

2.4.3.2.3 返回值

成功：返回 1。

失败：返回 0。

2.4.3.2.4 示例

```
#include <stdio.h>
#include "kspblas.h"

int main()
{
    KSPBLASVersion* ver =
(KSPBLASVersion*)malloc(sizeof(KSPBLASVersion));

    int status = KSPBLASGetVersion(ver);

    printf("Product Name           : %s\n", ver->productName);
    printf("Product Version          : %s\n", ver->productVersion);
    printf("Component name             : %s\n", ver->componentName);
    printf("Component Version          : %s\n", ver->componentVersion);
    printf("Component AppendInfo       : %s\n", ver->componentAppendInfo);
    printf("Software Name              : %s\n", ver->softwareName);
    printf("Software Version           : %s\n", ver->softwareVersion);
    free(ver);
    return 0;
}
```

运行结果：

```
Product Name           : Kunpeng Boostkit
Product Version          : 23.0.RC2
Component name             : BoostKit-kml
Component Version          : 1.8.0
Component AppendInfo       : gcc
Software Name              : boostkit-kml-spbblas
Software Version           : 1.8.0
```

3. 算法替换案例

科学计算与系统建模仿真平台基于 Julia 语言开发的高性能科学计算软件，其中默认线性代数库底层算法程序为 OpenBLAS，同时 Julia 中提供 libblastrampoline 作为"跳板"机制，可以实现外部 BLAS/LAPACK 库的识别、替换操作。主要替换步骤如下：



图 3-1 科学计算算法替换流程

BLAS（英语：Basic Linear Algebra Subprograms，基础线性代数程序集）是一个应用程序接口（API）标准，用以规范发布基础线性代数操作的数值库（如矢量或矩阵乘法）。该程序集最初发布于 1979 年，并用于创建更大的数值程序包（如 LAPACK）。在高性能计算领域，BLAS 被广泛使用。例如，LINPACK 的运算成绩则很大程度上取决于 BLAS 中子程序 DGEMM 的表现。

为提高性能，各软硬件厂商则针对其产品对 BLAS 接口实现进行高度优化。目前已有相关 BLAS 库实现如下：

- Netlib BLAS：官方参考实现，程序语言为 Fortran 77

- ACML (AMD Core Math Library): 厂商 AMD 的 BLAS 实现。
- ATLAS: BSD 许可证开源的 BLAS 实现。
- CUDA SDK: NVIDIA CUDA SDK 包含了 BLAS 功能, 通过 C 编程实现在 GeForce 8 系列或更新一代显卡上运行。
- GotoBLAS: 德克萨斯高级计算中心后藤和茂开发的 BSD 许可证开源的 BLAS 实现, 但已停止了活跃开发, 后继者为 OpenBLAS。
- OpenBLAS: 继任 GotoBLAS 的开源 BLAS 的实现, 主要由中国科学院软件研究所并行软件与计算科学实验室进行开发。
- ESSL: IBM 的科学工程数值库 ESSL, 支持 AIX 和 Linux 系统下的 PowerPC 架构。
- Intel MKL: Intel 核心数学库, 支持 Pentium, Intel Core 与 Itanium CPU 系列。实现平台包括 Linux, Windows 及 OS X。
- GSL: GNU 科学数值库 (GNU Scientific Library) 包含了 GNU 下的多平台 C 语言实现。
- RenderScript IntrinsicBLAS: 基于 Renderscript 的 Android 移动终端高性能 BLAS 实现。

此处主要以 OpenBLAS 库为案例进行替换。

接下来以 OpenBLAS 开源库作为示例展示如何切换 BLAS/LAPACK。

3.1 替换案例一: OpenBLAS@0.3.21

该案例展示在 windows 操作系统上, 编译 OpenBLAS 库并在 Syslab 中切换底层算法, 由于 OpenBLAS 默认将 BLAS/LAPACK 打包, 所以该案例同样适用于 LAPACK 替换。

为了编译 OpenBLAS 开源库并获取所需的动态库, 需先进行环境搭建。

A. 下载并安装 MSYS

Minimal GNU (POSIX) system on Windows, 是 MinGW 提供的一个小型的 GNU 环境, 包括基本的 bash, make 等等。与 Cygwin 大致相当。简单说 MSYS 就相当于一个在 Windows 下运行的 linux bash shell 环境, 支持绝大部分 linux 常用命令。而 MSYS2 是一个独立项目, 它重写了 MSYS, MSYS2 安装更简单, 使用更方便, 还提供 pacman 工具进行软件包的安装管理(就像 ubuntu 的 apt-get, centos 的 yum)。

下载地址: <https://repo.msys2.org/distrib/>

Index of /distrib/

../	18-Sep-2022 07:25	-
i686/	13-Feb-2023 08:28	-
x86_64/	05-Jul-2021 20:58	70M
msys2-i686-latest.sfx.exe	06-Jul-2021 16:24	566
msys2-i686-latest.sfx.exe.sig	05-Jul-2021 20:58	84M
msys2-i686-latest.tar.xz	06-Jul-2021 16:24	566
msys2-i686-latest.tar.xz.sig	27-Jan-2023 10:22	84M
msys2-x86_64-latest.exe	27-Jan-2023 10:22	566
msys2-x86_64-latest.exe.sig	27-Jan-2023 10:22	59M
msys2-x86_64-latest.sfx.exe	27-Jan-2023 10:22	566
msys2-x86_64-latest.sfx.exe.sig	27-Jan-2023 10:22	56M
msys2-x86_64-latest.tar.xz	27-Jan-2023 10:22	566
msys2-x86_64-latest.tar.xz.sig		

图 3-2 下载 MSYS

选择 msys2-x86_64-latest.exe 进行安装。
安装完成后，点击如下图标进行后续操作：



图 3-3 运行 MSYS2

B. 安装 MinGW gcc 编译器

输入命令安装 gcc 编译器：

```
pacman -S mingw-w64-x86_64-gcc //64 位
pacman -S mingw-w64-i686-gcc // 32 位
```

C. 安装 fortran 编译器

安装后，才可编译 LAPACK。

安装命令如下：

```
pacman -S mingw-w64-x86_64-gcc-fortran
```

D. 安装 perl

安装命令如下：

```
pacman -S --noconfirm perl
```

E. 安装 MSYS2 的 make

安装命令如下：

```
pacman -S make
```

F. 下载 OpenBLAS 源码

由于 Syslab 使用的 Julia 默认使用 64 位整数字长，所以要求 BLAS 库进行编译成 64 位整数字长，即 ILP64。这里选用最新发布的 OpenBLAS 作为替换目标。

详情请见：<https://github.com/xianyi/OpenBLAS/releases/tag/v0.3.21> %

源码下载地址：

<https://github.com/xianyi/OpenBLAS/releases/download/v0.3.21/OpenBLAS-0.3.21.zip> %

下载并解压后如下所示：

苏州同元软控信息投资有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。

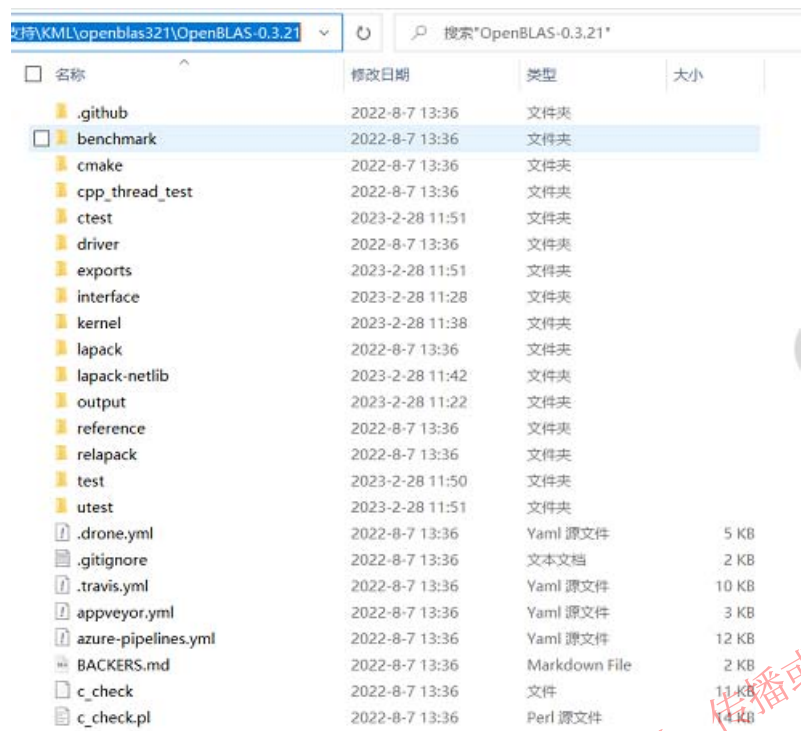


图 3-4 下载源码

G. 编译 OpenBLAS

运行下图所示的 MYSY2 MinGW64/32(根据系统选择), 解压下载好的 OpenBLAS 源码, 并利用 cd 命令切换至目标文件夹路径。

```
cd path_to_OpenBLAS
```

在通常情况下我们只需输入 make 即可开始编译, 但我们需要得到 ILP64 的 BLAS 库, 所以针对 OpenBLAS 我们需要利用如下方法进行编译:

```
make BINARY=64 INTERFACE64=1
```

编译结束后可在当前文件夹下找到 libopenblas.dll 文件如下:

名称	修改日期	类型	大小
Jenkinsfile	2022-8-7 13:36	文件	1 KB
l1param.h	2022-8-7 13:36	C/C++ Header	3 KB
l2param.h	2022-8-7 13:36	C/C++ Header	4 KB
lib.grd	2023-2-28 11:42	GRD 文件	0 KB
libopenblas.dll	2023-2-28 11:42	应用程序扩展	16,342 KB
libopenblas.dll.a	2023-2-28 11:42	A 文件	5,621 KB
libopenblas_haswellp-r0.3.21.a	2023-2-28 11:42	A 文件	21,384 KB
LICENSE	2022-8-7 13:36	文件	2 KB
make.inc	2022-8-7 13:36	Include File	1 KB
Makefile	2022-8-7 13:36	文件	12 KB
Makefile.alpha	2022-8-7 13:36	ALPHA 文件	2 KB
Makefile.arm	2022-8-7 13:36	ARM 文件	1 KB
Makefile.arm64	2022-8-7 13:36	ARM64 文件	7 KB
Makefile.conf	2023-2-28 11:25	CONF 文件	2 KB
Makefile.conf_last	2023-2-28 11:42	CONF_LAST 文件	2 KB
Makefile.e2k	2022-8-7 13:36	E2K 文件	1 KB
Makefile.generic	2022-8-7 13:36	GENERIC 文件	1 KB
Makefile.ia64	2022-8-7 13:36	IA64 文件	1 KB
Makefile.install	2022-8-7 13:36	INSTALL 文件	10 KB
Makefile.loongarch64	2022-8-7 13:36	LOONGARCH64 ...	1 KB
Makefile.mips	2022-8-7 13:36	MIPS 文件	1 KB
Makefile.mips64	2022-8-7 13:36	MIPS64 文件	1 KB
Makefile.power	2022-8-7 13:36	POWER 文件	5 KB
Makefile.prebuild	2022-8-7 13:36	PREBUILD 文件	3 KB

图 3-5 编译成功的 dll 库

3.1.2 切换 BLAS/LAPACK 库

A. 替换算法库

打开 Syslab 后在 REPL 中利用 “BLAS.lbt_forward” 函数对目标动态链接库进行替换，输入如下代码进行替换，其中 “verbose = true” 表示打印替换信息，“clear = true” 表示清除其他 BLAS/LAPACK 库。

```
newblaspath = "path_to_newblasdll"
BLAS.lbt_forward(newblaspath,clear = true,verbose = true)

#=
Generating forwards to D:/KML/openblas321/OpenBLAS-0.3.21/libopenblas.dll
-> Autodetected symbol suffix ""
-> Autodetected interface ILP64 (64-bit)
-> Autodetected gfortran calling convention
Processed 4945 symbols; forwarded 4860 symbols with 64-bit interface and mangling
to a suffix of ""
4860
=#
```


B. 查看当前算法库

打开 Syslab 后在 REPL 中输入如下代码查看当前作用的底层算法库：

```
using TyMath
BLAS.get_config()
#=
julia> BLAS.get_config()
LinearAlgebra.BLAS.LBTConfig
Libraries:
└ [ILP64] libopenblas64_.dll
=#
```

3.2 替换案例二：BLIS

该案例讲解 BLIS 库在 linux 系统上编译并替换过程。

3.2.1 编译 BLIS 库

具体编译流程，详情请见：

<https://github.com/flame/blis#how-to-download-blis>

注意：要想获取满足 ILP64 规范的 BLIS 库请使用 -b 64 选项设置。

3.2.2 替换 BLIS 库

通过上述步骤编译成功后，在 lib 文件夹能找到动态链接库 libblis.so, 获取其绝对路径 libblispath。

A. 替换算法库

打开 Syslab 后在 REPL 中利用 “BLAS.lbt_forward” 函数对目标动态链接库进行替换，输入如下代码进行替换，其中 “verbose = true” 表示打印替换信息，“clear = true” 表示清除其他 BLAS/LAPACK 库。

```
newblaspath = "path_to_newblasdll"
BLAS.lbt_forward(newblaspath, clear = true, verbose = true)
```



```
julia> libblispath = "/home/lius/桌面/BLAS/blis-0.9.0/lib/haswell/libblis.so"
"/home/lius/桌面/BLAS/blis-0.9.0/lib/haswell/libblis.so"

julia> BLAS.lbt_forward(libblispath,clear = true,verbose = true)
Generating forwards to /home/lius/桌面/BLAS/blis-0.9.0/lib/haswell/libblis.so (c
lear: 1, verbose: 1, suffix_hint: '(null)')
-> Autodetected symbol suffix ""
-> Autodetected interface ILP64 (64-bit)
-> Autodetected normal complex return style
-> Autodetected gfortran calling convention
-> CBLAS not found
Processed 4949 symbols; forwarded 161 symbols with 64-bit interface and mangling
to a suffix of ""
161
```

B. 查看当前算法库

打开 Syslab 后在 REPL 中输入如下代码查看当前作用的底层算法库：

```
BLAS.get_config()

julia> BLAS.get_config()
LinearAlgebra.BLAS.LBTConfig
Libraries:
└ [ILP64] libblis.so
```

3.3 KMLBLAS 测试报告

接下来将以 KML 中 BLAS 算法库作为目标，测试其功能和性能。

3.3.1 测试环境

表 3-1 KML 验证环境

处理器	Architecture:	aarch64
	CPU op-mode(s):	64-bit
	Byte Order:	Little Endian
	CPU(s):	16
	On-line CPU(s) list:	0-15
	Thread(s) per core:	1
	Core(s) per socket:	16
	Socket(s):	1
	NUMA node(s):	1
	Vendor ID:	HiSilicon
	Model:	0

	Model name: Kunpeng-920 Stepping: 0x1 CPU max MHz: 2400.0000 CPU min MHz: 2400.0000 BogoMIPS: 200.00 L1d cache: 1 MiB L1i cache: 1 MiB L2 cache: 8 MiB L3 cache: 32 MiB NUMA node0 CPU(s): 0-15
内存	126G
操作系统	麒麟 V10

3.3.2 测试

3.3.2.1 测试方法

1. 利用 Julia 提供 `ccall` 函数对 KMLBLAS 提供动态链接库内函数进行直接调用并测试性能
2. 利用 `BLAS.lbt_forward` 函数进行底层替换，通过外部 `mul!`、`*`、`norm` 等函数接口进行测试验证

3.3.2.2 测试对象与标准选取

KML 中提供 LP64(32 位)BLAS 库，不支持直接传入 Float64 数进行计算和，经过调研和测试发现在选用测试方法一时，创建数据类型为 Float32 的向量或矩阵进行计算，在选用测试方法二时，可以创建数据类型为 UInt32 的向量或矩阵进行计算，故测试对象选取为：

1. 数据类型为 Float32 的向量或矩阵，且使用 `s` 前缀标记为单精度浮点数进行计算，如：`sgemm_`、`ssymm_` 等
2. 数据类型为 UInt32 的向量或矩阵

利用 BenchmarkTools 库提供的 `@btime` 宏对测试用例进行性能测试并对比结果。

3.3.3 函数列表

针对 KML 提供的动态链接库，对其中部分函数进行测试，测试列表如下

表所示。

表 3-2 动态链接库测试目标函数功能列表

编号	函数名称	功能定义
1.	GEMM	通用矩阵乘法(General Matrix Multiply)
2.	SYMM	对称矩阵乘法(SYMMetric Matrix Multiply)
3.	SYR2K	对称矩阵乘法的双重乘积(SYmmetric Rank-2K update)
4.	TRMM	三角矩阵与一般矩阵的乘法(Triangular Matrix Matrix Multiply)
5.	TRSM	三角矩阵与多个右侧向量或矩阵的线性方程组(TRiangular Solve with Multiple right-hand sides)
6.	ROTG	计算 Givens 旋转矩阵(ROtate Givens)
7.	ROTMG	特定的 Givens 变换(ROtate Matrix Givens)
8.	ROT	计算多维反离散余弦变换阵列 A 的 (DCT) (Double ROtate)
9.	ROTM	执行数组 A 类型的多维实数输入/实数输出 (r2r) 变换 (Double ROtate Matrix)
10.	SCAL	将一个向量的每个元素乘以一个标量值
11.	AXPY	将一个向量乘以一个标量并加到另一个向量上 (Alpha X Plus Y)
12.	DOT	向量内积函数 (Dot Product)
13.	NRM2	计算向量二范数 (Norm 2)
14.	ASUM	向量的绝对值之和函数(计算向量的 L1 范数(曼哈顿范数)) (Absolute Sum)
15.	IDAMAX	向量最大值索引 (Index of Absolute Maximum)
16.	GEMV	矩阵向量乘法 (Generalized Matrix-Vector)

17.	GBMV	带状矩阵和向量之间的乘法 (Generalized Banded Matrix-Vector)
18.	SYMV	对称矩阵向量乘法 (Symmetric Matrix-Vector)
19.	SBMV	对称带状矩阵向量乘法 (Scaled Band Matrix-Vector)
20.	SPMV	稀疏矩阵向量乘法 (Sparse Matrix-Vector)
21.	TRMV	上/下三角矩阵向量乘法 (Triangular Matrix-Vector)
22.	TRSV	上/下三角矩阵求解函数, 用于解三角线性方程组 (Triangular Solve Vector)
23.	GER	计算两个向量的外积或外积更新 (Generalized Rank-1 Update)
24.	SYR	对称矩阵外积更新 (Symmetric Rank-1)
25.	SYR2K	对称矩阵的秩-2K 更新 (Symmetric Rank-2K)

表 3-3 BLAS 功能 Julia 函数列表

编号	函数名称	功能定义
1.	*	矩阵乘法 (Matrix Multiplication)
2.	*	向量乘法 (Vector Multiplication)
3.	norm	范数 (Norm)
4.	\	矩阵左除 (Matrix Left Division)
5.	symm	对称矩阵和一般矩阵的乘法运算 (Symmetric Matrix Multiplication)
6.	symm	对称矩阵和一般矩阵的乘法运算 (Symmetric Matrix Multiplication)
7.	SYR2K	对称矩阵与一般矩阵的乘法和加法运算 (Symmetric Rank-2K)

8.	TRMM	三角矩阵与一般矩阵的乘法 (Triangular Matrix-Matrix Multiplication)
9.	TRMM	三角矩阵与一般矩阵的乘法 (Triangular Matrix-Matrix Multiplication)
10.	scal	向量进行标量缩放 (Scale)
11.	Copy	将一个向量的元素复制到另一个向量中 (Copy)
12.	AXPY	向量乘以一个标量并与另一个向量相加 (A times X plus Y)
13.	DOT	向量的点积运算 (Dot product)
14.	NRM2	向量的二范数计算 (2-Norm)
15.	ASUM	向量的绝对值求和 (Absolute Sum)
16.	GEMV	一般矩阵与向量的乘法 (General Matrix-Vector Multiplication)
17.	symv	对称矩阵与向量的乘法 (Symmetric Matrix-Vector Multiplication)
18.	SPMV	稀疏矩阵与向量的乘法 (Sparse Matrix-Vector Multiplication)
19.	TRMV	三角矩阵与向量的乘法 (Triangular Matrix-Vector multiplication)
20.	TBMV	带状三角矩阵与向量的乘法 (Triangular Banded Matrix-Vector multiplication)
21.	GER	一般矩阵的秩-1 更新 (General Matrix Rank-1 Update)
22.	SYR	对称矩阵的秩-1 更新 (Symmetric Matrix Rank-1 Update)

3.3.4 单元测试结果及对比

如表 3-1 所示，展示了 KML 与 OpenBLAS 在相同示例情况下的性能对比，其中红色标记字体表示性能占优的数据。

表 3-4 动态链接库函数性能对比测试

编号	函数	示例	KML	OpenBLAS
1.	GEMM	<pre> rng = MersenneTwister(1234) A = rand(rng,Float32,1000,1000) B = rand(rng,Float32,1000,1000) C = zeros(Float32,1000,1000) transA = 'N' transB = 'N' m = size(A, transA == 'N' ? 1 : 2) ka = size(A, transA == 'N' ? 2 : 1) kb = size(B, transB == 'N' ? 1 : 2) n = size(B, transB == 'N' ? 2 : 1) alpha = Float32(1.0) beta = Float32(0.0) @btime ccall(:sgemm_,pwd() * libblaspath),Cvoid,(Ref{UInt8}, Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt}, Ref{Float32}, Ptr{Float32}, Ref{BLAS.BlasInt},Ptr{Float32}, Ref{BLAS.BlasInt}, Ref{Float32}, Ptr{Float32}, Ref{BLAS.BlasInt}, Clong, Clong),transA, transB,m,n,ka,alpha,A,max(1, stride(A,2)),B,max(1, stride(B,2)), beta,C,max(1, stride(C,2)),1,1); </pre>	4.263 ms (37 allocations: 592 bytes)	10.305 ms (37 allocations: 592 bytes)
2.	SYMM	<pre> side = 'L' uplo = 'L' m, n = size(C) elty =Float32 alpha = Float32(1.0) </pre>	13.200 μs (23 allocations: 368 bytes)	13.150 μs (23 allocations: 368 bytes)

		<pre> beta = Float32(0.0) rng = MersenneTwister(1234) A = rand(rng,Float32,3,4) B = rand(rng,Float32,4,2) C = zeros(Float32,3,2) ccall(:ssymm_, libblaspath), Cvoid, (Ref{UInt8}, Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt},Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty},Ref{BLAS.BlasInt}, Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt},Clong, Clong), side, uplo, m, n, Float32(alpha), A, max(1, stride(A,2)), B, max(1, stride(B,2)), Float32(beta), C, max(1, stride(C,2)), 1, 1) </pre>		
3.	SYR2K	<pre> side = 'L' uplo = 'N' m, n = size(C) elty = Float32 alpha = Float32(1.0) beta = Float32(0.0) rng = MersenneTwister(1234) A = rand(rng,Float32,3,4) B = rand(rng,Float32,4,2) C = zeros(Float32,3,2) ccall(:ssyr2k_, libblaspath), Cvoid, (Ref{UInt8}, Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt},Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty},Ref{BLAS.BlasInt}, Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt},Clong, Clong), side, uplo, m, n, Float32(alpha), A, max(1, stride(A,2)), B, max(1, stride(B,2)), Float32(beta), C, max(1, stride(C,2)), 1, 1) </pre>	/	13.311 μ s (23 allocations: 368 bytes)
4.	TRMM	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) </pre>	9.710 μ s (17 allocations: 272 bytes	10.041 μ s (17 allocations:

		<pre> A = rand(rng,Float32,30,30) A = diag(A) B = rand(rng,Float32,30,30) C = zeros(Float32,3,2) side = 'L' uplo = 'U' m, n = size(C) elty = Float32 alpha = Float32(1.0) beta = Float32(0.0) ccall(:strmm_, libblaspath), Cvoid, (Ref{UInt8}, Ref{UInt8}, Ref{UInt8}, Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt},Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty},Ref{BLAS.BlasInt}, Clong, Clong,Clong, Clong), side, uplo,'C','N', m, n,Float32(alpha), A, max(1, stride(A,2)), B, max(1, stride(B,2)),1, 1, 1, 1) </pre>		272 bytes)
5.	TRSM	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,30,30) A = diag(A) B = rand(rng,Float32,30,30) C = zeros(Float32,3,2) side = 'L' uplo = 'U' m, n = size(C) elty = Float32 alpha = Float32(1.0) beta = Float32(0.0) ccall(:strsm_, libblaspath), Cvoid,(Ref{UInt8}, Ref{UInt8}, Ref{UInt8}, Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt},Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty},Ref{BLAS.BlasInt}, Clong, Clong,Clong, Clong), side, uplo,'C','N', m, n, Float32(alpha), A, max(1, stride(A,2)), B, max(1, stride(B,2)),1, 1, 1, 1) </pre>	10.100 μ s (17 allocations: 272 bytes)	10.181 μ s (17 allocations: 272 bytes)

6.	ROTG	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300) B = rand(rng,Float32,300) c = rand(rng, Float32, 300) s = rand(rng, Float32, 300) elty = Float32 @btime ccall((:srotg_, libblaspath), Cvoid, (Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ref{elty}, Ref{elty})),1, A, 1, B, 1, c, s) </pre>	5.038 μ s (6 allocations: 128 bytes)	5.027 μ s (6 allocations: 128 bytes)
7.	ROTMG	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300) B = rand(rng,Float32,300) c = rand(rng, Float32, 300) s = rand(rng, Float32, 300) elty = Float32 @btime ccall((:srotmg_, libblaspath), Cvoid, (Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ref{elty}, Ref{elty})),1, A, 1, B, 1, c, s) </pre>	5.012 μ s (6 allocations: 128 bytes)	5.038 μ s (6 allocations: 128 bytes)
8.	ROT	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300) B = rand(rng,Float32,300) c = rand(rng, Float32, 300) s = rand(rng, Float32, 300) elty = Float32 ccall((:srot_, libblaspath), Cvoid, (Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ref{elty}, Ref{elty})),1, A, 1, B, 1, c, s) </pre>	5.075 μ s (6 allocations: 128 bytes)	5.058 μ s (6 allocations: 128 bytes)
9.	ROTM	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300) </pre>	5.027 μ s (6 allocations: 128 bytes)	4.969 μ s (6 allocations: 128 bytes)

		<pre> B = rand(rng,Float32,300) c = rand(rng, Float32, 300) s = rand(rng, Float32, 300) elty = Float32 @btime ccall(:srotm_, libblaspath), Cvoid, (Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ref{elty}, Ref{elty}),1, A, 1, B, 1, c, s) </pre>		
10.	SCAL	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300) elty = Float32 @time ccall(:sscal_, libblaspath), Cvoid, (Ref{BLAS.BlasInt}, Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt}),length(A),Fl oat32(5),A,stride(A,1)); </pre>	0.000039 seconds (8 allocations: 128 bytes)	0.000032 seconds (8 allocations: 128 bytes)
11.	AXPY	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300) B = rand(rng,Float32,300) elty = Float32 @btime ccall(:saxpy_, libblaspath), Cvoid, (Ref{BLAS.BlasInt}, Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}),length(A),Fl oat32(0.5),A,stride(A,1),B,stri de(B,1)); </pre>	5.787 μ s (11 allocations: 176 bytes)	5.587 μ s (11 allocations: 176 bytes)
12.	DOT	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300) B = rand(rng,Float32,300) elty = Float32 @time ccall(:saxpy_, libblaspath), Cvoid, (Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}),length(A),A, </pre>	/	/

		<code>stride(A,1),B, stride(B,1));</code>		
13.	NRM2	<pre>using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300) B = rand(rng,Float32,300) elty = Float32 @btime ccall(:snrm2_, libblaspath), elty, (Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}),length(A),A, stride(A,1));</pre>	3.423 μ s (5 allocations: 80 bytes)	4.015 μ s (5 allocations: 80 bytes)
14.	ASUM	<pre>using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300) B = rand(rng,Float32,300) elty = Float32 @btime ccall(:sasum_, libblaspath), elty, (Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}),length(A),A, stride(A,1));</pre>	2.859 μ s (5 allocations: 80 bytes)	2.836 μ s (5 allocations: 80 bytes)
15.	IDAMAX	<pre>using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300) B = rand(rng,Float32,300) elty = Float32 @btime ccall(:isamax_, libblaspath), elty, (Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}),length(A),A, stride(A,1));</pre>	2.933 μ s (5 allocations: 80 bytes)	2.979 μ s (5 allocations: 80 bytes)
16.	GEMV	<pre>using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300,300) X = rand(rng,Float32,300) elty = Float32 alpha = true beta = false m = 300 n = 300 trans = 'N'</pre>	29.921 μ s (19 allocations: 304 bytes)	29.400 μ s (19 allocations: 304 bytes)

		<pre> Y = rand(rng,Float32,300) @btime ccall(:sgemv_, libblaspath), Cvoid,(Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt}, Ref{elty},Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt},Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt}, Clong), trans, size(A,1), size(A,2), alpha, A, max(1, stride(A,2)), X, stride(X,1), beta, Y, stride(Y,1), 1); </pre>		
17.	GBMV	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300,300) X = rand(rng,Float32,300) elty = Float32 alpha = Float32(0) beta = Float32(0) m = 300 n = 300 trans = 'N' kl = 300 ku = 300 Y = rand(rng,Float32,300) @btime ccall(:sgbmv_, libblaspath),Cvoid,(Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt}, Ref{elty},Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt}, Clong), trans, m,size(A,1),kl,ku, alpha,A, max(1, stride(A,2)), X, stride(X,1),beta, Y, stride(Y,1), 1); </pre>	/	/
18.	SYMV	<pre> using LinearAlgebra using Random </pre>	17.250 μ s (19 allocations:	16.930 μ s (19

		<pre> rng = MersenneTwister(1234) A = rand(rng,Float32,300,300) x = rand(rng,Float32,300) elty = Float32 alpha = Float32(1) beta = Float32(1) m = 300 n = 300 y = rand(rng,Float32,300) uplo = 'L' @btime ccall(:ssymv_, libblaspath),Cvoid,(Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{elty}, Ptr{elty},Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ref{elty},Ptr{elty}, Ref{BLAS.BlasInt}, Clong), uplo, n, alpha, A, max(1, stride(A,2)), x, stride(x,1), beta, y, stride(y,1), 1); </pre>	304 bytes)	allocations: 304 bytes)
19.	SBMV	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300,300) x = rand(rng,Float32,300) elty = Float32 alpha = Float32(0) beta = Float32(0) m = 300 n = 300 y = rand(rng,Float32,300) uplo = 'L' @btime ccall(:ssbmv_, libblaspath),Cvoid,(Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt}, Ref{elty}, Ptr{elty},Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ref{elty},Ptr{elty}, Ref{BLAS.BlasInt}, Clong), uplo, n, 300,alpha, A, max(1, stride(A,2)), x, stride(x,1), beta, y, stride(y,1), 1); </pre>	/	/
20.	SPMV	<pre> using LinearAlgebra </pre>	10.010 μ s (17	10.130 μ s

		<pre> using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300,300) x = rand(rng,Float32,300) elty = Float32 alpha = Float32(0) beta = Float32(0) m = 300 n = 300 y = rand(rng,Float32,300) uplo = 'L' @btime ccall(:sspmv_, libblaspath),Cvoid,(Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{elty}, Ptr{elty},Ptr{elty}, Ref{BLAS.BlasInt}, Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt}, Clong), uplo, n, alpha, A,x, max(1, stride(x,1)), beta, y, stride(y,1), 1); </pre>	allocations: 272 bytes)	(17 allocations: 272 bytes)
21.	TRMV	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300,300) x = rand(rng,Float32,300) elty = Float32 alpha = Float32(0) beta = Float32(0) m = 300 n = 300 y = rand(rng,Float32,300) uplo = 'L' trans = 'N' diag = 'N' @btime ccall(:strmv_, libblaspath),Cvoid,(Ref{UInt8}, Ref{UInt8}, Ref{UInt8}, Ref{BLAS.BlasInt}, Ptr{elty},Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Clong, Clong, Clong), uplo, trans, diag, n, A, max(1, stride(A,2)), x, max(1, stride(x, 1)), 1, 1, 1); </pre>	21.520 μ s (14 allocations: 224 bytes)	12.360 μ s (14 allocations: 224 bytes)
22.	TRSV	<pre> using LinearAlgebra using Random </pre>	21.091 μs (14 allocations:	21.140 μ s (14

		<pre> rng = MersenneTwister(1234) A = rand(rng,Float32,300,300) x = rand(rng,Float32,300) elty = Float32 alpha = Float32(0) beta = Float32(0) m = 300 n = 300 y = rand(rng,Float32,300) uplo = 'L' trans = 'N' diag = 'N' @btime ccall(:strsv_, libblaspath),Cvoid,(Ref{UInt8}, Ref{UInt8}, Ref{UInt8}, Ref{BLAS.BlasInt}, Ptr{elty},Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Clong, Clong, Clong), uplo, trans, diag, n, A, max(1, stride(A,2)), x, max(1, stride(x, 1)), 1, 1, 1); </pre>	224 bytes)	allocations: 224 bytes)
23.	GER	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300,300) x = rand(rng,Float32,300) elty = Float32 alpha = Float32(0) beta = Float32(0) m = 300 n = 300 y = rand(rng,Float32,300) uplo = 'L' trans = 'N' diag = 'N' @btime ccall(:sger_, libblaspath),Cvoid,(Ref{BLAS.B lasInt},Ref{BLAS.BlasInt}, Ref{elty},Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty},Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}), m,n,alpha,x, stride(x,1),y, strid e(y,1),A,max(1, stride(A,2))); </pre>	9.260 μ s (16 allocations: 256 bytes)	9.370 μ s (16 allocations: 256 bytes)
24.	SYR	<pre> using LinearAlgebra </pre>	7.555 μ s (13	7.675 μ s (13

		<pre> using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300,300) x = rand(rng,Float32,300) elty = Float32 alpha = Float32(0) beta = Float32(0) m = 300 n = 300 y = rand(rng,Float32,300) uplo = 'L' trans = 'N' diag = 'N' @btime ccall(:ssyr_, libblaspath),Cvoid,(Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{elty},Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty},Ref{BLAS.BlasInt}), uplo,n,alpha,x,Stride(x,1),A,ma x(1,Stride(A,2))); </pre>	allocations: 208 bytes)	allocations: 208 bytes)
25.	SYR2K	<pre> using LinearAlgebra using Random rng = MersenneTwister(1234) A = rand(rng,Float32,300,300) B = rand(rng,Float32,300,300) C = rand(rng,Float32,300,300) x = rand(rng,Float32,300) elty = Float32 alpha = Float32(0) beta = Float32(0) m = 300 n = 300 k = 300 y = rand(rng,Float32,300) uplo = 'L' trans = 'N' diag = 'N' @btime ccall(:ssyr2k_, libblaspath),Cvoid,(Ref{UInt8}, Ref{UInt8}, Ref{BLAS.BlasInt}, Ref{BLAS.BlasInt}, Ref{elty}, Ptr{elty}, Ref{BLAS.BlasInt}, Ptr{elty}, Ref{BLAS.BlasInt}, Ref{elty},Ptr{elty}, Ref{BLAS.BlasInt}, Clong, Clong), uplo, trans, n, k, </pre>	17.310 μs (23 allocations: 368 bytes)	20.970 μ s (23 allocations: 368 bytes)

		alpha, A, max(1, stride(A, 2)), B, max(1, stride(B, 2)), beta, C, max(1, stride(C, 2)), 1, 1);		
--	--	--	--	--

表 3-5 BLAS 功能 Julia 函数性能对比测试

编号	函数	示例	KML	OpenBLAS
1.	*	<pre>using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10, 10) b = rand(rng, UInt32, 10, 10) using BenchmarkTools @btime \$a * \$b</pre>	1.617 μ s (3 allocations: 21.11 KiB)	2.410 μ s (3 allocations: 21.11 KiB)
		<pre>using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10) b = rand(rng, UInt32, 1, 10) using BenchmarkTools @btime \$a * \$b</pre>	1.074 μ s (5 allocations: 21.20 KiB)	1.070 μ s (5 allocations: 21.20 KiB)
2.	norm	<pre>using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10, 10) @btime norm(\$a);</pre>	557.425 ns (0 allocations: 0 bytes)	558.769 ns (0 allocations: 0 bytes)
3.	\	<pre>using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10, 10) b = rand(rng, UInt32, 10, 10) using BenchmarkTools @btime \$a \ \$b</pre>	报错	4.037 μ s (3 allocations: 1.89 KiB)
4.	symm	<pre>using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10, 10) b = rand(rng, UInt32, 10, 10) a = Symmetric(a) c = similar(b); using BenchmarkTools @btime mul!(c, a, b);</pre>	1.597 μ s (2 allocations: 20.62 KiB)	1.773 μ s (2 allocations: 20.62 KiB)

		<pre> using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10, 10) b = rand(rng, UInt32, 10, 10) a = Symmetric(a) using BenchmarkTools @btime \$a * \$b </pre>	1.686 μ s (3 allocations: 21.11 KiB)	2.746 μ s (3 allocations: 21.11 KiB)
5.	SYR2K	<pre> using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10, 10) b = rand(rng, UInt32, 10, 10) a = Symmetric(a) using BenchmarkTools @btime \$a * \$b + 0.5 * b; </pre>	2.382 μ s (5 allocations: 22.86 KiB)	3.470 μ s (5 allocations: 22.86 KiB)
6.	TRMM	<pre> using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10, 10) b = rand(rng, UInt32, 10, 10) a = Tridiagonal(a) c = similar(b) using BenchmarkTools @btime mul!(c, a, b); </pre>	213.687 ns (0 allocations: 0 bytes)	212.432 ns (0 allocations: 0 bytes)
		<pre> using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10, 10) b = rand(rng, UInt32, 10, 10) a = Tridiagonal(a) using BenchmarkTools @btime \$a * \$b; </pre>	245.132 ns (1 allocation: 496 bytes)	238.468 ns (1 allocation: 496 bytes)
7.	scal	<pre> using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10) b = rand(rng, UInt32, 10) using BenchmarkTools @btime b[1] .* a; </pre>	930.000 ns (4 allocations: 176 bytes)	841.452 ns (4 allocations: 176 bytes)
8.	Copy	<pre> using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10) </pre>	51.582 ns (1 allocation: 96 bytes)	51.672 ns (1 allocation: 96 bytes)

		<code>using BenchmarkTools @btime copy(a);</code>		
9.	AXPY	<code>using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 10) b = rand(rng, UInt32, 10) c = UInt32(3) using BenchmarkTools @btime c .* b + a;</code>	997.000 ns (4 allocations: 256 bytes)	925.344 ns (4 allocations: 256 bytes)
10.	DOT	<code>using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 1000) b = rand(rng, UInt32, 1000) c = UInt32(3) using BenchmarkTools @btime dot(a, b);</code>	367.633 ns (1 allocation: 16 bytes)	367.826 ns (1 allocation: 16 bytes)
11.	NRM2	<code>using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 1000) b = rand(rng, UInt32, 1000) c = UInt32(3) using BenchmarkTools @btime norm(a, 2);</code>	3.781 μ s (1 allocation: 16 bytes)	3.779 μ s (1 allocation: 16 bytes)
12.	ASUM	<code>using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 1000) b = rand(rng, UInt32, 1000) c = UInt32(3) using BenchmarkTools @btime norm(a, 1);</code>	669.120 ns (1 allocation: 16 bytes)	706.436 ns (1 allocation: 16 bytes)
13.	GEMV	<code>using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 1000, 1000) b = rand(rng, UInt32, 1000) c = similar(b) using BenchmarkTools @btime mul!(c, a, b);</code>	358.623 μ s (0 allocations: 0 bytes)	366.753 μ s (0 allocations: 0 bytes)
14.	symv	<code>using Random</code>	3.549 ms (0	3.542 ms (0

		<pre> rng = MersenneTwister(1234) a = rand(rng, UInt32, 1000, 1000) b = rand(rng, UInt32, 1000) a = Symmetric(a) c = similar(b) using BenchmarkTools @btime mul!(c, a, b); </pre>	allocations: 0 bytes)	allocations: 0 bytes)
15.	SPMV	<pre> using Random using SparseArrays rng = MersenneTwister(1234) a = sprand(rng, UInt32, 1000, 1000, 0.05) b = rand(rng, UInt32, 1000) a = Symmetric(a) c = similar(b) using BenchmarkTools @btime mul!(c, a, b); </pre>	85.221 μ s (0 allocations: 0 bytes)	84.901 μ s (0 allocations: 0 bytes)
16.	TRMV	<pre> using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 1000, 1000) b = rand(rng, UInt32, 1000) a = Tridiagonal(a) c = similar(b) using BenchmarkTools @btime mul!(c, a, b); </pre>	1.686 μ s (0 allocations: 0 bytes)	1.684 μ s (0 allocations: 0 bytes)
17.	TBMV	<pre> using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 1000, 1000) b = rand(rng, UInt32, 1000) a = LowerTriangular(a) c = similar(b) using BenchmarkTools @btime mul!(c, a, b); </pre>	768.207 μ s (0 allocations: 0 bytes)	804.047 μ s (0 allocations: 0 bytes)
18.	GER	<pre> using Random rng = MersenneTwister(1234) a = </pre>	无 32 位接口	无 32 位接口

		<pre> rand(rng, UInt32, 1000, 1000) b = rand(rng, UInt32, 1000) c = rand(rng, UInt32, 1000) alpha = 1.5 using BenchmarkTools @btime BLAS.ger!(alpha, b, c, a); </pre>		
19.	SYR	<pre> using Random rng = MersenneTwister(1234) a = rand(rng, UInt32, 1000, 1000) b = rand(rng, UInt32, 1000) c = rand(rng, UInt32, 1000) alpha = 1.5 a = UpperTriangular(a) using BenchmarkTools @btime BLAS.syr!(a, alpha, b); </pre>	/	/

3.3.5 测试结论

截止 2023/05/22，通过对单元函数测试，以动态链接库中函数和 BLAS 功能 Julia 函数作为目标，对 KMLBLAS 和 OpenBLAS 在鲲鹏提供 Kunpeng-920 芯片、麒麟 V10 操作系统下进行性能测试，现具体分析如下：

直接调用动态链接库中函数测试总数 25 个，除去无法对比或调用出错后 21 个，其中 KML 性能占优 4 个、性能持平 16 个、OpenBLAS 性能占优 1 个。

直接调用动态链接库中函数测试总数 19 个，除去无法对比或调用出错后 16 个，其中 KML 性能占优 5 个、性能持平 12 个、OpenBLAS 性能占有 2 个。

3.4 切换 FFTW 库

该章节以 KMLFFT 为例，讲述如何切换 FFTW 库。其主要方法为通过构建依赖 KML 的函数库，实现 KMLFFT 的调用和切换。

3.4.1 构建依赖 KML 库

基于已有 FFTW.jl 库进行修改构建出依赖 KML 的函数库。

3.4.1.1 符号与变量名修改

KML 中 libkfft.so 导出 C 接口名称前缀统一为:”fft”, 与 FFTW 中前缀”fftw”存在差异, 故将库中 “fftw_” 一键替换为 “fft_”, “fftwf_” 替换为 “fftf_”, 注意区分大小写。

3.4.1.2 函数注释与修改

1. KML 中未提供 `fft_set_timelimit`、`fftf_set_timelimit` 函数, 故注释 `unsafe_set_timelimit` 函数实现与调用, 可将 `unsafe_set_timelimit` 一键替换为 “# unsafe_set_timelimit”。
2. 注释 `fft_init_threads()` 调用。
3. 将文件中 “mkl” 替换为 “kml”, 区分大小写。
4. KML 中提供版本接口函数为 `KFFTGetVersion`, 将 “const version” 替换为 “# const version”, “(:KFFTGetVersion ” 替换为 “# (:KFFTGetVersion”。
5. “version >= v"3.3.4" &&” 更改为 “#= version >= v"3.3.4" && =#”。
6. `fft.jl` 文件中 212–217 行注释。
7. `fft.jl` 文件中 643 行 “fftw” 和 “fftwf” 分别修改为 “fft” 和 “fftf”。

3.4.1.3 路径修改

1. 将 `libffft3`、`libffft3f` 修改为 `libkfft.so` 的路径, 类型为字符串(String)。
2. “MKL_jll.librt_path” 替换为 `libkfft.so` 路径。

3.4.2 函数库使用

3.4.2.1 安装函数库

使用 `Pkg.dev` 函数或 `Pkg` 模式下使用 `dev` 功能进行函数库安装。

3.4.2.2 底层切换

利用 `FFTW.set_provider!("kml")` 方法将依赖切换至 KML。

3.4.2.3 重新启动

重新启动 Julia 以实现底层调用，利用 `FFTW.get_provider!()` 函数查看底层依赖。

```
julia> FFTW.libfftw3
Base.RefValue{String}("/home/HPCBase/libs/kml/1.7.0_gcc/lib/libkfft.so")
```

3.4.3 KMLFFT 测试报告

基于上述替换方法，切换 KMLFFT 底层算法依赖，并对其性能进行测试。

3.4.3.1 测试环境

表 3-6 KML 验证环境

处理器	Architecture: aarch64 CPU op-mode(s): 64-bit Byte Order: Little Endian CPU(s): 16 On-line CPU(s) list: 0-15 Thread(s) per core: 1 Core(s) per socket: 16 Socket(s): 1 NUMA node(s): 1 Vendor ID: HiSilicon Model: 0 Model name: Kunpeng-920 Stepping: 0x1 CPU max MHz: 2400.0000 CPU min MHz: 2400.0000 BogoMIPS: 200.00 L1d cache: 1 MiB L1i cache: 1 MiB L2 cache: 8 MiB L3 cache: 32 MiB NUMA node0 CPU(s): 0-15
内存	126G
操作系统	麒麟 V10

3.4.3.2 函数列表

测试目标函数选取原则：Syslab 基础数学库中提供如下表所示函数接口且在科学计算和工程应用中下表函数均为高频函数，故以如下函数接口作为目标测试函数。

表 3-7 测试目标函数功能列表

编号	函数名称	功能定义
1.	fft	快速傅里叶变换
2.	fftshift	将零频分量移到频谱中心
3.	ifft	快速傅里叶逆变换
4.	ifftshift	逆零频平移
5.	bfft	类似 ifft,计算非归一化逆（向后）变换
6.	rfft	实数组 A 的多维 FFT
7.	dct	执行多维 II 型离散余弦变换 (DCT) 数组 A，使用 DCT 的酉归一化。
8.	idct	计算多维反离散余弦变换阵列 A 的 (DCT)
9.	r2r	执行数组 A 类型的多维实数输入/实数输出 (r2r) 变换

3.4.3.3 测试

在 FFT 功能中，plan 对性能的加速是很明显的，由于该测试目的为给出 KMLFFT 功能的性能测试，故所有测试均源自默认 plan，以展示在直接使用默认条件下的性能差异对比。

每个函数选取一维(1000)、二维(1000 × 1000)、三维(1000 × 1000 × 100)数据进行性能测试。

3.4.3.3.1 单元测试结果及对比

如表 3-1 所示，展示了 KML 与 FFTW 在相同示例情况下的性能对比，其中红色标记字体表示性能占有的数据。

表 3-8 性能对比测试

编号	函数	示例	FFTW	KML
1.	fft	<pre>rng = MersenneTwister(1234) a = rand(rng,1000) @btime fft(a);</pre>	36.910 μ s	50.831 μ s
		<pre>rng = MersenneTwister(1234) a = rand(rng,1000,1000) @btime fft(a);</pre>	48.897 ms	28.608 ms
		<pre>using Random rng = MersenneTwister(1234) a = rand(rng,1000,1000,100) @btime fft(a);</pre>	7.527 s (8 allocations: 2.98 GiB)	7.081 s (31 allocations: 2.98 GiB)
2.	fftshift	<pre>rng = MersenneTwister(1234) a = rand(rng,1000) @btime fftshift(a);</pre>	875.135 ns	1.012 μ s
		<pre>rng = MersenneTwister(1234) a = rand(rng,1000,1000) @btime fftshift(a);</pre>	1.251 ms	1.293 ms
		<pre>using Random rng = MersenneTwister(1234) a = rand(rng,1000,1000,100) @btime fftshift(a);</pre>	178.239 ms (10 allocations: 762.94 MiB)	177.492 ms (10 allocations: 762.94 MiB)
3.	ifft	<pre>rng = MersenneTwister(1234) a = rand(rng,1000) @btime ifft(a);</pre>	38.300 μ s	52.070 μ s
		<pre>rng = MersenneTwister(1234) a = rand(rng,1000,1000) @btime ifft(a);</pre>	50.584 ms	29.896 ms
		<pre>using Random rng = MersenneTwister(1234) a = rand(rng,1000,1000,100) @btime ifft(a);</pre>	8.344 s (8 allocations: 2.98 GiB)	7.025 s (31 allocations: 2.98 GiB)
4.	ifftshift	<pre>rng = MersenneTwister(1234) a = rand(rng,1000) @btime ifftshift(a);</pre>	839.118 ns	864.561 ns
		<pre>rng = MersenneTwister(1234) a = rand(rng,1000,1000)</pre>	1.291 ms	1.270 ms

		@btime ifftshift(a);		
		<pre>using Random rng = MersenneTwister(1234) a = rand(rng,1000,1000,100) @btime ifftshift(a);</pre>	193.375 ms (10 allocations: 762.94 MiB)	177.540 ms (10 allocations: 762.94 MiB)
5.	bfft	<pre>rng = MersenneTwister(1234) a = rand(rng,1000) @btime bfft(a);</pre>	36.970 μ s	50.900 μ s
		<pre>rng = MersenneTwister(1234) a = rand(rng,1000,1000) @btime bfft(a);</pre>	51.413 ms	27.957 ms
		<pre>using Random rng = MersenneTwister(1234) a = rand(rng,1000,1000,100) @btime bfft(a);</pre>	7.894 s (8 allocations: 2.98 GiB)	6.850 s (31 allocations: 2.98 GiB)
6.	rfft	<pre>rng = MersenneTwister(1234) a = rand(rng,1000) @btime rfft(a);</pre>	42.200 μ s	35.390 μ s
		<pre>rng = MersenneTwister(1234) a = rand(rng,1000,1000) @btime rfft(a);</pre>	22.255 ms	13.988 ms
		<pre>using Random rng = MersenneTwister(1234) a = rand(rng,1000,1000,100) @btime rfft(a);</pre>	1.915 s (8 allocations: 764.47 MiB)	3.144 s (31 allocations: 764.47 MiB)
7.	dct	<pre>rng = MersenneTwister(1234) a = rand(rng,1000) @btime dct(a,1);</pre>	69.091 μ s	83.020 μ s
		<pre>rng = MersenneTwister(1234) a = rand(rng,1000,1000) @btime dct(a,1);</pre>	12.439 ms	8.705 ms
		<pre>using Random rng = MersenneTwister(1234) a = rand(rng,1000,1000,100) @btime dct(a,1);</pre>	951.359 ms (20 allocations: 764.47 MiB)	1.171 s (40 allocations: 764.47 MiB)

8.	idct	<pre>rng = MersenneTwister(1234) a = rand(rng,1000) @btime idct(a,1);</pre>	70.100 μ s	82.501 μ s
		<pre>rng = MersenneTwister(1234) a = rand(rng,1000,1000) @btime idct(a,1);</pre>	11.933 ms	9.052 ms
		<pre>using Random rng = MersenneTwister(1234) a = rand(rng,1000,1000,100) @btime idct(a,1);</pre>	1.061 s (22 allocations: 1.49 GiB)	1.323 s (42 allocations: 1.49 GiB)
9.	r2r	<pre>rng = MersenneTwister(1234) a = rand(rng,1000) @btime FFTW.r2r(a,1);</pre>	58.881 μ s	39.800 μ s
		<pre>rng = MersenneTwister(1234) a = rand(rng,1000,1000) @btime FFTW.r2r(a,1);</pre>	24.106 ms	13.930 ms
		<pre>using Random rng = MersenneTwister(1234) a = rand(rng,1000,1000,100) @btime FFTW.r2r(a,1);</pre>	6.031 s (6 allocations: 762.94 MiB)	3.856 s (26 allocations: 762.94 MiB)

3.4.3.3.2 综合示例结果对比

示例一

选取利用傅里叶变换将原始时域信号转换为频谱信号并重构过程作为示例一，对整个过程进行对比测试。

示例代码：

```
fun(x) = 5 * cos(1.9 * pi * x + pi / 4) + 3 * cos(4.97 * 2 *
pi * x + pi / 6) + 8 * cos(10 * 2 * pi * x + pi / 3) + 6 *
cos(15.99 * 2 * pi * x + pi / 3)
t_0 = 0:1/1000:8 # 采样步长 1/1000, 采样时间为 8s
y_0 = fun.(t_0)
N_0 = length(t_0) # 原始采样点数

# 数据采样处理
Fs = 256 # 信号采样频率,根据采样定理, 信号采样频率应大于信号频
率的 2 倍以上
t = LinRange(0, 5pi, Int(floor(Fs * 5pi))) # 采样点区间
N = length(t) - 1 # 采样点数
df = Fs / N # 频率分辨率
```

```

# FFT 变换
f = Fs / N * (0:N-1)
YFFT = fft(y_0)
mag = abs.(YFFT) # 获取原始模值
phase = angle.(YFFT) * 180 / pi #*相位角
# 获取原始信号幅值
magy = mag / N
magy = magy[1:Int(N / 2 + 1)]
magy[2:end-1] = magy[2:end-1] * 2
subplot(4, 1, 1)
plot(t_0, y_0)
grid("on")
xlim([0, pi])
xlabel("time(s)")
ylabel("Value")
title("原始时域信号")

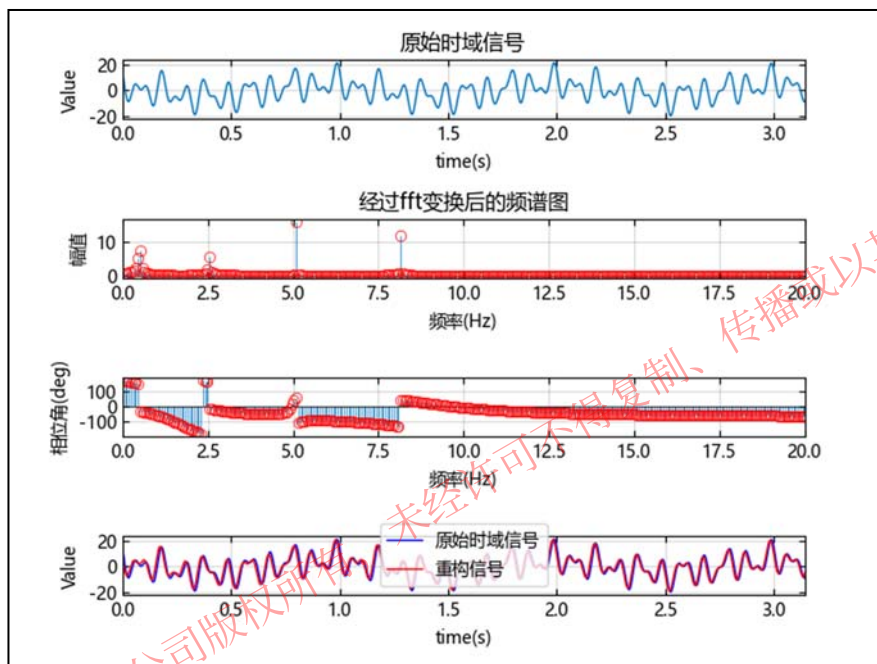
subplot(4, 1, 2)
stem(f[1:Int(N / 2)], magy[1:Int(N / 2)], "r",
      markeredgecolor="r")
xlabel("频率(Hz)")
ylabel("幅值")
title("经过 fft 变换后的频谱图")
grid("on")
xlim([0, 20])

subplot(4, 1, 3)
stem(f[1:Int(N / 2)], phase[1:Int(N / 2)], "r",
      markeredgecolor="r")
grid("on")
xlim([0, 20])
xlabel("频率(Hz)")
ylabel("相位角(deg)")
# 重构信号
fo1(t) = 4.96cos(6.03t + 0.56)
fo2(t) = 2.98cos(31.21t + 0.80)
fo3(t) = 7.83cos(62.82t + 1.42)
fo4(t) = 5.47cos(100.41t + 1.78)
yfft = fo1.(t_0) + fo2.(t_0) + fo3.(t_0) + fo4.(t_0)
# 绘制重构信号与原始信号对比图
subplot(4, 1, 4)
plot(t_0, y_0, "b")
hold("on")
plot(t_0, yfft, "r")
grid("on")
xlim([0, pi])
xlabel("time(s)")

```

```
ylabel("Value")
legend(["原始时域信号", "重构信号"])
hold("off")
tightlayout()
```

运行后图像:



排除绘图功能后代码:

```
fun(x) = 5 * cos(1.9 * pi * x + pi / 4) + 3 * cos(4.97 * 2 *
pi * x + pi / 6) + 8 * cos(10 * 2 * pi * x + pi / 3) + 6 *
cos(15.99 * 2 * pi * x + pi / 3)
t_0 = 0:1/1000:8 # 采样步长 1/1000, 采样时间为 8s
y_0 = fun.(t_0)
N_0 = length(t_0) # 原始采样点数

# 数据采样处理
Fs = 256 # 信号采样频率,根据采样定理,信号采样频率应大于信号频
率的 2 倍以上
t = LinRange(0, 5pi, Int(floor(Fs * 5pi))) # 采样点区间
N = length(t) - 1 # 采样点数
df = Fs / N #*频率分辨率
# FFT 变换
```

```
f = Fs / N * (0:N-1)
YFFT = fft(y_0)
mag = abs.(YFFT) # 获取原始模值
phase = angle.(YFFT) * 180 / pi #*相位角
# 获取原始信号幅值
magy = mag / N
magy = magy[1:Int(N / 2 + 1)]
magy[2:end-1] = magy[2:end-1] * 2
```

利用 include 加载文件、@benchmark 宏对整体进行性能测试：

FFTW:

```
julia> include(pwd()*"/KML/FFT/FFTy Zheng/FFTW.jl-master/examples/fft_eg1.jl")
BenchmarkTools.Trial: 2373 samples with 1 evaluation.
Range (min ... max): 2.057 ms ... 3.165 ms | GC (min ... max): 0.00% ... 18.04%
Time (median): 2.082 ms | GC (median): 0.00%
Time (mean ± σ): 2.100 ms ± 106.633 μs | GC (mean ± σ): 0.39% ± 2.27%

Histogram: log(frequency) by time
2.06 ms 2.82 ms <

Memory estimate: 675.22 KiB, allocs estimate: 44.
```

KML:

```
BenchmarkTools.Trial: 1233 samples with 1 evaluation.
Range (min ... max): 4.002 ms ... 4.877 ms | GC (min ... max): 0.00% ... 14.00%
Time (median): 4.031 ms | GC (median): 0.00%
Time (mean ± σ): 4.050 ms ± 93.243 μs | GC (mean ± σ): 0.23% ± 1.59%

Histogram: frequency by time
4 ms 4.76 ms <

Memory estimate: 673.80 KiB, allocs estimate: 23.
```

示例二

选取帮助文档中余弦波示例，比较时域和频域中的余弦波。

示例代码：

```
Fs = 1000
T = 1 / Fs
L = 1000
t = (0:L-1) * T
x1 = cos.(2 * pi * 50 * t)
x2 = cos.(2 * pi * 150 * t)
x3 = cos.(2 * pi * 300 * t)

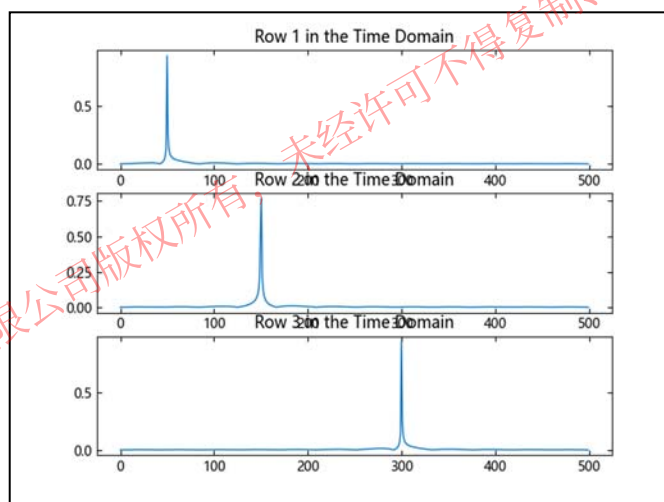
X = hcat(x1, x2, x3)
for i = 1:3
    ax = subplot(3, 1, i)
```

```

plot(ax, t[1:100], X[1:100, i])
title("Row $i in the Time Domain")
end
n = 2^nextpow2(L)
dim = 1
X = vcat(X, zeros(n - L, 3))
Y = fft(X, dim)
P2 = abs.(Y / L)
P1 = P2[1:Int(n / 2)+1, :]
P1[2:end-1, :] = 2 * P1[2:end-1, :]
for i = 1:3
    ax = subplot(3, 1, i)
    plot(ax, 0:(Fs/n):(Fs/2-Fs/n), P1[1:Int(n / 2), i])
    title("Row $i in the Time Domain")
end

```

根据上述代码得到结果图像为：



排除绘图功能后代码：

```

function nextpow2(x)
    if x == 0
        return 0
    end
    n, f = nextpow2_alg(abs(x))
    if f == 0.5
        n -= 1
    end
    return n
end
function nextpow2_alg(x)

```



```

z = log2(x)
e = floor(Int, z) + 1
f = x / exp2(e)
return e, f
end

@benchmark begin
  Fs = 1000
  T = 1 / Fs
  L = 1000
  t = (0:L-1) * T
  x1 = cos.(2 * pi * 50 * t)
  x2 = cos.(2 * pi * 150 * t)
  x3 = cos.(2 * pi * 300 * t)

  X = hcat(x1, x2, x3)
  n = 2^nextpow2(L)
  dim = 1
  X = vcat(X, zeros(n - L, 3))
  Y = fft(X, dim)
  P2 = abs.(Y / L)
  P1 = P2[1:Int(n / 2)+1, :]
  P1[2:end-1, :] = 2 * P1[2:end-1, :]
end

```

FFTW:

```

julia> include(pwd()*"/KML/FFT/FFTyangzheng/FFTW.jl-master/examples/fft_eg2.jl")
BenchmarkTools.Trial: 10000 samples with 1 evaluation.
Range (min ... max): 307.083 μs ... 1.623 ms | GC (min ... max): 0.00% ... 66.49%
Time (median): 314.963 μs | GC (median): 0.00%
Time (mean ± σ): 323.997 μs ± 91.364 μs | GC (mean ± σ): 1.77% ± 4.90%

Histogram: frequency by time
307 μs 363 μs <

Memory estimate: 278.41 KiB, allocs estimate: 42.

```

KML:

```

julia> include(pwd()*"/KML/FFT/FFTyangzheng/FFTW.jl-master/examples/fft_eg2.jl")
BenchmarkTools.Trial: 10000 samples with 1 evaluation.
Range (min ... max): 321.473 μs ... 1.710 ms | GC (min ... max): 0.00% ... 67.26%
Time (median): 326.793 μs | GC (median): 0.00%
Time (mean ± σ): 336.287 μs ± 93.322 μs | GC (mean ± σ): 1.87% ± 5.28%

Histogram: log(frequency) by time
321 μs 370 μs <

Memory estimate: 276.89 KiB, allocs estimate: 22.

```

示例三

选取总数据量为一千万的向量、矩阵、三维数组分别测试在执行 fft(快速傅

里叶变换)、傅里叶逆变换(iff)后的精度和性能。

精度测试:

类型	示例	KML	FFTW
向量	<pre>using Test rng = MersenneTwister(1234) a = rand(rng,100000000); @test isapprox(a,iff(fft(a)),atol = 1e-8)</pre>	通过	通过
矩阵	<pre>using Test rng = MersenneTwister(1234) a = rand(rng,10000,10000); @test isapprox(a,iff(fft(a)),atol = 1e-8)</pre>	通过	通过
三维数组	<pre>using Test rng = MersenneTwister(1234) a = rand(rng,10000,100,100); @test isapprox(a,iff(fft(a)),atol = 1e-8)</pre>	通过	通过

性能测试:

类型	示例	KML	FFTW
向量	<pre>using Test rng = MersenneTwister(1234) a = rand(rng,100000000); @btime iff(fft(a));</pre>	17.251 s (14 allocations: 4.47 GiB)	16.171 s (56 allocations: 4.47 GiB)
矩阵	<pre>using Test rng = MersenneTwister(1234) a = rand(rng,10000,10000); @btime iff(fft(a));</pre>	7.564 s (14 allocations: 4.47 GiB)	15.733 s (58 allocations: 4.47 GiB)
三维数组	<pre>using Test rng = MersenneTwister(1234) a = rand(rng,10000,100,100); @btime iff(fft(a));</pre>	12.792 s (14 allocations: 4.47 GiB)	10.783 s (60 allocations: 4.47 GiB)

示例四

傅里叶变换在工业应用中有一个常见场景：频谱分析。

频谱分析是通过对信号进行傅里叶变换，将信号从时域转换到频域，以获取信号中各个频率分量的信息。在工业领域中，频谱分析被广泛应用于以下方面：

1. 故障诊断和预测：频谱分析可用于监测和分析机械系统、电气设备或其他工业设备的振动或声音信号。通过对信号进行频谱分析，可以检测到异常的频率分量，识别设备故障，并预测设备的寿命。
2. 声学分析：频谱分析可用于处理声音信号，如音频处理、噪音分析和语音识别。通过分析声音信号的频谱特征，可以实现音频增强、噪音消除、声纹识别等应用。
3. 通信系统：频谱分析在无线通信系统中起着重要作用。通过分析信号的频谱特征，可以确定信道的带宽、检测信号干扰、优化信号调制和解调方法等，以提高通信系统的性能和可靠性。
4. 信号处理和滤波：频谱分析可用于信号处理和滤波应用。通过了解信号的频谱特征，可以设计滤波器来滤除不需要的频率分量，提取感兴趣的频率分量，实现信号的降噪、压缩、调制等处理操作。

示例三通过傅里叶变换获取信号的频谱信息，并对频谱进行分析。在这个示例中，我们首先定义了三个函数：`generate_signal` 用于生成模拟信号，`perform_fft` 用于对信号进行傅里叶变换，`analyze_spectrum` 用于对频谱进行分析。我们生成了一个包含多个频率分量的模拟信号，并对信号进行傅里叶变换和频谱分析。最后，我们打印出信号的频率分析结果。

实际代码：

```
# 生成模拟信号
function generate_signal(t, freqs, amplitudes)
    signal = zeros(length(t))
    for i in 1:length(freqs)
        signal += amplitudes[i] * sin.(2π * freqs[i] * t)
    end
    return signal
end

# 对信号进行傅里叶变换
function perform_fft(signal)
```

```

        return fft(signal)
    end

# 频谱分析
function analyze_spectrum(freqs, spectrum)
    max_freq_index = argmax(abs.(spectrum[1:end ÷ 2])) #
    只分析正频率部分
    max_freq = freqs[max_freq_index]
    return max_freq
end

# 工业应用实例
function industrial_application()
    # 模拟信号参数
    fs = 100000 # 采样频率
    duration = 1000.0 # 信号时长
    t = 0:1/fs:duration # 时间序列

    # 信号频率和幅度
    freqs = [50, 120, 250] # 信号频率
    amplitudes = [1.0, 0.8, 0.5] # 信号幅度

    # 生成模拟信号
    signal = generate_signal(t, freqs, amplitudes)

    # 傅里叶变换
    spectrum = perform_fft(signal)

    # 频谱分析
    max_freq =
    analyze_spectrum(fftshift(fftshift(length(signal), 1/fs)),
    spectrum)

    println("工业应用实例: ")
    println("信号频率分量: ", freqs)
    println("信号频率分析结果: ")
    println("最大频率: ", max_freq)
end

# 运行工业应用实例
industrial_application()

```

对 industrial_application 函数进行性能对比测试:

KML	FTTW
-----	------

71.144 s (99 allocations: 11.55 GiB)	72.269 s (121 allocations: 11.55 GiB)
--------------------------------------	---------------------------------------

3.4.3.4 测试结论

截止 2023/05/30，通过对单元函数和综合示例测试，主要进行了傅里叶变换主要接库函数和利用 fft 函数撰写的两个综合示例在鲲鹏提供 Kunpeng-920 芯片、麒麟 V10 操作系统下进行性能测试，汇总统计如下：

- 1. 性能测试目标函数 9 个。
- 2. 单元测试用例 27 个，性能占优 9 个，基本持平 4 个，性能略差 14 个。
- 3. 综合案例 4 类，测试用例 6 个，性能占优 1 个，基本持平 3 个，性能略差 2 个。

具体分析如下：

- 1. 向量情况下，变换类函数如：fft、ifft、bfft、dct，FFTW 性能略高于 KML
- 2. 矩阵情况下，变换类函数如：fft、ifft、bfft、dct，KML 性能略高于 FFTW
- 3. 三维数组情况下，变换类函数如：fft、ifft、bfft,r2r，KML 性能略高于 FFTW
- 4. 实数变换 KML 性能略高于 FFTW，如：rfft、r2r
- 5. 移位函数 FFTW 性能略高于 KML，如：fftshift、ifftshift

苏州同元软控信息技术有限公司版权所有，未经许可不得复制、传播或以其他方式使用；
版权所有，侵权必究。